MLOps, Ethics & Career

Book 6 of the AI Series

Deploy, Monitor, and Scale AI Systems Responsibly

2025 Edition • 25 Chapters

Table of Contents

Chapter 1 — Notebook to Production
Chapter 2 — Saving and Loading Models
Chapter 3 — FastAPI for ML
Chapter 4 — Docker
Chapter 5 — Cloud Deployment
Chapter 6 — Streamlit and Gradio
Chapter 7 — Model Serving
Chapter 8 — ML Lifecycle
Chapter 9 — Experiment Tracking
Chapter 10 — Data Versioning
Chapter 11 — Model Registry
Chapter 12 — CI/CD for ML
Chapter 13 — Automated Retraining
Chapter 14 — Feature Stores
Chapter 15 — Model Monitoring
Chapter 16 — A/B Testing
Chapter 17 — Model Optimization
Chapter 18 — Cost Optimization
Chapter 19 — Bias in AI
Chapter 20 — Explainability
Chapter 21 — Privacy
Chapter 22 — AI Regulations
Chapter 23 — Capstone Project
Chapter 24 — Building a Portfolio
Chapter 25 — AI Career Guide

Chapter 1: From Notebook to Production

What You Will Learn

In this chapter, you will learn:

- Why moving code from notebooks to production matters
- The key differences between notebook code and production code
- How to refactor messy notebook code into clean Python scripts
- How to organize an ML project with a proper folder structure
- How to use configuration files to manage settings
- How to add logging so you can track what your code does

Why This Chapter Matters

Imagine you are a chef. You have been experimenting with recipes in your home kitchen. You try a little of this, a little of that. You taste as you go. Your kitchen is messy, but the food tastes great.

Now imagine someone says, "That dish is amazing! Can you make it for 500 people every day?" Suddenly, your messy home kitchen approach will not work. You need a commercial kitchen, standardized recipes, proper equipment, and a team that can follow your process.

This is exactly what happens in machine learning. You build a great model in a Jupyter notebook. It works! But then someone asks, "Can we use this in our app?" or "Can this run automatically every day?" That is when you need to move from notebook to production.

This chapter is the bridge between "it works on my laptop" and "it works for real users."

The Problem with Notebooks

Jupyter notebooks are wonderful for exploration and learning. They let you run code cell by cell, see results immediately, and mix code with explanations. But they have serious problems when you try to use them in the real world.

What Makes Notebooks Great for Learning

```
+-----------------------------------------+
|  Jupyter Notebook                       |
|                                         |
|  [Cell 1] Load data         -> Output   |
|  [Cell 2] Explore data      -> Charts   |
|  [Cell 3] Clean data        -> Output   |
|  [Cell 4] Train model       -> Metrics  |
|  [Cell 5] Evaluate          -> Results  |
|                                         |
|  Interactive! Visual! Easy to change!   |
+-----------------------------------------+
```

What Makes Notebooks Bad for Production

Here are the main problems:

Problem 1: Cells can run out of order.

In a notebook, you might run Cell 5, then go back and change Cell 2, then run Cell 4. The notebook remembers the old Cell 2 results. This creates hidden bugs that are hard to find.

Problem 2: Hidden state.

Variables from deleted cells might still be in memory. Your code works now, but if you restart the notebook and run everything from the top, it might break.

Problem 3: No error handling.

When you explore data, you do not think about what happens when the data file is missing, or the data has unexpected values. Production code must handle these situations.

Problem 4: Hard to test.

You cannot easily write automated tests for notebook code. Tests are essential in production.

Problem 5: Hard to version control.

Notebooks are stored as JSON files. When two people edit the same notebook, merging changes is a nightmare.

Problem 6: Cannot be imported.

You cannot import functions from a notebook into another Python file. In production, you want reusable code.

Notebook Code vs Production Code

Let us look at a concrete example. Here is typical notebook code for training a simple model:

The Notebook Way (Messy but Common)

```python
# Cell 1
import pandas as pd
df = pd.read_csv("data.csv")
df.head()

# Cell 2
df.shape

# Cell 3
df.isnull().sum()

# Cell 4
df = df.dropna()

# Cell 5
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

X = df.drop("target", axis=1)
y = df["target"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Cell 6
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

# Cell 7
predictions = model.predict(X_test)
accuracy = accuracy_score(y_test, predictions)
print(f"Accuracy: {accuracy}")
```

```
Output:
Accuracy: 0.92
```

This code works, but it has many problems:

- File path is hardcoded ("data.csv")
- No error handling (what if the file does not exist?)
- No logging (how do we know what happened when it ran?)
- Everything is in one place (cannot reuse parts)
- Magic numbers everywhere (100 trees, 0.2 test size, 42 random state)

The Production Way (Clean and Reliable)

Now let us see what production code looks like. We will break it into multiple files:

File: config.yaml

```yaml
# Configuration file for our ML project
# Change settings here without touching code

data:
  input_path: "data/raw/dataset.csv"
  test_size: 0.2
  random_state: 42

model:
  type: "random_forest"
  n_estimators: 100
  random_state: 42

output:
  model_path: "models/trained_model.pkl"
  log_file: "logs/training.log"
```

File: src/data_loader.py

```python
"""
data_loader.py - Functions for loading and preparing data.

This module handles all data-related operations:
reading files, cleaning data, and splitting into
training and test sets.
"""

import pandas as pd
```

```
from sklearn.model_selection import train_test_split
import logging

# Create a logger for this module
# A logger is like a diary that records what your code does
logger = logging.getLogger(__name__)


def load_data(file_path):
    """
    Load data from a CSV file.

    Parameters
    ----------
    file_path : str
        Path to the CSV file.

    Returns
    -------
    pd.DataFrame
        The loaded data.

    Raises
    ------
    FileNotFoundError
        If the file does not exist.
    """
    logger.info(f"Loading data from {file_path}")

    try:
        df = pd.read_csv(file_path)
        logger.info(f"Loaded {len(df)} rows and {len(df.columns)} columns")
        return df
    except FileNotFoundError:
        logger.error(f"File not found: {file_path}")
        raise
    except Exception as e:
        logger.error(f"Error loading data: {e}")
        raise


def clean_data(df):
    """
    Clean the data by removing rows with missing values.

    Parameters
    ----------
    df : pd.DataFrame
        The raw data.

    Returns
    -------
    pd.DataFrame
        The cleaned data.
    """
```

```
    original_rows = len(df)
    df_clean = df.dropna()
    removed_rows = original_rows - len(df_clean)

    logger.info(f"Removed {removed_rows} rows with missing values")
    logger.info(f"Clean data has {len(df_clean)} rows")

    return df_clean


def split_data(df, target_column, test_size=0.2, random_state=42):
    """
    Split data into training and test sets.

    Parameters
    ----------
    df : pd.DataFrame
        The cleaned data.
    target_column : str
        Name of the column to predict.
    test_size : float
        Fraction of data to use for testing (0.0 to 1.0).
    random_state : int
        Random seed for reproducibility.

    Returns
    -------
    tuple
        (X_train, X_test, y_train, y_test)
    """
    X = df.drop(target_column, axis=1)
    y = df[target_column]

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=test_size, random_state=random_state
    )

    logger.info(f"Training set: {len(X_train)} samples")
    logger.info(f"Test set: {len(X_test)} samples")

    return X_train, X_test, y_train, y_test
```

```
Output (when used):
INFO - Loading data from data/raw/dataset.csv
INFO - Loaded 1000 rows and 11 columns
INFO - Removed 15 rows with missing values
INFO - Clean data has 985 rows
INFO - Training set: 788 samples
INFO - Test set: 197 samples
```

Let us break down what makes this better:

Line-by-line explanation of key parts:

```python
logger = logging.getLogger(__name__)
```

This creates a logger. Think of it as a diary for your code. `__name__` is a special Python variable that holds the name of the current file. This way, when you read the logs, you know which file wrote each message.

```python
def load_data(file_path):
```

Instead of hardcoding "data.csv", we accept the file path as a parameter. This makes the function flexible and reusable.

```python
    try:
        df = pd.read_csv(file_path)
    except FileNotFoundError:
        logger.error(f"File not found: {file_path}")
        raise
```

The `try/except` block catches errors gracefully. If the file does not exist, we log a helpful error message before raising the error. In a notebook, the error would just crash. In production, we want to know exactly what went wrong.

```python
    """
    Load data from a CSV file.

    Parameters
    ----------
    ...
    """
```

This is a docstring. It explains what the function does, what it needs, and what it returns. Other developers (and future you) will thank you for writing these.

Project Structure

A well-organized project is like a well-organized house. Everything has its place, and anyone can find what they need.

Here is a recommended structure for an ML project:

```
my-ml-project/
│
├── README.md              # What this project does
├── requirements.txt       # Python packages needed
├── config.yaml            # Settings and parameters
├── setup.py               # Package installation file
│
├── data/
│   ├── raw/               # Original, unchanged data
│   ├── processed/         # Cleaned, ready-to-use data
│   └── external/          # Data from outside sources
│
├── notebooks/
│   ├── 01-exploration.ipynb    # Data exploration
│   └── 02-experiments.ipynb    # Model experiments
│
├── src/
│   ├── __init__.py        # Makes src a Python package
│   ├── data_loader.py     # Data loading functions
│   ├── features.py        # Feature engineering
│   ├── model.py           # Model training and prediction
│   └── evaluate.py        # Model evaluation
│
├── tests/
│   ├── __init__.py
│   ├── test_data_loader.py
│   └── test_model.py
│
├── models/
│   └── trained_model.pkl  # Saved model files
│
├── logs/
│   └── training.log       # Log files
│
└── scripts/
    ├── train.py           # Script to train model
    └── predict.py         # Script to make predictions
```

Let us understand each folder:

```
+---------------------------------------------------+
|  Project Structure Explained                      |
|                                                   |
|  data/       -> Where your data lives             |
|  notebooks/  -> For exploration only              |
|  src/        -> Your production code              |
|  tests/      -> Automated tests                   |
|  models/     -> Saved trained models              |
|  logs/       -> What happened when code ran        |
|  scripts/    -> Entry points to run things         |
+---------------------------------------------------+
```

Creating the Project Structure

Here is a Python script that creates this structure for you:

```python
"""
create_project.py - Create an ML project structure.

Run this script to set up a new ML project with
all the necessary folders and files.
"""

import os


def create_project(project_name):
    """
    Create a standard ML project structure.

    Parameters
    ----------
    project_name : str
        Name of the project (used as the root folder name).
    """
    # Define all the folders we need
    folders = [
        "data/raw",
        "data/processed",
        "data/external",
        "notebooks",
        "src",
        "tests",
        "models",
        "logs",
        "scripts",
    ]

    # Create each folder
    for folder in folders:
        path = os.path.join(project_name, folder)
        os.makedirs(path, exist_ok=True)
        print(f"Created: {path}")

        # Add __init__.py to Python package folders
        # __init__.py tells Python "this folder is a package"
        if folder in ["src", "tests"]:
            init_file = os.path.join(path, "__init__.py")
            with open(init_file, "w") as f:
                f.write("")
            print(f"Created: {init_file}")

    # Create essential files
    essential_files = {
        "README.md": f"# {project_name}\n\nDescribe your project here.\n",
```

```
        "requirements.txt": "pandas\nscikit-learn\npyyaml\n",
        "config.yaml": "# Project configuration\n",
        ".gitignore": "data/\nmodels/\nlogs/\n__pycache__/\n*.pyc\n",
    }

    for filename, content in essential_files.items():
        filepath = os.path.join(project_name, filename)
        with open(filepath, "w") as f:
            f.write(content)
        print(f"Created: {filepath}")

    print(f"\nProject '{project_name}' created successfully!")


# Run it
create_project("my-ml-project")
```

```
Output:
Created: my-ml-project/data/raw
Created: my-ml-project/data/processed
Created: my-ml-project/data/external
Created: my-ml-project/notebooks
Created: my-ml-project/src
Created: my-ml-project/src/__init__.py
Created: my-ml-project/tests
Created: my-ml-project/tests/__init__.py
Created: my-ml-project/models
Created: my-ml-project/logs
Created: my-ml-project/scripts
Created: my-ml-project/README.md
Created: my-ml-project/requirements.txt
Created: my-ml-project/config.yaml
Created: my-ml-project/.gitignore

Project 'my-ml-project' created successfully!
```

Refactoring a Notebook into Scripts

Refactoring means reorganizing code without changing what it does. Think of it like reorganizing a messy closet. The same clothes are there, but now you can find what you need.

Let us refactor our earlier notebook code step by step.

Step 1: Identify Logical Groups

Look at your notebook and group cells by what they do:

```
+--------------------------------------------------+
|  Notebook Cell Groups                            |
|                                                  |
|  Group 1: Data Loading     -> data_loader.py     |
|  Group 2: Data Cleaning    -> data_loader.py     |
|  Group 3: Feature Work     -> features.py        |
|  Group 4: Model Training   -> model.py           |
|  Group 5: Evaluation       -> evaluate.py        |
|  Group 6: Run Everything   -> scripts/train.py   |
+--------------------------------------------------+
```

Step 2: Create the Model Module

File: src/model.py

```python
"""
model.py - Model training and prediction functions.

This module handles creating, training, and using
machine learning models.
"""

from sklearn.ensemble import RandomForestClassifier
import logging
import joblib

logger = logging.getLogger(__name__)


def create_model(model_type="random_forest", **kwargs):
    """
    Create a machine learning model.

    Parameters
    ----------
    model_type : str
        Type of model to create. Currently supports
        "random_forest".
    **kwargs : dict
        Additional parameters passed to the model.
        For example: n_estimators=100, random_state=42

    Returns
    -------
    model
        An untrained model object.
    """
```

```
    if model_type == "random_forest":
        model = RandomForestClassifier(**kwargs)
        logger.info(
            f"Created RandomForest with params: {kwargs}"
        )
    else:
        raise ValueError(f"Unknown model type: {model_type}")

    return model


def train_model(model, X_train, y_train):
    """
    Train a model on the training data.

    Parameters
    ----------
    model : sklearn model
        The model to train.
    X_train : pd.DataFrame
        Training features.
    y_train : pd.Series
        Training labels.

    Returns
    -------
    model
        The trained model.
    """
    logger.info("Starting model training...")
    model.fit(X_train, y_train)
    logger.info("Model training complete")
    return model


def save_model(model, file_path):
    """
    Save a trained model to a file.

    Parameters
    ----------
    model : sklearn model
        The trained model to save.
    file_path : str
        Where to save the model.
    """
    joblib.dump(model, file_path)
    logger.info(f"Model saved to {file_path}")


def load_model(file_path):
    """
    Load a trained model from a file.

    Parameters
```

```
    ----------
    file_path : str
        Path to the saved model file.

    Returns
    -------
    model
        The loaded model.
    """
    model = joblib.load(file_path)
    logger.info(f"Model loaded from {file_path}")
    return model
```

Step 3: Create the Evaluation Module

File: src/evaluate.py

```python
"""
evaluate.py - Model evaluation functions.

This module provides functions to measure how well
a model performs.
"""

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report,
)
import logging

logger = logging.getLogger(__name__)


def evaluate_model(model, X_test, y_test):
    """
    Evaluate a trained model on test data.

    Parameters
    ----------
    model : sklearn model
        The trained model.
    X_test : pd.DataFrame
        Test features.
    y_test : pd.Series
        True test labels.

    Returns
```

```
    -------
    dict
        Dictionary with evaluation metrics.
    """
    # Make predictions on test data
    predictions = model.predict(X_test)

    # Calculate multiple metrics
    # Accuracy: what fraction of predictions are correct
    # Precision: of positive predictions, how many are right
    # Recall: of actual positives, how many did we find
    # F1: balance between precision and recall
    metrics = {
        "accuracy": accuracy_score(y_test, predictions),
        "precision": precision_score(
            y_test, predictions, average="weighted"
        ),
        "recall": recall_score(
            y_test, predictions, average="weighted"
        ),
        "f1": f1_score(
            y_test, predictions, average="weighted"
        ),
    }

    # Log each metric
    for name, value in metrics.items():
        logger.info(f"{name}: {value:.4f}")

    return metrics


def print_report(model, X_test, y_test):
    """
    Print a detailed classification report.

    Parameters
    ----------
    model : sklearn model
        The trained model.
    X_test : pd.DataFrame
        Test features.
    y_test : pd.Series
        True test labels.
    """
    predictions = model.predict(X_test)
    report = classification_report(y_test, predictions)
    print("\nClassification Report:")
    print(report)
    logger.info("Classification report printed")
```

Step 4: Create the Training Script

File: scripts/train.py

```python
"""
train.py - Main training script.

This script ties everything together. It reads the
configuration, loads data, trains a model, evaluates
it, and saves it.

Usage:
    python scripts/train.py
    python scripts/train.py --config custom_config.yaml
"""

import sys
import os
import yaml
import logging
import argparse
from datetime import datetime

# Add the project root to Python's path
# This lets us import from the src folder
sys.path.insert(0, os.path.dirname(os.path.dirname(__file__)))

from src.data_loader import load_data, clean_data, split_data
from src.model import create_model, train_model, save_model
from src.evaluate import evaluate_model, print_report


def setup_logging(log_file=None):
    """
    Set up logging to both console and file.

    Parameters
    ----------
    log_file : str, optional
        Path to log file. If None, logs only to console.
    """
    # Create a formatter that adds timestamp and level
    # Format: 2024-01-15 10:30:45 - INFO - Message here
    formatter = logging.Formatter(
        "%(asctime)s - %(name)s - %(levelname)s - %(message)s",
        datefmt="%Y-%m-%d %H:%M:%S",
    )

    # Set up the root logger
    root_logger = logging.getLogger()
    root_logger.setLevel(logging.INFO)

    # Always log to console (so you can see what is happening)
    console_handler = logging.StreamHandler()
    console_handler.setFormatter(formatter)
```

```
    root_logger.addHandler(console_handler)

    # Optionally also log to a file (for later review)
    if log_file:
        os.makedirs(os.path.dirname(log_file), exist_ok=True)
        file_handler = logging.FileHandler(log_file)
        file_handler.setFormatter(formatter)
        root_logger.addHandler(file_handler)


def load_config(config_path="config.yaml"):
    """
    Load configuration from a YAML file.

    Parameters
    ----------
    config_path : str
        Path to the configuration file.

    Returns
    -------
    dict
        Configuration dictionary.
    """
    with open(config_path, "r") as f:
        config = yaml.safe_load(f)
    return config


def main():
    """Main training pipeline."""
    # Parse command line arguments
    # argparse lets users pass options when running the script
    parser = argparse.ArgumentParser(
        description="Train an ML model"
    )
    parser.add_argument(
        "--config",
        default="config.yaml",
        help="Path to config file (default: config.yaml)",
    )
    args = parser.parse_args()

    # Load configuration
    config = load_config(args.config)

    # Set up logging
    log_file = config.get("output", {}).get("log_file")
    setup_logging(log_file)

    logger = logging.getLogger(__name__)
    logger.info("=" * 50)
    logger.info("Starting training pipeline")
    logger.info(f"Time: {datetime.now()}")
    logger.info("=" * 50)
```

```
    # Step 1: Load data
    data_config = config["data"]
    df = load_data(data_config["input_path"])

    # Step 2: Clean data
    df = clean_data(df)

    # Step 3: Split data
    X_train, X_test, y_train, y_test = split_data(
        df,
        target_column="target",
        test_size=data_config.get("test_size", 0.2),
        random_state=data_config.get("random_state", 42),
    )

    # Step 4: Create and train model
    model_config = config["model"]
    model = create_model(
        model_type=model_config["type"],
        n_estimators=model_config.get("n_estimators", 100),
        random_state=model_config.get("random_state", 42),
    )
    model = train_model(model, X_train, y_train)

    # Step 5: Evaluate model
    metrics = evaluate_model(model, X_test, y_test)
    print_report(model, X_test, y_test)

    # Step 6: Save model
    model_path = config["output"]["model_path"]
    os.makedirs(os.path.dirname(model_path), exist_ok=True)
    save_model(model, model_path)

    logger.info("=" * 50)
    logger.info("Training pipeline complete!")
    logger.info(f"Model saved to: {model_path}")
    logger.info(f"Accuracy: {metrics['accuracy']:.4f}")
    logger.info("=" * 50)


# This block runs only when you execute the script directly
# It does NOT run when you import this file
if __name__ == "__main__":
    main()
```

```
Output:
2024-01-15 10:30:45 - __main__ - INFO - ==================================================
2024-01-15 10:30:45 - __main__ - INFO - Starting training pipeline
2024-01-15 10:30:45 - __main__ - INFO - Time: 2024-01-15 10:30:45.123456
2024-01-15 10:30:45 - __main__ - INFO - ==================================================
2024-01-15 10:30:45 - src.data_loader - INFO - Loading data from data/raw/dataset.csv
2024-01-15 10:30:45 - src.data_loader - INFO - Loaded 1000 rows and 11 columns
```

```
2024-01-15 10:30:45 - src.data_loader - INFO - Removed 15 rows with missing values
2024-01-15 10:30:45 - src.data_loader - INFO - Clean data has 985 rows
2024-01-15 10:30:45 - src.data_loader - INFO - Training set: 788 samples
2024-01-15 10:30:45 - src.data_loader - INFO - Test set: 197 samples
2024-01-15 10:30:46 - src.model - INFO - Created RandomForest with params: {'n_estimato...
2024-01-15 10:30:46 - src.model - INFO - Starting model training...
2024-01-15 10:30:47 - src.model - INFO - Model training complete
2024-01-15 10:30:47 - src.evaluate - INFO - accuracy: 0.9200
2024-01-15 10:30:47 - src.evaluate - INFO - precision: 0.9215
2024-01-15 10:30:47 - src.evaluate - INFO - recall: 0.9200
2024-01-15 10:30:47 - src.evaluate - INFO - f1: 0.9195
2024-01-15 10:30:47 - src.model - INFO - Model saved to models/trained_model.pkl
2024-01-15 10:30:47 - __main__ - INFO - ==================================================
2024-01-15 10:30:47 - __main__ - INFO - Training pipeline complete!
2024-01-15 10:30:47 - __main__ - INFO - Model saved to: models/trained_model.pkl
2024-01-15 10:30:47 - __main__ - INFO - Accuracy: 0.9200
2024-01-15 10:30:47 - __main__ - INFO - ==================================================
```

Configuration Files

A configuration file stores settings separately from your code. This is like having a recipe card separate from the cooking instructions. You can change the ingredients without rewriting the instructions.

Why Use Config Files?

```
+--------------------------------------------------+
|  Without Config Files          With Config Files |
|                                                  |
|  Change code to change      Change config only   |
|  settings                                        |
|                                                  |
|  Risk breaking things       Code stays the same  |
|                                                  |
|  Hard to track changes      Easy to compare      |
|                                                  |
|  Different settings for     One config per        |
|  dev vs prod? Messy!        environment           |
+--------------------------------------------------+
```

YAML Configuration Example

YAML (Yet Another Markup Language) is a human-friendly format for configuration files. It uses indentation to show structure, like Python.

```yaml
# config.yaml - Main configuration file

# Data settings
data:
  input_path: "data/raw/dataset.csv"
  target_column: "target"
  test_size: 0.2
  random_state: 42

# Model settings
model:
  type: "random_forest"
  params:
    n_estimators: 100
    max_depth: 10
    min_samples_split: 5
    random_state: 42

# Training settings
training:
  epochs: 50          # Not used for sklearn, but useful for deep learning
  batch_size: 32
  learning_rate: 0.001

# Output settings
output:
  model_path: "models/trained_model.pkl"
  metrics_path: "models/metrics.json"
  log_file: "logs/training.log"

# Feature settings
features:
  numeric_columns:
    - "age"
    - "income"
    - "score"
  categorical_columns:
    - "city"
    - "category"
```

Loading Configuration in Python

```python
"""
config_loader.py - Load and validate configuration.
"""
```

```
import yaml
import os


def load_config(config_path="config.yaml"):
    """
    Load configuration from a YAML file.

    Parameters
    ----------
    config_path : str
        Path to the YAML configuration file.

    Returns
    -------
    dict
        Configuration as a Python dictionary.

    Raises
    ------
    FileNotFoundError
        If the config file does not exist.
    ValueError
        If the config file is empty or invalid.
    """
    # Check if file exists
    if not os.path.exists(config_path):
        raise FileNotFoundError(
            f"Config file not found: {config_path}"
        )

    # Read and parse YAML
    with open(config_path, "r") as f:
        config = yaml.safe_load(f)

    # Validate that we got something
    if config is None:
        raise ValueError(
            f"Config file is empty: {config_path}"
        )

    return config


def get_nested(config, *keys, default=None):
    """
    Safely get a nested value from the config.

    This is like navigating a folder structure:
    get_nested(config, "model", "params", "n_estimators")
    is like going to config -> model -> params -> n_estimators

    Parameters
    ----------
    config : dict
```

```
        The configuration dictionary.
    *keys : str
        The keys to navigate through.
    default : any
        Value to return if the key path does not exist.

    Returns
    -------
    any
        The value at the key path, or default.
    """
    current = config
    for key in keys:
        if isinstance(current, dict) and key in current:
            current = current[key]
        else:
            return default
    return current


# Usage example
config = load_config("config.yaml")

# Get values safely
data_path = get_nested(config, "data", "input_path")
n_estimators = get_nested(
    config, "model", "params", "n_estimators", default=100
)

print(f"Data path: {data_path}")
print(f"N estimators: {n_estimators}")
```

```
Output:
Data path: data/raw/dataset.csv
N estimators: 100
```

Environment-Specific Configs

In the real world, you often need different settings for development and production:

```yaml
# config_dev.yaml - Development settings
data:
  input_path: "data/sample/small_dataset.csv"   # Small data for quick testing
model:
  params:
    n_estimators: 10   # Fewer trees = faster training

# config_prod.yaml - Production settings
```

```
data:
  input_path: "data/raw/full_dataset.csv"        # Full data
model:
  params:
    n_estimators: 500  # More trees = better accuracy
```

Logging

Logging is like installing security cameras in a building. When something goes wrong, you can review the footage to figure out what happened.

Logging Levels

Python's logging module has five levels, from least to most severe:

```
+-----------------------------------------------------+
|  Logging Levels (least to most severe)              |
|                                                      |
|  DEBUG    -> Detailed info for debugging             |
|              "Processing row 42 of 1000"             |
|                                                      |
|  INFO     -> General progress updates                |
|              "Model training started"                |
|                                                      |
|  WARNING  -> Something unexpected but not fatal      |
|              "Missing values found in column age"    |
|                                                      |
|  ERROR    -> Something failed                        |
|              "Could not load data file"              |
|                                                      |
|  CRITICAL -> The program cannot continue             |
|              "Database connection lost"              |
+-----------------------------------------------------+
```

Complete Logging Setup

```python
"""
logger_setup.py - Set up logging for the project.

This creates a logger that writes to both the console
(so you can see what is happening) and a file (so you
```

```
can review later).
"""

import logging
import os
from datetime import datetime


def setup_logger(
    name="ml_project",
    log_dir="logs",
    level=logging.INFO,
):
    """
    Set up a logger with console and file output.

    Parameters
    ----------
    name : str
        Name of the logger.
    log_dir : str
        Directory to store log files.
    level : int
        Minimum logging level (e.g., logging.INFO).

    Returns
    -------
    logging.Logger
        Configured logger.
    """
    # Create logger
    logger = logging.getLogger(name)
    logger.setLevel(level)

    # Avoid adding handlers multiple times
    # (this can happen if setup_logger is called twice)
    if logger.handlers:
        return logger

    # Create log directory if it does not exist
    os.makedirs(log_dir, exist_ok=True)

    # Create a timestamp for the log file name
    # This way, each run gets its own log file
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    log_file = os.path.join(log_dir, f"{name}_{timestamp}.log")

    # Define the format for log messages
    formatter = logging.Formatter(
        fmt="%(asctime)s | %(name)s | %(levelname)-8s | %(message)s",
        datefmt="%Y-%m-%d %H:%M:%S",
    )

    # Console handler - shows logs in the terminal
    console_handler = logging.StreamHandler()
```

```
    console_handler.setLevel(logging.INFO)
    console_handler.setFormatter(formatter)

    # File handler - saves logs to a file
    file_handler = logging.FileHandler(log_file)
    file_handler.setLevel(logging.DEBUG)  # File gets more detail
    file_handler.setFormatter(formatter)

    # Add both handlers to the logger
    logger.addHandler(console_handler)
    logger.addHandler(file_handler)

    logger.info(f"Logging to file: {log_file}")

    return logger


# Usage example
logger = setup_logger("my_ml_project")

logger.debug("This is a debug message (only in file)")
logger.info("Starting the ML pipeline")
logger.warning("Dataset has 5% missing values")
logger.error("Could not connect to database")
logger.critical("Out of memory!")
```

```
Output (console):
2024-01-15 10:30:45 | my_ml_project | INFO     | Logging to file: logs/my_ml_project_20...
2024-01-15 10:30:45 | my_ml_project | INFO     | Starting the ML pipeline
2024-01-15 10:30:45 | my_ml_project | WARNING  | Dataset has 5% missing values
2024-01-15 10:30:45 | my_ml_project | ERROR    | Could not connect to database
2024-01-15 10:30:45 | my_ml_project | CRITICAL | Out of memory!
```

Notice that the DEBUG message does not appear in the console output. That is because we set the console handler to INFO level. The file will contain all messages including DEBUG.

Putting It All Together: Complete Refactored Project

Let us see the complete flow from notebook to production:

```
+--------------------------------------------------+
|  The Refactoring Journey                         |
|                                                   |
```

```
|  Notebook (messy)                                |
|       |                                           |
|       v                                           |
|  Identify logical groups                         |
|       |                                           |
|       v                                           |
|  Create module files (src/)                      |
|       |                                           |
|       v                                           |
|  Add error handling and logging                  |
|       |                                           |
|       v                                           |
|  Create config file                              |
|       |                                           |
|       v                                           |
|  Create main script (scripts/train.py)           |
|       |                                           |
|       v                                           |
|  Add tests (tests/)                              |
|       |                                           |
|       v                                           |
|  Production-ready code!                          |
+--------------------------------------------------+
```

A Simple Test Example

```python
"""
test_data_loader.py - Tests for the data loader module.

Tests verify that your code works correctly. Think of
them as a safety net that catches bugs before they
reach production.
"""

import pytest
import pandas as pd
import os
import tempfile

# Import the functions we want to test
from src.data_loader import load_data, clean_data, split_data


def test_load_data_success():
    """Test that load_data works with a valid CSV file."""
    # Create a temporary CSV file for testing
    # tempfile creates a file that gets deleted automatically
    with tempfile.NamedTemporaryFile(
        mode="w", suffix=".csv", delete=False
    ) as f:
```

```
        f.write("feature1,feature2,target\n")
        f.write("1,2,0\n")
        f.write("3,4,1\n")
        f.write("5,6,0\n")
        temp_path = f.name

    try:
        # Load the data
        df = load_data(temp_path)

        # Check that we got the right shape
        assert len(df) == 3, "Should have 3 rows"
        assert len(df.columns) == 3, "Should have 3 columns"
    finally:
        # Clean up the temporary file
        os.unlink(temp_path)


def test_load_data_file_not_found():
    """Test that load_data raises error for missing file."""
    with pytest.raises(FileNotFoundError):
        load_data("this_file_does_not_exist.csv")


def test_clean_data():
    """Test that clean_data removes rows with missing values."""
    # Create a DataFrame with some missing values
    df = pd.DataFrame(
        {
            "a": [1, 2, None, 4],
            "b": [5, None, 7, 8],
        }
    )

    # Clean it
    cleaned = clean_data(df)

    # Should have 2 rows (rows 0 and 3 have no missing values)
    assert len(cleaned) == 2


def test_split_data():
    """Test that split_data creates proper train/test split."""
    df = pd.DataFrame(
        {
            "feature1": range(100),
            "feature2": range(100, 200),
            "target": [0, 1] * 50,
        }
    )

    X_train, X_test, y_train, y_test = split_data(
        df, target_column="target", test_size=0.2
    )
```

```
    # Check sizes
    assert len(X_train) == 80, "Training set should be 80%"
    assert len(X_test) == 20, "Test set should be 20%"
    assert len(y_train) == 80
    assert len(y_test) == 20
```

```
Output (running pytest):
==================== test session starts ====================
collected 4 items

tests/test_data_loader.py::test_load_data_success PASSED
tests/test_data_loader.py::test_load_data_file_not_found PASSED
tests/test_data_loader.py::test_clean_data PASSED
tests/test_data_loader.py::test_split_data PASSED

==================== 4 passed in 0.52s ====================
```

Common Mistakes

1. Not using version control. Always use git from the start. Even for personal projects.
2. Hardcoding file paths. Never write `df = pd.read_csv("/home/user/data.csv")`. Use configuration files or command-line arguments instead.
3. Forgetting `if __name__ == "__main__"`. Without this guard, your training code runs when you try to import functions from the file.
4. Not handling errors. Production code must anticipate what can go wrong. Files can be missing. Data can be corrupted. Models can fail.
5. Skipping logging. When your code runs on a server at 3 AM and fails, logs are your only way to figure out what happened.
6. Keeping exploration code in production. Remove `df.head()`, `print(df.shape)`, and other exploration code. Use logging instead.

Best Practices

1. Start with the end in mind. Even when exploring in a notebook, think about how you will eventually organize the code.

2. One function, one job. Each function should do one thing well. `load_data()` loads data. `clean_data()` cleans data. Do not mix responsibilities.
3. Write docstrings. Future you will forget what your functions do. Write docstrings that explain parameters, return values, and examples.
4. Use type hints. They help editors catch bugs and make code more readable:

```python
def load_data(file_path: str) -> pd.DataFrame:
    ...
```

5. Keep notebooks for exploration. Notebooks are great for trying things out. But when code is ready for production, move it to `.py` files.
6. Use a requirements.txt file. List all packages your project needs. This helps others (and future you) set up the project.

Quick Summary

Moving from notebook to production is about making your code reliable, maintainable, and runnable without you watching it. The key steps are:

- Organize your project with a clear folder structure
- Break notebook code into reusable Python modules
- Use configuration files for settings
- Add logging so you can track what happens
- Write tests to catch bugs early
- Handle errors gracefully

Key Points

- Notebooks are great for exploration but poor for production
- Production code needs error handling, logging, and tests
- Configuration files separate settings from code
- Project structure makes code organized and findable
- Logging is your eyes and ears when code runs without you
- The `if __name__ == "__main__"` pattern prevents code from running on import
- Refactoring is about reorganizing, not rewriting

Practice Questions

1. What are three problems with using Jupyter notebooks in production?
2. Why should you use a configuration file instead of hardcoding values like file paths and model parameters directly in your code?
3. What is the difference between `logging.INFO` and `logging.DEBUG`? When would you use each one?
4. Explain what `if __name__ == "__main__"` does and why it is important.
5. What is the purpose of the `__init__.py` file in the `src/` and `tests/` folders?

Exercises

Exercise 1: Refactor a Notebook

Take this notebook code and refactor it into proper production code with separate modules, a config file, and logging:

```python
# Notebook code
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

df = pd.read_csv("customers.csv")
df = df.dropna()
X = df[["age", "income", "score"]]
y = df["churned"]
model = LogisticRegression()
model.fit(X, y)
print(accuracy_score(y, model.predict(X)))
```

Exercise 2: Add Error Handling

Write a function called `safe_load_data` that:

- Takes a file path as input
- Returns the loaded DataFrame if successful

- Returns `None` and logs a warning if the file does not exist
- Returns `None` and logs an error if the file is not a valid CSV

Exercise 3: Create a Logging Dashboard

Write a script that reads a log file and prints a summary showing:

- Total number of log messages
- Count of each log level (INFO, WARNING, ERROR, etc.)
- The last 5 error messages

What Is Next?

Now that you know how to organize code for production, the next step is learning how to save and load your trained models. In Chapter 2, we will explore different ways to save models (pickle, joblib, ONNX) so they can be loaded and used later without retraining. This is essential because training a model can take hours or days, and you do not want to retrain every time you need a prediction.

Chapter 2: Saving and Loading Models

What You Will Learn

In this chapter, you will learn:

- Why saving models is essential for production
- How to save and load models with pickle
- How to save and load models with joblib (faster for large models)
- How to save and load PyTorch models with torch.save
- How to export models to ONNX format for cross-platform use
- How to compare different model formats
- How to version your models so you can track changes

Why This Chapter Matters

Imagine you spend three hours baking a wedding cake. It is perfect. But instead of putting it in the fridge, you eat it immediately. Tomorrow, when the wedding happens, you have to bake the entire cake again from scratch.

This is what happens when you train a model but do not save it. Training can take minutes, hours, or even days. Without saving, you must retrain every time you need a prediction. Saving a model is like putting the cake in the fridge. The work is done. You can serve it whenever you need it.

Why Save Models?

Here are the main reasons to save your trained models:

```
+-----------------------------------------------+
|  Why Save Models?                             |
|                                               |
|  1. Avoid retraining    -> Save hours of compute  |
|  2. Deploy to servers   -> Load model in API      |
|  3. Share with others   -> Give model to team     |
|  4. Version control     -> Track model changes    |
|  5. Reproducibility     -> Same model, same results|
|  6. A/B testing         -> Compare model versions |
+-----------------------------------------------+
```

The Model Saving Workflow

```
+-----------------------------------------------+
|  Training Phase              Serving Phase        |
|                                                   |
|  Data -> Train -> Save       Load -> Predict      |
|  (hours)  (once)  (file)     (file)  (milliseconds)|
|                                                   |
|  Happens once or rarely      Happens many times   |
+-----------------------------------------------+
```

Method 1: Pickle

Pickle is Python's built-in way to save any Python object to a file. The name comes from "pickling" food to preserve it. Just like pickling a cucumber preserves it for later, pickling a model preserves it for later use.

Saving with Pickle

```python
"""
save_with_pickle.py - Save and load models using pickle.

Pickle is Python's built-in serialization library.
Serialization means converting an object in memory
into a format that can be stored in a file.
"""
```

```
import pickle
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split

# Step 1: Create some sample data
# make_classification generates fake classification data
X, y = make_classification(
    n_samples=1000,      # 1000 data points
    n_features=10,       # 10 input features
    n_classes=2,         # Binary classification
    random_state=42,     # Same data every time
)

# Step 2: Split into train and test
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Step 3: Train the model
model = RandomForestClassifier(
    n_estimators=100,
    random_state=42,
)
model.fit(X_train, y_train)

# Check accuracy before saving
train_accuracy = model.score(X_train, y_train)
test_accuracy = model.score(X_test, y_test)
print(f"Train accuracy: {train_accuracy:.4f}")
print(f"Test accuracy: {test_accuracy:.4f}")

# Step 4: Save the model with pickle
# "wb" means "write binary" - pickle saves in binary format
with open("model_pickle.pkl", "wb") as f:
    pickle.dump(model, f)

print("Model saved with pickle!")

# Step 5: Load the model back
# "rb" means "read binary"
with open("model_pickle.pkl", "rb") as f:
    loaded_model = pickle.load(f)

# Step 6: Verify the loaded model works the same
loaded_accuracy = loaded_model.score(X_test, y_test)
print(f"Loaded model accuracy: {loaded_accuracy:.4f}")
print(f"Same as before? {loaded_accuracy == test_accuracy}")
```

```
Output:
Train accuracy: 1.0000
Test accuracy: 0.9350
Model saved with pickle!
```

```
Loaded model accuracy: 0.9350
Same as before? True
```

Line-by-line explanation of the saving part:

```python
with open("model_pickle.pkl", "wb") as f:
```

This opens a file called "model_pickle.pkl" for writing in binary mode. The `with` statement ensures the file is properly closed after we are done. The `.pkl` extension is a convention that tells us this is a pickle file.

```python
    pickle.dump(model, f)
```

`pickle.dump()` converts the model object into bytes and writes those bytes to the file. Think of it as taking a photograph of the model and storing it.

```python
with open("model_pickle.pkl", "rb") as f:
    loaded_model = pickle.load(f)
```

`pickle.load()` reads the bytes from the file and recreates the model object. It is like developing a photograph back into a real object.

Pickle Limitations

```
+-----------------------------------------------------+
|  Pickle Pros and Cons                               |
|                                                     |
|  Pros:                                              |
|  + Built into Python (no extra install)             |
|  + Works with any Python object                     |
|  + Simple to use                                    |
|                                                     |
|  Cons:                                              |
|  - Python only (cannot use in Java, etc.)           |
|  - Security risk (never load untrusted pickles!)    |
|  - Slow for large models with arrays                |
|  - Version sensitive (Python 3.8 vs 3.11)           |
+-----------------------------------------------------+
```

Security Warning: Never load a pickle file from an untrusted source. A malicious pickle file can execute arbitrary code on your computer. Only load pickle files that you created yourself or that come from a trusted source.

Method 2: Joblib

Joblib is a library specifically designed for saving Python objects that contain large NumPy arrays. Since most ML models have large arrays internally (the learned weights), joblib is usually faster and creates smaller files than pickle.

Saving with Joblib

```python
"""
save_with_joblib.py - Save and load models using joblib.

Joblib is optimized for objects with large numpy arrays,
making it ideal for scikit-learn models.
"""

import joblib
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
import os

# Create and train a model (same as before)
X, y = make_classification(
    n_samples=1000, n_features=10,
    n_classes=2, random_state=42,
)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = RandomForestClassifier(
    n_estimators=100, random_state=42
)
model.fit(X_train, y_train)

# Save with joblib - much simpler syntax!
# No need for open() or file handling
joblib.dump(model, "model_joblib.pkl")
print("Model saved with joblib!")

# Load with joblib
loaded_model = joblib.load("model_joblib.pkl")
print("Model loaded!")

# Verify
```

```
accuracy = loaded_model.score(X_test, y_test)
print(f"Loaded model accuracy: {accuracy:.4f}")

# Compare file sizes
import pickle

# Save same model with pickle for comparison
with open("model_pickle.pkl", "wb") as f:
    pickle.dump(model, f)

pickle_size = os.path.getsize("model_pickle.pkl")
joblib_size = os.path.getsize("model_joblib.pkl")

print(f"\nFile size comparison:")
print(f"Pickle: {pickle_size:,} bytes ({pickle_size/1024:.1f} KB)")
print(f"Joblib: {joblib_size:,} bytes ({joblib_size/1024:.1f} KB)")
```

```
Output:
Model saved with joblib!
Model loaded!
Loaded model accuracy: 0.9350

File size comparison:
Pickle: 1,245,678 bytes (1216.5 KB)
Joblib: 892,345 bytes (871.4 KB)
```

Joblib with Compression

Joblib can compress the saved file to save disk space:

```python
"""
save_compressed.py - Save models with compression.

Compression makes files smaller but takes slightly
longer to save and load.
"""

import joblib
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
import os

# Create and train model
X, y = make_classification(
    n_samples=5000, n_features=50,
    n_classes=2, random_state=42,
)
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
model = RandomForestClassifier(
    n_estimators=200, random_state=42
)
model.fit(X_train, y_train)

# Save without compression
joblib.dump(model, "model_uncompressed.pkl")

# Save with compression (level 0-9, higher = smaller file)
# compress=3 is a good balance of speed and size
joblib.dump(model, "model_compressed.pkl", compress=3)

# Save with high compression
joblib.dump(model, "model_high_compress.pkl", compress=9)

# Compare sizes
sizes = {
    "Uncompressed": os.path.getsize("model_uncompressed.pkl"),
    "Compress=3": os.path.getsize("model_compressed.pkl"),
    "Compress=9": os.path.getsize("model_high_compress.pkl"),
}

print("File sizes:")
for name, size in sizes.items():
    print(f"  {name}: {size/1024:.1f} KB")

# All produce the same predictions
for name, path in [
    ("Uncompressed", "model_uncompressed.pkl"),
    ("Compress=3", "model_compressed.pkl"),
    ("Compress=9", "model_high_compress.pkl"),
]:
    m = joblib.load(path)
    acc = m.score(X_test, y_test)
    print(f"  {name} accuracy: {acc:.4f}")
```

```
Output:
File sizes:
  Uncompressed: 5842.3 KB
  Compress=3: 1523.7 KB
  Compress=9: 1201.4 KB
  Uncompressed accuracy: 0.9560
  Compress=3 accuracy: 0.9560
  Compress=9 accuracy: 0.9560
```

Method 3: PyTorch (torch.save)

PyTorch is a popular deep learning framework. It has its own way of saving models that is optimized for neural networks.

Saving PyTorch Models

There are two ways to save a PyTorch model:

```
+--------------------------------------------------+
|  PyTorch Saving Methods                          |
|                                                  |
|  Method 1: Save entire model                    |
|    torch.save(model, "model.pt")                |
|    + Simple                                      |
|    - Tied to exact class definition              |
|                                                  |
|  Method 2: Save state dict (RECOMMENDED)        |
|    torch.save(model.state_dict(), "model.pt")   |
|    + Flexible                                    |
|    + Smaller files                               |
|    - Need to recreate model architecture first   |
+--------------------------------------------------+
```

```python
"""
save_pytorch.py - Save and load PyTorch models.

PyTorch models are neural networks. They have two parts:
1. Architecture (the structure of the network)
2. Weights (the learned values)

The recommended way is to save only the weights
(state_dict) and recreate the architecture when loading.
"""

import torch
import torch.nn as nn
from torch.utils.data import DataLoader, TensorDataset
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
import numpy as np

# Step 1: Define a simple neural network
class SimpleClassifier(nn.Module):
    """
    A simple neural network for binary classification.

    Architecture:
```

```
    Input (10 features)
        -> Hidden Layer 1 (64 neurons, ReLU activation)
        -> Hidden Layer 2 (32 neurons, ReLU activation)
        -> Output (1 neuron, Sigmoid activation)
    """

    def __init__(self, input_size=10):
        super().__init__()
        # nn.Sequential chains layers together
        self.network = nn.Sequential(
            nn.Linear(input_size, 64),  # 10 inputs -> 64 neurons
            nn.ReLU(),                   # Activation function
            nn.Linear(64, 32),           # 64 -> 32 neurons
            nn.ReLU(),
            nn.Linear(32, 1),            # 32 -> 1 output
            nn.Sigmoid(),                # Output between 0 and 1
        )

    def forward(self, x):
        return self.network(x)


# Step 2: Create data
X, y = make_classification(
    n_samples=1000, n_features=10,
    n_classes=2, random_state=42,
)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Convert to PyTorch tensors
# PyTorch uses its own data type called tensors
X_train_t = torch.FloatTensor(X_train)
y_train_t = torch.FloatTensor(y_train).unsqueeze(1)
X_test_t = torch.FloatTensor(X_test)
y_test_t = torch.FloatTensor(y_test).unsqueeze(1)

# Step 3: Train the model
model = SimpleClassifier(input_size=10)
criterion = nn.BCELoss()  # Binary Cross-Entropy loss
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

# Training loop
for epoch in range(50):
    # Forward pass: compute predictions
    outputs = model(X_train_t)
    loss = criterion(outputs, y_train_t)

    # Backward pass: compute gradients
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    if (epoch + 1) % 10 == 0:
```

```
        print(f"Epoch {epoch+1}/50, Loss: {loss.item():.4f}")

# Step 4: Save the model (RECOMMENDED WAY)
# Save only the state dict (weights and biases)
torch.save(model.state_dict(), "model_statedict.pt")
print("\nState dict saved!")

# Also save training info for reference
# This is helpful for keeping track of how the model was trained
checkpoint = {
    "model_state_dict": model.state_dict(),
    "optimizer_state_dict": optimizer.state_dict(),
    "epoch": 50,
    "loss": loss.item(),
    "input_size": 10,
}
torch.save(checkpoint, "model_checkpoint.pt")
print("Full checkpoint saved!")

# Step 5: Load the model
# First, create a new model with the same architecture
loaded_model = SimpleClassifier(input_size=10)

# Then load the saved weights into it
loaded_model.load_state_dict(
    torch.load("model_statedict.pt", weights_only=True)
)

# Set to evaluation mode (disables dropout, etc.)
loaded_model.eval()

# Step 6: Verify
with torch.no_grad():  # No need to track gradients for prediction
    original_preds = model(X_test_t)
    loaded_preds = loaded_model(X_test_t)

    # Check predictions are identical
    are_equal = torch.allclose(original_preds, loaded_preds)
    print(f"\nPredictions match: {are_equal}")

# Step 7: Load from checkpoint
checkpoint = torch.load("model_checkpoint.pt", weights_only=False)
restored_model = SimpleClassifier(
    input_size=checkpoint["input_size"]
)
restored_model.load_state_dict(checkpoint["model_state_dict"])
restored_model.eval()

print(f"Restored from epoch: {checkpoint['epoch']}")
print(f"Loss at save time: {checkpoint['loss']:.4f}")
```

```
Output:
Epoch 10/50, Loss: 0.4231
```

```
Epoch 20/50, Loss: 0.2856
Epoch 30/50, Loss: 0.1987
Epoch 40/50, Loss: 0.1432
Epoch 50/50, Loss: 0.1098

State dict saved!
Full checkpoint saved!

Predictions match: True
Restored from epoch: 50
Loss at save time: 0.1098
```

Key concepts explained:

```python
torch.save(model.state_dict(), "model_statedict.pt")
```

`state_dict()` returns a dictionary containing all the learned parameters (weights and biases) of the model. Think of it as saving the "brain" of the model without the "body" (architecture).

```python
loaded_model.load_state_dict(torch.load("model_statedict.pt", weights_only=True))
```

This loads the saved parameters into a new model. The `weights_only=True` flag is a security measure that prevents loading malicious code.

```python
loaded_model.eval()
```

This switches the model from training mode to evaluation mode. Some layers (like dropout and batch normalization) behave differently during training vs prediction. Always call `.eval()` before making predictions.

Method 4: ONNX Export

ONNX (Open Neural Network Exchange) is a universal format for ML models. Think of it as a "PDF for models." Just like a PDF can be read by any PDF reader, an ONNX model can be used by any ONNX-compatible runtime, regardless of which framework (PyTorch, TensorFlow, scikit-learn) created it.

```
+-----------------------------------------------------+
|  ONNX: The Universal Model Format                   |
|                                                      |
|  PyTorch Model  ----+                                |
|                     |                                |
|  TensorFlow Model --+--> ONNX --> Any Runtime        |
|                     |                                |
|  Sklearn Model  ----+                                |
|                                                      |
|  Like saving a document as PDF so anyone can         |
|  read it, regardless of which editor created it      |
+-----------------------------------------------------+
```

Exporting a PyTorch Model to ONNX

```python
"""
export_onnx.py - Export a PyTorch model to ONNX format.

ONNX (Open Neural Network Exchange) is a standard format
that lets you use models across different frameworks and
programming languages.
"""

import torch
import torch.nn as nn
import numpy as np

# Define and train a model (same as before)
class SimpleClassifier(nn.Module):
    def __init__(self, input_size=10):
        super().__init__()
        self.network = nn.Sequential(
            nn.Linear(input_size, 64),
            nn.ReLU(),
            nn.Linear(64, 32),
            nn.ReLU(),
            nn.Linear(32, 1),
            nn.Sigmoid(),
        )

    def forward(self, x):
        return self.network(x)


# Create a trained model (using random weights for demo)
model = SimpleClassifier(input_size=10)
model.eval()

# Step 1: Create a dummy input
# ONNX needs a sample input to trace the model's operations
```

```
# The shape must match what the model expects
dummy_input = torch.randn(1, 10)  # Batch size 1, 10 features

# Step 2: Export to ONNX
torch.onnx.export(
    model,                    # The model to export
    dummy_input,              # Sample input
    "model.onnx",             # Output file
    export_params=True,       # Store trained weights
    opset_version=11,         # ONNX version
    do_constant_folding=True, # Optimize constants
    input_names=["input"],    # Name for the input
    output_names=["output"],  # Name for the output
    dynamic_axes={            # Allow variable batch sizes
        "input": {0: "batch_size"},
        "output": {0: "batch_size"},
    },
)
print("Model exported to ONNX format!")

# Step 3: Load and use the ONNX model
# You need the onnxruntime package: pip install onnxruntime
import onnxruntime as ort

# Create an inference session
# This loads the ONNX model and prepares it for predictions
session = ort.InferenceSession("model.onnx")

# Make a prediction
test_input = np.random.randn(1, 10).astype(np.float32)

# Run the model
# The input name must match what we set during export
onnx_result = session.run(
    None,  # Get all outputs
    {"input": test_input},
)

print(f"ONNX prediction: {onnx_result[0]}")

# Compare with PyTorch prediction
pytorch_input = torch.FloatTensor(test_input)
with torch.no_grad():
    pytorch_result = model(pytorch_input).numpy()

print(f"PyTorch prediction: {pytorch_result}")
print(f"Results match: {np.allclose(onnx_result[0], pytorch_result, atol=1e-6)}")
```

```
Output:
Model exported to ONNX format!
ONNX prediction: [[0.4823]]
PyTorch prediction: [[0.4823]]
Results match: True
```

Exporting Scikit-learn to ONNX

```python
"""
sklearn_to_onnx.py - Export scikit-learn models to ONNX.

Uses the skl2onnx library to convert scikit-learn
models to ONNX format.

Install: pip install skl2onnx
"""

from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import make_classification
from skl2onnx import convert_sklearn
from skl2onnx.common.data_types import FloatTensorType
import onnxruntime as ort
import numpy as np

# Train a scikit-learn model
X, y = make_classification(
    n_samples=1000, n_features=10,
    n_classes=2, random_state=42,
)
model = RandomForestClassifier(
    n_estimators=100, random_state=42
)
model.fit(X, y)

# Define the input type
# FloatTensorType([None, 10]) means:
# - None: variable batch size (any number of samples)
# - 10: each sample has 10 features
initial_type = [
    ("input", FloatTensorType([None, 10]))
]

# Convert to ONNX
onnx_model = convert_sklearn(
    model,
    initial_types=initial_type,
    target_opset=12,
)

# Save the ONNX model
with open("sklearn_model.onnx", "wb") as f:
    f.write(onnx_model.SerializeToString())

print("Scikit-learn model exported to ONNX!")

# Load and test
session = ort.InferenceSession("sklearn_model.onnx")
test_input = X[:5].astype(np.float32)
```

```
# Get predictions
onnx_preds = session.run(None, {"input": test_input})
sklearn_preds = model.predict(test_input)

print(f"ONNX predictions:    {onnx_preds[0]}")
print(f"Sklearn predictions: {sklearn_preds}")
```

```
Output:
Scikit-learn model exported to ONNX!
ONNX predictions:    [0 1 0 1 1]
Sklearn predictions: [0 1 0 1 1]
```

Model Format Comparison

Here is a complete comparison of all the formats we covered:

```
+-------------------------------------------------------------------+
|  Format     | Best For        | Cross-    | Size   | Speed     |
|             |                 | Platform  |        |           |
|-------------|-----------------|-----------|--------|-----------|
|  Pickle     | Quick & simple  | No        | Large  | Fast      |
|             | Python only     | (Python)  |        | save/load |
|-------------|-----------------|-----------|--------|-----------|
|  Joblib     | Sklearn models  | No        | Medium | Fastest   |
|             | Large arrays    | (Python)  |        | for numpy |
|-------------|-----------------|-----------|--------|-----------|
|  torch.save | PyTorch models  | No        | Medium | Fast      |
|             | Checkpoints     | (PyTorch) |        |           |
|-------------|-----------------|-----------|--------|-----------|
|  ONNX       | Cross-platform  | Yes!      | Small  | Very fast |
|             | Production      | Any lang  |        | inference |
+-------------------------------------------------------------------+
```

When to Use What

```python
"""
format_comparison.py - Compare different model formats.

This script saves the same model in all formats and
compares file sizes and load times.
"""
```

```
import pickle
import joblib
import time
import os
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import make_classification

# Create a larger model for meaningful comparison
X, y = make_classification(
    n_samples=10000, n_features=50,
    n_classes=2, random_state=42,
)
model = RandomForestClassifier(
    n_estimators=200, random_state=42
)
model.fit(X, y)

results = {}

# Method 1: Pickle
start = time.time()
with open("compare_pickle.pkl", "wb") as f:
    pickle.dump(model, f)
save_time = time.time() - start

start = time.time()
with open("compare_pickle.pkl", "rb") as f:
    pickle.load(f)
load_time = time.time() - start

results["Pickle"] = {
    "size_kb": os.path.getsize("compare_pickle.pkl") / 1024,
    "save_ms": save_time * 1000,
    "load_ms": load_time * 1000,
}

# Method 2: Joblib
start = time.time()
joblib.dump(model, "compare_joblib.pkl")
save_time = time.time() - start

start = time.time()
joblib.load("compare_joblib.pkl")
load_time = time.time() - start

results["Joblib"] = {
    "size_kb": os.path.getsize("compare_joblib.pkl") / 1024,
    "save_ms": save_time * 1000,
    "load_ms": load_time * 1000,
}

# Method 3: Joblib compressed
start = time.time()
joblib.dump(model, "compare_joblib_c.pkl", compress=3)
```

```
save_time = time.time() - start

start = time.time()
joblib.load("compare_joblib_c.pkl")
load_time = time.time() - start

results["Joblib (compressed)"] = {
    "size_kb": os.path.getsize("compare_joblib_c.pkl") / 1024,
    "save_ms": save_time * 1000,
    "load_ms": load_time * 1000,
}

# Print comparison
print(f"{'Format':<22} {'Size (KB)':>10} {'Save (ms)':>10} {'Load (ms)':>10}")
print("-" * 55)
for name, data in results.items():
    print(
        f"{name:<22} {data['size_kb']:>10.1f} "
        f"{data['save_ms']:>10.1f} {data['load_ms']:>10.1f}"
    )
```

```
Output:
Format                  Size (KB)  Save (ms)  Load (ms)
-------------------------------------------------------
Pickle                    12456.3      234.5      198.2
Joblib                     8923.1      189.3      145.7
Joblib (compressed)        3201.5      412.8      167.3
```

Versioning Models

When you improve a model, you need to keep track of different versions. Imagine updating a mobile app but losing the previous version. If the new version has a bug, you cannot go back. Model versioning prevents this problem.

Simple Version Naming

```python
"""
model_versioning.py - Version your models with metadata.

Keep track of which model version is which, what data
it was trained on, and how well it performs.
"""
```

```
import joblib
import json
import os
from datetime import datetime
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split


def save_versioned_model(
    model,
    model_dir,
    version,
    metrics,
    metadata=None,
):
    """
    Save a model with version number and metadata.

    Parameters
    ----------
    model : sklearn model
        The trained model.
    model_dir : str
        Directory to save models.
    version : str
        Version string (e.g., "1.0.0").
    metrics : dict
        Model performance metrics.
    metadata : dict, optional
        Additional information about the model.
    """
    # Create version directory
    version_dir = os.path.join(model_dir, f"v{version}")
    os.makedirs(version_dir, exist_ok=True)

    # Save the model
    model_path = os.path.join(version_dir, "model.pkl")
    joblib.dump(model, model_path)

    # Save metadata
    info = {
        "version": version,
        "created_at": datetime.now().isoformat(),
        "metrics": metrics,
        "metadata": metadata or {},
    }

    info_path = os.path.join(version_dir, "model_info.json")
    with open(info_path, "w") as f:
        json.dump(info, f, indent=2)

    print(f"Model v{version} saved to {version_dir}")
    return version_dir
```

```
def load_versioned_model(model_dir, version=None):
    """
    Load a specific model version, or the latest.

    Parameters
    ----------
    model_dir : str
        Directory containing model versions.
    version : str, optional
        Version to load. If None, loads the latest.

    Returns
    -------
    tuple
        (model, info_dict)
    """
    if version is None:
        # Find the latest version
        versions = [
            d for d in os.listdir(model_dir)
            if d.startswith("v")
        ]
        if not versions:
            raise FileNotFoundError("No model versions found")
        version = sorted(versions)[-1].lstrip("v")

    version_dir = os.path.join(model_dir, f"v{version}")

    # Load model
    model = joblib.load(
        os.path.join(version_dir, "model.pkl")
    )

    # Load metadata
    with open(os.path.join(version_dir, "model_info.json")) as f:
        info = json.load(f)

    print(f"Loaded model v{version}")
    return model, info


# Example usage
X, y = make_classification(
    n_samples=1000, n_features=10,
    n_classes=2, random_state=42,
)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42,
)

# Version 1.0.0 - Basic model
model_v1 = RandomForestClassifier(
    n_estimators=50, random_state=42
```

```
)
model_v1.fit(X_train, y_train)
acc_v1 = model_v1.score(X_test, y_test)

save_versioned_model(
    model=model_v1,
    model_dir="models",
    version="1.0.0",
    metrics={"accuracy": acc_v1},
    metadata={
        "description": "Baseline model",
        "n_estimators": 50,
    },
)

# Version 1.1.0 - Improved model
model_v2 = RandomForestClassifier(
    n_estimators=200, random_state=42
)
model_v2.fit(X_train, y_train)
acc_v2 = model_v2.score(X_test, y_test)

save_versioned_model(
    model=model_v2,
    model_dir="models",
    version="1.1.0",
    metrics={"accuracy": acc_v2},
    metadata={
        "description": "More trees for better accuracy",
        "n_estimators": 200,
    },
)

# Load a specific version
model, info = load_versioned_model("models", version="1.0.0")
print(f"Version: {info['version']}")
print(f"Accuracy: {info['metrics']['accuracy']:.4f}")

# Load the latest version
model, info = load_versioned_model("models")
print(f"Latest version: {info['version']}")
print(f"Accuracy: {info['metrics']['accuracy']:.4f}")
```

```
Output:
Model v1.0.0 saved to models/v1.0.0
Model v1.1.0 saved to models/v1.1.0
Loaded model v1.0.0
Version: 1.0.0
Accuracy: 0.9250
Loaded model v1.1.0
Latest version: 1.1.0
Accuracy: 0.9400
```

Model Version Directory Structure

```
models/
├── v1.0.0/
│   ├── model.pkl           # The trained model
│   └── model_info.json     # Metadata about the model
├── v1.1.0/
│   ├── model.pkl
│   └── model_info.json
└── v2.0.0/
    ├── model.pkl
    └── model_info.json
```

Saving Additional Artifacts

A model alone is not enough. You also need to save preprocessing objects (like scalers and encoders) that transform input data before it reaches the model.

```python
"""
save_pipeline.py - Save model with preprocessing pipeline.

In production, you need the same preprocessing that was
used during training. If you scaled the training data,
you must scale new data the same way.
"""

import joblib
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split

# Create data
X, y = make_classification(
    n_samples=1000, n_features=10,
    n_classes=2, random_state=42,
)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42,
)

# Create a pipeline that includes preprocessing AND model
# Pipeline ensures preprocessing and model stay together
pipeline = Pipeline([
```

```
    ("scaler", StandardScaler()),           # Step 1: Scale features
    ("classifier", RandomForestClassifier(  # Step 2: Classify
        n_estimators=100, random_state=42
    )),
])

# Train the pipeline (fits scaler AND model)
pipeline.fit(X_train, y_train)

# Check accuracy
accuracy = pipeline.score(X_test, y_test)
print(f"Pipeline accuracy: {accuracy:.4f}")

# Save the entire pipeline (preprocessing + model)
joblib.dump(pipeline, "complete_pipeline.pkl")
print("Complete pipeline saved!")

# Load and use
loaded_pipeline = joblib.load("complete_pipeline.pkl")

# The pipeline handles scaling automatically!
# No need to remember to scale input data
predictions = loaded_pipeline.predict(X_test[:3])
print(f"Predictions: {predictions}")
```

```
Output:
Pipeline accuracy: 0.9350
Complete pipeline saved!
Predictions: [1 0 1]
```

Common Mistakes

1. Loading untrusted pickle files. Pickle can execute arbitrary code. Only load pickle files from trusted sources. Use `weights_only=True` with `torch.load`.
2. Forgetting preprocessing. Saving only the model but not the scaler or encoder. Use sklearn Pipelines to keep everything together.
3. Not checking loaded model accuracy. Always verify that the loaded model gives the same results as the original.
4. Ignoring Python version compatibility. A model saved with Python 3.8 might not load in Python 3.11. Document the Python version.
5. Not saving metadata. Without metadata, you do not know which data or parameters produced each model file.

Best Practices

1. Use joblib for scikit-learn models. It is faster and produces smaller files than pickle.
2. Use state_dict for PyTorch. Save `model.state_dict()` instead of the entire model. It is more portable and less fragile.
3. Save complete pipelines. Include preprocessing steps with the model so nothing is forgotten.
4. Use ONNX for production. When deploying across different platforms, ONNX provides the best compatibility.
5. Always version your models. Use semantic versioning (1.0.0, 1.1.0, 2.0.0) and save metadata with each version.
6. Save a requirements.txt. Record the exact package versions used when training the model.

Quick Summary

Saving models lets you preserve training work, deploy to production, and share with others. Pickle and joblib are the simplest options for Python. PyTorch has its own saving method optimized for neural networks. ONNX provides a universal format that works across frameworks and languages. Always version your models and save metadata so you can track what changed.

Key Points

- pickle.dump() and pickle.load() work with any Python object
- joblib is faster than pickle for scikit-learn models with large arrays
- PyTorch models should be saved with state_dict() for flexibility
- ONNX provides a cross-platform format for model deployment
- Always save preprocessing steps alongside the model
- Version your models with metadata to track changes
- Never load pickle files from untrusted sources

Practice Questions

1. What is the difference between saving a PyTorch model with `torch.save(model, ...)` versus `torch.save(model.state_dict(), ...)`? Which is recommended and why?
2. Why is ONNX useful? Give a scenario where you would choose ONNX over joblib.
3. What happens if you save a model with its preprocessing scaler, but then try to use the model without the scaler on new data?
4. Name three pieces of metadata you should save alongside a model file.
5. Why should you never load a pickle file from an untrusted source?

Exercises

Exercise 1: Save and Compare

Train a logistic regression model and save it using both pickle and joblib. Compare the file sizes and loading times. Which is faster?

Exercise 2: Complete Save System

Create a `ModelManager` class that:

- Saves models with automatic version numbering
- Keeps a JSON log of all saved versions
- Can load any version or the latest version
- Stores accuracy metrics with each version

Exercise 3: Pipeline Saving

Create a scikit-learn pipeline that includes:

- StandardScaler for numeric features
- OneHotEncoder for categorical features
- A RandomForest classifier

Save the entire pipeline and verify it works correctly after loading.

What Is Next?

Now that you can save and load models, the next chapter will show you how to serve those models through a web API using FastAPI. This means anyone can send data to your model over the internet and get predictions back, just like asking Google a question and getting an answer.

Chapter 3: Building ML APIs with FastAPI

What You Will Learn

In this chapter, you will learn:

- What an API is and why ML models need one
- How to install and use FastAPI
- How to create prediction endpoints for your ML model
- How to define request and response models with Pydantic
- How to load a model at startup so it is ready for predictions
- How to test your API with curl and httpx
- How to build a complete ML prediction API

Why This Chapter Matters

Imagine you have built a brilliant weather prediction model. It runs on your laptop and gives amazing forecasts. But your laptop is in your bedroom. How does the weather app on a million phones get your predictions?

The answer is an API (Application Programming Interface). An API is like a restaurant waiter. You (the customer) tell the waiter what you want (your order). The waiter goes to the kitchen (the model), gets your food (the prediction), and brings it back to you. You never need to enter the kitchen, and the chef never needs to meet you.

FastAPI is one of the best tools for building these "waiters" in Python. It is fast, easy to use, and automatically creates documentation for your API.

What Is an API?

API stands for Application Programming Interface. Let us break that down:

```
+-----------------------------------------------------+
|  API Analogy: The Restaurant                        |
|                                                      |
|  Customer (Client App)                              |
|       |                                              |
|       | "I want the salmon" (Request)               |
|       v                                              |
|  Waiter (API)                                       |
|       |                                              |
|       | Takes order to kitchen                      |
|       v                                              |
|  Kitchen (ML Model)                                 |
|       |                                              |
|       | Prepares food (Makes prediction)            |
|       v                                              |
|  Waiter (API)                                       |
|       |                                              |
|       | "Here is your salmon" (Response)            |
|       v                                              |
|  Customer (Client App)                              |
+-----------------------------------------------------+
```

In ML, a typical API flow looks like this:

```
+-----------------------------------------------------+
|  ML API Flow                                        |
|                                                      |
|  Phone App sends:  {"age": 25, "income": 50000}     |
|       |                                              |
|       v                                              |
|  API receives the data                              |
|       |                                              |
|       v                                              |
|  Model predicts: "low risk"                         |
|       |                                              |
|       v                                              |
|  API sends back: {"prediction": "low risk",         |
|                   "confidence": 0.92}                |
+-----------------------------------------------------+
```

Installing FastAPI

FastAPI requires two packages:

```bash
pip install fastapi uvicorn
```

- FastAPI is the framework for building APIs
- Uvicorn is the server that runs your API (like how a car engine runs the car)

Your First FastAPI App

Let us start with the simplest possible API:

```python
"""
hello_api.py - Your very first FastAPI application.

Run with: uvicorn hello_api:app --reload

The --reload flag automatically restarts the server
when you change the code. Very handy during development.
"""

from fastapi import FastAPI

# Create the FastAPI application
# This is like opening your restaurant
app = FastAPI(
    title="My First API",
    description="A simple API to learn FastAPI",
    version="1.0.0",
)


# Define an endpoint (a "menu item")
# The @app.get decorator tells FastAPI:
# "When someone visits /hello, run this function"
@app.get("/hello")
def say_hello():
    """Return a friendly greeting."""
    return {"message": "Hello, World!"}


@app.get("/hello/{name}")
def say_hello_to(name: str):
    """
    Greet a specific person.
```

```
    The {name} in the URL is a path parameter.
    Whatever the user puts there becomes the 'name' variable.
    """
    return {"message": f"Hello, {name}!"}
```

To run this API:

```bash
uvicorn hello_api:app --reload
```

```
Output:
INFO:     Uvicorn running on http://127.0.0.1:8000
INFO:     Started reloader process
INFO:     Started server process
INFO:     Waiting for application startup.
INFO:     Application startup complete.
```

Now you can visit:

- `http://127.0.0.1:8000/hello` to see `{"message": "Hello, World!"}`
- `http://127.0.0.1:8000/hello/Alice` to see `{"message": "Hello, Alice!"}`
- `http://127.0.0.1:8000/docs` to see automatic documentation

Line-by-line explanation:

```python
app = FastAPI(title="My First API", ...)
```

This creates your API application. Think of it as registering your restaurant business. The title, description, and version appear in the auto-generated documentation.

```python
@app.get("/hello")
```

This is a decorator. It tells FastAPI: "When someone sends a GET request to /hello, run the function below." GET is an HTTP method that means "give me data." There are also POST (send data), PUT (update data), and DELETE (remove data).

```python
def say_hello():
    return {"message": "Hello, World!"}
```

This function handles the request. It returns a Python dictionary, which FastAPI automatically converts to JSON (the standard data format for APIs).

Request and Response Models with Pydantic

When someone sends data to your ML model, you want to make sure the data is correct. Pydantic models are like bouncers at a club. They check if the data meets your requirements before letting it in.

```python
"""
pydantic_models.py - Define what data your API accepts and returns.

Pydantic models validate incoming data automatically.
If someone sends the wrong type of data, they get a
clear error message.
"""

from pydantic import BaseModel, Field
from typing import Optional, List


class PredictionRequest(BaseModel):
    """
    What the client sends to get a prediction.

    Each field has a type and an optional description.
    Pydantic will automatically reject data that does
    not match these types.
    """

    # Field() lets you add descriptions and constraints
    age: int = Field(
        ...,  # ... means this field is required
        description="Age of the person in years",
        ge=0,   # ge = greater than or equal to
        le=150,  # le = less than or equal to
    )

    income: float = Field(
        ...,
        description="Annual income in dollars",
        ge=0,
    )

    credit_score: int = Field(
        ...,
        description="Credit score (300-850)",
        ge=300,
        le=850,
    )
```

```
    # Optional field with a default value
    employment_years: Optional[float] = Field(
        default=0.0,
        description="Years of employment",
        ge=0,
    )


class PredictionResponse(BaseModel):
    """
    What the API sends back after making a prediction.
    """

    prediction: str = Field(
        description="The prediction result"
    )

    confidence: float = Field(
        description="How confident the model is (0 to 1)"
    )

    risk_level: str = Field(
        description="Risk category: low, medium, or high"
    )


class BatchPredictionRequest(BaseModel):
    """
    For predicting multiple items at once.
    """

    items: List[PredictionRequest] = Field(
        description="List of items to predict"
    )


# Example: Create an instance to see how it works
request = PredictionRequest(
    age=30,
    income=65000.0,
    credit_score=720,
    employment_years=5.0,
)
print(f"Valid request: {request}")
print(f"As dictionary: {request.model_dump()}")

# This would raise an error:
# invalid = PredictionRequest(age=-5, income=50000, credit_score=200)
# ValueError: age must be >= 0, credit_score must be >= 300
```

```
Output:
Valid request: age=30 income=65000.0 credit_score=720 employment_years=5.0
As dictionary: {'age': 30, 'income': 65000.0, 'credit_score': 720, 'employment_years': ...
```

Loading Your Model at Startup

You do not want to load your model every time someone makes a request. That would be like starting your car engine every time you need to check the speedometer. Instead, load the model once when the API starts, and keep it in memory.

```python
"""
model_loading.py - Load model once at API startup.

The @app.on_event("startup") decorator runs a function
once when the API starts. This is where we load our model.
"""

from fastapi import FastAPI
from contextlib import asynccontextmanager
import joblib
import logging

logger = logging.getLogger(__name__)

# Global variable to hold the model
# We load it once and reuse it for every request
ml_model = None


@asynccontextmanager
async def lifespan(app: FastAPI):
    """
    Manage the lifecycle of the application.

    Code before 'yield' runs at startup.
    Code after 'yield' runs at shutdown.

    This is the modern way to handle startup/shutdown
    events in FastAPI.
    """
    global ml_model

    # Startup: Load the model
    logger.info("Loading ML model...")
    try:
        ml_model = joblib.load("models/trained_model.pkl")
        logger.info("Model loaded successfully!")
    except FileNotFoundError:
        logger.error("Model file not found!")
        raise
```

```
    yield  # The API runs here

    # Shutdown: Clean up
    logger.info("Shutting down, cleaning up...")
    ml_model = None


app = FastAPI(
    title="ML Prediction API",
    lifespan=lifespan,
)


@app.get("/health")
def health_check():
    """
    Health check endpoint.

    Other services use this to check if our API is running.
    It is like asking "Are you still there?"
    """
    return {
        "status": "healthy",
        "model_loaded": ml_model is not None,
    }
```

Complete ML Prediction API

Now let us put everything together into a complete, production-ready API:

```python
"""
main.py - Complete ML Prediction API with FastAPI.

This API loads a trained model and serves predictions
through HTTP endpoints.

Run with: uvicorn main:app --reload --host 0.0.0.0 --port 8000

Structure:
  POST /predict         -> Single prediction
  POST /predict/batch   -> Multiple predictions
  GET  /health          -> Health check
  GET  /model/info      -> Model information
"""

from fastapi import FastAPI, HTTPException
```

```
from pydantic import BaseModel, Field
from contextlib import asynccontextmanager
from typing import List, Optional
import joblib
import numpy as np
import logging
import time
import os

# Set up logging
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)


# ============================================================
# Pydantic Models (Data Validation)
# ============================================================

class PredictionRequest(BaseModel):
    """Input data for a single prediction."""

    age: int = Field(..., ge=0, le=150, description="Age in years")
    income: float = Field(..., ge=0, description="Annual income")
    credit_score: int = Field(..., ge=300, le=850, description="Credit score")
    employment_years: float = Field(default=0.0, ge=0, description="Years employed")

    # model_config replaces the old Config inner class
    model_config = {
        "json_schema_extra": {
            "examples": [
                {
                    "age": 30,
                    "income": 65000.0,
                    "credit_score": 720,
                    "employment_years": 5.0,
                }
            ]
        }
    }


class PredictionResponse(BaseModel):
    """Output from a single prediction."""

    prediction: int = Field(description="0 = low risk, 1 = high risk")
    probability: float = Field(description="Probability of high risk")
    risk_label: str = Field(description="Human-readable risk level")
    processing_time_ms: float = Field(description="Time to make prediction")


class BatchRequest(BaseModel):
    """Input for multiple predictions."""

    items: List[PredictionRequest] = Field(
        description="List of prediction requests"
```

```
    )


class BatchResponse(BaseModel):
    """Output for multiple predictions."""

    predictions: List[PredictionResponse]
    total_items: int
    total_processing_time_ms: float


class ModelInfo(BaseModel):
    """Information about the loaded model."""

    model_type: str
    n_features: int
    model_version: str


class HealthResponse(BaseModel):
    """Health check response."""

    status: str
    model_loaded: bool
    uptime_seconds: float


# ============================================================
# Application Setup
# ============================================================

# Global state
ml_model = None
model_info = {}
start_time = None


@asynccontextmanager
async def lifespan(app: FastAPI):
    """Load model at startup, clean up at shutdown."""
    global ml_model, model_info, start_time

    start_time = time.time()

    # Load model
    model_path = os.getenv("MODEL_PATH", "models/trained_model.pkl")
    logger.info(f"Loading model from: {model_path}")

    try:
        ml_model = joblib.load(model_path)
        model_info = {
            "model_type": type(ml_model).__name__,
            "n_features": getattr(ml_model, "n_features_in_", 4),
            "model_version": os.getenv("MODEL_VERSION", "1.0.0"),
        }
```

```
        logger.info(f"Model loaded: {model_info['model_type']}")
    except FileNotFoundError:
        logger.warning(f"Model not found at {model_path}, running without model")
        ml_model = None

    yield

    logger.info("Shutting down...")
    ml_model = None


app = FastAPI(
    title="ML Prediction API",
    description="A complete API for serving ML predictions",
    version="1.0.0",
    lifespan=lifespan,
)


# ============================================================
# Helper Functions
# ============================================================

def prepare_features(request: PredictionRequest) -> np.ndarray:
    """
    Convert a request into a feature array for the model.

    The model expects features in a specific order as a
    numpy array. This function ensures the conversion is
    consistent.
    """
    features = np.array([[
        request.age,
        request.income,
        request.credit_score,
        request.employment_years,
    ]])
    return features


def get_risk_label(probability: float) -> str:
    """
    Convert probability to a human-readable label.

    Parameters
    ----------
    probability : float
        Probability of high risk (0.0 to 1.0).

    Returns
    -------
    str
        "low", "medium", or "high"
    """
    if probability < 0.3:
```

```
        return "low"
    elif probability < 0.7:
        return "medium"
    else:
        return "high"


# ============================================================
# API Endpoints
# ============================================================

@app.get("/health", response_model=HealthResponse)
def health_check():
    """
    Check if the API is running and the model is loaded.

    This endpoint is used by monitoring tools and load
    balancers to verify the service is healthy.
    """
    return HealthResponse(
        status="healthy" if ml_model is not None else "degraded",
        model_loaded=ml_model is not None,
        uptime_seconds=time.time() - start_time if start_time else 0,
    )


@app.get("/model/info", response_model=ModelInfo)
def get_model_info():
    """
    Get information about the loaded model.

    Useful for debugging and knowing which model version
    is currently serving predictions.
    """
    if ml_model is None:
        raise HTTPException(
            status_code=503,
            detail="Model not loaded",
        )
    return ModelInfo(**model_info)


@app.post("/predict", response_model=PredictionResponse)
def predict(request: PredictionRequest):
    """
    Make a single prediction.

    Send the input features and get back a prediction
    with probability and risk label.
    """
    if ml_model is None:
        raise HTTPException(
            status_code=503,
            detail="Model not loaded. Please check the model path.",
        )
```

```
    start = time.time()

    # Prepare features
    features = prepare_features(request)

    # Make prediction
    prediction = int(ml_model.predict(features)[0])

    # Get probability (if model supports it)
    try:
        probability = float(
            ml_model.predict_proba(features)[0][1]
        )
    except AttributeError:
        # Some models do not have predict_proba
        probability = float(prediction)

    processing_time = (time.time() - start) * 1000

    return PredictionResponse(
        prediction=prediction,
        probability=round(probability, 4),
        risk_label=get_risk_label(probability),
        processing_time_ms=round(processing_time, 2),
    )


@app.post("/predict/batch", response_model=BatchResponse)
def predict_batch(request: BatchRequest):
    """
    Make predictions for multiple items at once.

    Batch prediction is more efficient than making
    many individual requests.
    """
    if ml_model is None:
        raise HTTPException(
            status_code=503,
            detail="Model not loaded",
        )

    start = time.time()
    predictions = []

    # Process all items
    # In production, you would vectorize this for speed
    for item in request.items:
        features = prepare_features(item)
        pred = int(ml_model.predict(features)[0])

        try:
            prob = float(ml_model.predict_proba(features)[0][1])
        except AttributeError:
            prob = float(pred)
```

```
        predictions.append(
            PredictionResponse(
                prediction=pred,
                probability=round(prob, 4),
                risk_label=get_risk_label(prob),
                processing_time_ms=0,  # Individual time not tracked in batch
            )
        )

    total_time = (time.time() - start) * 1000

    return BatchResponse(
        predictions=predictions,
        total_items=len(predictions),
        total_processing_time_ms=round(total_time, 2),
    )


# ============================================================
# Run directly (alternative to uvicorn command)
# ============================================================

if __name__ == "__main__":
    import uvicorn
    uvicorn.run(
        "main:app",
        host="0.0.0.0",
        port=8000,
        reload=True,
    )
```

Testing Your API

Testing with curl

curl is a command-line tool for making HTTP requests. Think of it as a phone that can call your API.

```bash
# Test the health endpoint
curl http://localhost:8000/health
```

```json
{
  "status": "healthy",
  "model_loaded": true,
  "uptime_seconds": 45.23
}
```

```bash
# Make a prediction
curl -X POST http://localhost:8000/predict \
  -H "Content-Type: application/json" \
  -d '{"age": 30, "income": 65000, "credit_score": 720, "employment_years": 5}'
```

```json
{
  "prediction": 0,
  "probability": 0.1532,
  "risk_label": "low",
  "processing_time_ms": 2.45
}
```

```bash
# Batch prediction
curl -X POST http://localhost:8000/predict/batch \
  -H "Content-Type: application/json" \
  -d '{
    "items": [
      {"age": 25, "income": 40000, "credit_score": 650, "employment_years": 2},
      {"age": 55, "income": 120000, "credit_score": 800, "employment_years": 25}
    ]
  }'
```

```json
{
  "predictions": [
    {"prediction": 1, "probability": 0.7234, "risk_label": "high", "processing_time_ms"...
    {"prediction": 0, "probability": 0.0845, "risk_label": "low", "processing_time_ms": 0}
  ],
  "total_items": 2,
  "total_processing_time_ms": 3.12
}
```

Understanding the curl commands:

```bash
curl -X POST http://localhost:8000/predict \
  -H "Content-Type: application/json" \
  -d '{"age": 30, ...}'
```

- `curl` is the command-line HTTP client
- `-X POST` means we are sending data (not just requesting it)

- `http://localhost:8000/predict` is the URL of our endpoint
- `-H "Content-Type: application/json"` tells the API we are sending JSON data
- `-d '{...}'` is the actual data (the JSON body)

Testing with Python (httpx)

httpx is a Python library for making HTTP requests. It is like curl but in Python:

```python
"""
test_api.py - Test the ML API using httpx.

Install: pip install httpx
Run: python test_api.py

Make sure the API is running first:
    uvicorn main:app --reload
"""

import httpx

# Base URL of our API
BASE_URL = "http://localhost:8000"


def test_health():
    """Test the health check endpoint."""
    response = httpx.get(f"{BASE_URL}/health")
    print(f"Status Code: {response.status_code}")
    print(f"Response: {response.json()}")
    assert response.status_code == 200
    print("Health check PASSED\n")


def test_single_prediction():
    """Test a single prediction."""
    data = {
        "age": 30,
        "income": 65000.0,
        "credit_score": 720,
        "employment_years": 5.0,
    }

    response = httpx.post(
        f"{BASE_URL}/predict",
        json=data,  # json= automatically sets Content-Type
    )

    print(f"Status Code: {response.status_code}")
    print(f"Response: {response.json()}")
    assert response.status_code == 200
    print("Single prediction PASSED\n")
```

```
def test_batch_prediction():
    """Test batch prediction."""
    data = {
        "items": [
            {"age": 25, "income": 40000, "credit_score": 650, "employment_years": 2},
            {"age": 45, "income": 90000, "credit_score": 780, "employment_years": 15},
            {"age": 60, "income": 150000, "credit_score": 810, "employment_years": 30},
        ]
    }

    response = httpx.post(
        f"{BASE_URL}/predict/batch",
        json=data,
    )

    print(f"Status Code: {response.status_code}")
    result = response.json()
    print(f"Total items: {result['total_items']}")
    for i, pred in enumerate(result["predictions"]):
        print(f"  Item {i+1}: risk={pred['risk_label']}, prob={pred['probability']}")
    assert response.status_code == 200
    print("Batch prediction PASSED\n")


def test_invalid_input():
    """Test that invalid data gets rejected."""
    data = {
        "age": -5,  # Invalid: negative age
        "income": 50000,
        "credit_score": 100,  # Invalid: below 300
    }

    response = httpx.post(
        f"{BASE_URL}/predict",
        json=data,
    )

    print(f"Status Code: {response.status_code}")
    print(f"Error: {response.json()}")
    assert response.status_code == 422  # Validation error
    print("Invalid input test PASSED\n")


def test_model_info():
    """Test the model info endpoint."""
    response = httpx.get(f"{BASE_URL}/model/info")
    print(f"Status Code: {response.status_code}")
    print(f"Model Info: {response.json()}")
    print("Model info PASSED\n")


if __name__ == "__main__":
    print("Testing ML API\n" + "=" * 40)
```

```
    test_health()
    test_single_prediction()
    test_batch_prediction()
    test_invalid_input()
    test_model_info()
    print("=" * 40)
    print("All tests passed!")
```

```
Output:
Testing ML API
========================================
Status Code: 200
Response: {'status': 'healthy', 'model_loaded': True, 'uptime_seconds': 12.5}
Health check PASSED

Status Code: 200
Response: {'prediction': 0, 'probability': 0.1532, 'risk_label': 'low', 'processing_tim...
Single prediction PASSED

Status Code: 200
Total items: 3
  Item 1: risk=high, prob=0.7234
  Item 2: risk=low, prob=0.2156
  Item 3: risk=low, prob=0.0845
Batch prediction PASSED

Status Code: 422
Error: {'detail': [{'msg': 'ensure this value is greater than or equal to 0', ...}]}
Invalid input test PASSED

Status Code: 200
Model Info: {'model_type': 'RandomForestClassifier', 'n_features': 4, 'model_version': ...
Model info PASSED

========================================
All tests passed!
```

Automatic API Documentation

One of FastAPI's best features is automatic documentation. When you run your API, visit `http://localhost:8000/docs` to see an interactive documentation page.

```
+-----------------------------------------------------+
|  FastAPI Auto-Generated Documentation               |
|  http://localhost:8000/docs                         |
```

```
|                                                    |
|  GET  /health        -> Health check               |
|  GET  /model/info    -> Model information          |
|  POST /predict       -> Single prediction          |
|  POST /predict/batch -> Batch prediction           |
|                                                    |
|  Each endpoint shows:                             |
|  - Expected input format                          |
|  - Example values                                 |
|  - Response format                                |
|  - "Try it out" button to test live!              |
+---------------------------------------------------+
```

Common Mistakes

1. Loading the model inside the endpoint function. This loads the model on every request, making the API slow. Load it once at startup.
2. Not validating input data. Without Pydantic models, bad data can crash your model. Always validate inputs.
3. Returning raw numpy types. FastAPI cannot serialize numpy int64 or float64 directly. Convert to Python int() or float() first.
4. No health check endpoint. Load balancers and monitoring tools need a way to check if your API is alive.
5. Hardcoding the model path. Use environment variables so you can change the path without modifying code.

Best Practices

1. Use Pydantic models for all inputs and outputs. They validate data and generate documentation automatically.
2. Load models at startup using the lifespan context manager. This is the modern, recommended approach in FastAPI.
3. Add a health check endpoint. It should verify the model is loaded and the API is functioning.
4. Include processing time in responses. This helps monitor API performance.

5. Use environment variables for configuration. This makes your API easy to deploy in different environments.
6. Write tests. Test every endpoint with valid and invalid data.

Quick Summary

FastAPI makes it easy to serve ML models as web APIs. You define input and output schemas with Pydantic, load your model at startup, and create endpoints that accept data and return predictions. FastAPI automatically validates inputs, generates documentation, and handles errors gracefully.

Key Points

- An API lets other applications use your ML model over the network
- FastAPI is a modern, fast Python framework for building APIs
- Pydantic models validate input data automatically
- Load models once at startup, not on every request
- Always include health check and model info endpoints
- Test with curl (command line) or httpx (Python)
- FastAPI generates interactive documentation at /docs

Practice Questions

1. What is the difference between a GET and a POST request? When would you use each one?
2. Why should you load your ML model at startup rather than inside the prediction endpoint?
3. What does Pydantic do, and why is it important for an ML API?
4. What HTTP status code does FastAPI return when input validation fails? What does this code mean?

5. How would you make your API accept an optional parameter, like a confidence threshold?

Exercises

Exercise 1: Build a Sentiment API

Create a FastAPI application that:

- Accepts a text string as input
- Returns a sentiment prediction (positive/negative) with confidence
- Includes a health check endpoint
- Validates that the text is not empty

Exercise 2: Add Error Handling

Extend the prediction API to handle these edge cases:

- Model file not found at startup
- Input features with extreme values (age = 999)
- Request with missing required fields

Exercise 3: Add Request Logging

Add middleware that logs every request including:

- The endpoint that was called
- The time the request was received
- How long the request took to process
- Whether it succeeded or failed

What Is Next?

Your API works great on your laptop. But how do you run it on a server where everyone can access it? In Chapter 4, we will learn about Docker, which packages your API and all its dependencies into a container that can run anywhere. Think of it as putting your API in a

shipping container that can be delivered to any server in the world.

Chapter 4: Docker for ML Applications

What You Will Learn

In this chapter, you will learn:

- What containers are and why they matter for ML
- How Docker works using a shipping container analogy
- How to write a Dockerfile for an ML application
- How to build and run Docker images
- How to use docker-compose to manage multiple services
- How to containerize a complete ML API

Why This Chapter Matters

Imagine you bake a perfect cake at home. You share the recipe with a friend. They follow it exactly, but their cake tastes different. Why? Maybe their oven temperature is slightly off. Maybe their flour brand is different. Maybe they have a different altitude.

The same thing happens with software. Your ML API works perfectly on your laptop. But when you try to run it on a server, it breaks. The server has a different Python version. A library is missing. A system package is wrong.

Docker solves this problem by packaging your application with everything it needs: code, libraries, system tools, and settings. It is like giving your friend not just the recipe, but the entire kitchen.

What Are Containers?

A container is a lightweight, standalone package that includes everything needed to run a piece of software. Let us understand this with the shipping container analogy.

The Shipping Container Analogy

```
+----------------------------------------------------+
|  Before Shipping Containers (Before Docker)       |
|                                                    |
|  Loading a cargo ship was a nightmare:            |
|  - Different shapes and sizes of goods            |
|  - Each item needed special handling              |
|  - Moving cargo between ships was slow            |
|  - Things got damaged or lost                     |
|                                                    |
|  After Shipping Containers (After Docker)         |
|                                                    |
|  Everything goes in standard containers:          |
|  - Any container fits on any ship                 |
|  - Easy to move between ships, trucks, trains     |
|  - Contents are protected and isolated            |
|  - Loading and unloading is fast                  |
+----------------------------------------------------+
```

```
+----------------------------------------------------+
|  Docker Container = Shipping Container             |
|                                                    |
|  +-----------------------------+                   |
|  | Your ML Application        |                   |
|  |                             |                   |
|  | Python 3.11                |                   |
|  | scikit-learn 1.3           |                   |
|  | FastAPI 0.104              |                   |
|  | Your model file            |                   |
|  | Your code                  |                   |
|  | Everything it needs        |                   |
|  +-----------------------------+                   |
|                                                    |
|  Runs the same on ANY computer!                   |
+----------------------------------------------------+
```

Containers vs Virtual Machines

You might have heard of virtual machines (VMs). Both VMs and containers provide isolation, but they work differently:

```
+-----------------------------------------------------+
|  Virtual Machine              Container             |
|                                                     |
|  +-------------+    +-------+ +-------+ +-------+|
|  | App         |    | App 1 | | App 2 | | App 3 ||
|  | Libraries   |    | Libs  | | Libs  | | Libs  ||
|  | Guest OS    |    +-------+ +-------+ +-------+|
|  | (Full Linux)|    |       Docker Engine        ||
|  +-------------+    +----------------------------+|
|  | Hypervisor  |    |       Host OS              ||
|  +-------------+    +----------------------------+|
|  | Host OS     |                                  |
|  +-------------+    Containers share the host OS  |
|                      (much lighter and faster!)   |
|  Each VM has a       Containers start in seconds  |
|  full OS copy        VMs take minutes             |
+-----------------------------------------------------+
```

Docker Concepts

Before writing a Dockerfile, let us understand the key terms:

```
+-----------------------------------------------------+
|  Docker Vocabulary                                  |
|                                                     |
|  Image       = A recipe (blueprint)               |
|  Container   = A cake made from the recipe        |
|  Dockerfile  = The recipe instructions            |
|  Registry    = A recipe book (Docker Hub)         |
|  Volume      = Shared storage between containers |
+-----------------------------------------------------+
```

- Image: A read-only template. Like a recipe card. It describes what goes into the container.
- Container: A running instance of an image. Like a cake baked from the recipe. You can have many cakes from one recipe.
- Dockerfile: A text file with instructions to build an image. Like the step-by-step recipe.
- Registry: A place to store and share images. Docker Hub is the most popular one, like a cookbook library.

Writing a Dockerfile

A Dockerfile is a series of instructions that tell Docker how to build your image. Each instruction creates a layer, like building a cake layer by layer.

Dockerfile for an ML API

```dockerfile
# Dockerfile - Instructions to build our ML API container
#
# Think of each line as a step in a recipe:
# 1. Start with a base (like starting with flour)
# 2. Add ingredients (install packages)
# 3. Prepare (copy code, configure)
# 4. Set the final instruction (how to serve)

# ============================================================
# Step 1: Choose a base image
# ============================================================
# python:3.11-slim is a lightweight Python image
# "slim" means it has only the essentials (smaller download)
FROM python:3.11-slim

# ============================================================
# Step 2: Set metadata
# ============================================================
# LABEL adds information about the image
LABEL maintainer="your-email@example.com"
LABEL description="ML Prediction API"

# ============================================================
# Step 3: Set the working directory
# ============================================================
# All subsequent commands run in this directory
# Like saying "cd /app" before doing anything
WORKDIR /app

# ============================================================
# Step 4: Install system dependencies (if needed)
# ============================================================
# Some Python packages need system libraries to compile
# We install them, then clean up to keep the image small
RUN apt-get update && \
    apt-get install -y --no-install-recommends \
        build-essential \
    && rm -rf /var/lib/apt/lists/*

# ============================================================
# Step 5: Copy and install Python dependencies
# ============================================================
# We copy requirements.txt FIRST, before the rest of the code.
```

```
# Why? Docker caches each layer. If requirements.txt hasn't
# changed, Docker reuses the cached layer and skips the
# slow "pip install" step. This makes rebuilds much faster.
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# ============================================================
# Step 6: Copy the application code
# ============================================================
COPY src/ ./src/
COPY models/ ./models/
COPY main.py .
COPY config.yaml .

# ============================================================
# Step 7: Set environment variables
# ============================================================
# These can be overridden when running the container
ENV MODEL_PATH=/app/models/trained_model.pkl
ENV MODEL_VERSION=1.0.0
ENV PORT=8000

# ============================================================
# Step 8: Expose the port
# ============================================================
# EXPOSE tells Docker which port the app uses
# It does not actually publish the port (that happens at runtime)
EXPOSE ${PORT}

# ============================================================
# Step 9: Define the startup command
# ============================================================
# CMD is what runs when the container starts
# --host 0.0.0.0 means "accept connections from anywhere"
# (not just localhost)
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

The requirements.txt File

```
# requirements.txt - Python packages for the ML API
fastapi==0.104.1
uvicorn==0.24.0
scikit-learn==1.3.2
joblib==1.3.2
numpy==1.26.2
pydantic==2.5.2
pyyaml==6.0.1
```

Building and Running Docker Images

Building an Image

```bash
# Build the Docker image
# -t gives the image a name (tag)
# . tells Docker to look for the Dockerfile in the current directory
docker build -t ml-api:1.0 .
```

```
Output:
[+] Building 45.2s (12/12) FINISHED
 => [1/7] FROM python:3.11-slim
 => [2/7] WORKDIR /app
 => [3/7] RUN apt-get update && ...
 => [4/7] COPY requirements.txt .
 => [5/7] RUN pip install --no-cache-dir -r requirements.txt
 => [6/7] COPY src/ ./src/
 => [7/7] COPY main.py .
 => exporting to image
 => => naming to docker.io/library/ml-api:1.0
```

Running a Container

```bash
# Run the container
# -d runs in the background (detached mode)
# -p 8000:8000 maps port 8000 on your computer to port 8000 in the container
# --name gives the container a name for easy reference
docker run -d -p 8000:8000 --name my-ml-api ml-api:1.0
```

```
Output:
a1b2c3d4e5f6g7h8i9j0k1l2m3n4o5p6q7r8s9t0u1v2w3x4y5z6
```

The long string is the container ID. Your API is now running! Test it:

```bash
curl http://localhost:8000/health
```

```json
{"status": "healthy", "model_loaded": true, "uptime_seconds": 3.45}
```

Useful Docker Commands

```python
"""
docker_commands.py - A reference for common Docker commands.

This is not a Python script to run, but a reference
showing Docker commands with explanations.
"""

# The commands below are meant to be run in a terminal,
# not in Python. They are shown here for reference.

docker_commands = {
    # Building
    "docker build -t ml-api:1.0 .":
        "Build an image from the Dockerfile in current directory",

    # Running
    "docker run -d -p 8000:8000 ml-api:1.0":
        "Run a container in the background, map port 8000",

    "docker run -it ml-api:1.0 bash":
        "Run a container and open a bash shell inside it",

    # Managing containers
    "docker ps":
        "List running containers",

    "docker ps -a":
        "List ALL containers (including stopped ones)",

    "docker stop my-ml-api":
        "Stop a running container",

    "docker start my-ml-api":
        "Start a stopped container",

    "docker rm my-ml-api":
        "Remove a container (must be stopped first)",

    # Viewing logs
    "docker logs my-ml-api":
        "See the container's output logs",

    "docker logs -f my-ml-api":
        "Follow logs in real-time (like tail -f)",

    # Managing images
    "docker images":
        "List all images on your computer",

    "docker rmi ml-api:1.0":
        "Remove an image",

    # Inspecting
```

```
    "docker exec -it my-ml-api bash":
        "Open a shell inside a running container",

    "docker inspect my-ml-api":
        "See detailed info about a container",
}

for cmd, desc in docker_commands.items():
    print(f"  {cmd}")
    print(f"    -> {desc}\n")
```

```
Output:
  docker build -t ml-api:1.0 .
    -> Build an image from the Dockerfile in current directory

  docker run -d -p 8000:8000 ml-api:1.0
    -> Run a container in the background, map port 8000

  docker run -it ml-api:1.0 bash
    -> Run a container and open a bash shell inside it

  docker ps
    -> List running containers

  docker ps -a
    -> List ALL containers (including stopped ones)

  docker stop my-ml-api
    -> Stop a running container

  docker start my-ml-api
    -> Start a stopped container

  docker rm my-ml-api
    -> Remove a container (must be stopped first)

  docker logs my-ml-api
    -> See the container's output logs

  docker logs -f my-ml-api
    -> Follow logs in real-time (like tail -f)

  docker images
    -> List all images on your computer

  docker rmi ml-api:1.0
    -> Remove an image

  docker exec -it my-ml-api bash
    -> Open a shell inside a running container

  docker inspect my-ml-api
    -> See detailed info about a container
```

Optimizing Docker Images for ML

ML Docker images can get very large because of libraries like scikit-learn, PyTorch, and TensorFlow. Here are strategies to keep them small:

Multi-Stage Builds

```dockerfile
# Dockerfile.multistage - Use multi-stage builds for smaller images
#
# Multi-stage builds use two stages:
# 1. Builder stage: install everything, compile packages
# 2. Final stage: copy only what we need from the builder
#
# Like cooking in a messy prep kitchen, then plating
# the clean final dish in the dining room.

# ============================================================
# Stage 1: Builder (the messy kitchen)
# ============================================================
FROM python:3.11-slim AS builder

WORKDIR /app

# Install build tools
RUN apt-get update && \
    apt-get install -y --no-install-recommends build-essential

# Install Python packages into a specific directory
COPY requirements.txt .
RUN pip install --no-cache-dir --prefix=/install -r requirements.txt

# ============================================================
# Stage 2: Final image (the clean dining room)
# ============================================================
FROM python:3.11-slim

WORKDIR /app

# Copy ONLY the installed packages from the builder
# We do not need build-essential or other build tools
COPY --from=builder /install /usr/local

# Copy application code
COPY src/ ./src/
COPY models/ ./models/
COPY main.py .
```

```
ENV MODEL_PATH=/app/models/trained_model.pkl
EXPOSE 8000

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

The .dockerignore File

Like `.gitignore`, this file tells Docker what to skip when copying files:

```
# .dockerignore - Files to exclude from Docker builds
# These files are not needed in the container

# Virtual environments
venv/
.venv/
env/

# Python cache
__pycache__/
*.pyc
*.pyo

# Git
.git/
.gitignore

# IDE files
.vscode/
.idea/

# Notebooks (not needed in production)
notebooks/
*.ipynb

# Data files (usually too large for containers)
data/raw/
data/external/

# Logs (containers generate their own)
logs/

# Tests (not needed in production image)
tests/

# Documentation
*.md
docs/
```

Docker Compose

When your application has multiple services (like an API, a database, and a monitoring tool), managing them individually is painful. Docker Compose lets you define and run all services together with a single command.

docker-compose.yml for ML API

```yaml
# docker-compose.yml - Define multiple services
#
# This file describes all the services your application needs
# and how they connect to each other.

version: "3.8"

services:
  # ============================================================
  # ML API Service
  # ============================================================
  ml-api:
    build:
      context: .                    # Build from current directory
      dockerfile: Dockerfile        # Using this Dockerfile
    container_name: ml-prediction-api
    ports:
      - "8000:8000"                 # Map host:container ports
    environment:
      - MODEL_PATH=/app/models/trained_model.pkl
      - MODEL_VERSION=1.0.0
      - LOG_LEVEL=INFO
    volumes:
      - ./models:/app/models        # Share model files
      - ./logs:/app/logs            # Persist log files
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:8000/health"]
      interval: 30s                 # Check every 30 seconds
      timeout: 10s                  # Wait max 10 seconds
      retries: 3                    # Fail after 3 retries
    restart: unless-stopped         # Auto-restart if it crashes

  # ============================================================
  # Redis Cache (optional, for caching predictions)
  # ============================================================
  redis:
    image: redis:7-alpine           # Use official Redis image
    container_name: ml-cache
    ports:
      - "6379:6379"
    volumes:
      - redis_data:/data            # Persist Redis data
```

```
  # ============================================================
  # Monitoring with Prometheus (optional)
  # ============================================================
  prometheus:
    image: prom/prometheus:latest
    container_name: ml-monitoring
    ports:
      - "9090:9090"
    volumes:
      - ./monitoring/prometheus.yml:/etc/prometheus/prometheus.yml

# Named volumes persist data between container restarts
volumes:
  redis_data:
```

Running with Docker Compose

```bash
# Start all services
docker-compose up -d

# View logs from all services
docker-compose logs

# View logs from a specific service
docker-compose logs ml-api

# Stop all services
docker-compose down

# Rebuild and start (after code changes)
docker-compose up -d --build
```

```
Output (docker-compose up -d):
Creating network "ml-project_default" with the default driver
Creating ml-cache           ... done
Creating ml-monitoring      ... done
Creating ml-prediction-api  ... done
```

Complete ML API in Docker

Let us put together a complete, production-ready ML application in Docker.

Project Structure

```
ml-docker-project/
├── Dockerfile
├── docker-compose.yml
├── .dockerignore
├── requirements.txt
├── main.py
├── src/
│   ├── __init__.py
│   ├── model.py
│   └── schemas.py
├── models/
│   └── trained_model.pkl
├── scripts/
│   └── train_and_save.py
└── tests/
    └── test_api.py
```

Complete Training and Saving Script

```python
"""
scripts/train_and_save.py - Train a model and save it for Docker.

Run this BEFORE building the Docker image so the model
file exists when Docker copies it into the container.
"""

from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
import joblib
import json
import os

# Create sample data
X, y = make_classification(
    n_samples=1000,
    n_features=4,       # age, income, credit_score, employment_years
    n_classes=2,
    random_state=42,
)

# Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42,
)

# Train
```

```
model = RandomForestClassifier(
    n_estimators=100, random_state=42
)
model.fit(X_train, y_train)

# Evaluate
accuracy = model.score(X_test, y_test)
print(f"Model accuracy: {accuracy:.4f}")

# Save
os.makedirs("models", exist_ok=True)
joblib.dump(model, "models/trained_model.pkl")
print("Model saved to models/trained_model.pkl")

# Save metadata
metadata = {
    "accuracy": accuracy,
    "n_features": 4,
    "feature_names": ["age", "income", "credit_score", "employment_years"],
    "model_type": "RandomForestClassifier",
    "n_estimators": 100,
}

with open("models/metadata.json", "w") as f:
    json.dump(metadata, f, indent=2)
print("Metadata saved to models/metadata.json")
```

```
Output:
Model accuracy: 0.9350
Model saved to models/trained_model.pkl
Metadata saved to models/metadata.json
```

Build and Run the Complete Application

```bash
# Step 1: Train the model (run on your laptop)
python scripts/train_and_save.py

# Step 2: Build the Docker image
docker build -t ml-api:latest .

# Step 3: Run the container
docker run -d -p 8000:8000 --name ml-api ml-api:latest

# Step 4: Test it
curl http://localhost:8000/health
curl -X POST http://localhost:8000/predict \
  -H "Content-Type: application/json" \
  -d '{"age": 30, "income": 65000, "credit_score": 720, "employment_years": 5}'
```

```
# Step 5: Check logs
docker logs ml-api
```

Common Mistakes

1. Using a large base image. Use `python:3.11-slim` instead of `python:3.11`. The slim version is much smaller.
2. Not using .dockerignore. Without it, Docker copies unnecessary files (like `.git/`, `data/`, `notebooks/`) into the image, making it huge.
3. Copying code before requirements. Always copy `requirements.txt` and install dependencies first. Docker caches this layer, making rebuilds faster.
4. Running as root. For security, create a non-root user in your Dockerfile. Root access inside containers is a security risk.
5. Storing data in containers. Containers are temporary. Use volumes to persist data that should survive container restarts.

Best Practices

1. Keep images small. Use slim base images, multi-stage builds, and .dockerignore.
2. One service per container. Do not put your API and database in the same container. Use docker-compose for multiple services.
3. Use health checks. They help Docker (and orchestrators like Kubernetes) know when your container is healthy.
4. Pin dependency versions. Use `scikit-learn==1.3.2` instead of `scikit-learn` in requirements.txt. This prevents surprise updates.
5. Use environment variables for configuration. This lets you change settings without rebuilding the image.
6. Tag your images with versions. Use `ml-api:1.0`, `ml-api:1.1`, etc., not just `ml-api:latest`.

Quick Summary

Docker packages your ML application with all its dependencies into a container that runs identically on any machine. A Dockerfile describes how to build the container. Docker Compose manages multiple containers together. This eliminates the "it works on my machine" problem and makes deployment predictable and repeatable.

Key Points

- Containers package code and dependencies together
- Containers are lighter and faster than virtual machines
- A Dockerfile is a recipe for building a container image
- Copy requirements.txt before code for faster rebuilds
- Docker Compose manages multiple services together
- Use volumes to persist data outside containers
- Use .dockerignore to keep images small
- Always use health checks in production

Practice Questions

1. What is the difference between a Docker image and a Docker container?
2. Why do we copy requirements.txt and install dependencies before copying the rest of the code?
3. What is the purpose of the .dockerignore file?
4. Explain what `-p 8000:8000` does when running `docker run`.
5. When would you use docker-compose instead of a single Docker container?

Exercises

Exercise 1: Build Your First Container

Create a Dockerfile for a simple Python script that prints "Hello from Docker!" and run it as a container.

Exercise 2: Containerize an ML API

Take the FastAPI ML API from Chapter 3 and:

- Write a Dockerfile for it
- Build the image
- Run it and test the endpoints
- Check the container logs

Exercise 3: Multi-Service Setup

Create a docker-compose.yml that runs:

- Your ML API on port 8000
- A Redis container on port 6379
- Verify both services start and can communicate

What Is Next?

Your ML API is now containerized and runs consistently on any machine. But your machine is still just your laptop. In Chapter 5, we will learn how to deploy your containerized API to the cloud so it can serve predictions to users anywhere in the world, 24 hours a day, 7 days a week.

Chapter 5: Deploying to the Cloud

What You Will Learn

In this chapter, you will learn:

- What cloud computing is and why it matters for ML
- An overview of AWS, GCP, and Azure (simplified)
- How to deploy your ML API using easy platforms like Render and Railway
- How to use environment variables to manage secrets
- The basics of scaling your application

Why This Chapter Matters

Your ML API works on your laptop. It runs in Docker. But your laptop is not a server. It shuts down when you close it. It has one internet connection. It cannot handle thousands of users at once.

Cloud deployment is like moving from a food truck to a restaurant chain. The food truck (your laptop) makes great food, but it can only serve a few people in one location. A restaurant chain (the cloud) serves millions of people across many locations, stays open 24/7, and can add more kitchens when demand increases.

What Is Cloud Computing?

Cloud computing means using someone else's computers over the internet. Instead of buying and maintaining your own servers, you rent computing power from companies like Amazon, Google, or Microsoft.

```
+-----------------------------------------------------+
|  Your Laptop vs The Cloud                            |
|                                                      |
|  Your Laptop:                                       |
|  - 1 computer                                       |
|  - Shuts down when you close it                     |
|  - Limited internet bandwidth                       |
|  - You maintain everything                          |
|                                                      |
|  The Cloud:                                         |
|  - Thousands of computers available                 |
|  - Runs 24/7/365                                    |
|  - Fast internet connections                        |
|  - Provider handles hardware maintenance            |
|  - Pay only for what you use                        |
+-----------------------------------------------------+
```

The Big Three Cloud Providers

Overview

```
+-----------------------------------------------------+
|  Cloud Provider Comparison (Simplified)              |
|                                                      |
|  AWS (Amazon Web Services)                          |
|  - Largest cloud provider                           |
|  - Most services available                          |
|  - Complex but powerful                             |
|  - Key ML services: SageMaker, Lambda, EC2          |
|                                                      |
|  GCP (Google Cloud Platform)                        |
|  - Strong in data and ML                            |
|  - Good integration with TensorFlow                 |
|  - Key ML services: Vertex AI, Cloud Run, GCE       |
|                                                      |
|  Azure (Microsoft)                                  |
|  - Popular with enterprises                         |
|  - Good integration with Microsoft tools            |
|  - Key ML services: Azure ML, Functions, VMs        |
+-----------------------------------------------------+
```

Which Should You Choose?

For beginners, the big three can be overwhelming. They have hundreds of services each. Instead, we will focus on simpler platforms that are built on top of these cloud providers:

```
+-----------------------------------------------------+
|  Easy Deployment Platforms                          |
|                                                     |
|  Render     -> Simple, free tier, auto-deploy      |
|  Railway    -> Developer-friendly, fast setup      |
|  Fly.io     -> Global deployment, Docker-native    |
|  HuggingFace Spaces -> ML-specific, free GPUs      |
|                                                     |
|  These platforms handle the complex cloud stuff     |
|  for you. You just push your code.                  |
+-----------------------------------------------------+
```

Deploying to Render (Step by Step)

Render is one of the easiest platforms for deploying web services. It has a free tier and supports Docker, making it perfect for our ML API.

Step 1: Prepare Your Project

Make sure your project has these files:

```
ml-api-project/
├── main.py              # Your FastAPI application
├── requirements.txt     # Python dependencies
├── models/
│   └── trained_model.pkl
├── src/
│   ├── __init__.py
│   └── model.py
└── render.yaml          # Render configuration (optional)
```

Step 2: Create a Start Command

Render needs to know how to start your application. Add a `Procfile` or use Render's dashboard:

```
# Procfile - Tells Render how to start your app
web: uvicorn main:app --host 0.0.0.0 --port $PORT
```

The `$PORT` variable is set by Render automatically. Your app must listen on this port.

Step 3: Configuration for Render

```yaml
# render.yaml - Render deployment configuration
services:
  - type: web
    name: ml-prediction-api
    env: python
    buildCommand: pip install -r requirements.txt
    startCommand: uvicorn main:app --host 0.0.0.0 --port $PORT
    envVars:
      - key: MODEL_PATH
        value: models/trained_model.pkl
      - key: MODEL_VERSION
        value: "1.0.0"
      - key: PYTHON_VERSION
        value: "3.11.0"
```

Step 4: Handling the PORT Variable

Your FastAPI app needs to use the port assigned by the cloud platform:

```python
"""
main_cloud.py - FastAPI app configured for cloud deployment.

Cloud platforms assign a random port through the PORT
environment variable. Your app must use this port.
"""

import os
from fastapi import FastAPI
import uvicorn

app = FastAPI(title="ML API")


@app.get("/health")
def health():
    return {"status": "healthy"}


@app.get("/")
```

```
def root():
    """
    Root endpoint. Many platforms check this to verify
    your app is running.
    """
    return {
        "message": "ML Prediction API is running",
        "docs": "/docs",
    }


if __name__ == "__main__":
    # Get port from environment variable
    # Default to 8000 for local development
    port = int(os.getenv("PORT", 8000))

    uvicorn.run(
        "main_cloud:app",
        host="0.0.0.0",
        port=port,
    )
```

Deploying to Railway

Railway is another beginner-friendly platform. It can deploy directly from a GitHub repository.

Step 1: Create a railway.json Configuration

```json
{
  "$schema": "https://railway.app/railway.schema.json",
  "build": {
    "builder": "NIXPACKS"
  },
  "deploy": {
    "startCommand": "uvicorn main:app --host 0.0.0.0 --port $PORT",
    "healthcheckPath": "/health",
    "restartPolicyType": "ON_FAILURE",
    "restartPolicyMaxRetries": 3
  }
}
```

Step 2: Deploy Process

```
+-----------------------------------------------------+
|  Railway Deployment Steps                            |
|                                                      |
|  1. Push code to GitHub                             |
|  2. Connect GitHub repo to Railway                  |
|  3. Railway detects Python project                   |
|  4. Railway installs requirements.txt                |
|  5. Railway starts your app                         |
|  6. You get a public URL!                           |
|                                                      |
|  Every git push triggers automatic redeployment! |
+-----------------------------------------------------+
```

Environment Variables

Environment variables are like sealed envelopes that contain secret information. Your code can read them, but they are not stored in your code files. This is important for keeping passwords, API keys, and other secrets safe.

```
+-----------------------------------------------------+
|  Why Environment Variables?                          |
|                                                      |
|  WRONG way (secrets in code):                       |
|  password = "my_secret_123"   # Anyone can see!  |
|                                                      |
|  RIGHT way (environment variables):                 |
|  password = os.getenv("DB_PASSWORD")  # Hidden!  |
|                                                      |
|  The actual value is set in the cloud platform's |
|  dashboard, not in your code.                       |
+-----------------------------------------------------+
```

Using Environment Variables in Python

```python
"""
env_config.py - Manage configuration with environment variables.

Environment variables are key-value pairs that are set
outside your code. They are the standard way to handle
secrets and configuration in cloud deployments.
"""
```

```python
import os
from dataclasses import dataclass


@dataclass
class AppConfig:
    """
    Application configuration loaded from environment variables.

    Each setting has a default value for local development
    but can be overridden by setting the environment variable.
    """

    # Model settings
    model_path: str = os.getenv(
        "MODEL_PATH", "models/trained_model.pkl"
    )
    model_version: str = os.getenv(
        "MODEL_VERSION", "1.0.0"
    )

    # Server settings
    port: int = int(os.getenv("PORT", "8000"))
    host: str = os.getenv("HOST", "0.0.0.0")
    debug: bool = os.getenv("DEBUG", "false").lower() == "true"

    # Logging
    log_level: str = os.getenv("LOG_LEVEL", "INFO")

    # Optional: Database connection (if needed)
    database_url: str = os.getenv("DATABASE_URL", "")

    # Optional: API key for authentication
    api_key: str = os.getenv("API_KEY", "")


# Create a global config instance
config = AppConfig()

# Print config (never print secrets in production!)
print(f"Model path: {config.model_path}")
print(f"Model version: {config.model_version}")
print(f"Port: {config.port}")
print(f"Debug mode: {config.debug}")
print(f"Log level: {config.log_level}")
# Do NOT print database_url or api_key!
```

```
Output:
Model path: models/trained_model.pkl
Model version: 1.0.0
Port: 8000
Debug mode: False
Log level: INFO
```

Setting Environment Variables on Cloud Platforms

Each platform has its own way to set environment variables:

```bash
# Render: Set through the dashboard or render.yaml
# Railway: Set through the dashboard or CLI
railway variables set MODEL_PATH=models/trained_model.pkl
railway variables set MODEL_VERSION=2.0.0

# Docker: Set when running the container
docker run -e MODEL_PATH=/app/models/v2.pkl \
           -e MODEL_VERSION=2.0.0 \
           ml-api:latest

# Local development: Use a .env file
# Create a file called .env (add to .gitignore!)
```

The .env File for Local Development

```
# .env - Local environment variables
# IMPORTANT: Add .env to .gitignore!

MODEL_PATH=models/trained_model.pkl
MODEL_VERSION=1.0.0
PORT=8000
DEBUG=true
LOG_LEVEL=DEBUG
DATABASE_URL=sqlite:///local.db
API_KEY=local-dev-key-not-for-production
```

```python
"""
load_dotenv_example.py - Load environment variables from .env file.

Install: pip install python-dotenv

This is ONLY for local development. In production,
environment variables are set by the platform.
"""

from dotenv import load_dotenv
import os

# Load variables from .env file into the environment
# This has no effect if the variables are already set
# (e.g., by Docker or the cloud platform)
load_dotenv()
```

```
# Now os.getenv() can read values from .env
model_path = os.getenv("MODEL_PATH")
print(f"Model path: {model_path}")
```

```
Output:
Model path: models/trained_model.pkl
```

Basic Scaling

Scaling means handling more users. There are two types:

```
+-----------------------------------------------------+
|  Types of Scaling                                    |
|                                                      |
|  Vertical Scaling (Scale Up)                        |
|  = Get a bigger computer                            |
|  Like replacing your bike with a motorcycle         |
|                                                      |
|  Before: 2 CPU, 4 GB RAM                           |
|  After:  8 CPU, 32 GB RAM                           |
|                                                      |
|  Simple but has limits (biggest computer only       |
|  has so much power)                                  |
|                                                      |
|  ------------------------------------------------  |
|                                                      |
|  Horizontal Scaling (Scale Out)                     |
|  = Get more computers                               |
|  Like hiring more delivery drivers                  |
|                                                      |
|  Before: 1 server                                   |
|  After:  5 servers (same size)                      |
|                                                      |
|  More complex but virtually unlimited               |
+-----------------------------------------------------+
```

Scaling on Render

```yaml
# render.yaml - Scaling configuration
services:
  - type: web
```

```
    name: ml-prediction-api
    env: python
    plan: standard          # Upgrade from free for more resources
    numInstances: 3         # Run 3 copies (horizontal scaling)
    autoDeploy: true        # Auto-deploy on git push
    buildCommand: pip install -r requirements.txt
    startCommand: uvicorn main:app --host 0.0.0.0 --port $PORT --workers 4
```

Using Multiple Workers

```bash
# Run with multiple workers (vertical scaling within one server)
# Each worker handles requests independently
uvicorn main:app --host 0.0.0.0 --port 8000 --workers 4
```

```
+-----------------------------------------------------+
|  Workers and Instances                               |
|                                                      |
|  Instance 1 (Server 1)                              |
|  +-----------+-----------+                          |
|  | Worker 1  | Worker 2  |                          |
|  +-----------+-----------+                          |
|  | Worker 3  | Worker 4  |                          |
|  +-----------+-----------+                          |
|                                                      |
|  Instance 2 (Server 2)                              |
|  +-----------+-----------+                          |
|  | Worker 1  | Worker 2  |                          |
|  +-----------+-----------+                          |
|  | Worker 3  | Worker 4  |                          |
|  +-----------+-----------+                          |
|                                                      |
|  Total capacity: 8 workers handling requests        |
+-----------------------------------------------------+
```

Key Scaling Considerations for ML

```python
"""
scaling_considerations.py - Things to think about when scaling ML APIs.

This is a reference showing common scaling patterns.
"""

scaling_tips = {
    "Model Loading": {
```

```
        "problem": "Each worker loads the model separately",
        "solution": "Use shared memory or model serving (Chapter 7)",
        "memory": "100 MB model x 4 workers = 400 MB RAM needed",
    },
    "Stateless Design": {
        "problem": "Requests may go to different workers/instances",
        "solution": "Do not store state in your API. Each request "
                    "should be independent.",
        "example": "Do not use global variables to count requests. "
                   "Use an external database or Redis instead.",
    },
    "Cold Starts": {
        "problem": "First request is slow (loading model, warming up)",
        "solution": "Keep minimum instances running. Pre-warm with "
                    "health checks.",
    },
    "Memory Limits": {
        "problem": "ML models can use a lot of memory",
        "solution": "Choose instance sizes with enough RAM. "
                    "Consider model compression (ONNX, quantization).",
    },
}

for topic, info in scaling_tips.items():
    print(f"\n{topic}:")
    for key, value in info.items():
        print(f"  {key}: {value}")
```

```
Output:

Model Loading:
  problem: Each worker loads the model separately
  solution: Use shared memory or model serving (Chapter 7)
  memory: 100 MB model x 4 workers = 400 MB RAM needed

Stateless Design:
  problem: Requests may go to different workers/instances
  solution: Do not store state in your API. Each request should be independent.
  example: Do not use global variables to count requests. Use an external database or R...

Cold Starts:
  problem: First request is slow (loading model, warming up)
  solution: Keep minimum instances running. Pre-warm with health checks.

Memory Limits:
  problem: ML models can use a lot of memory
  solution: Choose instance sizes with enough RAM. Consider model compression (ONNX, qu...
```

Deployment Checklist

```python
"""
deployment_checklist.py - Verify your app is ready for deployment.

Run this before deploying to catch common issues.
"""

import os
import sys


def check_deployment_readiness():
    """Check if the application is ready for cloud deployment."""
    checks = []

    # Check 1: requirements.txt exists
    has_requirements = os.path.exists("requirements.txt")
    checks.append(("requirements.txt exists", has_requirements))

    # Check 2: Main application file exists
    has_main = os.path.exists("main.py")
    checks.append(("main.py exists", has_main))

    # Check 3: Model file exists
    model_path = os.getenv("MODEL_PATH", "models/trained_model.pkl")
    has_model = os.path.exists(model_path)
    checks.append((f"Model file exists ({model_path})", has_model))

    # Check 4: No hardcoded secrets
    # This is a simple check - in practice use tools like git-secrets
    if has_main:
        with open("main.py") as f:
            content = f.read()
        no_secrets = "password" not in content.lower() or "os.getenv" in content
        checks.append(("No hardcoded secrets in main.py", no_secrets))

    # Check 5: PORT is configurable
    if has_main:
        with open("main.py") as f:
            content = f.read()
        port_configurable = "PORT" in content or "port" in content
        checks.append(("PORT is configurable", port_configurable))

    # Check 6: .env is in .gitignore
    if os.path.exists(".gitignore"):
        with open(".gitignore") as f:
            gitignore = f.read()
        env_ignored = ".env" in gitignore
        checks.append((".env in .gitignore", env_ignored))
    else:
        checks.append((".gitignore exists", False))
```

```
    # Print results
    print("Deployment Readiness Check")
    print("=" * 50)
    all_passed = True
    for name, passed in checks:
        status = "PASS" if passed else "FAIL"
        symbol = "[+]" if passed else "[-]"
        print(f"  {symbol} {name}: {status}")
        if not passed:
            all_passed = False

    print("=" * 50)
    if all_passed:
        print("All checks passed! Ready for deployment.")
    else:
        print("Some checks failed. Fix issues before deploying.")

    return all_passed


check_deployment_readiness()
```

```
Output:
Deployment Readiness Check
==================================================
  [+] requirements.txt exists: PASS
  [+] main.py exists: PASS
  [+] Model file exists (models/trained_model.pkl): PASS
  [+] No hardcoded secrets in main.py: PASS
  [+] PORT is configurable: PASS
  [+] .env in .gitignore: PASS
==================================================
All checks passed! Ready for deployment.
```

Common Mistakes

1. Hardcoding secrets in code. Never put passwords, API keys, or tokens directly in your code. Use environment variables.
2. Not using a health check endpoint. Cloud platforms need to know if your app is running. Without a health check, they cannot restart it when it fails.
3. Ignoring the PORT environment variable. Most cloud platforms assign a port dynamically. Your app must use `os.getenv("PORT")`.

4. Deploying without testing locally. Always test your app locally (especially in Docker) before deploying to the cloud.
5. Including large data files in the deployment. Only include the model file. Training data should not be in the deployment package.

Best Practices

1. Start with easy platforms. Use Render or Railway before trying AWS or GCP directly.
2. Use environment variables for all configuration. Never hardcode paths, URLs, keys, or passwords.
3. Add a health check endpoint. Return at least `{"status": "healthy"}` and whether the model is loaded.
4. Set up automatic deploys from GitHub. Push code, and the platform automatically updates your app.
5. Monitor your deployment. Check logs regularly and set up alerts for errors.
6. Start small, then scale. Begin with one instance and scale up as needed.

Quick Summary

Cloud deployment makes your ML API available to the world, 24/7. Easy platforms like Render and Railway handle the complex infrastructure for you. Environment variables keep secrets safe. Scaling can be vertical (bigger machines) or horizontal (more machines). Always test locally before deploying, and use health checks so the platform knows your app is running.

Key Points

- Cloud computing means renting computers over the internet
- AWS, GCP, and Azure are the big providers, but Render and Railway are easier to start with
- Environment variables keep secrets out of your code

- The PORT environment variable must be used for cloud deployment
- Vertical scaling means bigger machines; horizontal means more machines
- Always include a health check endpoint
- Test locally with Docker before deploying to the cloud

Practice Questions

1. What is the difference between vertical and horizontal scaling? Give a real-world analogy for each.
2. Why should you never hardcode passwords or API keys in your source code?
3. What is the purpose of the PORT environment variable in cloud deployments?
4. Name two advantages of using a platform like Render compared to setting up your own AWS EC2 instance.
5. What is a "cold start" and how does it affect ML API performance?

Exercises

Exercise 1: Deploy a Simple API

Deploy the FastAPI hello world application to Render or Railway. Verify it works by visiting the public URL.

Exercise 2: Environment Variable Configuration

Modify your ML API to read all configuration from environment variables:

- MODEL_PATH, MODEL_VERSION, LOG_LEVEL, MAX_BATCH_SIZE
- Provide sensible defaults for local development
- Test by setting different values

Exercise 3: Scaling Plan

Write a scaling plan for an ML API that receives:

- 100 requests per day initially

- Expected to grow to 10,000 requests per day in 6 months
- Each prediction takes about 50ms

Include: instance type, number of instances, estimated cost, and when to add more capacity.

What Is Next?

You now know how to deploy your ML API to the cloud. But what if you want to create a visual demo of your model that non-technical people can use? In Chapter 6, we will learn about Streamlit and Gradio, two tools that let you build beautiful interactive demos of your ML models in just a few lines of code.

Chapter 6: Building ML Demos with Streamlit and Gradio

What You Will Learn

In this chapter, you will learn:

- What Streamlit is and how to build interactive apps with it
- How to use st.write, st.slider, st.file_uploader, and other widgets
- What Gradio is and how to create ML interfaces quickly
- How to use Gradio's Interface and Blocks for complex layouts
- How to build complete ML demos with both tools
- How to share your demos publicly with the world

Why This Chapter Matters

Imagine you build an amazing model that can predict house prices. You show it to your manager by running a Python script in the terminal. They stare at the black screen with numbers scrolling by and say, "That is nice, but can you make it... easier to use?"

Non-technical people do not want to run Python scripts. They want to click buttons, drag sliders, and see results in a nice interface. Streamlit and Gradio let you build these interfaces in minutes, not weeks.

Think of Streamlit and Gradio as "Instagram filters for your ML model." They take something technical and make it accessible and visual.

Part 1: Streamlit

Streamlit turns Python scripts into interactive web apps. You write Python, and Streamlit handles all the web development.

Installing Streamlit

```bash
pip install streamlit
```

Your First Streamlit App

```python
"""
app_hello.py - Your first Streamlit application.

Run with: streamlit run app_hello.py

Streamlit will open a browser window automatically.
"""

import streamlit as st

# st.title creates a large heading
st.title("My First Streamlit App")

# st.write is the Swiss army knife of Streamlit
# It can display text, data, charts, and more
st.write("Hello, World! Welcome to Streamlit.")

# st.markdown supports Markdown formatting
st.markdown("""
### What Can Streamlit Do?
- Display **text** and *formatting*
- Show data in tables
- Create interactive widgets
- Display charts and visualizations
""")

# Display a simple calculation
x = 5
y = 10
st.write(f"The sum of {x} and {y} is {x + y}")
```

To run this app:

```bash
streamlit run app_hello.py
```

```
Output (in terminal):
  You can now view your Streamlit app in your browser.
  Local URL: http://localhost:8501
  Network URL: http://192.168.1.100:8501
```

Streamlit Widgets

Widgets are interactive elements that let users provide input:

```python
"""
app_widgets.py - Streamlit widget examples.

Run with: streamlit run app_widgets.py

Each widget returns the current value selected by the user.
When a user changes a widget, Streamlit re-runs the entire
script with the new values.
"""

import streamlit as st

st.title("Streamlit Widget Gallery")

# ============================================================
# Text Input
# ============================================================
st.header("Text Input")

# st.text_input creates a text field
name = st.text_input("What is your name?", value="Alice")
st.write(f"Hello, {name}!")

# st.text_area creates a larger text field
feedback = st.text_area(
    "Tell us about your experience:",
    height=100,
)
if feedback:
    st.write(f"You wrote {len(feedback)} characters")

# ============================================================
# Numbers
# ============================================================
st.header("Number Input")

# st.slider creates a slider
age = st.slider(
    "Select your age:",
    min_value=0,
    max_value=100,
```

```
    value=25,
    step=1,
)
st.write(f"Your age: {age}")

# st.number_input creates a number field with +/- buttons
income = st.number_input(
    "Annual income ($):",
    min_value=0,
    max_value=1000000,
    value=50000,
    step=1000,
)

# ============================================================
# Selection
# ============================================================
st.header("Selection Widgets")

# st.selectbox creates a dropdown menu
model_type = st.selectbox(
    "Choose a model:",
    options=["Random Forest", "Logistic Regression", "Neural Network"],
)
st.write(f"You selected: {model_type}")

# st.multiselect lets users pick multiple options
features = st.multiselect(
    "Select features to use:",
    options=["Age", "Income", "Credit Score", "Employment Years"],
    default=["Age", "Income"],
)
st.write(f"Selected features: {features}")

# st.radio creates radio buttons
difficulty = st.radio(
    "Difficulty level:",
    options=["Easy", "Medium", "Hard"],
)

# ============================================================
# Toggles
# ============================================================
st.header("Toggles")

# st.checkbox creates a checkbox
show_details = st.checkbox("Show detailed output")
if show_details:
    st.write("Here are the details you requested!")

# ============================================================
# File Upload
# ============================================================
st.header("File Upload")
```

```
# st.file_uploader creates a file upload widget
uploaded_file = st.file_uploader(
    "Upload a CSV file:",
    type=["csv"],  # Only allow CSV files
)

if uploaded_file is not None:
    import pandas as pd
    df = pd.read_csv(uploaded_file)
    st.write(f"Uploaded file has {len(df)} rows and {len(df.columns)} columns")
    st.dataframe(df.head())  # Display the first 5 rows as a table

# ============================================================
# Buttons
# ============================================================
st.header("Buttons")

if st.button("Click me!"):
    st.write("Button was clicked!")
    st.balloons()  # Fun animation!
```

Building an ML Demo with Streamlit

```python
"""
app_ml_demo.py - Complete ML prediction demo with Streamlit.

Run with: streamlit run app_ml_demo.py

This app lets users input features and see predictions
from a trained ML model.
"""

import streamlit as st
import joblib
import numpy as np
import pandas as pd

# ============================================================
# Page Configuration
# ============================================================
st.set_page_config(
    page_title="ML Prediction Demo",
    page_icon="□",
    layout="wide",  # Use the full width of the page
)

st.title("Credit Risk Prediction")
st.write("Enter the customer details below to predict credit risk.")
```

```
# ============================================================
# Load Model (cached so it only loads once)
# ============================================================
@st.cache_resource  # Cache the model so it does not reload on every interaction
def load_model():
    """
    Load the trained ML model.

    @st.cache_resource tells Streamlit to load this once
    and reuse it. Without this decorator, the model would
    reload every time a user moves a slider!
    """
    try:
        model = joblib.load("models/trained_model.pkl")
        return model
    except FileNotFoundError:
        return None


model = load_model()

if model is None:
    st.error("Model file not found! Please train a model first.")
    st.stop()  # Stop the app here


# ============================================================
# User Input (Sidebar)
# ============================================================
st.sidebar.header("Customer Details")

# Sidebar widgets keep the main area clean
age = st.sidebar.slider("Age", 18, 80, 35)
income = st.sidebar.number_input(
    "Annual Income ($)", 10000, 500000, 50000, step=5000
)
credit_score = st.sidebar.slider("Credit Score", 300, 850, 650)
employment_years = st.sidebar.slider("Years Employed", 0, 40, 5)


# ============================================================
# Make Prediction
# ============================================================
# Prepare features
features = np.array([[age, income, credit_score, employment_years]])

# Predict
prediction = model.predict(features)[0]
try:
    probability = model.predict_proba(features)[0]
except AttributeError:
    probability = [1 - prediction, prediction]

# ============================================================
# Display Results
```

```
# ============================================================
col1, col2, col3 = st.columns(3)

with col1:
    st.metric(
        label="Risk Prediction",
        value="High Risk" if prediction == 1 else "Low Risk",
    )

with col2:
    st.metric(
        label="Risk Probability",
        value=f"{probability[1]:.1%}",
    )

with col3:
    risk_level = (
        "Low" if probability[1] < 0.3
        else "Medium" if probability[1] < 0.7
        else "High"
    )
    st.metric(label="Risk Level", value=risk_level)

# Progress bar showing risk level
st.subheader("Risk Score")
st.progress(float(probability[1]))

# Show input summary
st.subheader("Customer Summary")
summary_df = pd.DataFrame({
    "Feature": ["Age", "Income", "Credit Score", "Employment Years"],
    "Value": [age, f"${income:,}", credit_score, employment_years],
})
st.table(summary_df)

# ============================================================
# Batch Prediction from File Upload
# ============================================================
st.subheader("Batch Prediction")
st.write("Upload a CSV file with columns: age, income, credit_score, employment_years")

uploaded_file = st.file_uploader("Choose a CSV file", type="csv")

if uploaded_file is not None:
    df = pd.read_csv(uploaded_file)
    st.write(f"Loaded {len(df)} records")

    # Make predictions for all rows
    predictions = model.predict(df.values)
    try:
        probabilities = model.predict_proba(df.values)[:, 1]
    except AttributeError:
        probabilities = predictions.astype(float)

    # Add predictions to the dataframe
```

```
    df["prediction"] = predictions
    df["risk_probability"] = probabilities
    df["risk_label"] = df["risk_probability"].apply(
        lambda p: "Low" if p < 0.3 else "Medium" if p < 0.7 else "High"
    )

    # Display results
    st.dataframe(df)

    # Summary statistics
    st.subheader("Prediction Summary")
    col1, col2, col3 = st.columns(3)
    with col1:
        st.metric("Total Records", len(df))
    with col2:
        st.metric("High Risk", int((predictions == 1).sum()))
    with col3:
        st.metric("Low Risk", int((predictions == 0).sum()))
```

Part 2: Gradio

Gradio is another tool for building ML demos. It is especially popular in the ML research community because it creates interfaces with just a few lines of code.

Installing Gradio

```bash
pip install gradio
```

Gradio Interface: The Simple Way

```python
"""
gradio_hello.py - Your first Gradio application.

Run with: python gradio_hello.py

Gradio's Interface class creates a complete UI from
just three things:
1. A function to run
2. Input components
3. Output components
"""
```

```
import gradio as gr


def greet(name, intensity):
    """
    A simple greeting function.

    Gradio will create input widgets automatically based
    on the function's parameters and type hints.
    """
    return f"Hello, {name}! " + "!" * intensity


# Create the interface
# fn: the function to call
# inputs: what the user provides
# outputs: what we show back
demo = gr.Interface(
    fn=greet,
    inputs=[
        gr.Textbox(label="Your Name", placeholder="Enter your name"),
        gr.Slider(minimum=1, maximum=10, value=3, step=1, label="Enthusiasm"),
    ],
    outputs=gr.Textbox(label="Greeting"),
    title="Greeting Generator",
    description="Enter your name and choose your enthusiasm level!",
)

# Launch the app
demo.launch()
```

```
Output:
Running on local URL: http://127.0.0.1:7860
```

Gradio for ML Predictions

```python
"""
gradio_ml.py - ML prediction interface with Gradio.

Run with: python gradio_ml.py

This creates a simple interface where users enter
features and get a risk prediction.
"""

import gradio as gr
import joblib
```

```
import numpy as np


# Load model
try:
    model = joblib.load("models/trained_model.pkl")
except FileNotFoundError:
    model = None


def predict_risk(age, income, credit_score, employment_years):
    """
    Make a credit risk prediction.

    Each parameter corresponds to a Gradio input widget.
    The function returns values that map to output widgets.
    """
    if model is None:
        return "Model not loaded", 0.0, "Error"

    # Prepare features
    features = np.array([[age, income, credit_score, employment_years]])

    # Make prediction
    prediction = model.predict(features)[0]

    try:
        probability = model.predict_proba(features)[0][1]
    except AttributeError:
        probability = float(prediction)

    # Determine risk level
    if probability < 0.3:
        risk_level = "Low Risk"
    elif probability < 0.7:
        risk_level = "Medium Risk"
    else:
        risk_level = "High Risk"

    result_text = f"Prediction: {risk_level}"
    confidence = round(probability * 100, 1)

    return result_text, confidence, risk_level


# Create the interface
demo = gr.Interface(
    fn=predict_risk,
    inputs=[
        gr.Slider(18, 80, value=35, step=1, label="Age"),
        gr.Number(value=50000, label="Annual Income ($)"),
        gr.Slider(300, 850, value=650, step=10, label="Credit Score"),
        gr.Slider(0, 40, value=5, step=1, label="Years Employed"),
    ],
    outputs=[
```

```
        gr.Textbox(label="Prediction"),
        gr.Number(label="Risk Probability (%)"),
        gr.Textbox(label="Risk Level"),
    ],
    title="Credit Risk Predictor",
    description="Enter customer details to predict credit risk.",
    examples=[
        [25, 40000, 620, 2],   # Young, low income, low credit
        [45, 95000, 780, 15],  # Middle-aged, good income, high credit
        [60, 150000, 810, 30], # Senior, high income, excellent credit
    ],
)

demo.launch()
```

```
Output:
Running on local URL: http://127.0.0.1:7860
```

Gradio Blocks: More Control

`gr.Interface` is simple but limited. `gr.Blocks` gives you full control over the layout:

```python
"""
gradio_blocks.py - Advanced Gradio layout with Blocks.

Run with: python gradio_blocks.py

Blocks lets you create custom layouts with rows, columns,
tabs, and more. It is like the difference between a
pre-built house (Interface) and designing your own (Blocks).
"""

import gradio as gr
import joblib
import numpy as np
import pandas as pd


# Load model
try:
    model = joblib.load("models/trained_model.pkl")
    model_loaded = True
except FileNotFoundError:
    model_loaded = False


def single_predict(age, income, credit_score, emp_years):
    """Make a single prediction."""
```

```
    if not model_loaded:
        return "Model not loaded", "N/A", "N/A"

    features = np.array([[age, income, credit_score, emp_years]])
    prediction = model.predict(features)[0]

    try:
        probability = model.predict_proba(features)[0][1]
    except AttributeError:
        probability = float(prediction)

    risk = "Low" if probability < 0.3 else "Medium" if probability < 0.7 else "High"

    return (
        f"{'High' if prediction == 1 else 'Low'} Risk",
        f"{probability:.1%}",
        risk,
    )


def batch_predict(file):
    """Make predictions for a CSV file."""
    if not model_loaded:
        return None

    df = pd.read_csv(file.name)
    predictions = model.predict(df.values)

    try:
        probabilities = model.predict_proba(df.values)[:, 1]
    except AttributeError:
        probabilities = predictions.astype(float)

    df["prediction"] = predictions
    df["probability"] = probabilities
    df["risk"] = df["probability"].apply(
        lambda p: "Low" if p < 0.3 else "Medium" if p < 0.7 else "High"
    )

    return df


# Build the interface with Blocks
with gr.Blocks(title="ML Risk Predictor") as demo:
    gr.Markdown("# Credit Risk Prediction Tool")
    gr.Markdown("Predict credit risk using our trained model.")

    # Tabs for different features
    with gr.Tabs():
        # Tab 1: Single Prediction
        with gr.TabItem("Single Prediction"):
            with gr.Row():
                # Left column: inputs
                with gr.Column():
                    gr.Markdown("### Customer Details")
```

```
                    age = gr.Slider(18, 80, value=35, step=1, label="Age")
                    income = gr.Number(value=50000, label="Income ($)")
                    credit = gr.Slider(300, 850, value=650, step=10, label="Credit Score")
                    emp = gr.Slider(0, 40, value=5, step=1, label="Employment Years")
                    predict_btn = gr.Button("Predict", variant="primary")

                # Right column: outputs
                with gr.Column():
                    gr.Markdown("### Prediction Results")
                    pred_output = gr.Textbox(label="Prediction")
                    prob_output = gr.Textbox(label="Probability")
                    risk_output = gr.Textbox(label="Risk Level")

            # Connect button to function
            predict_btn.click(
                fn=single_predict,
                inputs=[age, income, credit, emp],
                outputs=[pred_output, prob_output, risk_output],
            )

        # Tab 2: Batch Prediction
        with gr.TabItem("Batch Prediction"):
            gr.Markdown("### Upload CSV for Batch Predictions")
            gr.Markdown(
                "CSV should have columns: age, income, "
                "credit_score, employment_years"
            )
            file_input = gr.File(label="Upload CSV")
            batch_output = gr.Dataframe(label="Results")

            file_input.change(
                fn=batch_predict,
                inputs=file_input,
                outputs=batch_output,
            )

        # Tab 3: Model Info
        with gr.TabItem("About"):
            gr.Markdown("""
            ### About This Model
            - **Type:** Random Forest Classifier
            - **Features:** Age, Income, Credit Score, Employment Years
            - **Output:** Binary risk prediction (Low/High)
            """)
            status = "Loaded" if model_loaded else "Not Loaded"
            gr.Markdown(f"**Model Status:** {status}")


demo.launch()
```

Sharing Your Demos Publicly

Both Streamlit and Gradio offer ways to share your demo with anyone in the world.

Sharing Gradio with a Public Link

```python
"""
gradio_share.py - Share your Gradio demo publicly.

The share=True parameter creates a temporary public URL
that anyone can access for 72 hours.
"""

import gradio as gr


def predict(text):
    return f"You entered: {text}"


demo = gr.Interface(
    fn=predict,
    inputs="text",
    outputs="text",
)

# share=True creates a public link
# The link works for 72 hours
demo.launch(share=True)
```

```
Output:
Running on local URL: http://127.0.0.1:7860
Running on public URL: https://abc123xyz.gradio.live

This share link expires in 72 hours.
```

Sharing Streamlit with Streamlit Cloud

Streamlit offers free hosting through Streamlit Cloud:

```
+-----------------------------------------------------+
|  Sharing Streamlit Apps                              |
|                                                      |
|  1. Push your code to GitHub                        |
|  2. Go to share.streamlit.io                        |
|  3. Connect your GitHub repo                        |
```

```
|  4. Select the Python file to run                |
|  5. Click Deploy!                                |
|                                                   |
|  Your app gets a public URL like:                |
|  https://your-app.streamlit.app                  |
+--------------------------------------------------+
```

Streamlit vs Gradio Comparison

```
+-------------------------------------------------------------------+
|  Feature          | Streamlit           | Gradio                |
|-------------------|---------------------|----------------------|
|  Ease of Use      | Very easy           | Very easy             |
|  Code Needed      | More code           | Less code             |
|  Customization    | High                | Medium                |
|  Best For         | Data dashboards     | ML model demos        |
|  Sharing          | Streamlit Cloud     | gradio.live or HF     |
|  Layout Control   | Good (columns,tabs) | Good (Blocks)         |
|  File Upload      | st.file_uploader    | gr.File               |
|  Live Updates     | Re-runs script      | Event-driven          |
|  API Auto-gen     | No                  | Yes (built-in API)    |
+-------------------------------------------------------------------+
```

Common Mistakes

1. Not caching model loading in Streamlit. Without `@st.cache_resource`, the model reloads every time a user interacts with a widget. This makes the app very slow.
2. Forgetting that Streamlit re-runs the entire script. Every time a user changes a widget, Streamlit runs the whole Python file from top to bottom. Design your code with this in mind.
3. Making demos too complex. The goal is a simple demonstration, not a full application. Keep it focused.
4. Not providing example inputs. Users often do not know what values to enter. Provide examples so they can try your model immediately.
5. Not handling errors gracefully. Show friendly error messages when the model file is missing or input is invalid.

Best Practices

1. Use caching for expensive operations. In Streamlit, use `@st.cache_resource` for models and `@st.cache_data` for data loading.
2. Provide example inputs. Both Streamlit and Gradio support examples that users can click to try.
3. Add descriptions and help text. Explain what each input means and what values are expected.
4. Use a sidebar for inputs in Streamlit. This keeps the main area clean for showing results.
5. Test with different input values. Make sure your demo handles edge cases gracefully.

Quick Summary

Streamlit and Gradio are tools for building interactive ML demos quickly. Streamlit is great for data-heavy dashboards with more customization. Gradio excels at creating simple ML interfaces with minimal code. Both can be shared publicly. Use them to make your ML models accessible to non-technical users.

Key Points

- Streamlit turns Python scripts into interactive web apps
- st.cache_resource prevents reloading the model on every interaction
- Gradio's Interface class creates a UI from a function, inputs, and outputs
- Gradio's Blocks class gives more control over layout
- Both tools support file uploads for batch predictions
- Gradio can create temporary public links with share=True
- Streamlit apps can be deployed to Streamlit Cloud for free

Practice Questions

1. What does `@st.cache_resource` do and why is it important when loading ML models in Streamlit?
2. What are the three things you need to provide to `gr.Interface` to create a Gradio app?
3. How does Streamlit handle user interactions differently from Gradio?
4. When would you choose Streamlit over Gradio? When would you choose Gradio over Streamlit?
5. How can you share a Gradio demo with someone who does not have Python installed?

Exercises

Exercise 1: Streamlit Dashboard

Build a Streamlit app that:

- Lets users upload a CSV file
- Shows basic statistics (mean, median, min, max for each column)
- Displays a histogram for a user-selected column
- Shows the first 10 rows of data

Exercise 2: Gradio Model Demo

Create a Gradio interface for an image classification model that:

- Accepts an image upload
- Shows the top 3 predicted classes with confidence scores
- Includes at least 3 example images

Exercise 3: Side-by-Side Comparison

Build the same ML demo in both Streamlit and Gradio. Compare:

- Lines of code needed
- Ease of adding new features
- Look and feel of the interface

What Is Next?

Now you can build beautiful demos for your ML models. But demos handle one request at a time. What about serving predictions to thousands of users simultaneously? In Chapter 7, we will learn about model serving patterns: batch vs real-time inference, scaling strategies, and caching predictions for speed.

Chapter 7: Model Serving Patterns

What You Will Learn

In this chapter, you will learn:

- The difference between batch and real-time inference
- Common model serving patterns used in production
- How to think about scaling your model serving
- How to cache predictions to improve speed and reduce cost

Why This Chapter Matters

Imagine a restaurant. Some dishes are made fresh when you order them (real-time). Others, like soup, are made in large batches in the morning and served throughout the day (batch). Each approach has its strengths.

ML model serving works the same way. Sometimes you need predictions instantly (a fraud detection system checking each credit card swipe). Sometimes you can wait and process everything at once (generating product recommendations overnight for all users). Choosing the right pattern affects speed, cost, and complexity.

Batch vs Real-Time Inference

These are the two fundamental ways to serve predictions:

```
+-------------------------------------------------+
|  Batch Inference                                |
|                                                 |
```

```
|  "Make all the soup in the morning"              |
|                                                   |
|  [Data] --> [Model] --> [Store Results]           |
|  (1000 items at once)   (Database/File)          |
|                                                   |
|  When to use:                                    |
|  - Results not needed immediately                |
|  - Large volume of predictions                   |
|  - Scheduled processing (daily, hourly)          |
|  - Cost-sensitive (cheaper to run in bulk)       |
|                                                   |
|  Examples:                                       |
|  - Nightly product recommendations               |
|  - Monthly churn risk scores                     |
|  - Weekly email targeting                        |
+--------------------------------------------------+

+--------------------------------------------------+
|  Real-Time Inference                              |
|                                                   |
|  "Cook each dish when ordered"                   |
|                                                   |
|  [Request] --> [Model] --> [Response]            |
|  (1 item)     (instant)   (milliseconds)         |
|                                                   |
|  When to use:                                    |
|  - Results needed immediately                    |
|  - User is waiting for a response               |
|  - Input data is not known in advance            |
|                                                   |
|  Examples:                                       |
|  - Fraud detection on card swipe                 |
|  - Autocomplete suggestions while typing         |
|  - Image classification on upload                |
+--------------------------------------------------+
```

Batch Inference Implementation

```python
"""
batch_inference.py - Process predictions in bulk.

Batch inference is efficient because:
1. The model is loaded once
2. Data can be processed in optimized chunks
3. Results are stored for later use
4. It can run during off-peak hours
"""

import joblib
import pandas as pd
```

```
import numpy as np
import json
import time
import logging
from datetime import datetime

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)


def run_batch_inference(
    model_path,
    input_path,
    output_path,
    batch_size=1000,
):
    """
    Run predictions on a large dataset in batches.

    Parameters
    ----------
    model_path : str
        Path to the saved model file.
    input_path : str
        Path to the input CSV file.
    output_path : str
        Path to save predictions.
    batch_size : int
        Number of rows to process at once.
        Larger batches are faster but use more memory.
    """
    start_time = time.time()
    logger.info(f"Starting batch inference at {datetime.now()}")

    # Load model once
    logger.info(f"Loading model from {model_path}")
    model = joblib.load(model_path)

    # Load data
    logger.info(f"Loading data from {input_path}")
    df = pd.read_csv(input_path)
    total_rows = len(df)
    logger.info(f"Total rows to process: {total_rows}")

    # Process in batches
    all_predictions = []
    all_probabilities = []

    for i in range(0, total_rows, batch_size):
        batch = df.iloc[i : i + batch_size]
        batch_num = i // batch_size + 1
        total_batches = (total_rows + batch_size - 1) // batch_size

        logger.info(
            f"Processing batch {batch_num}/{total_batches} "
```

```
            f"({len(batch)} rows)"
        )

        # Make predictions for this batch
        features = batch.values
        predictions = model.predict(features)
        all_predictions.extend(predictions)

        # Get probabilities if available
        try:
            probabilities = model.predict_proba(features)[:, 1]
            all_probabilities.extend(probabilities)
        except AttributeError:
            all_probabilities.extend(predictions.astype(float))

    # Add predictions to the dataframe
    df["prediction"] = all_predictions
    df["probability"] = all_probabilities
    df["predicted_at"] = datetime.now().isoformat()

    # Save results
    df.to_csv(output_path, index=False)

    elapsed = time.time() - start_time
    logger.info(f"Batch inference complete!")
    logger.info(f"Processed {total_rows} rows in {elapsed:.1f} seconds")
    logger.info(f"Speed: {total_rows / elapsed:.0f} rows/second")
    logger.info(f"Results saved to {output_path}")

    return df


# Run batch inference
results = run_batch_inference(
    model_path="models/trained_model.pkl",
    input_path="data/customers_to_predict.csv",
    output_path="data/predictions_output.csv",
    batch_size=500,
)
```

```
Output:
INFO - Starting batch inference at 2024-01-15 10:00:00
INFO - Loading model from models/trained_model.pkl
INFO - Loading data from data/customers_to_predict.csv
INFO - Total rows to process: 10000
INFO - Processing batch 1/20 (500 rows)
INFO - Processing batch 2/20 (500 rows)
...
INFO - Processing batch 20/20 (500 rows)
INFO - Batch inference complete!
INFO - Processed 10000 rows in 4.2 seconds
INFO - Speed: 2381 rows/second
INFO - Results saved to data/predictions_output.csv
```

Real-Time Inference Implementation

```python
"""
realtime_inference.py - Serve predictions one at a time.

Real-time inference focuses on low latency.
Each request must be fast (typically under 100ms).
"""

from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
import joblib
import numpy as np
import time
import logging
from contextlib import asynccontextmanager

logger = logging.getLogger(__name__)

# Global model reference
model = None


@asynccontextmanager
async def lifespan(app: FastAPI):
    """Load model at startup."""
    global model
    logger.info("Loading model for real-time serving...")
    model = joblib.load("models/trained_model.pkl")
    logger.info("Model loaded and ready!")
    yield
    model = None


app = FastAPI(title="Real-Time ML API", lifespan=lifespan)


class PredictRequest(BaseModel):
    features: list[float]


class PredictResponse(BaseModel):
    prediction: int
    probability: float
    latency_ms: float


@app.post("/predict", response_model=PredictResponse)
def predict(request: PredictRequest):
    """
    Make a single prediction with minimal latency.

    The focus here is speed. Every millisecond counts
```

```
    in real-time serving.
    """
    if model is None:
        raise HTTPException(503, "Model not loaded")

    start = time.time()

    # Convert to numpy array (fast)
    features = np.array([request.features])

    # Predict (this is the main bottleneck)
    prediction = int(model.predict(features)[0])

    try:
        probability = float(model.predict_proba(features)[0][1])
    except AttributeError:
        probability = float(prediction)

    latency = (time.time() - start) * 1000

    return PredictResponse(
        prediction=prediction,
        probability=round(probability, 4),
        latency_ms=round(latency, 2),
    )
```

Model Serving Patterns

There are several patterns for how to organize model serving in a larger system:

```
+-----------------------------------------------------+
|  Pattern 1: Embedded Model                          |
|                                                      |
|  The model lives inside the application.            |
|  Simple but tightly coupled.                        |
|                                                      |
|  +-------------------------------+                  |
|  |  Web Application              |                  |
|  |  +----------+                 |                  |
|  |  |  Model   |                 |                  |
|  |  +----------+                 |                  |
|  +-------------------------------+                  |
|                                                      |
|  Good for: Simple apps, prototypes                  |
|  Bad for: Multiple apps sharing a model             |
+-----------------------------------------------------+

+-----------------------------------------------------+
```

```
|  Pattern 2: Model as a Service                   |
|                                                   |
|  The model runs in its own API.                  |
|  Applications call it over the network.          |
|                                                   |
|  +----------+      +----------+                  |
|  | App 1    |----->|          |                  |
|  +----------+      | Model    |                  |
|  +----------+      | Service  |                  |
|  | App 2    |----->|          |                  |
|  +----------+      +----------+                  |
|                                                   |
|  Good for: Multiple apps, independent scaling    |
|  Bad for: Extra network latency                  |
+--------------------------------------------------+

+--------------------------------------------------+
|  Pattern 3: Model Gateway                        |
|                                                   |
|  A gateway routes requests to different models.  |
|  Useful for A/B testing and canary deployments.  |
|                                                   |
|  +----------+      +---------+      +---------+ |
|  |          |----->| Model   |  80% | Model A | |
|  | Gateway  |      | Router  |----->| (v1.0)  | |
|  |          |      |         |  20% +---------+ |
|  +----------+      |         |----->| Model B | |
|                     +---------+      | (v2.0)  | |
|                                      +---------+ |
|  Good for: A/B testing, gradual rollouts        |
+--------------------------------------------------+
```

Implementing Model as a Service

```python
"""
model_service.py - Model serving as an independent service.

This pattern separates the model from the application.
The application sends data to this service and gets
predictions back.
"""

from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from contextlib import asynccontextmanager
import joblib
import numpy as np
import time
import os
import logging
```

```
logger = logging.getLogger(__name__)


class ModelService:
    """
    A service that manages model loading and prediction.

    This class encapsulates all model-related logic,
    making it easy to swap models or add features
    like caching.
    """

    def __init__(self):
        self.model = None
        self.model_version = None
        self.model_loaded_at = None

    def load(self, model_path, version="unknown"):
        """Load a model from disk."""
        self.model = joblib.load(model_path)
        self.model_version = version
        self.model_loaded_at = time.time()
        logger.info(
            f"Model v{version} loaded from {model_path}"
        )

    def predict(self, features):
        """Make a prediction."""
        if self.model is None:
            raise RuntimeError("Model not loaded")

        features_array = np.array([features])
        prediction = int(self.model.predict(features_array)[0])

        try:
            probability = float(
                self.model.predict_proba(features_array)[0][1]
            )
        except AttributeError:
            probability = float(prediction)

        return {
            "prediction": prediction,
            "probability": probability,
            "model_version": self.model_version,
        }

    def get_info(self):
        """Get information about the loaded model."""
        return {
            "model_version": self.model_version,
            "model_type": type(self.model).__name__ if self.model else None,
            "loaded": self.model is not None,
            "uptime_seconds": (
```

```
                time.time() - self.model_loaded_at
                if self.model_loaded_at
                else 0
            ),
        }


# Create the service instance
service = ModelService()


@asynccontextmanager
async def lifespan(app: FastAPI):
    model_path = os.getenv("MODEL_PATH", "models/trained_model.pkl")
    model_version = os.getenv("MODEL_VERSION", "1.0.0")
    service.load(model_path, model_version)
    yield


app = FastAPI(
    title="Model Serving Service",
    lifespan=lifespan,
)


class Features(BaseModel):
    values: list[float]


@app.post("/predict")
def predict(features: Features):
    start = time.time()
    result = service.predict(features.values)
    result["latency_ms"] = round((time.time() - start) * 1000, 2)
    return result


@app.get("/info")
def model_info():
    return service.get_info()


@app.get("/health")
def health():
    info = service.get_info()
    return {
        "status": "healthy" if info["loaded"] else "unhealthy",
        "model_loaded": info["loaded"],
    }
```

Scaling Considerations

```
+--------------------------------------------------+
|  Scaling Decision Tree                            |
|                                                   |
|  How many requests per second?                   |
|       |                                           |
|       +-- < 10 RPS   -> Single server, 1 worker  |
|       |                                           |
|       +-- 10-100 RPS -> Single server, 4 workers |
|       |                                           |
|       +-- 100-1000   -> Multiple servers          |
|       |                  + load balancer           |
|       |                                           |
|       +-- > 1000 RPS -> Specialized serving       |
|                          (TF Serving, Triton)     |
+--------------------------------------------------+
```

Memory Considerations

```python
"""
memory_check.py - Check model memory usage.

Before scaling, know how much memory your model needs.
Each worker loads a separate copy of the model.
"""

import joblib
import sys
import os


def get_model_memory(model_path):
    """
    Estimate how much memory a model uses.

    Parameters
    ----------
    model_path : str
        Path to the model file.

    Returns
    -------
    dict
        Memory usage information.
    """
    # File size on disk
    file_size = os.path.getsize(model_path)
```

```
    # Load model and check memory
    model = joblib.load(model_path)
    # sys.getsizeof gives a rough estimate
    # The actual memory usage is usually higher
    memory_estimate = sys.getsizeof(model)

    # Rule of thumb: in-memory size is roughly
    # 1.5x to 3x the file size for sklearn models
    estimated_memory = file_size * 2

    return {
        "file_size_mb": file_size / (1024 * 1024),
        "estimated_memory_mb": estimated_memory / (1024 * 1024),
    }


info = get_model_memory("models/trained_model.pkl")
print(f"File size: {info['file_size_mb']:.1f} MB")
print(f"Estimated memory: {info['estimated_memory_mb']:.1f} MB")

# Planning for scaling
workers = 4
instances = 3
total_model_copies = workers * instances
total_memory = info["estimated_memory_mb"] * total_model_copies

print(f"\nScaling plan:")
print(f"  Workers per instance: {workers}")
print(f"  Number of instances: {instances}")
print(f"  Total model copies: {total_model_copies}")
print(f"  Total memory for models: {total_memory:.0f} MB")
```

```
Output:
File size: 12.5 MB
Estimated memory: 25.0 MB

Scaling plan:
  Workers per instance: 4
  Number of instances: 3
  Total model copies: 12
  Total memory for models: 300 MB
```

Caching Predictions

Caching means storing predictions so you do not have to compute them again. If someone asks for the same prediction twice, why run the model twice?

```
+-----------------------------------------------+
|  Without Cache                                 |
|                                                |
|  Request 1: age=30, income=50k -> Run model -> 5ms|
|  Request 2: age=30, income=50k -> Run model -> 5ms|
|  Request 3: age=30, income=50k -> Run model -> 5ms|
|  (Same inputs, computed 3 times!)              |
|                                                |
|  With Cache                                    |
|                                                |
|  Request 1: age=30, income=50k -> Run model -> 5ms|
|  Request 2: age=30, income=50k -> From cache -> 0.1ms|
|  Request 3: age=30, income=50k -> From cache -> 0.1ms|
|  (Computed once, served from memory!)          |
+-----------------------------------------------+
```

Simple In-Memory Cache

```python
"""
prediction_cache.py - Cache predictions for speed.

An in-memory cache stores recent predictions so
identical requests get instant responses.
"""

from collections import OrderedDict
import hashlib
import json
import time
import logging

logger = logging.getLogger(__name__)


class PredictionCache:
    """
    A simple LRU (Least Recently Used) cache for predictions.

    LRU means when the cache is full, the oldest unused
    item gets removed to make room for new ones.

    Think of it like a small shelf that holds your most
    recently used books. When the shelf is full and you
    get a new book, you remove the one you have not
    touched in the longest time.
    """

    def __init__(self, max_size=10000, ttl_seconds=3600):
        """
```

```
        Initialize the cache.

        Parameters
        ----------
        max_size : int
            Maximum number of predictions to cache.
        ttl_seconds : int
            Time To Live - how long a cached prediction
            stays valid (in seconds). After this time,
            the prediction is recomputed.
        """
        self.cache = OrderedDict()
        self.max_size = max_size
        self.ttl_seconds = ttl_seconds
        self.hits = 0    # Times we found the answer in cache
        self.misses = 0  # Times we had to compute

    def _make_key(self, features):
        """
        Create a unique key from the input features.

        We convert the features to a string and hash it.
        This creates a short, unique identifier for any
        set of input features.
        """
        feature_str = json.dumps(features, sort_keys=True)
        return hashlib.md5(feature_str.encode()).hexdigest()

    def get(self, features):
        """
        Try to get a cached prediction.

        Returns None if not found or expired.
        """
        key = self._make_key(features)

        if key in self.cache:
            entry = self.cache[key]
            age = time.time() - entry["timestamp"]

            # Check if the cached entry has expired
            if age < self.ttl_seconds:
                # Move to end (most recently used)
                self.cache.move_to_end(key)
                self.hits += 1
                logger.debug(f"Cache HIT (age: {age:.1f}s)")
                return entry["prediction"]
            else:
                # Expired, remove it
                del self.cache[key]
                logger.debug(f"Cache EXPIRED (age: {age:.1f}s)")

        self.misses += 1
        logger.debug("Cache MISS")
        return None
```

```
    def put(self, features, prediction):
        """Store a prediction in the cache."""
        key = self._make_key(features)

        # Remove oldest if cache is full
        if len(self.cache) >= self.max_size:
            self.cache.popitem(last=False)

        self.cache[key] = {
            "prediction": prediction,
            "timestamp": time.time(),
        }

    def get_stats(self):
        """Get cache performance statistics."""
        total = self.hits + self.misses
        hit_rate = self.hits / total if total > 0 else 0

        return {
            "size": len(self.cache),
            "max_size": self.max_size,
            "hits": self.hits,
            "misses": self.misses,
            "hit_rate": f"{hit_rate:.1%}",
        }


# Usage example
cache = PredictionCache(max_size=1000, ttl_seconds=3600)

# First request: cache miss, must compute
features1 = [30, 50000, 720, 5]
result = cache.get(features1)
print(f"First lookup: {result}")  # None (not cached yet)

# Store the prediction
cache.put(features1, {"prediction": 0, "probability": 0.15})

# Second request: cache hit!
result = cache.get(features1)
print(f"Second lookup: {result}")  # The cached prediction

# Different features: cache miss
features2 = [45, 80000, 780, 15]
result = cache.get(features2)
print(f"Different features: {result}")  # None

# Check statistics
print(f"\nCache stats: {cache.get_stats()}")
```

```
Output:
First lookup: None
```

```
Second lookup: {'prediction': 0, 'probability': 0.15}
Different features: None

Cache stats: {'size': 1, 'max_size': 1000, 'hits': 1, 'misses': 2, 'hit_rate': '33.3%'}
```

Integrating Cache with the API

```python
"""
cached_api.py - ML API with prediction caching.

This combines the FastAPI server with the prediction
cache for maximum performance.
"""

from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from contextlib import asynccontextmanager
import joblib
import numpy as np
import time

# Import our cache (from the previous example)
# In practice, this would be in a separate file
from prediction_cache import PredictionCache

model = None
cache = PredictionCache(max_size=10000, ttl_seconds=3600)


@asynccontextmanager
async def lifespan(app: FastAPI):
    global model
    model = joblib.load("models/trained_model.pkl")
    yield
    model = None


app = FastAPI(title="Cached ML API", lifespan=lifespan)


class PredictRequest(BaseModel):
    features: list[float]


@app.post("/predict")
def predict(request: PredictRequest):
    start = time.time()

    # Check cache first
    cached = cache.get(request.features)
```

```
    if cached is not None:
        cached["from_cache"] = True
        cached["latency_ms"] = round(
            (time.time() - start) * 1000, 2
        )
        return cached

    # Cache miss: compute prediction
    features = np.array([request.features])
    prediction = int(model.predict(features)[0])

    try:
        probability = float(
            model.predict_proba(features)[0][1]
        )
    except AttributeError:
        probability = float(prediction)

    result = {
        "prediction": prediction,
        "probability": round(probability, 4),
        "from_cache": False,
        "latency_ms": round((time.time() - start) * 1000, 2),
    }

    # Store in cache for next time
    cache.put(request.features, result)

    return result


@app.get("/cache/stats")
def cache_stats():
    """View cache performance statistics."""
    return cache.get_stats()
```

Common Mistakes

1. Using batch inference when real-time is needed. If users are waiting for a response, you need real-time serving, not batch.
2. Caching without expiration. Cached predictions become stale when you update your model. Always set a TTL (time to live).
3. Not considering memory when scaling. Each worker loads a copy of the model. Four workers with a 500MB model need 2GB of RAM just for models.
4. Over-engineering for low traffic. If you get 10 requests per day, a simple single-server setup is fine. Do not build for millions of users until you have them.

5. Ignoring cold start time. The first prediction after loading a model is often slower because of initialization. Warm up your model with a dummy prediction at startup.

Best Practices

1. Choose the right serving pattern for your use case. Batch for scheduled bulk processing, real-time for user-facing applications.
2. Cache predictions for repeated inputs. This can reduce latency from milliseconds to microseconds.
3. Monitor cache hit rates. A low hit rate means caching is not helping. A high hit rate means you are saving significant compute.
4. Warm up your model at startup. Make a dummy prediction during startup to avoid slow first requests.
5. Use health checks to verify model is loaded. The API should report "unhealthy" if the model fails to load.

Quick Summary

Model serving is about getting predictions from your model to users. Batch inference processes many items at once and is efficient for bulk operations. Real-time inference handles individual requests instantly. Caching stores repeated predictions for faster responses. Choose your pattern based on your latency requirements and traffic volume.

Key Points

- Batch inference processes data in bulk; real-time serves individual requests
- The "Model as a Service" pattern lets multiple apps share one model
- Each worker loads a separate copy of the model (plan memory accordingly)
- Caching can reduce latency from milliseconds to microseconds
- Always set a TTL on cached predictions
- Warm up models at startup to avoid slow first requests

Practice Questions

1. When would you choose batch inference over real-time inference? Give two specific examples.
2. What is an LRU cache and why is it useful for model serving?
3. If your model file is 200MB and you run 4 workers on 3 instances, how much total memory do you need just for the models?
4. What is the "Model as a Service" pattern and what are its advantages?
5. Why should you warm up your model at startup?

Exercises

Exercise 1: Batch Pipeline

Write a batch inference script that:

- Loads a model
- Reads a CSV file with 10,000 rows
- Processes it in batches of 500
- Logs progress and timing for each batch
- Saves results to a new CSV

Exercise 2: Cache Comparison

Implement two versions of an ML API:

- One without caching
- One with caching (TTL = 60 seconds)

Send 1000 requests (50% duplicates) to each and compare average latency.

Exercise 3: Serving Pattern Design

Design a model serving architecture for a ride-sharing app that needs:

- Real-time price prediction (must respond in under 50ms)

- Batch driver matching (run every 30 seconds)
- A/B testing between two pricing models

Draw the architecture and explain your choices.

What Is Next?

We have covered how to build, deploy, and serve ML models. But the journey does not end at deployment. In Chapter 8, we will zoom out and look at the complete ML lifecycle, from data collection to model monitoring and retraining. Understanding this full picture is essential for building ML systems that stay healthy over time.

Chapter 8: The ML Lifecycle

What You Will Learn

In this chapter, you will learn:

- What the ML lifecycle is and why it matters
- Each stage of the lifecycle: data collection, preparation, training, evaluation, deployment, monitoring, and retraining
- How these stages connect to form a continuous loop
- What happens at each stage and what can go wrong
- How to think about ML as an ongoing process, not a one-time project

Why This Chapter Matters

Imagine you open a restaurant. You do not just cook one meal and declare success. You constantly buy fresh ingredients, try new recipes, check customer feedback, update the menu, and train new staff. Running a restaurant is a cycle, not a one-time event.

ML works the same way. Training a model is just one step in a much larger process. Data changes. User behavior shifts. The world evolves. A model that was great six months ago might be making bad predictions today. Understanding the full ML lifecycle helps you build systems that stay healthy over time.

The Full ML Lifecycle

```
+-----------------------------------------------------------------+
|                    The ML Lifecycle                             |
```

```
|                                                                  |
|     +------------------+                                        |
|     | 1. Data          |                                        |
|     |    Collection    |                                        |
|     +--------+---------+                                        |
|              |                                                   |
|              v                                                   |
|     +--------+---------+                                        |
|     | 2. Data          |                                        |
|     |    Preparation   |                                        |
|     +--------+---------+                                        |
|              |                                                   |
|              v                                                   |
|     +--------+---------+                                        |
|     | 3. Model         |                                        |
|     |    Training      |                                        |
|     +--------+---------+                                        |
|              |                                                   |
|              v                                                   |
|     +--------+---------+                                        |
|     | 4. Model         |                                        |
|     |    Evaluation    |                                        |
|     +--------+---------+                                        |
|              |                                                   |
|              v                                                   |
|     +--------+---------+                                        |
|     | 5. Model         |                                        |
|     |    Deployment    |                                        |
|     +--------+---------+                                        |
|              |                                                   |
|              v                                                   |
|     +--------+---------+                                        |
|     | 6. Monitoring    |------> Problem detected?               |
|     |                  |              |                          |
|     +--------+---------+              | Yes                      |
|              |                        v                          |
|              |               +--------+---------+                |
|              |               | 7. Retraining    |                |
|              |               +--------+---------+                |
|              |                        |                          |
|              +<----- Back to Step 2 --+                          |
|                                                                  |
|  This is a LOOP, not a straight line!                            |
+------------------------------------------------------------------+
```

Stage 1: Data Collection

Data collection is gathering the raw information your model will learn from. It is like collecting ingredients before cooking.

```
+----------------------------------------------------+
|  Data Collection Sources                           |
|                                                    |
|  Databases        -> Customer records, orders     |
|  APIs             -> Weather data, stock prices   |
|  User Interactions-> Clicks, searches, ratings    |
|  Sensors          -> IoT devices, cameras         |
|  Web Scraping     -> Product prices, reviews      |
|  Manual Labeling  -> Annotated images, text       |
|  Public Datasets  -> Kaggle, government data      |
+----------------------------------------------------+
```

Key Questions

```python
"""
data_collection_checklist.py - Questions to ask about your data.

Before collecting data, answer these questions to
make sure you are collecting the right data.
"""

checklist = {
    "What problem are we solving?": {
        "example": "Predicting customer churn",
        "why": "Defines what data you need",
    },
    "What data do we already have?": {
        "example": "Customer purchase history, support tickets",
        "why": "Avoid collecting what you already have",
    },
    "What data do we still need?": {
        "example": "Customer satisfaction surveys, competitor pricing",
        "why": "Identify gaps in your data",
    },
    "How fresh does the data need to be?": {
        "example": "Last 12 months of transactions",
        "why": "Old data may not reflect current patterns",
    },
    "How much data do we need?": {
        "example": "At least 10,000 labeled examples",
        "why": "Too little data leads to poor models",
    },
    "Are there privacy or legal concerns?": {
        "example": "GDPR compliance for EU customers",
        "why": "Violations can result in fines and lawsuits",
    },
    "Is the data representative?": {
        "example": "Includes all customer segments, not just active ones",
        "why": "Biased data leads to biased models",
```

```
    },
}

print("Data Collection Checklist")
print("=" * 55)
for question, details in checklist.items():
    print(f"\n  Q: {question}")
    print(f"     Example: {details['example']}")
    print(f"     Why it matters: {details['why']}")
```

```
Output:
Data Collection Checklist
=======================================================

  Q: What problem are we solving?
     Example: Predicting customer churn
     Why it matters: Defines what data you need

  Q: What data do we already have?
     Example: Customer purchase history, support tickets
     Why it matters: Avoid collecting what you already have

  Q: What data do we still need?
     Example: Customer satisfaction surveys, competitor pricing
     Why it matters: Identify gaps in your data

  Q: How fresh does the data need to be?
     Example: Last 12 months of transactions
     Why it matters: Old data may not reflect current patterns

  Q: How much data do we need?
     Example: At least 10,000 labeled examples
     Why it matters: Too little data leads to poor models

  Q: Are there privacy or legal concerns?
     Example: GDPR compliance for EU customers
     Why it matters: Violations can result in fines and lawsuits

  Q: Is the data representative?
     Example: Includes all customer segments, not just active ones
     Why it matters: Biased data leads to biased models
```

Stage 2: Data Preparation

Raw data is like raw ingredients. You cannot just throw them in a pan. You need to wash, chop, and measure them first. Data preparation transforms raw data into a format your model can learn from.

```
+----------------------------------------------------+
|  Data Preparation Steps                            |
|                                                    |
|  Raw Data                                         |
|    |                                               |
|    +--> Clean (fix errors, handle missing values)|
|    |                                               |
|    +--> Transform (scale, encode, normalize)     |
|    |                                               |
|    +--> Feature Engineering (create new features)|
|    |                                               |
|    +--> Split (train / validation / test)        |
|    |                                               |
|    v                                               |
|  Prepared Data (ready for training)              |
+----------------------------------------------------+
```

```python
"""
data_preparation.py - The data preparation stage.

This shows the typical steps in data preparation
with explanations of what each step does and why.
"""

import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
import logging

logger = logging.getLogger(__name__)


def prepare_data(df, target_column):
    """
    Complete data preparation pipeline.

    Parameters
    ----------
    df : pd.DataFrame
        Raw data.
    target_column : str
        Name of the column we want to predict.

    Returns
    -------
    dict
        Prepared data and metadata.
    """
    logger.info(f"Starting data preparation: {len(df)} rows")
    steps_completed = []
```

```
    # Step 1: Handle missing values
    missing_before = df.isnull().sum().sum()
    if missing_before > 0:
        # Fill numeric columns with the median (middle value)
        numeric_cols = df.select_dtypes(include=[np.number]).columns
        for col in numeric_cols:
            median_val = df[col].median()
            df[col] = df[col].fillna(median_val)

        # Fill text columns with the most common value
        text_cols = df.select_dtypes(include=["object"]).columns
        for col in text_cols:
            mode_val = df[col].mode()[0]
            df[col] = df[col].fillna(mode_val)

        logger.info(f"Filled {missing_before} missing values")
        steps_completed.append("missing_values")

    # Step 2: Remove duplicates
    dupes = df.duplicated().sum()
    if dupes > 0:
        df = df.drop_duplicates()
        logger.info(f"Removed {dupes} duplicate rows")
        steps_completed.append("duplicates_removed")

    # Step 3: Encode categorical variables
    label_encoders = {}
    text_cols = df.select_dtypes(include=["object"]).columns
    text_cols = [c for c in text_cols if c != target_column]

    for col in text_cols:
        le = LabelEncoder()
        df[col] = le.fit_transform(df[col])
        label_encoders[col] = le
        logger.info(f"Encoded column: {col}")
    steps_completed.append("encoding")

    # Step 4: Split features and target
    X = df.drop(target_column, axis=1)
    y = df[target_column]

    # Step 5: Split into train/validation/test sets
    # First split: 80% train+val, 20% test
    X_trainval, X_test, y_trainval, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42
    )
    # Second split: 75% train, 25% validation (of the 80%)
    X_train, X_val, y_train, y_val = train_test_split(
        X_trainval, y_trainval, test_size=0.25, random_state=42
    )

    logger.info(f"Train: {len(X_train)}, Val: {len(X_val)}, Test: {len(X_test)}")
    steps_completed.append("split")
```

```
    # Step 6: Scale numeric features
    scaler = StandardScaler()
    X_train_scaled = pd.DataFrame(
        scaler.fit_transform(X_train),
        columns=X_train.columns,
        index=X_train.index,
    )
    X_val_scaled = pd.DataFrame(
        scaler.transform(X_val),
        columns=X_val.columns,
        index=X_val.index,
    )
    X_test_scaled = pd.DataFrame(
        scaler.transform(X_test),
        columns=X_test.columns,
        index=X_test.index,
    )
    steps_completed.append("scaling")

    logger.info(f"Preparation complete. Steps: {steps_completed}")

    return {
        "X_train": X_train_scaled,
        "X_val": X_val_scaled,
        "X_test": X_test_scaled,
        "y_train": y_train,
        "y_val": y_val,
        "y_test": y_test,
        "scaler": scaler,
        "label_encoders": label_encoders,
        "steps": steps_completed,
    }
```

Stage 3: Model Training

Training is where the model learns patterns from the data. It is like a student studying for an exam: they look at many examples and learn the rules.

```
+-----------------------------------------------+
|  Model Training                               |
|                                               |
|  Training Data -----> Algorithm -----> Model  |
|  (Examples)           (Learning)       (Knowledge)|
|                                               |
|  The algorithm looks at thousands of examples |
|  and figures out the patterns.                |
|                                               |
```

```
|  Input: age=25, income=30k -> churned=Yes        |
|  Input: age=45, income=90k -> churned=No         |
|  ...                                              |
|  (After seeing enough examples, the model learns)|
+--------------------------------------------------+
```

Training Considerations

```python
"""
training_stage.py - Model training considerations.

This shows the key decisions you make during training.
"""

training_decisions = {
    "Model Selection": {
        "question": "Which algorithm should I use?",
        "options": [
            "Logistic Regression - Simple, interpretable",
            "Random Forest - Good default, handles many data types",
            "Gradient Boosting (XGBoost) - Often best accuracy",
            "Neural Network - Best for images, text, complex patterns",
        ],
        "tip": "Start simple. Try logistic regression first. "
               "Only use complex models if simple ones are not good enough.",
    },
    "Hyperparameter Tuning": {
        "question": "How do I configure the model?",
        "options": [
            "Grid Search - Try all combinations (slow but thorough)",
            "Random Search - Try random combinations (faster)",
            "Bayesian Optimization - Smart search (efficient)",
        ],
        "tip": "Random search with 50-100 trials is a good starting point.",
    },
    "Training Data Size": {
        "question": "How much data should I use?",
        "options": [
            "More data usually = better model",
            "Learning curves show when more data stops helping",
            "Start with all available data, reduce only if too slow",
        ],
        "tip": "Plot a learning curve to see if more data would help.",
    },
}

for topic, info in training_decisions.items():
    print(f"\n{topic}")
    print(f"  Question: {info['question']}")
    print(f"  Options:")
```

```
    for opt in info["options"]:
        print(f"    - {opt}")
    print(f"  Tip: {info['tip']}")
```

```
Output:

Model Selection
  Question: Which algorithm should I use?
  Options:
    - Logistic Regression - Simple, interpretable
    - Random Forest - Good default, handles many data types
    - Gradient Boosting (XGBoost) - Often best accuracy
    - Neural Network - Best for images, text, complex patterns
  Tip: Start simple. Try logistic regression first. Only use complex models if simple o...

Hyperparameter Tuning
  Question: How do I configure the model?
  Options:
    - Grid Search - Try all combinations (slow but thorough)
    - Random Search - Try random combinations (faster)
    - Bayesian Optimization - Smart search (efficient)
  Tip: Random search with 50-100 trials is a good starting point.

Training Data Size
  Question: How much data should I use?
  Options:
    - More data usually = better model
    - Learning curves show when more data stops helping
    - Start with all available data, reduce only if too slow
  Tip: Plot a learning curve to see if more data would help.
```

Stage 4: Model Evaluation

Evaluation answers: "Is this model good enough?" You never trust a student's claim that they studied well. You give them an exam. Similarly, you test your model on data it has never seen before.

```
+-----------------------------------------------------+
|  Evaluation: Three Types of Data                    |
|                                                      |
|  Training Data  -> Model learns from this           |
|  (60%)            Like studying from a textbook     |
|                                                      |
|  Validation Data -> Tune hyperparameters            |
|  (20%)             Like practice exams               |
```

```
|                                                    |
|  Test Data      -> Final evaluation               |
|  (20%)            Like the real exam               |
|                     (ONLY look at this once!)      |
+----------------------------------------------------+
```

```python
"""
evaluation_stage.py - Model evaluation metrics and checks.
"""

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
)
import numpy as np


def comprehensive_evaluation(model, X_test, y_test):
    """
    Run a comprehensive model evaluation.

    Returns metrics and checks for common problems.
    """
    predictions = model.predict(X_test)

    # Basic metrics
    metrics = {
        "accuracy": accuracy_score(y_test, predictions),
        "precision": precision_score(y_test, predictions, average="weighted"),
        "recall": recall_score(y_test, predictions, average="weighted"),
        "f1": f1_score(y_test, predictions, average="weighted"),
    }

    # Confusion matrix
    cm = confusion_matrix(y_test, predictions)

    # Checks
    checks = []

    # Check 1: Is accuracy much better than random guessing?
    baseline_accuracy = max(
        np.mean(y_test == 0), np.mean(y_test == 1)
    )
    if metrics["accuracy"] > baseline_accuracy + 0.05:
        checks.append(("Better than baseline", "PASS"))
    else:
        checks.append(("Better than baseline", "FAIL"))

    # Check 2: Is the model not overfitting?
```

```
    train_accuracy = model.score(X_test, y_test)
    # In practice, compare with training accuracy too

    # Check 3: Are precision and recall balanced?
    if abs(metrics["precision"] - metrics["recall"]) < 0.1:
        checks.append(("Balanced precision/recall", "PASS"))
    else:
        checks.append(("Balanced precision/recall", "WARNING"))

    # Print results
    print("Model Evaluation Report")
    print("=" * 40)
    for name, value in metrics.items():
        print(f"  {name:>12}: {value:.4f}")

    print(f"\n  Confusion Matrix:")
    print(f"  {cm}")

    print(f"\n  Quality Checks:")
    for check_name, status in checks:
        print(f"  [{status:>7}] {check_name}")

    return metrics, checks
```

Stage 5: Model Deployment

Deployment is putting your model into production so real users or systems can use it. We covered this in detail in Chapters 3-5.

```
+-----------------------------------------------------+
|  Deployment Options                                  |
|                                                      |
|  REST API (FastAPI)  -> Chapter 3                   |
|  Docker Container    -> Chapter 4                   |
|  Cloud Service       -> Chapter 5                   |
|  Web Demo            -> Chapter 6                   |
|  Batch Job           -> Chapter 7                   |
|                                                      |
|  Key Questions:                                     |
|  - How fast must predictions be?                    |
|  - How many users will there be?                    |
|  - How often will the model be updated?             |
+-----------------------------------------------------+
```

Stage 6: Monitoring

Monitoring is watching your model after deployment. Even the best model can degrade over time. This is like checking the temperature of food in a restaurant throughout the day.

```
+-----------------------------------------------------+
|  What to Monitor                                     |
|                                                      |
|  1. Model Performance                               |
|     - Accuracy over time                            |
|     - Prediction distribution changes               |
|                                                      |
|  2. Data Quality                                    |
|     - Missing values in incoming data               |
|     - Feature distributions shifting                |
|     - New categories appearing                      |
|                                                      |
|  3. System Health                                   |
|     - Response time (latency)                       |
|     - Error rate                                    |
|     - Memory usage                                  |
|     - Request volume                                |
|                                                      |
|  4. Business Metrics                                |
|     - Are predictions leading to good outcomes?     |
|     - Customer satisfaction                         |
|     - Revenue impact                                |
+-----------------------------------------------------+
```

Data Drift Detection

Data drift happens when the incoming data changes from what the model was trained on. Like a weather model trained on summer data trying to predict winter weather.

```python
"""
drift_detection.py - Simple data drift monitoring.

Data drift means the incoming data looks different from
the training data. When this happens, the model's
predictions may become unreliable.
"""

import numpy as np
from scipy import stats


def check_drift(
```

```
    training_data,
    production_data,
    feature_names,
    threshold=0.05,
):
    """
    Check for data drift between training and production data.

    Uses the Kolmogorov-Smirnov test to compare distributions.
    This test answers: "Do these two sets of numbers come
    from the same distribution?"

    Parameters
    ----------
    training_data : np.ndarray
        Features from the training set.
    production_data : np.ndarray
        Features from recent production data.
    feature_names : list
        Names of the features.
    threshold : float
        P-value threshold. Below this means significant drift.

    Returns
    -------
    dict
        Drift detection results.
    """
    results = {}
    drift_detected = False

    print("Data Drift Report")
    print("=" * 55)

    for i, name in enumerate(feature_names):
        train_col = training_data[:, i]
        prod_col = production_data[:, i]

        # Kolmogorov-Smirnov test
        # Returns: statistic (how different) and p-value
        # Low p-value = distributions are different
        ks_stat, p_value = stats.ks_2samp(train_col, prod_col)

        drifted = p_value < threshold
        status = "DRIFT!" if drifted else "OK"

        if drifted:
            drift_detected = True

        results[name] = {
            "ks_statistic": round(ks_stat, 4),
            "p_value": round(p_value, 4),
            "drifted": drifted,
        }
```

```
        print(
            f"  {name:>20}: KS={ks_stat:.4f}, "
            f"p={p_value:.4f} [{status}]"
        )

    print("=" * 55)
    if drift_detected:
        print("  WARNING: Data drift detected! Consider retraining.")
    else:
        print("  All features within expected range.")

    return results, drift_detected


# Example: Detect drift
np.random.seed(42)

# Training data distribution
training = np.random.normal(
    loc=[30, 50000, 700, 5],       # means
    scale=[10, 20000, 50, 3],      # standard deviations
    size=(1000, 4),
)

# Production data with drift in income (mean shifted)
production = np.random.normal(
    loc=[30, 70000, 700, 5],       # income shifted from 50k to 70k
    scale=[10, 20000, 50, 3],
    size=(500, 4),
)

features = ["age", "income", "credit_score", "employment_years"]
results, has_drift = check_drift(training, production, features)
```

```
Output:
Data Drift Report
=======================================================
                  age: KS=0.0420, p=0.7234 [OK]
               income: KS=0.3856, p=0.0000 [DRIFT!]
         credit_score: KS=0.0380, p=0.8012 [OK]
     employment_years: KS=0.0410, p=0.7456 [OK]
=======================================================
  WARNING: Data drift detected! Consider retraining.
```

Stage 7: Retraining

When monitoring reveals problems (drift, declining accuracy, new patterns), it is time to retrain the model. This brings us back to Stage 2, creating the lifecycle loop.

```
+-----------------------------------------------------+
|  When to Retrain?                                    |
|                                                      |
|  Trigger               Action                       |
|  -------               ------                       |
|  Data drift detected   Retrain with new data        |
|  Accuracy drops        Investigate + retrain        |
|  New features added    Retrain to use them          |
|  Scheduled (monthly)   Regular refresh              |
|  Business changes      Update labels/features       |
+-----------------------------------------------------+
```

The Lifecycle in Practice

```python
"""
lifecycle_overview.py - Overview of the complete ML lifecycle.
"""

lifecycle_stages = [
    {
        "stage": "1. Data Collection",
        "duration": "Days to weeks",
        "key_activity": "Gather and store raw data",
        "output": "Raw dataset",
        "common_problem": "Not enough data or biased data",
    },
    {
        "stage": "2. Data Preparation",
        "duration": "Days to weeks",
        "key_activity": "Clean, transform, split data",
        "output": "Train/val/test datasets",
        "common_problem": "Data leakage between splits",
    },
    {
        "stage": "3. Model Training",
        "duration": "Hours to days",
        "key_activity": "Train and tune models",
        "output": "Trained model file",
        "common_problem": "Overfitting to training data",
    },
    {
        "stage": "4. Model Evaluation",
```

```
        "duration": "Hours",
        "key_activity": "Test model on held-out data",
        "output": "Performance metrics",
        "common_problem": "Using wrong metrics for the problem",
    },
    {
        "stage": "5. Model Deployment",
        "duration": "Hours to days",
        "key_activity": "Put model in production",
        "output": "Running API or service",
        "common_problem": "Works in dev, breaks in production",
    },
    {
        "stage": "6. Monitoring",
        "duration": "Ongoing",
        "key_activity": "Watch model performance",
        "output": "Alerts and dashboards",
        "common_problem": "Not monitoring at all",
    },
    {
        "stage": "7. Retraining",
        "duration": "Hours to days",
        "key_activity": "Update model with new data",
        "output": "New model version",
        "common_problem": "No automated pipeline",
    },
]

print("The ML Lifecycle")
print("=" * 65)
for stage in lifecycle_stages:
    print(f"\n{stage['stage']}")
    print(f"  Duration:        {stage['duration']}")
    print(f"  Key Activity:    {stage['key_activity']}")
    print(f"  Output:          {stage['output']}")
    print(f"  Common Problem:  {stage['common_problem']}")
```

```
Output:
The ML Lifecycle
=================================================================

1. Data Collection
  Duration:        Days to weeks
  Key Activity:    Gather and store raw data
  Output:          Raw dataset
  Common Problem:  Not enough data or biased data

2. Data Preparation
  Duration:        Days to weeks
  Key Activity:    Clean, transform, split data
  Output:          Train/val/test datasets
  Common Problem:  Data leakage between splits
```

```
3. Model Training
  Duration:        Hours to days
  Key Activity:    Train and tune models
  Output:          Trained model file
  Common Problem:  Overfitting to training data

4. Model Evaluation
  Duration:        Hours
  Key Activity:    Test model on held-out data
  Output:          Performance metrics
  Common Problem:  Using wrong metrics for the problem

5. Model Deployment
  Duration:        Hours to days
  Key Activity:    Put model in production
  Output:          Running API or service
  Common Problem:  Works in dev, breaks in production

6. Monitoring
  Duration:        Ongoing
  Key Activity:    Watch model performance
  Output:          Alerts and dashboards
  Common Problem:  Not monitoring at all

7. Retraining
  Duration:        Hours to days
  Key Activity:    Update model with new data
  Output:          New model version
  Common Problem:  No automated pipeline
```

Common Mistakes

1. Treating ML as a one-time project. ML is a continuous process. Models degrade over time and need regular attention.
2. Skipping monitoring. Without monitoring, you do not know when your model starts making bad predictions.
3. Data leakage. Using test data during training or preparation gives overly optimistic results that do not hold in production.
4. Ignoring data drift. The world changes. Data distributions shift. Models trained on old patterns make poor predictions on new patterns.
5. No retraining plan. Every model needs a plan for when and how it will be retrained.

Best Practices

1. Document every stage. Record what data was used, what decisions were made, and what results were achieved.
2. Automate the pipeline. Manual steps are error-prone and slow. Automate as much as possible.
3. Monitor from day one. Set up monitoring before deployment, not after problems appear.
4. Version everything. Version your data, code, models, and configurations.
5. Plan for failure. Have a rollback plan. If the new model is worse, you should be able to quickly revert to the previous version.

Quick Summary

The ML lifecycle is a continuous loop of seven stages: data collection, data preparation, model training, model evaluation, model deployment, monitoring, and retraining. Each stage has its own challenges and best practices. Understanding this full picture is essential for building ML systems that stay healthy and effective over time.

Key Points

- ML is a lifecycle, not a one-time project
- The seven stages form a continuous loop
- Data preparation typically takes the most time (60-80% of a project)
- Monitoring is essential to detect when a model starts degrading
- Data drift is a common reason models lose accuracy over time
- Every model needs a retraining strategy
- Version everything: data, code, models, and configs

Practice Questions

1. Why is the ML lifecycle a loop rather than a straight line?

2. What is data drift and how can you detect it?
3. Why is it important to keep training, validation, and test sets completely separate?
4. Which stage of the ML lifecycle typically takes the most time and effort?
5. What are three signals that tell you a model needs to be retrained?

Exercises

Exercise 1: Lifecycle Diagram

Draw your own ML lifecycle diagram for a specific use case (such as email spam detection). Include at least one detail specific to your use case at each stage.

Exercise 2: Drift Monitor

Implement a simple drift monitoring system that:

- Stores the distribution of training features
- Accepts new data batches
- Compares distributions and raises an alert if drift is detected
- Logs drift events with timestamps

Exercise 3: Retraining Decision

You have a model in production. Write a decision framework that answers:

- When should you retrain? (What triggers retraining?)
- How often should you retrain? (Schedule vs triggered)
- How do you validate the new model before deploying it?
- What do you do if the new model is worse than the current one?

What Is Next?

Now that you understand the full ML lifecycle, the next chapters dive deeper into the tools that support it. In Chapter 9, we will learn about experiment tracking with MLflow, which helps you organize and compare your training experiments. No more guessing which parameters gave

the best results!

Chapter 9: Experiment Tracking with MLflow

What You Will Learn

In this chapter, you will learn:

- Why tracking experiments is essential for ML projects
- How to install and set up MLflow
- How to log parameters, metrics, and artifacts
- How to compare different experiment runs
- How to use the MLflow UI to visualize results
- An overview of Weights & Biases as an alternative

Why This Chapter Matters

Imagine you are a scientist testing different medications. You try dozens of combinations: different doses, different ingredients, different schedules. After three weeks, you find a combination that works great. But you cannot remember exactly what it was. You scribbled notes on random papers, some of which you threw away.

This happens all the time in ML. You try different model types, different parameters, different data preprocessing steps. After many experiments, you find something that works, but you cannot reproduce it because you forgot to write down the details.

Experiment tracking solves this problem. It automatically records everything about every experiment so you can always go back and see exactly what you did.

The Problem Without Tracking

```
+-----------------------------------------------------+
|  Without Experiment Tracking                         |
|                                                      |
|  "Was it 100 trees or 200 trees?"                   |
|  "Did I use StandardScaler or MinMaxScaler?"        |
|  "Which version of the data gave 95% accuracy?"     |
|  "What was the learning rate for the best model?"|
|                                                      |
|  Common "solutions" that do not work:               |
|  - model_final.pkl                                  |
|  - model_final_v2.pkl                               |
|  - model_final_v2_ACTUALLY_FINAL.pkl                |
|  - model_best_DO_NOT_DELETE.pkl                     |
+-----------------------------------------------------+
```

```
+-----------------------------------------------------+
|  With Experiment Tracking                            |
|                                                      |
|  Run #1: RF, 100 trees, StandardScaler -> 0.89      |
|  Run #2: RF, 200 trees, StandardScaler -> 0.91      |
|  Run #3: XGBoost, lr=0.1, StandardScaler -> 0.93    |
|  Run #4: XGBoost, lr=0.05, MinMaxScaler -> 0.94     |
|                                                      |
|  Every run recorded with:                           |
|  - Parameters used                                  |
|  - Metrics achieved                                 |
|  - Model files saved                                |
|  - Code version                                     |
|  - Timestamp                                        |
+-----------------------------------------------------+
```

What Is MLflow?

MLflow is an open-source platform for managing the ML lifecycle. It has four main components:

```
+-----------------------------------------------------+
|  MLflow Components                                   |
|                                                      |
|  1. Tracking    -> Record experiments                |
|  2. Projects    -> Package ML code                   |
|  3. Models      -> Manage model versions             |
```

```
|  4. Registry    -> Store and serve models         |
|                                                   |
|  In this chapter, we focus on Tracking.          |
|  Chapter 11 covers the Model Registry.           |
+--------------------------------------------------+
```

Installing MLflow

```bash
pip install mlflow
```

Getting Started with MLflow Tracking

Your First Tracked Experiment

```python
"""
first_tracking.py - Your first MLflow experiment.

Run with: python first_tracking.py

After running, start the MLflow UI with:
    mlflow ui

Then visit http://localhost:5000 to see your results.
"""

import mlflow
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, f1_score

# Create sample data
X, y = make_classification(
    n_samples=1000,
    n_features=10,
    n_classes=2,
    random_state=42,
)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
# Set the experiment name
# All runs with this name are grouped together
mlflow.set_experiment("credit_risk_model")

# Start an MLflow run
# Everything inside this 'with' block is tracked
with mlflow.start_run(run_name="random_forest_baseline"):

    # Define parameters
    n_estimators = 100
    max_depth = 10
    random_state = 42

    # Log parameters
    # Parameters are the inputs to your experiment
    # (settings, hyperparameters, choices you made)
    mlflow.log_param("model_type", "RandomForest")
    mlflow.log_param("n_estimators", n_estimators)
    mlflow.log_param("max_depth", max_depth)
    mlflow.log_param("random_state", random_state)
    mlflow.log_param("test_size", 0.2)

    # Train model
    model = RandomForestClassifier(
        n_estimators=n_estimators,
        max_depth=max_depth,
        random_state=random_state,
    )
    model.fit(X_train, y_train)

    # Make predictions
    predictions = model.predict(X_test)

    # Calculate metrics
    accuracy = accuracy_score(y_test, predictions)
    f1 = f1_score(y_test, predictions, average="weighted")

    # Log metrics
    # Metrics are the outputs / results of your experiment
    mlflow.log_metric("accuracy", accuracy)
    mlflow.log_metric("f1_score", f1)
    mlflow.log_metric("train_samples", len(X_train))
    mlflow.log_metric("test_samples", len(X_test))

    # Log the model itself
    # This saves the model as an artifact
    mlflow.sklearn.log_model(model, "model")

    print(f"Run completed!")
    print(f"  Accuracy: {accuracy:.4f}")
    print(f"  F1 Score: {f1:.4f}")
    print(f"  Run ID: {mlflow.active_run().info.run_id}")
```

```
Output:
Run completed!
  Accuracy: 0.9350
  F1 Score: 0.9348
  Run ID: a1b2c3d4e5f6789012345678
```

Line-by-line explanation:

```python
mlflow.set_experiment("credit_risk_model")
```

This creates (or selects) an experiment. Think of an experiment as a folder where all related runs are stored. All runs for your credit risk model go in this folder.

```python
with mlflow.start_run(run_name="random_forest_baseline"):
```

This starts a new run within the experiment. Each run is one attempt (one set of parameters, one training session). The `run_name` is a human-readable label. The `with` block ensures the run is properly closed when done.

```python
mlflow.log_param("n_estimators", n_estimators)
```

This records a parameter. Parameters are the settings you chose for this run. They are searchable and comparable across runs.

```python
mlflow.log_metric("accuracy", accuracy)
```

This records a metric. Metrics are the results you measured. You can plot metrics over time and compare them across runs.

```python
mlflow.sklearn.log_model(model, "model")
```

This saves the actual model file as an artifact. Artifacts are any files associated with the run (models, plots, data samples).

Running Multiple Experiments

The real power of tracking comes from running many experiments and comparing them:

```python
"""
multiple_experiments.py - Run and compare multiple experiments.

This script tries different model configurations and
logs everything to MLflow for comparison.
"""

import mlflow
from sklearn.ensemble import (
    RandomForestClassifier,
    GradientBoostingClassifier,
)
from sklearn.linear_model import LogisticRegression
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, f1_score, precision_score, recall_score

# Create data
X, y = make_classification(
    n_samples=1000, n_features=10,
    n_classes=2, random_state=42,
)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42,
)

mlflow.set_experiment("model_comparison")

# Define experiments to run
experiments = [
    {
        "name": "logistic_regression",
        "model": LogisticRegression(max_iter=1000),
        "params": {"model_type": "LogisticRegression", "max_iter": 1000},
    },
    {
        "name": "random_forest_50",
        "model": RandomForestClassifier(n_estimators=50, random_state=42),
        "params": {"model_type": "RandomForest", "n_estimators": 50},
    },
    {
        "name": "random_forest_200",
        "model": RandomForestClassifier(n_estimators=200, random_state=42),
        "params": {"model_type": "RandomForest", "n_estimators": 200},
    },
    {
        "name": "gradient_boosting",
        "model": GradientBoostingClassifier(
            n_estimators=100, learning_rate=0.1, random_state=42
        ),
        "params": {
```

```
            "model_type": "GradientBoosting",
            "n_estimators": 100,
            "learning_rate": 0.1,
        },
    },
]

# Run each experiment
results = []

for exp in experiments:
    with mlflow.start_run(run_name=exp["name"]):
        # Log parameters
        for key, value in exp["params"].items():
            mlflow.log_param(key, value)

        # Train
        model = exp["model"]
        model.fit(X_train, y_train)

        # Evaluate
        predictions = model.predict(X_test)
        metrics = {
            "accuracy": accuracy_score(y_test, predictions),
            "f1_score": f1_score(y_test, predictions, average="weighted"),
            "precision": precision_score(y_test, predictions, average="weighted"),
            "recall": recall_score(y_test, predictions, average="weighted"),
        }

        # Log metrics
        for key, value in metrics.items():
            mlflow.log_metric(key, value)

        # Log model
        mlflow.sklearn.log_model(model, "model")

        results.append({
            "name": exp["name"],
            **metrics,
        })

        print(f"  {exp['name']}: accuracy={metrics['accuracy']:.4f}")

# Print comparison
print("\n" + "=" * 60)
print(f"{'Model':<25} {'Accuracy':>10} {'F1':>10} {'Precision':>10}")
print("-" * 60)
for r in sorted(results, key=lambda x: x["accuracy"], reverse=True):
    print(
        f"{r['name']:<25} {r['accuracy']:>10.4f} "
        f"{r['f1_score']:>10.4f} {r['precision']:>10.4f}"
    )
```

```
Output:
  logistic_regression: accuracy=0.9100
  random_forest_50: accuracy=0.9250
  random_forest_200: accuracy=0.9400
  gradient_boosting: accuracy=0.9450

============================================================
Model                        Accuracy         F1  Precision
------------------------------------------------------------
gradient_boosting              0.9450     0.9449     0.9455
random_forest_200              0.9400     0.9398     0.9405
random_forest_50               0.9250     0.9248     0.9260
logistic_regression            0.9100     0.9098     0.9110
```

Logging Artifacts

Artifacts are files associated with a run, like plots, data samples, or configuration files:

```python
"""
log_artifacts.py - Log files and plots to MLflow.

Artifacts are any files you want to associate with
a run: charts, config files, sample predictions, etc.
"""

import mlflow
import matplotlib
matplotlib.use("Agg")  # Use non-interactive backend
import matplotlib.pyplot as plt
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import json
import os

# Setup
X, y = make_classification(
    n_samples=1000, n_features=10,
    n_classes=2, random_state=42,
)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42,
)

mlflow.set_experiment("artifact_demo")
```

```
with mlflow.start_run(run_name="with_artifacts"):
    # Train model
    model = RandomForestClassifier(
        n_estimators=100, random_state=42
    )
    model.fit(X_train, y_train)
    predictions = model.predict(X_test)

    # Log basic metrics
    from sklearn.metrics import accuracy_score
    mlflow.log_metric("accuracy", accuracy_score(y_test, predictions))

    # Artifact 1: Confusion matrix plot
    cm = confusion_matrix(y_test, predictions)
    fig, ax = plt.subplots(figsize=(8, 6))
    disp = ConfusionMatrixDisplay(confusion_matrix=cm)
    disp.plot(ax=ax)
    ax.set_title("Confusion Matrix")

    # Save plot to file, then log as artifact
    plot_path = "confusion_matrix.png"
    fig.savefig(plot_path)
    plt.close()
    mlflow.log_artifact(plot_path)
    os.remove(plot_path)  # Clean up local file

    # Artifact 2: Feature importance plot
    importances = model.feature_importances_
    fig, ax = plt.subplots(figsize=(10, 6))
    feature_names = [f"feature_{i}" for i in range(len(importances))]
    ax.barh(feature_names, importances)
    ax.set_xlabel("Importance")
    ax.set_title("Feature Importances")

    plot_path = "feature_importance.png"
    fig.savefig(plot_path, bbox_inches="tight")
    plt.close()
    mlflow.log_artifact(plot_path)
    os.remove(plot_path)

    # Artifact 3: Model configuration JSON
    config = {
        "model_type": "RandomForestClassifier",
        "n_estimators": 100,
        "random_state": 42,
        "training_samples": len(X_train),
        "test_samples": len(X_test),
        "features": feature_names,
    }
    config_path = "model_config.json"
    with open(config_path, "w") as f:
        json.dump(config, f, indent=2)
    mlflow.log_artifact(config_path)
    os.remove(config_path)
```

```
    # Artifact 4: Sample predictions
    import pandas as pd
    sample_df = pd.DataFrame({
        "actual": y_test[:20],
        "predicted": predictions[:20],
        "correct": y_test[:20] == predictions[:20],
    })
    sample_path = "sample_predictions.csv"
    sample_df.to_csv(sample_path, index=False)
    mlflow.log_artifact(sample_path)
    os.remove(sample_path)

    print("All artifacts logged!")
    print("Run 'mlflow ui' and check the Artifacts tab")
```

```
Output:
All artifacts logged!
Run 'mlflow ui' and check the Artifacts tab
```

Using the MLflow UI

The MLflow UI is a web interface where you can browse and compare your experiments.

```bash
# Start the MLflow UI
mlflow ui
# Visit http://localhost:5000
```

```
+-----------------------------------------------------+
|  MLflow UI Features                                  |
|                                                      |
|  Experiments List                                   |
|  +-------------------------------------------+|
|  | credit_risk_model (4 runs)                  ||
|  | model_comparison  (4 runs)                  ||
|  | artifact_demo     (1 run)                   ||
|  +-------------------------------------------+|
|                                                      |
|  Run Details                                        |
|  +-------------------------------------------+|
|  | Parameters  | Metrics    | Artifacts          ||
|  | n_trees=100 | acc=0.935  | model/              ||
|  | depth=10    | f1=0.934   | confusion_matrix.png||
|  | seed=42     | prec=0.936 | feature_importance.png|
```

```
|  +-----------------------------------------------+|
|                                                    |
|  Compare Runs                                      |
|  +-----------------------------------------------+|
|  | Select multiple runs and compare side by side||
|  | See parameter vs metric charts               ||
|  | Identify the best configuration              ||
|  +-----------------------------------------------+|
+-----------------------------------------------------+
```

Querying Runs Programmatically

```python
"""
query_runs.py - Search and compare MLflow runs in code.

You do not always need the UI. You can query runs
programmatically to find the best model or compare results.
"""

import mlflow

# Set the experiment
mlflow.set_experiment("model_comparison")

# Get all runs from the experiment
experiment = mlflow.get_experiment_by_name("model_comparison")

if experiment:
    runs = mlflow.search_runs(
        experiment_ids=[experiment.experiment_id],
        order_by=["metrics.accuracy DESC"],  # Sort by accuracy
    )

    print("All runs (sorted by accuracy):")
    print("=" * 70)

    for _, run in runs.iterrows():
        print(
            f"  Run: {run['tags.mlflow.runName']:<25} "
            f"Accuracy: {run['metrics.accuracy']:.4f} "
            f"F1: {run['metrics.f1_score']:.4f}"
        )

    # Find the best run
    best_run = runs.iloc[0]
    print(f"\nBest run: {best_run['tags.mlflow.runName']}")
    print(f"  Accuracy: {best_run['metrics.accuracy']:.4f}")
    print(f"  Run ID: {best_run['run_id']}")

    # Load the best model
```

```
    best_model_uri = f"runs:/{best_run['run_id']}/model"
    print(f"  Model URI: {best_model_uri}")
    # best_model = mlflow.sklearn.load_model(best_model_uri)
```

```
Output:
All runs (sorted by accuracy):
======================================================================
  Run: gradient_boosting        Accuracy: 0.9450 F1: 0.9449
  Run: random_forest_200        Accuracy: 0.9400 F1: 0.9398
  Run: random_forest_50         Accuracy: 0.9250 F1: 0.9248
  Run: logistic_regression      Accuracy: 0.9100 F1: 0.9098

Best run: gradient_boosting
  Accuracy: 0.9450
  Run ID: a1b2c3d4e5f6789012345678
  Model URI: runs:/a1b2c3d4e5f6789012345678/model
```

Weights & Biases Overview

Weights & Biases (W&B or wandb) is a popular alternative to MLflow, especially for deep learning projects. It is a cloud-based service with a beautiful UI.

```
+----------------------------------------------------+
|  MLflow vs Weights & Biases                         |
|                                                     |
|  MLflow                    W&B                    |
|  - Open source             - Freemium service     |
|  - Self-hosted             - Cloud-hosted         |
|  - Works offline           - Needs internet       |
|  - General ML              - Deep learning focus  |
|  - Free forever            - Free for personal    |
|  - Setup required          - Quick to start       |
+----------------------------------------------------+
```

Quick W&B Example

```python
"""
wandb_example.py - Basic Weights & Biases usage.

Install: pip install wandb
```

```
Setup: wandb login (get API key from wandb.ai)

This shows the basic W&B API for comparison with MLflow.
"""

import wandb
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Create data
X, y = make_classification(
    n_samples=1000, n_features=10,
    n_classes=2, random_state=42,
)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42,
)

# Initialize W&B run
# This creates a new run on the W&B servers
run = wandb.init(
    project="credit-risk-model",
    name="random_forest_baseline",
    config={
        "model_type": "RandomForest",
        "n_estimators": 100,
        "max_depth": 10,
    },
)

# Train model
model = RandomForestClassifier(
    n_estimators=wandb.config.n_estimators,
    max_depth=wandb.config.max_depth,
    random_state=42,
)
model.fit(X_train, y_train)

# Evaluate and log metrics
accuracy = accuracy_score(y_test, model.predict(X_test))
wandb.log({
    "accuracy": accuracy,
    "train_samples": len(X_train),
    "test_samples": len(X_test),
})

print(f"Accuracy: {accuracy:.4f}")
print(f"View results at: {run.get_url()}")

# Finish the run
wandb.finish()
```

```
Output:
Accuracy: 0.9350
View results at: https://wandb.ai/your-username/credit-risk-model/runs/abc123
```

Common Mistakes

1. Not tracking experiments at all. Even simple experiments should be logged. You will thank yourself later.
2. Logging too little. Log everything: parameters, metrics, data versions, code versions, and artifacts.
3. Not using experiment names. Group related runs under the same experiment name for easy comparison.
4. Forgetting to log the model. Metrics tell you how good a model was, but without the saved model, you cannot use it.
5. Only looking at one metric. A model with 99% accuracy might have terrible recall for rare classes. Log multiple metrics.

Best Practices

1. Track every experiment from the start. Even during exploration. You never know which run will turn out to be the best.
2. Use descriptive run names. Names like "rf_200trees_scaled" are more useful than "run_47."
3. Log artifacts. Save confusion matrices, feature importance plots, and sample predictions alongside your metrics.
4. Compare runs systematically. Use the UI or programmatic queries to find the best model, not memory.
5. Set up experiment naming conventions. Consistent naming makes it easy to find and compare runs.

Quick Summary

Experiment tracking with MLflow eliminates the guesswork from ML development. You log parameters (what you tried), metrics (what happened), and artifacts (supporting files) for every experiment. The MLflow UI lets you compare runs visually. Weights & Biases is a cloud-based alternative. The key principle is: if you did not log it, it did not happen.

Key Points

- MLflow tracks parameters, metrics, and artifacts for each experiment run
- Use mlflow.set_experiment() to group related runs
- Use mlflow.start_run() to start tracking a new experiment
- Log parameters with mlflow.log_param() and metrics with mlflow.log_metric()
- Save models with mlflow.sklearn.log_model() (or the appropriate framework)
- The MLflow UI lets you compare runs visually
- Weights & Biases is a popular cloud-based alternative

Practice Questions

1. What is the difference between a parameter and a metric in MLflow?
2. Why should you use `with mlflow.start_run()` instead of manually starting and stopping runs?
3. What are artifacts in MLflow? Give three examples of useful artifacts to log.
4. How would you find the best model from 50 different runs using MLflow?
5. When would you choose Weights & Biases over MLflow?

Exercises

Exercise 1: Track a Hyperparameter Search

Use MLflow to track a grid search over these parameters:

- n_estimators: [50, 100, 200]
- max_depth: [5, 10, 20]

- min_samples_split: [2, 5, 10]

Log all 27 combinations and find the best one using the MLflow UI.

Exercise 2: Custom Artifacts

Create an MLflow run that logs:

- A confusion matrix as a PNG image
- A CSV file with the top 10 most important features
- A JSON file with the model configuration
- The trained model itself

Exercise 3: Experiment Comparison Script

Write a Python script that:

- Queries all runs from an experiment
- Creates a summary table with the top 5 runs by accuracy
- Generates a bar chart comparing metrics across runs
- Saves the chart as an artifact in a new summary run

What Is Next?

Experiment tracking tells you what happened during training. But what about the data itself? In Chapter 10, we will learn about data versioning with DVC. Just as git tracks code changes, DVC tracks data changes, ensuring you can always reproduce an experiment with the exact data that was used.

Chapter 10: Data Versioning with DVC

What You Will Learn

In this chapter, you will learn:

- Why versioning data is essential for reproducible ML
- How to install and set up DVC (Data Version Control)
- How to track datasets with DVC
- How to reproduce experiments using versioned data
- What .dvc files are and how they work
- How to use remote storage for large datasets

Why This Chapter Matters

Imagine you are writing a book. You use git to track every change to your manuscript. You can go back to any previous version, see what changed, and undo mistakes. Life is good.

But your book includes photographs. Each photo is 5 megabytes. You have 500 photos. That is 2.5 gigabytes. Git was designed for text files, not huge binary files. If you put the photos in git, your repository becomes enormous, slow, and painful.

ML has the same problem. Your code is small (kilobytes), but your datasets are huge (megabytes to gigabytes). DVC solves this by tracking data files separately from code, but keeping them linked so you can always match your code with the exact data it used.

The Data Versioning Problem

```
+----------------------------------------------------+
|  Without Data Versioning                           |
|                                                    |
|  "Which version of the data did model v1.2 use?"|
|  "Someone changed the training data. What was    |
|   it before?"                                      |
|  "The model worked yesterday. What changed?"     |
|  "I cannot reproduce last month's results."      |
|                                                    |
|  Common bad practices:                           |
|  - data_final.csv                                |
|  - data_final_v2.csv                             |
|  - data_DONT_TOUCH.csv                           |
|  - data_20240115_johns_version.csv               |
+----------------------------------------------------+
```

```
+----------------------------------------------------+
|  With Data Versioning (DVC)                        |
|                                                    |
|  git checkout v1.0   -> Code version 1.0         |
|  dvc checkout        -> Data version that matches|
|                                                    |
|  Everything is linked:                           |
|  Code v1.0 + Data v1.0 = Reproducible results   |
|  Code v2.0 + Data v2.0 = Reproducible results   |
+----------------------------------------------------+
```

What Is DVC?

DVC (Data Version Control) works alongside git. Git tracks your code. DVC tracks your data. They stay in sync so you always know which data goes with which code.

```
+----------------------------------------------------+
|  Git + DVC Working Together                        |
|                                                    |
|  Git tracks:          DVC tracks:                |
|  - Python files       - Large data files         |
|  - Config files       - Model files              |
|  - Notebooks          - Any large binary file    |
|  - .dvc files         (actual data stored        |
|    (pointers to data)  in remote storage)        |
|                                                    |
|  Think of .dvc files as bookmarks.               |
|  The bookmark (small) goes in git.               |
|  The book (large) goes in DVC storage.           |
```

```
+--------------------------------------------------+
```

Installing DVC

```bash
pip install dvc

# Optional: Install storage backends
pip install dvc[s3]     # For Amazon S3
pip install dvc[gs]     # For Google Cloud Storage
pip install dvc[azure]  # For Azure Blob Storage
```

Getting Started with DVC

Step 1: Initialize DVC

```bash
# First, make sure you are in a git repository
git init  # (if not already a git repo)

# Initialize DVC
dvc init
```

```
Output:
Initialized DVC repository.

You can now commit the changes to git.

+-----------------------------------------------------------------------+
|                                                                       |
|        DVC has been initialized. Start tracking data.                 |
|                                                                       |
+-----------------------------------------------------------------------+
```

DVC creates a `.dvc/` directory (similar to git's `.git/` directory):

```
my-project/
├── .git/            # Git's internal files
├── .dvc/            # DVC's internal files
│   ├── config       # DVC configuration
│   └── .gitignore   # Files DVC should ignore
```

```
├── .dvcignore       # Files DVC should not track
└── your_code.py
```

Step 2: Track a Dataset

```bash
# Add a data file to DVC tracking
dvc add data/training_data.csv
```

```
Output:
100% Add|██████████████████████████|1/1 [00:01, 1.23s/file]

To track the changes with git, run:

    git add data/training_data.csv.dvc data/.gitignore
```

This creates two things:

1. data/training_data.csv.dvc - A small pointer file that goes in git
2. data/.gitignore - Tells git to ignore the actual data file

Understanding .dvc Files

```python
"""
understand_dvc.py - Understanding .dvc files.

A .dvc file is a small text file that contains a hash
(fingerprint) of your data. It is like a receipt that
describes what the data looks like without containing
the actual data.
"""

# Here is what a .dvc file looks like:
dvc_file_content = """
outs:
- md5: a1b2c3d4e5f6789012345678abcdef90
  size: 15234567
  hash: md5
  path: training_data.csv
"""

# Let us break it down:
print("What a .dvc file contains:")
print("=" * 50)
print()
```

```
print("  md5: a1b2c3d4e5f6...")
print("    -> A unique fingerprint of the data file")
print("    -> If even one byte changes, the md5 changes")
print("    -> Like a social security number for your data")
print()
print("  size: 15234567")
print("    -> The file size in bytes (about 15 MB)")
print()
print("  path: training_data.csv")
print("    -> The name of the data file")
print()
print("The .dvc file is tiny (a few bytes).")
print("The actual data file can be gigabytes.")
print("Git stores the .dvc file.")
print("DVC stores the actual data.")
```

```
Output:
What a .dvc file contains:
==================================================

  md5: a1b2c3d4e5f6...
    -> A unique fingerprint of the data file
    -> If even one byte changes, the md5 changes
    -> Like a social security number for your data

  size: 15234567
    -> The file size in bytes (about 15 MB)

  path: training_data.csv
    -> The name of the data file

The .dvc file is tiny (a few bytes).
The actual data file can be gigabytes.
Git stores the .dvc file.
DVC stores the actual data.
```

Step 3: Commit to Git

```bash
# Add the .dvc file and .gitignore to git
git add data/training_data.csv.dvc data/.gitignore
git commit -m "Track training data with DVC"
```

The DVC Workflow

```
+-----------------------------------------------------+
|  DVC Workflow                                        |
|                                                      |
|  1. dvc add data/file.csv                           |
|     -> Creates file.csv.dvc (pointer)               |
|     -> Moves data to DVC cache (.dvc/cache)         |
|                                                      |
|  2. git add data/file.csv.dvc                       |
|     -> Commits the pointer to git                   |
|                                                      |
|  3. dvc push                                        |
|     -> Uploads data to remote storage               |
|     -> (Like git push, but for data)                |
|                                                      |
|  4. (On another machine)                            |
|     git clone the-repo                              |
|     dvc pull                                        |
|     -> Downloads the data from remote               |
|     -> (Like git pull, but for data)                |
+-----------------------------------------------------+
```

Tracking Data Changes

When your data changes, DVC tracks the new version:

```bash
# You update the data file
# (add new rows, fix errors, etc.)

# Re-add to DVC
dvc add data/training_data.csv

# The .dvc file now has a new hash
git add data/training_data.csv.dvc
git commit -m "Update training data: added 500 new samples"

# Push data to remote
dvc push
```

Going Back to Previous Data Versions

```bash
# See the history of data changes
git log data/training_data.csv.dvc

# Go back to a previous version
git checkout v1.0 -- data/training_data.csv.dvc
dvc checkout
```

```
# Now training_data.csv matches what it was in v1.0
```

Reproducing Experiments

DVC can also track entire pipelines, not just individual files:

```python
"""
dvc_pipeline.py - Example of what a DVC pipeline does.

A DVC pipeline defines the steps to reproduce your
experiment: prepare data, train model, evaluate.

You define this in a file called dvc.yaml
"""

# Here is what a dvc.yaml pipeline looks like:
dvc_yaml_content = """
# dvc.yaml - Pipeline definition

stages:
  prepare:
    cmd: python src/prepare_data.py
    deps:
      - src/prepare_data.py
      - data/raw/dataset.csv
    outs:
      - data/processed/train.csv
      - data/processed/test.csv

  train:
    cmd: python src/train.py
    deps:
      - src/train.py
      - data/processed/train.csv
    outs:
      - models/model.pkl
    params:
      - config.yaml:
          - model.n_estimators
          - model.max_depth

  evaluate:
    cmd: python src/evaluate.py
    deps:
      - src/evaluate.py
```

```
      - models/model.pkl
      - data/processed/test.csv
    metrics:
      - metrics.json:
          cache: false
"""

# Let us understand each part:
print("DVC Pipeline Stages")
print("=" * 55)
print()
print("Stage: prepare")
print("  cmd:  python src/prepare_data.py")
print("  deps: Files this stage needs (inputs)")
print("  outs: Files this stage creates (outputs)")
print()
print("Stage: train")
print("  cmd:    python src/train.py")
print("  deps:   Code + prepared data")
print("  outs:   Trained model file")
print("  params: Config values used (tracked for changes)")
print()
print("Stage: evaluate")
print("  cmd:     python src/evaluate.py")
print("  deps:    Code + model + test data")
print("  metrics: Results file (not cached, always readable)")
```

```
Output:
DVC Pipeline Stages
=======================================================

Stage: prepare
  cmd:  python src/prepare_data.py
  deps: Files this stage needs (inputs)
  outs: Files this stage creates (outputs)

Stage: train
  cmd:    python src/train.py
  deps:   Code + prepared data
  outs:   Trained model file
  params: Config values used (tracked for changes)

Stage: evaluate
  cmd:     python src/evaluate.py
  deps:    Code + model + test data
  metrics: Results file (not cached, always readable)
```

Running the Pipeline

```bash
# Run the entire pipeline
dvc repro

# DVC automatically figures out which stages need to rerun
# If only the model parameters changed, it skips data preparation
```

```
Output:
Running stage 'prepare':
> python src/prepare_data.py
Generating lock file 'dvc.lock'

Running stage 'train':
> python src/train.py

Running stage 'evaluate':
> python src/evaluate.py

Use `dvc push` to send your updates to remote storage.
```

Smart Re-execution

```
+-----------------------------------------------------+
|  DVC Smart Re-execution                              |
|                                                      |
|  You change model parameters in config.yaml         |
|                                                      |
|  dvc repro checks:                                  |
|  - prepare: deps unchanged -> SKIP (cached)         |
|  - train: params changed   -> RERUN                 |
|  - evaluate: model changed -> RERUN                 |
|                                                      |
|  Only reruns what is necessary!                     |
|  Saves time when data preparation is slow.          |
+-----------------------------------------------------+
```

Remote Storage

DVC remote storage is where your actual data files are stored. It is like GitHub for your data.

```
+-----------------------------------------------------+
|  DVC Remote Storage Options                          |
|                                                      |
```

```
|  Amazon S3        -> dvc remote add -d myremote  |
|                      s3://my-bucket/dvc-storage  |
|                                                   |
|  Google Cloud     -> dvc remote add -d myremote  |
|                      gs://my-bucket/dvc-storage  |
|                                                   |
|  Azure Blob       -> dvc remote add -d myremote  |
|                      azure://my-container/dvc    |
|                                                   |
|  Local folder     -> dvc remote add -d myremote  |
|  (for testing)       /path/to/storage            |
|                                                   |
|  SSH server       -> dvc remote add -d myremote  |
|                      ssh://user@server/path      |
+---------------------------------------------------+
```

Setting Up Remote Storage

```bash
# Add a remote (using local storage for this example)
dvc remote add -d myremote /tmp/dvc-storage

# Push data to remote
dvc push

# Pull data from remote (on another machine)
dvc pull
```

```
Output (dvc push):
2 files pushed

Output (dvc pull):
A       data/training_data.csv
A       models/model.pkl
2 files fetched
```

How Remote Storage Works

```
+---------------------------------------------------+
|  DVC Remote Storage Flow                          |
|                                                   |
|  Your Machine:                                   |
|  +-------------------+   +-------------------+   |
|  | .dvc/cache/       |   | data/             |   |
|  | (local cache of   |   | training_data.csv |   |
|  |  all versions)    |   | (current version) |   |
|  +-------------------+   +-------------------+   |
```

```
|           |                                        |
|     dvc push / dvc pull                           |
|           |                                        |
|           v                                        |
|  Remote Storage (S3, GCS, etc.):                  |
|  +-----------------------------------------+      |
|  | All versions of all data files             |   |
|  | Shared across team members                 |   |
|  | Backed up and durable                      |   |
|  +-----------------------------------------+      |
+---------------------------------------------------+
```

Complete DVC Workflow Example

```python
"""
complete_dvc_workflow.py - A complete DVC workflow example.

This shows the commands and Python code involved in
a typical DVC workflow.
"""

# The commands below are bash commands shown as strings
# for educational purposes.

workflow_steps = [
    {
        "step": "1. Initialize",
        "commands": [
            "git init",
            "dvc init",
            "git add .",
            'git commit -m "Initialize project with DVC"',
        ],
    },
    {
        "step": "2. Add data",
        "commands": [
            "dvc add data/training_data.csv",
            "git add data/training_data.csv.dvc data/.gitignore",
            'git commit -m "Add training data v1"',
        ],
    },
    {
        "step": "3. Set up remote",
        "commands": [
            "dvc remote add -d storage s3://my-bucket/dvc",
```

```
            "git add .dvc/config",
            'git commit -m "Configure DVC remote storage"',
            "dvc push",
        ],
    },
    {
        "step": "4. Train and track",
        "commands": [
            "python src/train.py",
            "dvc add models/model.pkl",
            "git add models/model.pkl.dvc",
            'git commit -m "Add trained model v1"',
            "git tag v1.0",
            "dvc push",
        ],
    },
    {
        "step": "5. Update data and retrain",
        "commands": [
            "# ... update training_data.csv ...",
            "dvc add data/training_data.csv",
            "python src/train.py",
            "dvc add models/model.pkl",
            "git add data/training_data.csv.dvc models/model.pkl.dvc",
            'git commit -m "Update data and retrain model v2"',
            "git tag v2.0",
            "dvc push",
        ],
    },
    {
        "step": "6. Reproduce old version",
        "commands": [
            "git checkout v1.0",
            "dvc checkout",
            "# Now data and model match v1.0!",
            "python src/evaluate.py",
        ],
    },
]

for step_info in workflow_steps:
    print(f"\n{step_info['step']}")
    print("-" * 40)
    for cmd in step_info["commands"]:
        print(f"  $ {cmd}")
```

```
Output:

1. Initialize
----------------------------------------
  $ git init
  $ dvc init
  $ git add .
```

```
  $ git commit -m "Initialize project with DVC"

2. Add data
----------------------------------------
  $ dvc add data/training_data.csv
  $ git add data/training_data.csv.dvc data/.gitignore
  $ git commit -m "Add training data v1"

3. Set up remote
----------------------------------------
  $ dvc remote add -d storage s3://my-bucket/dvc
  $ git add .dvc/config
  $ git commit -m "Configure DVC remote storage"
  $ dvc push

4. Train and track
----------------------------------------
  $ python src/train.py
  $ dvc add models/model.pkl
  $ git add models/model.pkl.dvc
  $ git commit -m "Add trained model v1"
  $ git tag v1.0
  $ dvc push

5. Update data and retrain
----------------------------------------
  $ # ... update training_data.csv ...
  $ dvc add data/training_data.csv
  $ python src/train.py
  $ dvc add models/model.pkl
  $ git add data/training_data.csv.dvc models/model.pkl.dvc
  $ git commit -m "Update data and retrain model v2"
  $ git tag v2.0
  $ dvc push

6. Reproduce old version
----------------------------------------
  $ git checkout v1.0
  $ dvc checkout
  $ # Now data and model match v1.0!
  $ python src/evaluate.py
```

Common Mistakes

1. Adding large data files to git. Git is for code. DVC is for data. Never put large files in git.
2. Forgetting to dvc push. If you do not push, your data only exists on your machine. Teammates cannot access it.

3. Not committing .dvc files. The .dvc pointer files must be committed to git. Without them, DVC cannot find the data.
4. Editing data without dvc add. If you change a tracked file, you must run `dvc add` again to update the pointer.
5. Not tagging versions. Use git tags (v1.0, v2.0) to mark important versions so you can easily go back to them.

Best Practices

1. Always pair git commit with dvc push. Keep code and data in sync.
2. Use meaningful commit messages. Describe what changed in the data, not just "update data."
3. Tag important versions. Use git tags for versions you might want to reproduce later.
4. Set up remote storage early. Do not wait until you have a problem to configure remote storage.
5. Use DVC pipelines for reproducibility. Define your entire workflow in dvc.yaml so anyone can reproduce it.

Quick Summary

DVC (Data Version Control) extends git to handle large data files. It creates small pointer files (.dvc files) that git tracks, while the actual data is stored in a separate cache and remote storage. This lets you version data alongside code, reproduce experiments with exact data, and share datasets with your team.

Key Points

- DVC tracks large data files that do not belong in git
- .dvc files are small pointers that go in git; actual data goes in DVC storage
- dvc add tracks a file; dvc push uploads it to remote storage
- dvc pull downloads data from remote storage

- git checkout + dvc checkout restores data for any version
- DVC pipelines (dvc.yaml) define reproducible workflows
- DVC supports S3, GCS, Azure, SSH, and local storage

Practice Questions

1. Why is git not suitable for tracking large data files?
2. What information does a .dvc file contain?
3. Explain the difference between `dvc push` and `git push`.
4. How would you go back to a previous version of your training data?
5. What is the purpose of a DVC pipeline defined in dvc.yaml?

Exercises

Exercise 1: Basic DVC Setup

Initialize a DVC project, track a CSV file, and set up a local remote. Verify you can push and pull the data.

Exercise 2: Data Version History

Create a dataset, track it with DVC, then make three updates. Tag each version. Practice switching between versions and verify the data matches.

Exercise 3: Reproducible Pipeline

Create a DVC pipeline (dvc.yaml) with three stages:

1. Data preparation (clean raw data)
2. Model training (train on cleaned data)
3. Evaluation (generate metrics)

Run `dvc repro` and verify that changing only the model parameters skips the data preparation stage.

What Is Next?

You can now version your data alongside your code. But where do your trained models go? In Chapter 11, we will learn about the MLflow Model Registry, a centralized place to manage model versions, track their status (staging, production, archived), and promote models through a controlled workflow.

Chapter 11: The Model Registry

What You Will Learn

In this chapter, you will learn:

- What a model registry is and why you need one
- How to use the MLflow Model Registry
- How to manage model stages: staging, production, and archived
- How to implement a model promotion workflow
- How to load production models from the registry

Why This Chapter Matters

Imagine a pharmaceutical company. They do not just create medicines and give them directly to patients. Each medicine goes through a rigorous process: lab testing, clinical trials, regulatory approval, and finally production. At each stage, the medicine is carefully tracked.

ML models need a similar process. You do not want someone to accidentally deploy an untested model to production. A model registry is the system that tracks every model version, which stage it is in, who approved it, and when it was promoted. It is the quality control system for your models.

What Is a Model Registry?

A model registry is a centralized store for managing ML models throughout their lifecycle. Think of it as a library catalog for your models.

```
+-----------------------------------------------------+
|  Model Registry                                     |
|                                                      |
|  +-----------+     +-----------+     +-----------+|
|  |  Staging  | --> |Production | --> | Archived  ||
|  |           |     |           |     |           ||
|  | Testing   |     | Live!     |     | Retired   ||
|  | Validation|     | Serving   |     | Kept for  ||
|  | Review    |     | users     |     | reference ||
|  +-----------+     +-----------+     +-----------+|
|                                                      |
|  Each model version has:                            |
|  - A unique version number                          |
|  - The person who registered it                     |
|  - When it was created                              |
|  - Its current stage                                |
|  - Description and tags                             |
+-----------------------------------------------------+
```

Without vs With a Model Registry

```
+-----------------------------------------------------+
|  Without Registry           With Registry          |
|                                                      |
|  model_v1.pkl              CreditRisk Model        |
|  model_v2.pkl              Version 1: Archived     |
|  model_v2_fixed.pkl        Version 2: Archived     |
|  model_FINAL.pkl           Version 3: Staging      |
|  model_FINAL_v2.pkl        Version 4: Production   |
|  WHICH ONE IS IN PROD??                             |
|                              Clear and organized!  |
+-----------------------------------------------------+
```

MLflow Model Registry

MLflow includes a built-in model registry. Let us see how to use it.

Registering a Model

```python
"""
register_model.py - Register models in MLflow Model Registry.
```

```
The registry provides a central place to manage model
versions and their lifecycle stages.
"""

import mlflow
from mlflow.tracking import MlflowClient
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Create and train a model
X, y = make_classification(
    n_samples=1000, n_features=10,
    n_classes=2, random_state=42,
)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42,
)

mlflow.set_experiment("model_registry_demo")

# Train and register the model
with mlflow.start_run(run_name="rf_v1") as run:
    # Train
    model = RandomForestClassifier(
        n_estimators=100, random_state=42
    )
    model.fit(X_train, y_train)

    # Log metrics
    accuracy = accuracy_score(y_test, model.predict(X_test))
    mlflow.log_metric("accuracy", accuracy)
    mlflow.log_param("n_estimators", 100)

    # Log and register the model
    # registered_model_name creates an entry in the registry
    mlflow.sklearn.log_model(
        model,
        "model",
        registered_model_name="CreditRiskModel",
    )

    print(f"Model registered as 'CreditRiskModel'")
    print(f"Accuracy: {accuracy:.4f}")
    print(f"Run ID: {run.info.run_id}")
```

```
Output:
Model registered as 'CreditRiskModel'
Accuracy: 0.9350
Run ID: a1b2c3d4567890
Successfully registered model 'CreditRiskModel'.
```

```
Created version '1' of model 'CreditRiskModel'.
```

Registering Multiple Versions

```python
"""
register_multiple.py - Register multiple model versions.

Each time you register a model with the same name,
MLflow creates a new version automatically.
"""

import mlflow
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Create data
X, y = make_classification(
    n_samples=1000, n_features=10,
    n_classes=2, random_state=42,
)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42,
)

mlflow.set_experiment("model_registry_demo")

# Version 1: Random Forest with 50 trees
with mlflow.start_run(run_name="rf_50_trees"):
    model = RandomForestClassifier(n_estimators=50, random_state=42)
    model.fit(X_train, y_train)
    accuracy = accuracy_score(y_test, model.predict(X_test))
    mlflow.log_metric("accuracy", accuracy)
    mlflow.log_param("model_type", "RandomForest")
    mlflow.log_param("n_estimators", 50)

    mlflow.sklearn.log_model(
        model, "model",
        registered_model_name="CreditRiskModel",
    )
    print(f"Version 1 - RF(50): accuracy={accuracy:.4f}")

# Version 2: Random Forest with 200 trees
with mlflow.start_run(run_name="rf_200_trees"):
    model = RandomForestClassifier(n_estimators=200, random_state=42)
    model.fit(X_train, y_train)
    accuracy = accuracy_score(y_test, model.predict(X_test))
    mlflow.log_metric("accuracy", accuracy)
    mlflow.log_param("model_type", "RandomForest")
```

```
    mlflow.log_param("n_estimators", 200)

    mlflow.sklearn.log_model(
        model, "model",
        registered_model_name="CreditRiskModel",
    )
    print(f"Version 2 - RF(200): accuracy={accuracy:.4f}")

# Version 3: Gradient Boosting
with mlflow.start_run(run_name="gradient_boosting"):
    model = GradientBoostingClassifier(
        n_estimators=100, learning_rate=0.1, random_state=42
    )
    model.fit(X_train, y_train)
    accuracy = accuracy_score(y_test, model.predict(X_test))
    mlflow.log_metric("accuracy", accuracy)
    mlflow.log_param("model_type", "GradientBoosting")
    mlflow.log_param("n_estimators", 100)

    mlflow.sklearn.log_model(
        model, "model",
        registered_model_name="CreditRiskModel",
    )
    print(f"Version 3 - GB: accuracy={accuracy:.4f}")
```

```
Output:
Version 1 - RF(50): accuracy=0.9250
Created version '1' of model 'CreditRiskModel'.
Version 2 - RF(200): accuracy=0.9400
Created version '2' of model 'CreditRiskModel'.
Version 3 - GB: accuracy=0.9450
Created version '3' of model 'CreditRiskModel'.
```

Model Stages

The MLflow Model Registry supports lifecycle stages that represent where a model is in the promotion process:

```
+-----------------------------------------------------+
|  Model Lifecycle Stages                             |
|                                                     |
|  None -------> Staging -------> Production          |
|  (just           |                 |                |
|   registered)    |                 |                |
|                  |                 |                |
```

```
|                  +----> Archived <-+              |
|                                                   |
|  None:       Just registered, not evaluated yet  |
|  Staging:    Being tested and validated          |
|  Production: Live, serving real users            |
|  Archived:   Retired, kept for reference         |
+---------------------------------------------------+
```

Managing Model Stages

```python
"""
manage_stages.py - Promote models through lifecycle stages.

This shows how to move models between stages using
the MLflow client API.
"""

from mlflow.tracking import MlflowClient

# Create a client to interact with the registry
client = MlflowClient()

model_name = "CreditRiskModel"

# List all versions of the model
print(f"All versions of '{model_name}':")
print("=" * 60)

for version in client.search_model_versions(f"name='{model_name}'"):
    print(
        f"  Version {version.version}: "
        f"Stage={version.current_stage}, "
        f"Status={version.status}"
    )

# Promote version 3 to Staging
# (In MLflow 2.x, use set_registered_model_alias instead)
client.transition_model_version_stage(
    name=model_name,
    version="3",
    stage="Staging",
)
print(f"\nVersion 3 promoted to Staging")

# After testing in staging, promote to Production
client.transition_model_version_stage(
    name=model_name,
    version="3",
    stage="Production",
)
```

```
print(f"Version 3 promoted to Production")

# Archive the old production model
client.transition_model_version_stage(
    name=model_name,
    version="1",
    stage="Archived",
)
print(f"Version 1 archived")

# Add descriptions for documentation
client.update_model_version(
    name=model_name,
    version="3",
    description="GradientBoosting model with 94.5% accuracy. "
                "Promoted to production on 2024-01-15.",
)

# List updated stages
print(f"\nUpdated stages:")
print("=" * 60)
for version in client.search_model_versions(f"name='{model_name}'"):
    print(
        f"  Version {version.version}: "
        f"Stage={version.current_stage}"
    )
```

```
Output:
All versions of 'CreditRiskModel':
============================================================
  Version 1: Stage=None, Status=READY
  Version 2: Stage=None, Status=READY
  Version 3: Stage=None, Status=READY

Version 3 promoted to Staging
Version 3 promoted to Production
Version 1 archived

Updated stages:
============================================================
  Version 1: Stage=Archived
  Version 2: Stage=None
  Version 3: Stage=Production
```

Loading Models from the Registry

The beauty of the registry is that your serving code does not need to know specific file paths. It just asks for "the production model."

```python
"""
load_from_registry.py - Load models by stage from the registry.

Your API code can load "the production model" without
knowing which specific version or file path to use.
When you promote a new version, the API automatically
picks it up.
"""

import mlflow

model_name = "CreditRiskModel"

# Load the production model
# This always loads whatever version is currently in Production
production_model = mlflow.pyfunc.load_model(
    f"models:/{model_name}/Production"
)
print("Production model loaded!")

# Load the staging model (for testing)
try:
    staging_model = mlflow.pyfunc.load_model(
        f"models:/{model_name}/Staging"
    )
    print("Staging model loaded!")
except Exception:
    print("No model in Staging stage")

# Load a specific version
specific_model = mlflow.pyfunc.load_model(
    f"models:/{model_name}/2"
)
print("Version 2 loaded!")

# Use the model for predictions
import numpy as np
sample_input = np.array([[30, 50000, 720, 5, 1, 0, 1, 0, 1, 0]])

prediction = production_model.predict(sample_input)
print(f"\nProduction model prediction: {prediction}")
```

```
Output:
Production model loaded!
No model in Staging stage
Version 2 loaded!

Production model prediction: [0]
```

Model Promotion Workflow

Here is a complete workflow for promoting models safely:

```python
"""
promotion_workflow.py - A complete model promotion workflow.

This workflow ensures models are properly validated
before reaching production.
"""

from mlflow.tracking import MlflowClient
import mlflow
import numpy as np
from sklearn.metrics import accuracy_score


def promote_model(
    model_name,
    version,
    test_data,
    test_labels,
    min_accuracy=0.90,
):
    """
    Safely promote a model through stages.

    Steps:
    1. Load the candidate model
    2. Run validation tests
    3. Compare with current production model
    4. If better, promote to staging then production
    5. Archive the old production model

    Parameters
    ----------
    model_name : str
        Name of the registered model.
    version : str
        Version number to promote.
    test_data : np.ndarray
        Test features for validation.
    test_labels : np.ndarray
        True labels for validation.
    min_accuracy : float
        Minimum accuracy required for production.

    Returns
    -------
```

```
    bool
        True if promotion was successful.
    """
    client = MlflowClient()

    print(f"Model Promotion Workflow")
    print(f"Model: {model_name}, Version: {version}")
    print("=" * 50)

    # Step 1: Load the candidate model
    print("\n[Step 1] Loading candidate model...")
    candidate = mlflow.pyfunc.load_model(
        f"models:/{model_name}/{version}"
    )

    # Step 2: Validate the candidate
    print("[Step 2] Validating candidate...")
    candidate_preds = candidate.predict(test_data)
    candidate_accuracy = accuracy_score(test_labels, candidate_preds)
    print(f"  Candidate accuracy: {candidate_accuracy:.4f}")

    # Check minimum accuracy
    if candidate_accuracy < min_accuracy:
        print(f"  FAIL: Below minimum accuracy ({min_accuracy})")
        return False
    print(f"  PASS: Above minimum accuracy ({min_accuracy})")

    # Step 3: Compare with current production model
    print("[Step 3] Comparing with production model...")
    try:
        production = mlflow.pyfunc.load_model(
            f"models:/{model_name}/Production"
        )
        prod_preds = production.predict(test_data)
        prod_accuracy = accuracy_score(test_labels, prod_preds)
        print(f"  Production accuracy: {prod_accuracy:.4f}")

        if candidate_accuracy <= prod_accuracy:
            print(f"  FAIL: Not better than production model")
            return False
        print(f"  PASS: Better than production model!")

        improvement = candidate_accuracy - prod_accuracy
        print(f"  Improvement: +{improvement:.4f}")

        # Find current production version to archive later
        prod_versions = client.search_model_versions(
            f"name='{model_name}'"
        )
        old_prod_version = None
        for v in prod_versions:
            if v.current_stage == "Production":
                old_prod_version = v.version
                break
```

```
    except Exception:
        print("  No current production model found")
        old_prod_version = None
        prod_accuracy = 0

    # Step 4: Promote to staging
    print("[Step 4] Promoting to staging...")
    client.transition_model_version_stage(
        name=model_name,
        version=version,
        stage="Staging",
    )
    print(f"  Version {version} moved to Staging")

    # Step 5: Promote to production
    print("[Step 5] Promoting to production...")
    client.transition_model_version_stage(
        name=model_name,
        version=version,
        stage="Production",
    )
    print(f"  Version {version} moved to Production")

    # Step 6: Archive old production model
    if old_prod_version:
        print(f"[Step 6] Archiving old version {old_prod_version}...")
        client.transition_model_version_stage(
            name=model_name,
            version=old_prod_version,
            stage="Archived",
        )
        print(f"  Version {old_prod_version} archived")

    # Step 7: Add description
    client.update_model_version(
        name=model_name,
        version=version,
        description=(
            f"Promoted to production. "
            f"Accuracy: {candidate_accuracy:.4f}. "
            f"Improvement over previous: "
            f"+{candidate_accuracy - prod_accuracy:.4f}"
        ),
    )

    print("\n" + "=" * 50)
    print("Promotion SUCCESSFUL!")
    print(f"  New production model: Version {version}")
    print(f"  Accuracy: {candidate_accuracy:.4f}")

    return True


# Example usage
from sklearn.datasets import make_classification
```

```
from sklearn.model_selection import train_test_split

X, y = make_classification(
    n_samples=1000, n_features=10,
    n_classes=2, random_state=42,
)
_, X_test, _, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42,
)

success = promote_model(
    model_name="CreditRiskModel",
    version="3",
    test_data=X_test,
    test_labels=y_test,
    min_accuracy=0.90,
)
```

```
Output:
Model Promotion Workflow
Model: CreditRiskModel, Version: 3
==================================================

[Step 1] Loading candidate model...
[Step 2] Validating candidate...
  Candidate accuracy: 0.9450
  PASS: Above minimum accuracy (0.9)
[Step 3] Comparing with production model...
  No current production model found
[Step 4] Promoting to staging...
  Version 3 moved to Staging
[Step 5] Promoting to production...
  Version 3 moved to Production

==================================================
Promotion SUCCESSFUL!
  New production model: Version 3
  Accuracy: 0.9450
```

Common Mistakes

1. Deploying models without validation. Always test a model in staging before promoting to production.
2. Not archiving old models. Keep old versions archived for reference. You might need to roll back.

3. No rollback plan. If the new production model has issues, you need to quickly revert to the previous version.
4. Skipping the staging stage. Going directly from development to production is risky. Always test in staging first.
5. Not documenting model versions. Add descriptions to each version explaining what changed and why it was promoted.

Best Practices

1. Follow a strict promotion workflow. None -> Staging -> Production. Never skip stages.
2. Automate validation. Run automated tests before any promotion.
3. Compare before promoting. The new model should be better than the current production model on key metrics.
4. Keep a rollback plan. If the new model causes problems, you should be able to quickly revert to the previous production version.
5. Document every version. Record why each model was created, what data it used, and why it was promoted or archived.

Quick Summary

A model registry is a centralized system for managing ML model versions and their lifecycle stages. MLflow's Model Registry lets you register models, promote them through stages (None, Staging, Production, Archived), and load the production model without knowing specific file paths. A proper promotion workflow includes validation, comparison with the current production model, and archiving of old versions.

Key Points

- A model registry tracks model versions and their lifecycle stages
- MLflow supports four stages: None, Staging, Production, Archived
- Models should be validated in Staging before promoting to Production

- Loading models by stage (e.g., "Production") decouples serving from specific versions
- Always compare a candidate with the current production model before promoting
- Keep old versions archived for potential rollback

Practice Questions

1. What are the four lifecycle stages in the MLflow Model Registry?
2. Why should you never promote a model directly from development to production?
3. How does loading a model by stage (e.g., `models:/ModelName/Production`) help in production systems?
4. What should you do with the old production model when promoting a new one?
5. What validation checks should you perform before promoting a model to production?

Exercises

Exercise 1: Register and Manage

Register three different model versions in MLflow. Practice promoting one to staging, then to production, and archiving an old version.

Exercise 2: Automated Promotion

Write a function that:

- Takes a model version and test data
- Runs at least three validation checks (accuracy, precision, recall)
- Only promotes if ALL checks pass
- Logs the promotion decision with reasons

Exercise 3: Rollback System

Implement a rollback function that:

- Takes a model name as input
- Finds the most recently archived version

- Promotes it back to production
- Archives the current production model
- Logs all changes

What Is Next?

You now have a system for managing model versions. But how do you ensure models are automatically tested and deployed when code changes? In Chapter 12, we will learn about CI/CD (Continuous Integration and Continuous Deployment) for ML, using GitHub Actions to automate testing, training, and deployment.

Chapter 12: CI/CD for Machine Learning

What You Will Learn

In this chapter, you will learn:

- What CI/CD means and why ML projects need it
- How to use GitHub Actions for automated testing
- How to create automated training pipelines
- How to implement model validation gates that prevent bad models from reaching production

Why This Chapter Matters

Imagine you run a bakery with 10 bakers. Each baker makes cakes independently. One day, a baker uses salt instead of sugar, but nobody catches the mistake until customers complain. The next day, another baker changes the recipe without telling anyone.

CI/CD prevents this chaos. CI (Continuous Integration) means every change is automatically tested. CD (Continuous Deployment) means tested changes are automatically deployed. In ML, this means every code change is tested, every model is validated, and only good models make it to production.

What Is CI/CD?

```
+-----------------------------------------------------+
|  CI/CD Explained                                     |
|                                                      |
|  CI = Continuous Integration                        |
|  "Every code change is automatically tested"        |
|                                                      |
|  Developer pushes code                              |
|       |                                              |
|       v                                              |
|  Automated tests run                                |
|       |                                              |
|       +-- Tests pass?  --> Merge allowed            |
|       |                                              |
|       +-- Tests fail?  --> Fix before merging       |
|                                                      |
|  CD = Continuous Deployment                         |
|  "Tested code is automatically deployed"            |
|                                                      |
|  Tests pass                                         |
|       |                                              |
|       v                                              |
|  Automatically deploy to production                 |
+-----------------------------------------------------+
```

CI/CD for ML Is Special

ML CI/CD has extra challenges compared to regular software:

```
+-----------------------------------------------------+
|  Regular Software CI/CD    ML CI/CD                  |
|                                                      |
|  Test code                 Test code                 |
|  Deploy code               Test data quality         |
|                             Test model performance  |
|                             Validate predictions    |
|                             Deploy model + code     |
|                             Monitor model health    |
+-----------------------------------------------------+
```

GitHub Actions Basics

GitHub Actions is a CI/CD service built into GitHub. When you push code, it automatically runs tasks you define in a YAML file.

```
+-----------------------------------------------------+
|  How GitHub Actions Works                           |
|                                                     |
|  1. You push code to GitHub                        |
|  2. GitHub reads .github/workflows/*.yml files     |
|  3. GitHub creates a virtual machine               |
|  4. The VM runs your defined steps                 |
|  5. Results are shown on GitHub                    |
|                                                     |
|  It is like having a robot assistant that           |
|  checks your work every time you submit it.        |
+-----------------------------------------------------+
```

Your First GitHub Action

```yaml
# .github/workflows/test.yml
# This file tells GitHub Actions what to do

name: Run Tests  # Name shown in the GitHub UI

# When should this run?
on:
  push:
    branches: [main]      # On push to main branch
  pull_request:
    branches: [main]      # On pull requests to main

# What should it do?
jobs:
  test:
    runs-on: ubuntu-latest  # Use a Linux virtual machine

    steps:
      # Step 1: Get the code
      - name: Checkout code
        uses: actions/checkout@v4

      # Step 2: Set up Python
      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.11"

      # Step 3: Install dependencies
      - name: Install dependencies
        run: |
          pip install --upgrade pip
          pip install -r requirements.txt
          pip install pytest
```

```
      # Step 4: Run tests
      - name: Run tests
        run: pytest tests/ -v
```

Line-by-line explanation:

```yaml
name: Run Tests
```

This is the name of the workflow. It appears in the GitHub Actions tab.

```yaml
on:
  push:
    branches: [main]
  pull_request:
    branches: [main]
```

This defines when the workflow runs. It triggers on pushes to main and on pull requests targeting main.

```yaml
jobs:
  test:
    runs-on: ubuntu-latest
```

Jobs are the actual work. `runs-on: ubuntu-latest` means GitHub creates a fresh Ubuntu Linux machine for each run.

```yaml
    steps:
      - name: Checkout code
        uses: actions/checkout@v4
```

Steps are individual tasks within a job. `actions/checkout@v4` is a pre-built action that downloads your code into the virtual machine.

CI/CD Pipeline for ML Projects

Testing Data Quality

```yaml
# .github/workflows/ml-pipeline.yml
name: ML Pipeline

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  # Job 1: Test code quality
  code-quality:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.11"

      - name: Install dependencies
        run: |
          pip install -r requirements.txt
          pip install pytest flake8

      - name: Lint code
        run: flake8 src/ --max-line-length 100

      - name: Run unit tests
        run: pytest tests/ -v

  # Job 2: Validate data
  data-validation:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.11"

      - name: Install dependencies
        run: pip install -r requirements.txt

      - name: Validate data quality
        run: python scripts/validate_data.py

  # Job 3: Train and evaluate model
  train-model:
    runs-on: ubuntu-latest
    needs: [code-quality, data-validation]  # Wait for previous jobs
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.11"
```

```
      - name: Install dependencies
        run: pip install -r requirements.txt

      - name: Train model
        run: python scripts/train.py --config config.yaml

      - name: Evaluate model
        run: python scripts/evaluate.py

      - name: Check model quality gates
        run: python scripts/check_gates.py
```

Data Validation Script

```python
"""
scripts/validate_data.py - Validate data quality in CI/CD.

This script runs as part of the CI/CD pipeline to
ensure the data meets quality standards before training.
"""

import pandas as pd
import sys
import json


def validate_data(data_path, rules_path=None):
    """
    Validate a dataset against quality rules.

    Parameters
    ----------
    data_path : str
        Path to the dataset CSV.
    rules_path : str, optional
        Path to validation rules JSON.

    Returns
    -------
    tuple
        (passed: bool, report: dict)
    """
    print(f"Validating data: {data_path}")
    print("=" * 50)

    df = pd.read_csv(data_path)
    checks = []

    # Check 1: File is not empty
```

```
    if len(df) > 0:
        checks.append(("Not empty", True, f"{len(df)} rows"))
    else:
        checks.append(("Not empty", False, "File is empty!"))

    # Check 2: Required columns exist
    required_columns = ["age", "income", "credit_score", "target"]
    missing = [c for c in required_columns if c not in df.columns]
    if not missing:
        checks.append(("Required columns", True, "All present"))
    else:
        checks.append(("Required columns", False, f"Missing: {missing}"))

    # Check 3: No excessive missing values (max 5%)
    max_missing_pct = 0.05
    missing_pct = df.isnull().mean()
    worst_column = missing_pct.max()
    if worst_column <= max_missing_pct:
        checks.append((
            "Missing values",
            True,
            f"Max: {worst_column:.1%}",
        ))
    else:
        checks.append((
            "Missing values",
            False,
            f"Max: {worst_column:.1%} (limit: {max_missing_pct:.0%})",
        ))

    # Check 4: No duplicate rows (max 1%)
    dupe_pct = df.duplicated().mean()
    if dupe_pct <= 0.01:
        checks.append(("Duplicates", True, f"{dupe_pct:.2%}"))
    else:
        checks.append(("Duplicates", False, f"{dupe_pct:.2%} (limit: 1%)"))

    # Check 5: Target column has expected values
    if "target" in df.columns:
        unique_targets = df["target"].nunique()
        if unique_targets >= 2:
            checks.append((
                "Target classes",
                True,
                f"{unique_targets} classes",
            ))
        else:
            checks.append((
                "Target classes",
                False,
                f"Only {unique_targets} class(es)!",
            ))

    # Check 6: Minimum dataset size
    min_size = 100
```

```
    if len(df) >= min_size:
        checks.append(("Minimum size", True, f"{len(df)} >= {min_size}"))
    else:
        checks.append(("Minimum size", False, f"{len(df)} < {min_size}"))

    # Print results
    all_passed = True
    for name, passed, detail in checks:
        status = "PASS" if passed else "FAIL"
        symbol = "[+]" if passed else "[-]"
        print(f"  {symbol} {name}: {status} ({detail})")
        if not passed:
            all_passed = False

    print("=" * 50)
    if all_passed:
        print("All data validation checks PASSED")
    else:
        print("Data validation FAILED")

    return all_passed


if __name__ == "__main__":
    data_path = "data/training_data.csv"
    passed = validate_data(data_path)

    if not passed:
        print("\nExiting with error code 1")
        sys.exit(1)  # Non-zero exit code = CI/CD failure
```

```
Output (all checks pass):
Validating data: data/training_data.csv
==================================================
  [+] Not empty: PASS (1000 rows)
  [+] Required columns: PASS (All present)
  [+] Missing values: PASS (Max: 1.2%)
  [+] Duplicates: PASS (0.30%)
  [+] Target classes: PASS (2 classes)
  [+] Minimum size: PASS (1000 >= 100)
==================================================
All data validation checks PASSED
```

Model Validation Gates

A validation gate is a check that must pass before a model can be deployed. It is like a quality control checkpoint in a factory.

```
+-----------------------------------------------------+
|  Model Validation Gates                              |
|                                                      |
|  Gate 1: Minimum accuracy (e.g., > 0.85)            |
|  Gate 2: Better than current production model        |
|  Gate 3: No performance regression on key groups    |
|  Gate 4: Prediction latency under threshold          |
|  Gate 5: Model size within limits                    |
|                                                      |
|  ALL gates must pass for deployment!                 |
+-----------------------------------------------------+
```

```python
"""
scripts/check_gates.py - Model validation gates for CI/CD.

This script checks if a trained model meets all quality
requirements before it can be deployed.
"""

import joblib
import json
import sys
import time
import os
import numpy as np
from sklearn.metrics import accuracy_score, f1_score
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split


def check_model_gates(model_path, test_data_path=None):
    """
    Run all validation gates on a trained model.

    Parameters
    ----------
    model_path : str
        Path to the trained model file.
    test_data_path : str, optional
        Path to test data. If None, uses synthetic data.

    Returns
    -------
    tuple
        (all_passed: bool, results: list)
    """
    print("Model Validation Gates")
    print("=" * 55)

    gates = []
```

```
    # Load model
    model = joblib.load(model_path)

    # Load or create test data
    if test_data_path:
        import pandas as pd
        df = pd.read_csv(test_data_path)
        X_test = df.drop("target", axis=1).values
        y_test = df["target"].values
    else:
        X, y = make_classification(
            n_samples=1000, n_features=10,
            n_classes=2, random_state=42,
        )
        _, X_test, _, y_test = train_test_split(
            X, y, test_size=0.2, random_state=42,
        )

    predictions = model.predict(X_test)

    # Gate 1: Minimum accuracy
    accuracy = accuracy_score(y_test, predictions)
    min_accuracy = float(os.getenv("MIN_ACCURACY", "0.85"))
    gate1_passed = accuracy >= min_accuracy
    gates.append({
        "gate": "Minimum Accuracy",
        "passed": gate1_passed,
        "value": f"{accuracy:.4f}",
        "threshold": f">= {min_accuracy}",
    })

    # Gate 2: Minimum F1 score
    f1 = f1_score(y_test, predictions, average="weighted")
    min_f1 = 0.80
    gate2_passed = f1 >= min_f1
    gates.append({
        "gate": "Minimum F1 Score",
        "passed": gate2_passed,
        "value": f"{f1:.4f}",
        "threshold": f">= {min_f1}",
    })

    # Gate 3: Prediction latency
    # Time how long predictions take
    start = time.time()
    for _ in range(100):
        model.predict(X_test[:1])
    avg_latency_ms = (time.time() - start) / 100 * 1000

    max_latency_ms = 50.0
    gate3_passed = avg_latency_ms <= max_latency_ms
    gates.append({
        "gate": "Prediction Latency",
        "passed": gate3_passed,
        "value": f"{avg_latency_ms:.2f} ms",
```

```
        "threshold": f"<= {max_latency_ms} ms",
    })

    # Gate 4: Model file size
    model_size_mb = os.path.getsize(model_path) / (1024 * 1024)
    max_size_mb = 500.0
    gate4_passed = model_size_mb <= max_size_mb
    gates.append({
        "gate": "Model File Size",
        "passed": gate4_passed,
        "value": f"{model_size_mb:.1f} MB",
        "threshold": f"<= {max_size_mb} MB",
    })

    # Gate 5: No class with zero predictions
    unique_preds = set(predictions)
    all_classes_predicted = len(unique_preds) >= 2
    gates.append({
        "gate": "All Classes Predicted",
        "passed": all_classes_predicted,
        "value": f"{len(unique_preds)} classes",
        "threshold": ">= 2 classes",
    })

    # Print results
    all_passed = True
    for gate in gates:
        status = "PASS" if gate["passed"] else "FAIL"
        symbol = "[+]" if gate["passed"] else "[-]"
        print(
            f"  {symbol} {gate['gate']:<25} "
            f"{status:>4} | "
            f"Value: {gate['value']:<15} "
            f"Threshold: {gate['threshold']}"
        )
        if not gate["passed"]:
            all_passed = False

    print("=" * 55)
    if all_passed:
        print("ALL GATES PASSED - Model approved for deployment!")
    else:
        print("GATES FAILED - Model NOT approved for deployment.")

    # Save results as JSON (for CI/CD reporting)
    results = {
        "all_passed": all_passed,
        "gates": gates,
        "model_path": model_path,
    }
    with open("gate_results.json", "w") as f:
        json.dump(results, f, indent=2)

    return all_passed, gates
```

```
if __name__ == "__main__":
    model_path = os.getenv(
        "MODEL_PATH", "models/trained_model.pkl"
    )
    passed, _ = check_model_gates(model_path)

    if not passed:
        sys.exit(1)
```

```
Output:
Model Validation Gates
=========================================================
  [+] Minimum Accuracy          PASS | Value: 0.9350         Threshold: >= 0.85
  [+] Minimum F1 Score          PASS | Value: 0.9348         Threshold: >= 0.8
  [+] Prediction Latency        PASS | Value: 2.34 ms        Threshold: <= 50.0 ms
  [+] Model File Size           PASS | Value: 12.5 MB        Threshold: <= 500.0 MB
  [+] All Classes Predicted     PASS | Value: 2 classes      Threshold: >= 2 classes
=========================================================
ALL GATES PASSED - Model approved for deployment!
```

Automated Training Pipeline

```yaml
# .github/workflows/train-and-deploy.yml
name: Train and Deploy Model

on:
  # Run weekly on Mondays at 6 AM UTC
  schedule:
    - cron: "0 6 * * 1"

  # Also allow manual triggering
  workflow_dispatch:
    inputs:
      force_deploy:
        description: "Force deployment even if model is not better"
        required: false
        default: "false"

jobs:
  train:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v4
```

```
      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.11"

      - name: Install dependencies
        run: pip install -r requirements.txt

      - name: Validate data
        run: python scripts/validate_data.py

      - name: Train model
        run: python scripts/train.py --config config.yaml
        env:
          MLFLOW_TRACKING_URI: ${{ secrets.MLFLOW_TRACKING_URI }}

      - name: Run validation gates
        run: python scripts/check_gates.py
        env:
          MODEL_PATH: models/trained_model.pkl
          MIN_ACCURACY: "0.85"

      - name: Upload model artifact
        uses: actions/upload-artifact@v4
        with:
          name: trained-model
          path: |
            models/trained_model.pkl
            gate_results.json

  deploy:
    runs-on: ubuntu-latest
    needs: train  # Only runs after train succeeds
    if: github.ref == 'refs/heads/main'  # Only deploy from main

    steps:
      - uses: actions/checkout@v4

      - name: Download model artifact
        uses: actions/download-artifact@v4
        with:
          name: trained-model

      - name: Deploy to production
        run: |
          echo "Deploying model to production..."
          # In practice, this would:
          # - Push Docker image to registry
          # - Update the running service
          # - Register model in MLflow registry
          echo "Deployment complete!"
        env:
          DEPLOY_TOKEN: ${{ secrets.DEPLOY_TOKEN }}
```

Complete CI/CD Flow

```
+-----------------------------------------------------------------+
|  ML CI/CD Pipeline                                               |
|                                                                   |
|  Developer pushes code to GitHub                                 |
|       |                                                           |
|       v                                                           |
|  [Code Quality]   Run linter and unit tests                     |
|       |                                                           |
|       +-- Fail? --> Developer must fix and push again            |
|       |                                                           |
|       v  Pass                                                    |
|  [Data Validation] Check data quality                           |
|       |                                                           |
|       +-- Fail? --> Alert: data issue                            |
|       |                                                           |
|       v  Pass                                                    |
|  [Model Training]  Train the model                              |
|       |                                                           |
|       v                                                           |
|  [Validation Gates] Check accuracy, latency, size               |
|       |                                                           |
|       +-- Fail? --> Alert: model does not meet standards         |
|       |                                                           |
|       v  Pass                                                    |
|  [Deploy]          Push to production                           |
|       |                                                           |
|       v                                                           |
|  [Monitor]         Watch model performance                      |
+-----------------------------------------------------------------+
```

Common Mistakes

1. No tests at all. Even basic tests catch many bugs before they reach production.
2. Testing only code, not data or models. ML CI/CD must validate data quality and model performance, not just code.
3. Hard-coding secrets in YAML files. Use GitHub Secrets for API keys, tokens, and passwords. Never put them in code.
4. No validation gates. Without gates, a terrible model can be deployed automatically.

5. Deploying from any branch. Only deploy from the main branch. Use pull requests for code review.

Best Practices

1. Test at multiple levels. Test code (unit tests), data (validation), and models (performance gates).
2. Use GitHub Secrets for sensitive values. Never commit API keys, passwords, or tokens to your repository.
3. Make gates configurable. Use environment variables for thresholds so you can adjust them without changing code.
4. Run tests on pull requests. Catch problems before code is merged, not after.
5. Save artifacts. Upload trained models and test results as GitHub Actions artifacts for later review.
6. Use scheduled workflows. Retrain models on a schedule (weekly, monthly) to keep them fresh.

Quick Summary

CI/CD for ML automates testing, training, and deployment. GitHub Actions runs workflows automatically when code changes. Data validation checks ensure data quality. Model validation gates ensure model quality. Together, they create a pipeline where only tested code and validated models reach production.

Key Points

- CI ensures every code change is automatically tested
- CD automates deployment of tested changes
- ML CI/CD adds data validation and model validation gates
- GitHub Actions workflows are defined in YAML files
- Validation gates prevent bad models from reaching production

- Use GitHub Secrets for sensitive configuration
- Schedule automatic retraining to keep models fresh

Practice Questions

1. What is the difference between CI and CD?
2. Why does ML CI/CD need more than just code testing?
3. What is a validation gate? Give three examples of gates for an ML model.
4. Why should you use GitHub Secrets instead of putting API keys directly in your workflow file?
5. How does the `needs` keyword work in GitHub Actions?

Exercises

Exercise 1: Basic CI Pipeline

Create a GitHub Actions workflow that:

- Triggers on pull requests
- Installs Python dependencies
- Runs pytest
- Checks code style with flake8

Exercise 2: Model Quality Gates

Write a Python script with at least 5 validation gates:

- Minimum accuracy
- Minimum F1 score
- Maximum prediction latency
- Maximum model file size
- Balanced predictions across classes

The script should exit with code 1 if any gate fails.

Exercise 3: Full ML Pipeline

Design a complete CI/CD pipeline (as a GitHub Actions YAML file) that:

- Validates data quality
- Trains a model
- Runs validation gates
- Deploys only if all checks pass
- Sends a notification on failure

What Is Next?

CI/CD automates the process of testing and deploying models. But what triggers retraining in the first place? In Chapter 13, we will learn about automated retraining: how to detect when a model needs updating and how to orchestrate the retraining process automatically.

Chapter 13: Automated Retraining

What You Will Learn

In this chapter, you will learn:

- What triggers model retraining and when it is needed
- Three types of retraining triggers: schedule-based, drift-based, and performance-based
- How to orchestrate retraining pipelines
- How to build a simple scheduler for automated retraining
- How to validate new models before replacing old ones

Why This Chapter Matters

Imagine you have a navigation app on your phone. It works great when you first install it. But six months later, a new highway has been built, three roads have been closed, and a bridge is under construction. If the app never updates its maps, it will send you down closed roads.

ML models are the same. They are trained on historical data that reflects the world at a specific moment. But the world changes. Customer behavior shifts. New products appear. Economic conditions fluctuate. A model that was excellent six months ago might be giving poor recommendations today.

Automated retraining keeps your models fresh by detecting when they need updating and running the retraining process without human intervention.

When to Retrain

There are three main triggers for retraining:

```
+---------------------------------------------------+
|  Retraining Triggers                               |
|                                                    |
|  1. Schedule-Based                                |
|     "Retrain every Monday at 6 AM"                |
|     Simple and predictable                        |
|     May retrain when not needed                   |
|                                                    |
|  2. Drift-Based                                   |
|     "Retrain when data distribution changes"      |
|     Retrains only when needed                     |
|     Requires drift monitoring                     |
|                                                    |
|  3. Performance-Based                             |
|     "Retrain when accuracy drops below 85%"       |
|     Most meaningful trigger                       |
|     Requires ground truth labels                  |
+---------------------------------------------------+
```

Schedule-Based Retraining

The simplest approach. Like changing your car's oil every 5,000 miles, you retrain on a fixed schedule.

```python
"""
scheduled_retraining.py - Retrain on a fixed schedule.

This is the simplest retraining approach. You set a
schedule (daily, weekly, monthly) and the model
retrains automatically.

Schedule options:
- Daily:   For fast-changing data (stock prices, trending topics)
- Weekly:  For moderately changing data (user behavior)
- Monthly: For slowly changing data (demographics, climate)
"""

import schedule
import time
import logging
from datetime import datetime

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)


def retrain_model():
    """
    Complete retraining pipeline.
```

```
    This function:
    1. Loads the latest data
    2. Trains a new model
    3. Evaluates the new model
    4. Replaces the production model if the new one is better
    """
    logger.info("=" * 50)
    logger.info(f"Retraining started at {datetime.now()}")
    logger.info("=" * 50)

    try:
        # Step 1: Load latest data
        logger.info("Step 1: Loading latest data...")
        # In practice: query database, download files, etc.
        # data = load_latest_data()

        # Step 2: Prepare data
        logger.info("Step 2: Preparing data...")
        # X_train, X_test, y_train, y_test = prepare_data(data)

        # Step 3: Train new model
        logger.info("Step 3: Training new model...")
        # new_model = train_model(X_train, y_train)

        # Step 4: Evaluate new model
        logger.info("Step 4: Evaluating new model...")
        # new_accuracy = evaluate(new_model, X_test, y_test)

        # Step 5: Compare with current model
        logger.info("Step 5: Comparing with current model...")
        # current_accuracy = evaluate(current_model, X_test, y_test)

        # Step 6: Deploy if better
        # if new_accuracy > current_accuracy:
        #     deploy_model(new_model)
        #     logger.info(f"New model deployed! Accuracy: {new_accuracy:.4f}")
        # else:
        #     logger.info(f"Current model is still better. No deployment.")

        logger.info("Retraining complete!")

    except Exception as e:
        logger.error(f"Retraining failed: {e}")
        # In practice: send alert to team
        raise


# Schedule retraining
# This is a simple in-process scheduler
# For production, use cron, Airflow, or cloud schedulers
schedule.every().monday.at("06:00").do(retrain_model)
schedule.every().thursday.at("06:00").do(retrain_model)

print("Scheduler started. Waiting for next retraining...")
```

```
print("Scheduled times: Monday and Thursday at 06:00")

# Run the scheduler loop
while True:
    schedule.run_pending()
    time.sleep(60)  # Check every minute
```

Drift-Based Retraining

Smarter than scheduled retraining. Only retrain when the data actually changes.

```python
"""
drift_retraining.py - Retrain when data drift is detected.

This monitors incoming data and triggers retraining
only when the data distribution changes significantly.
"""

import numpy as np
from scipy import stats
import logging
from datetime import datetime

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)


class DriftMonitor:
    """
    Monitor data for drift and trigger retraining.

    This class stores the training data distribution
    and compares new data against it. When significant
    drift is detected, it triggers retraining.
    """

    def __init__(
        self,
        reference_data,
        feature_names,
        drift_threshold=0.05,
        min_drift_features=1,
    ):
        """
        Initialize the drift monitor.

        Parameters
        ----------
        reference_data : np.ndarray
            The training data (what the model was trained on).
```

```
        feature_names : list
            Names of the features.
        drift_threshold : float
            P-value threshold for the KS test.
            Below this = drift detected.
        min_drift_features : int
            Minimum number of features that must show drift
            before triggering retraining.
        """
        self.reference_data = reference_data
        self.feature_names = feature_names
        self.drift_threshold = drift_threshold
        self.min_drift_features = min_drift_features
        self.check_count = 0
        self.drift_count = 0

    def check_drift(self, new_data):
        """
        Check if new data has drifted from the reference.

        Parameters
        ----------
        new_data : np.ndarray
            Recent production data to check.

        Returns
        -------
        dict
            Drift check results including whether
            retraining is recommended.
        """
        self.check_count += 1
        drifted_features = []

        for i, name in enumerate(self.feature_names):
            ref_col = self.reference_data[:, i]
            new_col = new_data[:, i]

            # Kolmogorov-Smirnov test
            ks_stat, p_value = stats.ks_2samp(ref_col, new_col)

            if p_value < self.drift_threshold:
                drifted_features.append({
                    "feature": name,
                    "ks_statistic": round(ks_stat, 4),
                    "p_value": round(p_value, 6),
                })

        # Determine if retraining is needed
        should_retrain = (
            len(drifted_features) >= self.min_drift_features
        )

        if should_retrain:
            self.drift_count += 1
```

```
        result = {
            "timestamp": datetime.now().isoformat(),
            "check_number": self.check_count,
            "total_features": len(self.feature_names),
            "drifted_features": len(drifted_features),
            "drifted_details": drifted_features,
            "should_retrain": should_retrain,
        }

        # Log results
        if should_retrain:
            logger.warning(
                f"DRIFT DETECTED in {len(drifted_features)} features! "
                f"Retraining recommended."
            )
            for feat in drifted_features:
                logger.warning(
                    f"  {feat['feature']}: p={feat['p_value']}"
                )
        else:
            logger.info(
                f"No significant drift detected "
                f"({len(drifted_features)} features drifted, "
                f"threshold: {self.min_drift_features})"
            )

        return result


# Example usage
np.random.seed(42)

# Reference data (what the model was trained on)
reference = np.random.normal(
    loc=[30, 50000, 700, 5],
    scale=[10, 20000, 50, 3],
    size=(1000, 4),
)

feature_names = ["age", "income", "credit_score", "emp_years"]

# Create monitor
monitor = DriftMonitor(
    reference_data=reference,
    feature_names=feature_names,
    drift_threshold=0.05,
    min_drift_features=1,
)

# Check 1: Similar data (no drift expected)
similar_data = np.random.normal(
    loc=[30, 50000, 700, 5],
    scale=[10, 20000, 50, 3],
    size=(500, 4),
```

```
)
result1 = monitor.check_drift(similar_data)
print(f"\nCheck 1 - Should retrain: {result1['should_retrain']}")

# Check 2: Shifted data (drift expected)
shifted_data = np.random.normal(
    loc=[35, 70000, 680, 8],  # Income and emp_years shifted
    scale=[10, 20000, 50, 3],
    size=(500, 4),
)
result2 = monitor.check_drift(shifted_data)
print(f"Check 2 - Should retrain: {result2['should_retrain']}")
print(f"  Drifted features: {result2['drifted_features']}")
```

```
Output:
INFO - No significant drift detected (0 features drifted, threshold: 1)

Check 1 - Should retrain: False
WARNING - DRIFT DETECTED in 3 features! Retraining recommended.
WARNING -   age: p=0.001234
WARNING -   income: p=0.000001
WARNING -   emp_years: p=0.000023
Check 2 - Should retrain: True
  Drifted features: 3
```

Performance-Based Retraining

The most meaningful trigger. Retrain when the model's actual performance drops.

```python
"""
performance_retraining.py - Retrain when performance drops.

This monitors the model's actual predictions against
ground truth labels and triggers retraining when
accuracy drops below a threshold.
"""

import numpy as np
from collections import deque
from datetime import datetime
import logging

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)


class PerformanceMonitor:
    """
```

```
    Monitor model performance and trigger retraining.

    Uses a rolling window to track recent accuracy.
    When accuracy drops below the threshold, it
    triggers retraining.
    """

    def __init__(
        self,
        accuracy_threshold=0.85,
        window_size=100,
        alert_after_n_checks=3,
    ):
        """
        Initialize the performance monitor.

        Parameters
        ----------
        accuracy_threshold : float
            Minimum acceptable accuracy (0.0 to 1.0).
        window_size : int
            Number of recent predictions to consider.
        alert_after_n_checks : int
            Number of consecutive checks below threshold
            before triggering retraining.
        """
        self.accuracy_threshold = accuracy_threshold
        self.window_size = window_size
        self.alert_after_n_checks = alert_after_n_checks

        # Track recent predictions and actual outcomes
        # deque with maxlen automatically drops old entries
        self.predictions = deque(maxlen=window_size)
        self.actuals = deque(maxlen=window_size)

        self.consecutive_failures = 0
        self.total_checks = 0

    def record(self, prediction, actual):
        """Record a single prediction and its actual outcome."""
        self.predictions.append(prediction)
        self.actuals.append(actual)

    def record_batch(self, predictions, actuals):
        """Record a batch of predictions and actual outcomes."""
        for pred, actual in zip(predictions, actuals):
            self.record(pred, actual)

    def check_performance(self):
        """
        Check if performance is acceptable.

        Returns
        -------
        dict
```

```
        Performance check results.
    """
    self.total_checks += 1

    if len(self.predictions) < 10:
        return {
            "status": "insufficient_data",
            "samples": len(self.predictions),
            "should_retrain": False,
        }

    # Calculate rolling accuracy
    correct = sum(
        p == a for p, a in zip(self.predictions, self.actuals)
    )
    accuracy = correct / len(self.predictions)

    # Check if below threshold
    below_threshold = accuracy < self.accuracy_threshold

    if below_threshold:
        self.consecutive_failures += 1
    else:
        self.consecutive_failures = 0

    # Trigger retraining after N consecutive failures
    should_retrain = (
        self.consecutive_failures >= self.alert_after_n_checks
    )

    result = {
        "timestamp": datetime.now().isoformat(),
        "accuracy": round(accuracy, 4),
        "threshold": self.accuracy_threshold,
        "below_threshold": below_threshold,
        "consecutive_failures": self.consecutive_failures,
        "should_retrain": should_retrain,
        "window_size": len(self.predictions),
    }

    # Logging
    if should_retrain:
        logger.warning(
            f"PERFORMANCE DEGRADATION! Accuracy: {accuracy:.4f} "
            f"(threshold: {self.accuracy_threshold}). "
            f"Consecutive failures: {self.consecutive_failures}. "
            f"RETRAINING TRIGGERED!"
        )
    elif below_threshold:
        logger.warning(
            f"Performance below threshold: {accuracy:.4f} "
            f"(failure {self.consecutive_failures}/"
            f"{self.alert_after_n_checks})"
        )
    else:
```

```
            logger.info(
                f"Performance OK: {accuracy:.4f} "
                f"(threshold: {self.accuracy_threshold})"
            )

        return result


# Example: Simulate performance degradation
monitor = PerformanceMonitor(
    accuracy_threshold=0.85,
    window_size=50,
    alert_after_n_checks=3,
)

np.random.seed(42)

# Phase 1: Model performing well (90% accuracy)
print("Phase 1: Model performing well")
for _ in range(50):
    actual = np.random.choice([0, 1])
    # 90% chance of correct prediction
    correct = np.random.random() < 0.90
    prediction = actual if correct else 1 - actual
    monitor.record(prediction, actual)

result = monitor.check_performance()
print(f"  Accuracy: {result['accuracy']}, Retrain: {result['should_retrain']}")

# Phase 2: Model starting to degrade (75% accuracy)
print("\nPhase 2: Model degrading")
for _ in range(50):
    actual = np.random.choice([0, 1])
    correct = np.random.random() < 0.75
    prediction = actual if correct else 1 - actual
    monitor.record(prediction, actual)

for check in range(4):
    result = monitor.check_performance()
    print(
        f"  Check {check+1}: Accuracy={result['accuracy']}, "
        f"Failures={result['consecutive_failures']}, "
        f"Retrain={result['should_retrain']}"
    )
```

```
Output:
Phase 1: Model performing well
  Accuracy: 0.88, Retrain: False

Phase 2: Model degrading
  Check 1: Accuracy=0.76, Failures=1, Retrain=False
  Check 2: Accuracy=0.76, Failures=2, Retrain=False
  Check 3: Accuracy=0.76, Failures=3, Retrain=True
```

```
  Check 4: Accuracy=0.76, Failures=4, Retrain=True
```

Pipeline Orchestration

Orchestration means coordinating all the steps in the retraining pipeline. Think of it as a conductor directing an orchestra. Each musician (step) plays their part at the right time.

```
+-----------------------------------------------------+
|  Retraining Pipeline Steps                          |
|                                                     |
|  Trigger (drift/schedule/performance)              |
|       |                                             |
|       v                                             |
|  1. Collect latest data                            |
|       |                                             |
|       v                                             |
|  2. Validate data quality                          |
|       |                                             |
|       v                                             |
|  3. Prepare data (clean, split)                    |
|       |                                             |
|       v                                             |
|  4. Train new model                                |
|       |                                             |
|       v                                             |
|  5. Evaluate new model                             |
|       |                                             |
|       v                                             |
|  6. Compare with production model                  |
|       |                                             |
|       +-- New model worse? --> Keep current,       |
|       |                        log and alert        |
|       |                                             |
|       +-- New model better? --> Continue            |
|       |                                             |
|       v                                             |
|  7. Deploy new model                               |
|       |                                             |
|       v                                             |
|  8. Verify deployment (smoke test)                 |
|       |                                             |
|       v                                             |
|  9. Log results and notify team                    |
+-----------------------------------------------------+
```

Complete Retraining Pipeline

```python
"""
retraining_pipeline.py - A complete automated retraining pipeline.

This pipeline handles the entire process from data
collection to model deployment.
"""

import logging
import json
import time
import os
from datetime import datetime
from dataclasses import dataclass, field
from typing import Optional

import numpy as np
import joblib
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, f1_score

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)


@dataclass
class PipelineResult:
    """Results from a retraining pipeline run."""

    started_at: str = ""
    completed_at: str = ""
    trigger: str = ""
    data_samples: int = 0
    new_model_accuracy: float = 0.0
    current_model_accuracy: float = 0.0
    deployed: bool = False
    reason: str = ""
    duration_seconds: float = 0.0
    errors: list = field(default_factory=list)


class RetrainingPipeline:
    """
    Complete retraining pipeline.

    Coordinates all steps of the retraining process.
    """

    def __init__(
        self,
```

```
        model_path="models/production_model.pkl",
        data_source="data/latest.csv",
        min_accuracy=0.85,
        min_improvement=0.01,
    ):
        """
        Initialize the pipeline.

        Parameters
        ----------
        model_path : str
            Path to the current production model.
        data_source : str
            Where to get training data.
        min_accuracy : float
            Minimum accuracy for deployment.
        min_improvement : float
            Minimum improvement over current model.
        """
        self.model_path = model_path
        self.data_source = data_source
        self.min_accuracy = min_accuracy
        self.min_improvement = min_improvement

    def run(self, trigger="manual"):
        """
        Execute the complete retraining pipeline.

        Parameters
        ----------
        trigger : str
            What triggered this run (schedule, drift,
            performance, manual).

        Returns
        -------
        PipelineResult
            Results of the pipeline run.
        """
        result = PipelineResult(
            started_at=datetime.now().isoformat(),
            trigger=trigger,
        )
        start_time = time.time()

        logger.info("=" * 60)
        logger.info(f"RETRAINING PIPELINE STARTED")
        logger.info(f"Trigger: {trigger}")
        logger.info(f"Time: {result.started_at}")
        logger.info("=" * 60)

        try:
            # Step 1: Collect data
            logger.info("\n[1/7] Collecting latest data...")
            X, y = self._collect_data()
```

```
            result.data_samples = len(X)
            logger.info(f"  Collected {len(X)} samples")

            # Step 2: Validate data
            logger.info("\n[2/7] Validating data quality...")
            self._validate_data(X, y)
            logger.info("  Data validation passed")

            # Step 3: Prepare data
            logger.info("\n[3/7] Preparing data...")
            X_train, X_test, y_train, y_test = train_test_split(
                X, y, test_size=0.2, random_state=42
            )
            logger.info(
                f"  Train: {len(X_train)}, Test: {len(X_test)}"
            )

            # Step 4: Train new model
            logger.info("\n[4/7] Training new model...")
            new_model = self._train_model(X_train, y_train)
            logger.info("  Training complete")

            # Step 5: Evaluate new model
            logger.info("\n[5/7] Evaluating new model...")
            new_preds = new_model.predict(X_test)
            new_accuracy = accuracy_score(y_test, new_preds)
            new_f1 = f1_score(y_test, new_preds, average="weighted")
            result.new_model_accuracy = new_accuracy
            logger.info(
                f"  New model: accuracy={new_accuracy:.4f}, "
                f"f1={new_f1:.4f}"
            )

            # Step 6: Compare with current model
            logger.info("\n[6/7] Comparing with current model...")
            should_deploy = self._compare_models(
                new_model, new_accuracy, X_test, y_test, result
            )

            # Step 7: Deploy or skip
            if should_deploy:
                logger.info("\n[7/7] Deploying new model...")
                self._deploy_model(new_model)
                result.deployed = True
                result.reason = (
                    f"New model ({new_accuracy:.4f}) is better "
                    f"than current ({result.current_model_accuracy:.4f})"
                )
                logger.info("  Deployment complete!")
            else:
                logger.info("\n[7/7] Skipping deployment")
                result.deployed = False
                logger.info(
                    f"  Reason: {result.reason}"
                )
```

```
        except Exception as e:
            logger.error(f"Pipeline failed: {e}")
            result.errors.append(str(e))
            result.reason = f"Pipeline error: {e}"

        result.completed_at = datetime.now().isoformat()
        result.duration_seconds = round(time.time() - start_time, 2)

        # Log summary
        logger.info("\n" + "=" * 60)
        logger.info("PIPELINE SUMMARY")
        logger.info(f"  Duration: {result.duration_seconds}s")
        logger.info(f"  Deployed: {result.deployed}")
        logger.info(f"  Reason: {result.reason}")
        logger.info("=" * 60)

        # Save results
        self._save_results(result)

        return result

    def _collect_data(self):
        """Collect the latest training data."""
        # In practice, this would query a database or API
        X, y = make_classification(
            n_samples=2000,
            n_features=10,
            n_classes=2,
            random_state=int(time.time()) % 1000,
        )
        return X, y

    def _validate_data(self, X, y):
        """Validate data quality."""
        # Check minimum size
        if len(X) < 100:
            raise ValueError(
                f"Insufficient data: {len(X)} samples "
                f"(minimum: 100)"
            )

        # Check for NaN values
        if np.isnan(X).any():
            raise ValueError("Data contains NaN values")

        # Check class balance
        unique, counts = np.unique(y, return_counts=True)
        min_ratio = counts.min() / counts.max()
        if min_ratio < 0.1:
            logger.warning(
                f"Imbalanced classes: ratio={min_ratio:.2f}"
            )

    def _train_model(self, X_train, y_train):
```

```
        """Train a new model."""
        model = RandomForestClassifier(
            n_estimators=100,
            random_state=42,
        )
        model.fit(X_train, y_train)
        return model

    def _compare_models(
        self, new_model, new_accuracy, X_test, y_test, result
    ):
        """Compare new model with current production model."""
        # Check minimum accuracy
        if new_accuracy < self.min_accuracy:
            result.reason = (
                f"New model accuracy ({new_accuracy:.4f}) "
                f"below minimum ({self.min_accuracy})"
            )
            return False

        # Try to load current model
        try:
            current_model = joblib.load(self.model_path)
            current_preds = current_model.predict(X_test)
            current_accuracy = accuracy_score(y_test, current_preds)
            result.current_model_accuracy = current_accuracy

            logger.info(
                f"  Current model accuracy: {current_accuracy:.4f}"
            )
            logger.info(
                f"  New model accuracy: {new_accuracy:.4f}"
            )

            improvement = new_accuracy - current_accuracy
            logger.info(f"  Improvement: {improvement:+.4f}")

            if improvement < self.min_improvement:
                result.reason = (
                    f"Improvement ({improvement:+.4f}) below "
                    f"minimum ({self.min_improvement})"
                )
                return False

            return True

        except FileNotFoundError:
            logger.info("  No current model found. Deploying new model.")
            result.current_model_accuracy = 0.0
            return True

    def _deploy_model(self, model):
        """Deploy the new model."""
        os.makedirs(os.path.dirname(self.model_path), exist_ok=True)
        joblib.dump(model, self.model_path)
```

```
        logger.info(f"  Model saved to {self.model_path}")

    def _save_results(self, result):
        """Save pipeline results to a log file."""
        results_dir = "logs/retraining"
        os.makedirs(results_dir, exist_ok=True)

        timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
        filepath = os.path.join(
            results_dir, f"run_{timestamp}.json"
        )

        # Convert dataclass to dict for JSON
        result_dict = {
            "started_at": result.started_at,
            "completed_at": result.completed_at,
            "trigger": result.trigger,
            "data_samples": result.data_samples,
            "new_model_accuracy": result.new_model_accuracy,
            "current_model_accuracy": result.current_model_accuracy,
            "deployed": result.deployed,
            "reason": result.reason,
            "duration_seconds": result.duration_seconds,
            "errors": result.errors,
        }

        with open(filepath, "w") as f:
            json.dump(result_dict, f, indent=2)

        logger.info(f"Results saved to {filepath}")


# Run the pipeline
pipeline = RetrainingPipeline(
    model_path="models/production_model.pkl",
    min_accuracy=0.85,
    min_improvement=0.01,
)

result = pipeline.run(trigger="manual")
```

```
Output:
============================================================
RETRAINING PIPELINE STARTED
Trigger: manual
Time: 2024-01-15T10:00:00.000000
============================================================

[1/7] Collecting latest data...
  Collected 2000 samples

[2/7] Validating data quality...
  Data validation passed
```

```
[3/7] Preparing data...
  Train: 1600, Test: 400

[4/7] Training new model...
  Training complete

[5/7] Evaluating new model...
  New model: accuracy=0.9325, f1=0.9323

[6/7] Comparing with current model...
  No current model found. Deploying new model.

[7/7] Deploying new model...
  Model saved to models/production_model.pkl
  Deployment complete!

============================================================
PIPELINE SUMMARY
  Duration: 3.45s
  Deployed: True
  Reason: New model (0.9325) is better than current (0.0000)
============================================================
```

Simple Scheduler Example

Here is a complete example that combines drift monitoring with scheduled checks:

```python
"""
auto_retrain_scheduler.py - Automated retraining with monitoring.

This combines schedule-based checks with drift detection
and performance monitoring.
"""

import time
import logging
from datetime import datetime, timedelta

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s | %(levelname)-8s | %(message)s",
)
logger = logging.getLogger(__name__)
```

```
class AutoRetrainer:
    """
    Automated retraining scheduler.

    Combines multiple triggers:
    - Scheduled checks (e.g., daily)
    - Drift detection
    - Performance monitoring
    """

    def __init__(
        self,
        check_interval_hours=24,
        max_days_without_retrain=30,
    ):
        """
        Initialize the auto-retrainer.

        Parameters
        ----------
        check_interval_hours : int
            How often to check if retraining is needed.
        max_days_without_retrain : int
            Force retraining after this many days.
        """
        self.check_interval = timedelta(hours=check_interval_hours)
        self.max_interval = timedelta(days=max_days_without_retrain)
        self.last_retrain = datetime.now()
        self.last_check = datetime.now()
        self.retrain_count = 0

    def should_retrain(self, drift_detected=False, accuracy=None):
        """
        Decide whether to retrain.

        Parameters
        ----------
        drift_detected : bool
            Whether data drift was detected.
        accuracy : float, optional
            Current model accuracy (0.0 to 1.0).

        Returns
        -------
        tuple
            (should_retrain: bool, reason: str)
        """
        now = datetime.now()
        time_since_retrain = now - self.last_retrain

        # Reason 1: Maximum time exceeded
        if time_since_retrain > self.max_interval:
            return True, (
                f"Maximum interval exceeded "
                f"({time_since_retrain.days} days > "
```

```
                f"{self.max_interval.days} days)"
            )

        # Reason 2: Data drift detected
        if drift_detected:
            return True, "Data drift detected"

        # Reason 3: Performance degradation
        if accuracy is not None and accuracy < 0.85:
            return True, (
                f"Performance below threshold "
                f"({accuracy:.4f} < 0.85)"
            )

        return False, "No retraining needed"

    def run_check(self, drift_detected=False, accuracy=None):
        """
        Run a single check and potentially trigger retraining.

        Returns
        -------
        dict
            Check results.
        """
        self.last_check = datetime.now()

        should_retrain, reason = self.should_retrain(
            drift_detected=drift_detected,
            accuracy=accuracy,
        )

        result = {
            "timestamp": datetime.now().isoformat(),
            "should_retrain": should_retrain,
            "reason": reason,
            "days_since_retrain": (
                datetime.now() - self.last_retrain
            ).days,
        }

        if should_retrain:
            logger.warning(f"RETRAINING TRIGGERED: {reason}")
            self._execute_retrain(reason)
        else:
            logger.info(f"Check complete: {reason}")

        return result

    def _execute_retrain(self, reason):
        """Execute the retraining pipeline."""
        logger.info("Starting retraining pipeline...")

        # In practice, this would call your RetrainingPipeline
        # pipeline = RetrainingPipeline()
```

```
        # result = pipeline.run(trigger=reason)

        self.retrain_count += 1
        self.last_retrain = datetime.now()

        logger.info(
            f"Retraining complete! "
            f"Total retrains: {self.retrain_count}"
        )

    def get_status(self):
        """Get the current status of the auto-retrainer."""
        now = datetime.now()
        return {
            "last_retrain": self.last_retrain.isoformat(),
            "days_since_retrain": (now - self.last_retrain).days,
            "total_retrains": self.retrain_count,
            "next_check": (
                self.last_check + self.check_interval
            ).isoformat(),
        }


# Demo: Simulate the auto-retrainer
retrainer = AutoRetrainer(
    check_interval_hours=24,
    max_days_without_retrain=30,
)

# Simulate checks
print("Simulating automated retraining checks\n")

# Check 1: Everything is fine
result = retrainer.run_check(drift_detected=False, accuracy=0.92)
print(f"  Result: {result['reason']}\n")

# Check 2: Drift detected
result = retrainer.run_check(drift_detected=True, accuracy=0.90)
print(f"  Result: {result['reason']}\n")

# Check 3: Performance drop
result = retrainer.run_check(drift_detected=False, accuracy=0.80)
print(f"  Result: {result['reason']}\n")

# Show status
status = retrainer.get_status()
print(f"\nRetrainer Status:")
for key, value in status.items():
    print(f"  {key}: {value}")
```

```
Output:
Simulating automated retraining checks
```

```
2024-01-15 10:00:00 | INFO     | Check complete: No retraining needed
  Result: No retraining needed

2024-01-15 10:00:00 | WARNING  | RETRAINING TRIGGERED: Data drift detected
2024-01-15 10:00:00 | INFO     | Starting retraining pipeline...
2024-01-15 10:00:00 | INFO     | Retraining complete! Total retrains: 1
  Result: Data drift detected

2024-01-15 10:00:00 | WARNING  | RETRAINING TRIGGERED: Performance below threshold (0.8...
2024-01-15 10:00:00 | INFO     | Starting retraining pipeline...
2024-01-15 10:00:00 | INFO     | Retraining complete! Total retrains: 2
  Result: Performance below threshold (0.8000 < 0.85)

Retrainer Status:
  last_retrain: 2024-01-15T10:00:00.654321
  days_since_retrain: 0
  total_retrains: 2
  next_check: 2024-01-16T10:00:00.654321
```

Common Mistakes

1. Retraining too frequently. If nothing has changed, retraining wastes compute and risks introducing instability. Use triggers, not just schedules.
2. Not comparing with the current model. Always verify the new model is actually better before deploying it.
3. No rollback mechanism. If the newly deployed model causes problems, you need to be able to quickly revert.
4. Ignoring data quality. Retraining on bad data produces a bad model. Always validate data before training.
5. No logging or notification. When retraining happens automatically, you need logs and alerts to know what happened.

Best Practices

1. Use multiple triggers. Combine schedule-based, drift-based, and performance-based triggers for comprehensive coverage.
2. Always validate before deploying. Run validation gates before replacing the production model.

3. Keep a history of retraining runs. Log every run with its trigger, data, metrics, and outcome.
4. Set up alerts. Notify the team when retraining happens, especially when it fails.
5. Test the pipeline itself. Run the retraining pipeline in a test environment before relying on it in production.
6. Have a manual override. Sometimes you need to force retraining or skip it. Build in manual controls.

Quick Summary

Automated retraining keeps ML models fresh by detecting when they need updating and running the retraining process automatically. Three triggers drive retraining: schedule-based (regular intervals), drift-based (data distribution changes), and performance-based (accuracy drops). A complete retraining pipeline collects data, validates it, trains a new model, compares it with the current model, and deploys it only if it is better.

Key Points

- Models degrade over time as the world changes
- Schedule-based retraining is simple but may retrain unnecessarily
- Drift-based retraining detects when data distributions change
- Performance-based retraining triggers when accuracy drops
- Always compare new models with the current production model
- Never deploy a new model without validation
- Log every retraining run for debugging and auditing
- Have a rollback plan in case the new model causes problems

Practice Questions

1. What are the three types of retraining triggers? When would you use each one?

2. Why is it important to compare a new model with the current production model before deploying?
3. What is data drift and how can it affect model performance?
4. What should happen if the retraining pipeline produces a model that is worse than the current one?
5. Why should you log every retraining run, even successful ones?

Exercises

Exercise 1: Drift Monitor

Build a drift monitor that:

- Accepts reference data during initialization
- Checks new data batches for drift using the KS test
- Tracks drift history over time
- Returns a report with which features drifted and by how much

Exercise 2: Complete Retraining Pipeline

Implement a retraining pipeline that:

- Loads data from a CSV file
- Validates data quality (at least 3 checks)
- Trains a model
- Evaluates against 3 metrics (accuracy, precision, recall)
- Compares with the current production model
- Deploys only if better
- Saves a JSON report of the run

Exercise 3: Multi-Trigger Scheduler

Create a retraining system that combines all three triggers:

- Runs a check every hour
- Checks for data drift
- Checks model performance
- Forces retraining if more than 7 days have passed

- Logs all decisions and actions

What Is Next?

Congratulations! You have completed the MLOps journey from notebook to production. You now know how to:

- Organize code for production (Chapter 1)
- Save and load models (Chapter 2)
- Build ML APIs with FastAPI (Chapter 3)
- Containerize with Docker (Chapter 4)
- Deploy to the cloud (Chapter 5)
- Build demos with Streamlit and Gradio (Chapter 6)
- Serve models at scale (Chapter 7)
- Understand the full ML lifecycle (Chapter 8)
- Track experiments with MLflow (Chapter 9)
- Version data with DVC (Chapter 10)
- Manage models in a registry (Chapter 11)
- Automate with CI/CD (Chapter 12)
- Set up automated retraining (Chapter 13)

These skills form the foundation of MLOps, the discipline of deploying and maintaining ML models in production. Keep practicing, keep building, and keep learning. The field is evolving rapidly, and the best way to stay current is to build real projects.

Chapter 14: Feature Stores — Centralized Feature Management

What You Will Learn

In this chapter, you will learn:

- What feature stores are and why they exist
- The problems feature stores solve in ML systems
- How to set up and use Feast (an open source feature store)
- How to define, register, and serve features
- How to use features for both training and real-time inference
- Best practices for organizing and managing features at scale

Why This Chapter Matters

Imagine you work at a company with 50 data scientists. Each one builds their own features from raw data. One person calculates "average purchase amount" by including returns. Another excludes returns. A third uses a different time window. The same concept has three different implementations, and nobody knows which one is correct.

This is the feature management problem. Without a central system, features get duplicated, definitions conflict, and bugs hide in production for months.

A feature store is like a library for ML features. Just as a library organizes books so everyone can find and borrow them, a feature store organizes features so every model can find and use them consistently.

Think of it this way. A restaurant kitchen has a prep station where ingredients are washed, chopped, and organized before cooking begins. The feature store is that prep station for machine learning. Raw data comes in, gets transformed into useful features, and sits ready

for any model that needs it.

What Is a Feature?

Before diving into feature stores, let us make sure we understand features.

A feature is a piece of information that a model uses to make predictions. If you are predicting whether a customer will buy a product, features might include:

- Customer age
- Number of past purchases
- Average order value
- Days since last visit
- Whether they opened the last email

```
Raw Data                    Features (Model Inputs)
+------------------+       +----------------------+
| Transaction logs | ----> | avg_purchase_amount  |
| User profiles    | ----> | customer_age         |
| Click streams    | ----> | pages_visited_7d     |
| Email events     | ----> | email_open_rate      |
+------------------+       +----------------------+
```

Feature engineering is the process of transforming raw data into these features. It often involves calculations, aggregations, and transformations.

The Problem: Why Feature Stores Exist

Without a feature store, ML teams face several painful problems.

Problem 1: Training-Serving Skew

Training-serving skew means your model sees different feature values during training than during prediction.

```
TRAINING TIME (Python script):
    avg_purchase = purchases.mean()    # Uses pandas

SERVING TIME (Java API):
    avg_purchase = sum / count          # Reimplemented in Java

RESULT: Slightly different calculations = wrong predictions!
```

Think of this like a student who practices with one textbook but takes an exam based on a different textbook. Even small differences cause wrong answers.

Problem 2: Duplicated Work

```
WITHOUT FEATURE STORE:

Data Scientist A:
    "I need customer lifetime value"
    --> Writes SQL query (2 hours)
    --> Writes Python transform (3 hours)

Data Scientist B (next week):
    "I also need customer lifetime value"
    --> Writes different SQL query (2 hours)
    --> Writes different Python transform (3 hours)
    --> Gets slightly different numbers!

Total wasted: 10+ hours, inconsistent results
```

Problem 3: Slow Feature Serving

During real-time prediction, you cannot run a complex SQL query that takes 30 seconds. You need features served in milliseconds.

```
User clicks "Buy" --> Model needs prediction in 50ms
                      |
                      +--> Cannot run: SELECT AVG(amount)
                           FROM purchases
                           WHERE customer_id = 123
                           AND date > NOW() - INTERVAL 90 DAY
                           (Takes 2 seconds!)
```

How a Feature Store Solves These Problems

```
+------------------+     +------------------+     +------------------+
|   Raw Data       |     |  FEATURE STORE   |     |    ML Models     |
|                  |     |                  |     |                  |
| - Databases      |---->| - Define once    |---->| - Training       |
| - Event streams  |     | - Compute once   |     | - Prediction     |
| - Files          |     | - Serve fast     |     | - Monitoring     |
|                  |     | - Version track  |     |                  |
+------------------+     +------------------+     +------------------+

ONE definition. ONE computation. CONSISTENT everywhere.
```

Feature Store Architecture

A feature store has several key components.

```
FEATURE STORE ARCHITECTURE:

+-------------------------------------------------------------------+
|                        FEATURE STORE                              |
|                                                                   |
|  +------------------+    +------------------+    +---------------+ |
|  | Feature          |    | Offline Store    |    | Online Store  | |
|  | Definitions      |    | (for training)   |    | (for serving) | |
|  |                  |    |                  |    |               | |
|  | - Names          |    | - All history    |    | - Latest      | |
|  | - Types          |    | - Batch access   |    |   values only | |
|  | - Sources        |    | - Parquet/DB     |    | - Fast lookup | |
|  | - Entities       |    |                  |    | - Redis/DDB   | |
|  +------------------+    +------------------+    +---------------+ |
|                                                                   |
|  +------------------+    +------------------+                     |
|  | Feature          |    | Feature          |                     |
|  | Registry         |    | Serving API      |                     |
|  |                  |    |                  |                     |
|  | - Metadata       |    | - Get features   |                     |
|  | - Versions       |    | - for entity     |                     |
|  | - Lineage        |    | - Low latency    |                     |
|  +------------------+    +------------------+                     |
+-------------------------------------------------------------------+
```

Let us break down each component:

Offline Store — Stores all historical feature values. Used when training models because you need lots of data going back in time. Think of this as the warehouse where you keep everything.

Online Store — Stores only the latest feature values. Used during real-time prediction because you need fast access. Think of this as the shelf right next to the cash register with only what you need right now.

Feature Registry — A catalog of all features with their definitions, types, and metadata. Think of this as the library card catalog.

Feature Serving API — The interface that models use to request features. Think of this as the librarian who fetches books for you.

Introduction to Feast

Feast (Feature Store) is the most popular open source feature store. It is free, well-documented, and used by many companies.

```
WHY FEAST:

+------------------+------------------+------------------+
|  Open Source      |  Python-native   |  Cloud-ready     |
|  Free to use      |  Easy to learn   |  Scales up       |
|  Active community |  Familiar tools  |  Production-ready|
+------------------+------------------+------------------+
```

Installing Feast

```python
# Install Feast
# Run this in your terminal:
# pip install feast

# Verify installation
import feast
print(f"Feast version: {feast.__version__}")
```

```
Output:
Feast version: 0.37.0
```

Creating a Feast Project

Let us build a feature store for an e-commerce company that predicts whether customers will make a purchase.

```python
# Step 1: Create project directory structure
import os

# Create directory for our feature store project
project_dir = "ecommerce_features"
os.makedirs(project_dir, exist_ok=True)
os.makedirs(f"{project_dir}/data", exist_ok=True)

print(f"Created project directory: {project_dir}")
print(f"Created data directory: {project_dir}/data")
```

```
Output:
Created project directory: ecommerce_features
Created data directory: ecommerce_features/data
```

Preparing Sample Data

Before defining features, we need some data to work with.

```python
import pandas as pd
import numpy as np
from datetime import datetime, timedelta

# Create sample customer feature data
# In real life, this would come from your data warehouse
np.random.seed(42)

num_customers = 1000
num_records = 5000  # Multiple records per customer over time

# Generate timestamps over the last 30 days
base_time = datetime(2024, 1, 1)
timestamps = [
    base_time + timedelta(
        days=np.random.randint(0, 30),
        hours=np.random.randint(0, 24)
    )
    for _ in range(num_records)
]

# Create feature data
customer_features = pd.DataFrame({
    "customer_id": np.random.randint(1, num_customers + 1, num_records),
    "event_timestamp": timestamps,
```

```
    "total_purchases": np.random.randint(0, 100, num_records),
    "avg_order_value": np.round(np.random.uniform(10, 500, num_records), 2),
    "days_since_last_purchase": np.random.randint(0, 365, num_records),
    "email_open_rate": np.round(np.random.uniform(0, 1, num_records), 3),
    "pages_visited_7d": np.random.randint(0, 50, num_records),
    "cart_abandonment_rate": np.round(np.random.uniform(0, 1, num_records), 3),
    "is_loyalty_member": np.random.choice([0, 1], num_records),
    "customer_segment": np.random.choice(
        ["new", "active", "dormant", "vip"], num_records
    ),
})

# Sort by timestamp
customer_features = customer_features.sort_values("event_timestamp")

# Save to parquet file (Feast works well with parquet)
output_path = f"{project_dir}/data/customer_features.parquet"
customer_features.to_parquet(output_path)

print(f"Created {len(customer_features)} feature records")
print(f"For {customer_features['customer_id'].nunique()} unique customers")
print(f"Saved to: {output_path}")
print(f"\nSample data:")
print(customer_features.head(3).to_string(index=False))
```

```
Output:
Created 5000 feature records
For 999 unique customers
Saved to: ecommerce_features/data/customer_features.parquet

Sample data:
 customer_id  event_timestamp  total_purchases  avg_order_value  days_since_last_purcha...
         847 2024-01-01 01:00               15           203.45                        1...
         293 2024-01-01 01:00               67            89.23                         ...
         512 2024-01-01 02:00                3           412.67                        2...
```

Defining Features in Feast

In Feast, you define features using Python code. The key concepts are:

- Entity — The thing you are tracking (customer, product, driver)
- Feature View — A group of related features from the same data source
- Data Source — Where the raw data lives (file, database, stream)

```
FEAST CONCEPTS:

Entity (WHO)          Feature View (WHAT)        Data Source (WHERE)
+-------------+      +-------------------+      +------------------+
| customer_id |----->| Customer Features |----->| Parquet file     |
|             |      |  - total_purchases|      | or Database      |
|             |      |  - avg_order_value|      | or Stream        |
+-------------+      |  - email_open_rate|      +------------------+
                     +-------------------+
```

Writing Feature Definitions

```python
# feature_store_definitions.py
# This file defines all features for our e-commerce feature store

from datetime import timedelta
from feast import Entity, FeatureView, Field, FileSource
from feast.types import Float32, Int32, Int64, String

# ============================================================
# Step 1: Define the Entity
# ============================================================
# An Entity is the "thing" you track features for.
# Think of it as the primary key in a database.

customer = Entity(
    name="customer_id",       # Name of the entity
    description="Unique identifier for each customer",
)

# ============================================================
# Step 2: Define the Data Source
# ============================================================
# This tells Feast where to find the raw feature data.
# We are using a local parquet file. In production, this could
# be a database table, S3 bucket, or streaming source.

customer_source = FileSource(
    path="data/customer_features.parquet",  # Path to data file
    timestamp_field="event_timestamp",       # Column with timestamps
)

# ============================================================
# Step 3: Define the Feature View
# ============================================================
# A Feature View groups related features together.
# It connects an entity to a data source and defines
# which columns are features and their data types.

customer_features_view = FeatureView(
```

```
    name="customer_features",   # Name for this feature group
    entities=[customer],        # Which entity these features belong to
    ttl=timedelta(days=30),     # Time-to-live: how old data can be
    schema=[
        # Define each feature with its name and type
        Field(name="total_purchases", dtype=Int32),
        Field(name="avg_order_value", dtype=Float32),
        Field(name="days_since_last_purchase", dtype=Int32),
        Field(name="email_open_rate", dtype=Float32),
        Field(name="pages_visited_7d", dtype=Int32),
        Field(name="cart_abandonment_rate", dtype=Float32),
        Field(name="is_loyalty_member", dtype=Int32),
        Field(name="customer_segment", dtype=String),
    ],
    source=customer_source,     # Where the data comes from
    online=True,                # Make available for real-time serving
)

print("Feature definitions created successfully!")
print(f"Entity: {customer.name}")
print(f"Feature View: {customer_features_view.name}")
print(f"Number of features: {len(customer_features_view.schema)}")
print(f"TTL: {customer_features_view.ttl}")
```

```
Output:
Feature definitions created successfully!
Entity: customer_id
Feature View: customer_features
Number of features: 8
TTL: 30 days, 0:00:00
```

Let us explain each part:

```
LINE-BY-LINE BREAKDOWN:

Entity(name="customer_id")
    |
    +--> "customer_id" is the column that identifies each customer.
         Every feature is associated with a customer_id.

FileSource(path="...", timestamp_field="event_timestamp")
    |
    +--> Tells Feast where the data file is.
    +--> timestamp_field tells Feast which column has the time.
         This is critical for point-in-time correctness.

FeatureView(name="customer_features", ...)
    |
    +--> Groups related features together.
    +--> entities=[customer] links features to customers.
    +--> ttl=timedelta(days=30) means features older than 30 days
```

```
         are considered stale and will not be served.
    +--> schema=[...] defines each feature and its data type.
    +--> online=True means these features should be available
         for real-time serving (not just batch training).
```

Setting Up the Feast Feature Store

Now let us create a complete Feast project and apply our definitions.

```python
# Create the feature_store.yaml configuration file
import yaml

feast_config = {
    "project": "ecommerce",
    "provider": "local",
    "registry": "data/registry.db",
    "online_store": {
        "type": "sqlite",
        "path": "data/online_store.db"
    },
    "offline_store": {
        "type": "file"
    },
    "entity_key_serialization_version": 2
}

config_path = f"{project_dir}/feature_store.yaml"
with open(config_path, "w") as f:
    yaml.dump(feast_config, f, default_flow_style=False)

print("feature_store.yaml created:")
print(yaml.dump(feast_config, default_flow_style=False))
```

```
Output:
feature_store.yaml created:
project: ecommerce
provider: local
registry: data/registry.db
online_store:
  type: sqlite
  path: data/online_store.db
offline_store:
  type: file
entity_key_serialization_version: 2
```

```
CONFIGURATION EXPLAINED:

project: "ecommerce"
    +--> Name of your project. All features belong to this project.

provider: "local"
    +--> Where Feast runs. "local" means on your machine.
         In production, this would be "gcp", "aws", or "azure".

registry: "data/registry.db"
    +--> Where Feast stores feature definitions (metadata).

online_store: sqlite
    +--> Where latest feature values are stored for fast serving.
         In production, this would be Redis, DynamoDB, etc.

offline_store: file
    +--> Where historical feature data is stored for training.
         In production, this would be BigQuery, Redshift, etc.
```

Applying Feature Definitions

After defining features, you need to "apply" them to register with Feast.

```python
from feast import FeatureStore
from feast.repo_config import RepoConfig
from feast.infra.online_stores.sqlite import SqliteOnlineStoreConfig

# In a real project, you would run "feast apply" from the command line.
# Here we demonstrate the concept programmatically.

# Simulate applying feature definitions
print("Applying feature definitions to the feature store...")
print()
print("Step 1: Reading feature definitions...")
print("  - Found entity: customer_id")
print("  - Found feature view: customer_features (8 features)")
print()
print("Step 2: Registering with the registry...")
print("  - Entity 'customer_id' registered")
print("  - Feature view 'customer_features' registered")
print()
print("Step 3: Creating online store tables...")
print("  - Table 'customer_features' created in SQLite")
print()
print("Feature store is ready!")
```

```
print()

# Show what the registry contains
print("=" * 50)
print("FEATURE REGISTRY CONTENTS")
print("=" * 50)
print()
print("Entities:")
print(f"  - customer_id (type: INT64)")
print()
print("Feature Views:")
print(f"  - customer_features")
print(f"    Source: data/customer_features.parquet")
print(f"    Features:")

features = [
    ("total_purchases", "INT32", "Total number of purchases"),
    ("avg_order_value", "FLOAT32", "Average order value in dollars"),
    ("days_since_last_purchase", "INT32", "Days since last purchase"),
    ("email_open_rate", "FLOAT32", "Fraction of emails opened"),
    ("pages_visited_7d", "INT32", "Pages visited in last 7 days"),
    ("cart_abandonment_rate", "FLOAT32", "Cart abandonment rate"),
    ("is_loyalty_member", "INT32", "1 if loyalty member, 0 if not"),
    ("customer_segment", "STRING", "Customer segment category"),
]

for name, dtype, desc in features:
    print(f"      {name} ({dtype}): {desc}")
```

```
Output:
Applying feature definitions to the feature store...

Step 1: Reading feature definitions...
  - Found entity: customer_id
  - Found feature view: customer_features (8 features)

Step 2: Registering with the registry...
  - Entity 'customer_id' registered
  - Feature view 'customer_features' registered

Step 3: Creating online store tables...
  - Table 'customer_features' created in SQLite

Feature store is ready!

==================================================
FEATURE REGISTRY CONTENTS
==================================================

Entities:
  - customer_id (type: INT64)

Feature Views:
```

```
  - customer_features
    Source: data/customer_features.parquet
    Features:
      total_purchases (INT32): Total number of purchases
      avg_order_value (FLOAT32): Average order value in dollars
      days_since_last_purchase (INT32): Days since last purchase
      email_open_rate (FLOAT32): Fraction of emails opened
      pages_visited_7d (INT32): Pages visited in last 7 days
      cart_abandonment_rate (FLOAT32): Cart abandonment rate
      is_loyalty_member (INT32): 1 if loyalty member, 0 if not
      customer_segment (STRING): Customer segment category
```

Serving Features for Training

When training a model, you need historical features at specific points in time. This is called point-in-time correct feature retrieval.

Why Point-in-Time Matters

```
THE PROBLEM WITHOUT POINT-IN-TIME:

Timeline:
Jan 1          Jan 15         Feb 1
  |              |              |
  Customer       Customer       Customer
  has 5          has 12         has 20
  purchases      purchases      purchases

If you train a model to predict Feb 1 purchase:
  WRONG: Use Feb 1 features (20 purchases) - that includes the answer!
  RIGHT: Use Jan 15 features (12 purchases) - what you knew BEFORE Feb 1

This is called "data leakage" and it makes your model
look great in training but fail in production.
```

Think of it like a detective solving a crime. You cannot use evidence discovered after the crime to predict it. You can only use information available at the time.

Getting Training Features

```python
import pandas as pd
import numpy as np
from datetime import datetime, timedelta

# Load our feature data
feature_data = pd.read_parquet(f"{project_dir}/data/customer_features.parquet")

# Create an entity dataframe (the "spine" of your training data)
# This represents the prediction events you want features for
entity_df = pd.DataFrame({
    "customer_id": [101, 202, 303, 404, 505],
    "event_timestamp": [
        datetime(2024, 1, 15),  # Want features as of Jan 15
        datetime(2024, 1, 20),  # Want features as of Jan 20
        datetime(2024, 1, 10),  # Want features as of Jan 10
        datetime(2024, 1, 25),  # Want features as of Jan 25
        datetime(2024, 1, 18),  # Want features as of Jan 18
    ],
    "label": [1, 0, 1, 0, 1],  # Did they purchase? (target variable)
})

print("Entity DataFrame (prediction events):")
print(entity_df.to_string(index=False))
```

```
Output:
Entity DataFrame (prediction events):
 customer_id event_timestamp  label
         101      2024-01-15      1
         202      2024-01-20      0
         303      2024-01-10      1
         404      2024-01-25      0
         505      2024-01-18      1
```

```python
# Simulate point-in-time correct feature retrieval
# In real Feast: store.get_historical_features(entity_df, features)

def get_historical_features(entity_df, feature_data, ttl_days=30):
    """
    Simulate Feast's get_historical_features.

    For each row in entity_df, find the most recent feature values
    that existed BEFORE the event_timestamp (point-in-time correct).
    """
    results = []

    for _, entity_row in entity_df.iterrows():
        customer_id = entity_row["customer_id"]
        event_time = entity_row["event_timestamp"]
        cutoff_time = event_time - timedelta(days=ttl_days)
```

```
        # Get features for this customer BEFORE the event time
        # and AFTER the TTL cutoff
        mask = (
            (feature_data["customer_id"] == customer_id) &
            (feature_data["event_timestamp"] <= event_time) &
            (feature_data["event_timestamp"] >= cutoff_time)
        )

        customer_features = feature_data[mask]

        if len(customer_features) > 0:
            # Get the most recent record
            latest = customer_features.iloc[-1]
            result = {
                "customer_id": customer_id,
                "event_timestamp": event_time,
                "label": entity_row["label"],
                "total_purchases": latest["total_purchases"],
                "avg_order_value": latest["avg_order_value"],
                "days_since_last_purchase": latest["days_since_last_purchase"],
                "email_open_rate": latest["email_open_rate"],
                "pages_visited_7d": latest["pages_visited_7d"],
            }
        else:
            # No features available (customer not found or outside TTL)
            result = {
                "customer_id": customer_id,
                "event_timestamp": event_time,
                "label": entity_row["label"],
                "total_purchases": None,
                "avg_order_value": None,
                "days_since_last_purchase": None,
                "email_open_rate": None,
                "pages_visited_7d": None,
            }

        results.append(result)

    return pd.DataFrame(results)

# Get training features
training_df = get_historical_features(entity_df, feature_data)

print("Training DataFrame with point-in-time correct features:")
print(training_df.to_string(index=False))
print(f"\nShape: {training_df.shape}")
print(f"Null values: {training_df.isnull().sum().sum()}")
```

```
Output:
Training DataFrame with point-in-time correct features:
 customer_id event_timestamp  label  total_purchases  avg_order_value  days_since_last_...
         101      2024-01-15      1               23           145.67                   ...
         202      2024-01-20      0               67            89.23                   ...
```

```
         303      2024-01-10      1               12           234.56                  ...
         404      2024-01-25      0               45           178.90                  ...
         505      2024-01-18      1                8           312.45                  ...

Shape: (5, 8)
Null values: 0
```

```
POINT-IN-TIME RETRIEVAL VISUALIZED:

Timeline for Customer 101:

Jan 1        Jan 10       Jan 15       Jan 20
  |            |            |            |
  Features     Features     QUERY        Features
  version 1    version 2    TIME         version 3
                            |
                            +--> Returns version 2
                                 (most recent BEFORE query time)

                                 Does NOT return version 3
                                 (that is in the future!)
```

Serving Features for Real-Time Inference

During real-time prediction (when a user visits your website), you need features served in milliseconds.

```python
# Simulate materializing features to the online store
# In real Feast: store.materialize(start_date, end_date)

def materialize_to_online_store(feature_data):
    """
    Simulate Feast's materialize command.

    This takes the latest feature values for each customer
    and stores them in a fast-access store (like Redis).
    """
    # Get the latest record for each customer
    latest_features = (
        feature_data
        .sort_values("event_timestamp")
        .groupby("customer_id")
        .last()
        .reset_index()
```

```
    )

    print(f"Materialized {len(latest_features)} customer records")
    print(f"to the online store.")
    print(f"Each customer has their LATEST feature values.")

    return latest_features

online_store = materialize_to_online_store(feature_data)
print(f"\nOnline store contains {len(online_store)} records")
```

```
Output:
Materialized 999 customer records
to the online store.
Each customer has their LATEST feature values.

Online store contains 999 records
```

```python
# Simulate real-time feature serving
# In real Feast: store.get_online_features(entity_rows, features)

def get_online_features(online_store, customer_ids, feature_names):
    """
    Simulate Feast's get_online_features.

    Returns the latest feature values for given customers.
    This is what happens when your API needs predictions.
    """
    import time

    start_time = time.time()

    results = {}
    for cid in customer_ids:
        customer_data = online_store[online_store["customer_id"] == cid]
        if len(customer_data) > 0:
            row = customer_data.iloc[0]
            results[cid] = {feat: row[feat] for feat in feature_names}
        else:
            results[cid] = {feat: None for feat in feature_names}

    elapsed_ms = (time.time() - start_time) * 1000

    return results, elapsed_ms

# Simulate a real-time request
customer_ids = [101, 202, 303]
feature_names = [
    "total_purchases",
    "avg_order_value",
```

```
    "email_open_rate"
]

features, latency = get_online_features(
    online_store, customer_ids, feature_names
)

print("Real-time Feature Serving Results:")
print(f"Latency: {latency:.2f} ms")
print()
for cid, feats in features.items():
    print(f"Customer {cid}:")
    for name, value in feats.items():
        print(f"  {name}: {value}")
    print()
```

```
Output:
Real-time Feature Serving Results:
Latency: 1.23 ms

Customer 101:
  total_purchases: 45
  avg_order_value: 234.56
  email_open_rate: 0.678

Customer 202:
  total_purchases: 12
  avg_order_value: 89.34
  email_open_rate: 0.345

Customer 303:
  total_purchases: 78
  avg_order_value: 156.78
  email_open_rate: 0.891
```

```
TRAINING vs SERVING COMPARISON:

TRAINING (Offline Store):                SERVING (Online Store):
+---------------------------+           +---------------------------+
| Get historical features   |           | Get latest features       |
| for many customers        |           | for one customer          |
| at specific times         |           | right now                 |
|                           |           |                           |
| Latency: seconds-minutes  |           | Latency: milliseconds     |
| Volume: millions of rows  |           | Volume: one row at a time |
| Used for: model training  |           | Used for: predictions     |
+---------------------------+           +---------------------------+
```

Using Features in a Model Training Pipeline

Let us put it all together and show how features flow from the store into model training.

```python
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report
import numpy as np

# Step 1: Create training dataset using the feature store
# Generate a larger entity dataframe for training
np.random.seed(42)
n_samples = 500

training_entities = pd.DataFrame({
    "customer_id": np.random.randint(1, 1000, n_samples),
    "event_timestamp": [
        datetime(2024, 1, 1) + timedelta(days=np.random.randint(5, 25))
        for _ in range(n_samples)
    ],
    "purchased": np.random.choice([0, 1], n_samples, p=[0.6, 0.4]),
})

# Step 2: Get features from the feature store
training_data = get_historical_features(
    training_entities, feature_data, ttl_days=30
)

# Drop rows with missing features
training_data = training_data.dropna()
print(f"Training samples with features: {len(training_data)}")

# Step 3: Prepare features and labels
feature_columns = [
    "total_purchases",
    "avg_order_value",
    "days_since_last_purchase",
    "email_open_rate",
    "pages_visited_7d"
]

X = training_data[feature_columns].values
y = training_data["label"].values

# Step 4: Split and train
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)
```

```
# Step 5: Evaluate
y_pred = model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)

print(f"\nModel trained with features from the feature store!")
print(f"Accuracy: {accuracy:.3f}")
print(f"\nFeature Importance:")
for name, importance in sorted(
    zip(feature_columns, model.feature_importances_),
    key=lambda x: x[1],
    reverse=True
):
    bar = "█" * int(importance * 50)
    print(f"  {name:30s} {importance:.3f} {bar}")
```

```
Output:
Training samples with features: 487

Model trained with features from the feature store!
Accuracy: 0.536

Feature Importance:
  avg_order_value                0.234 ██████████
  days_since_last_purchase       0.221 ██████████
  total_purchases                0.198 ████████
  email_open_rate                0.189 ████████
  pages_visited_7d               0.158 ██████
```

Using Features in a Prediction API

Here is how you would use the feature store in a real-time prediction service.

```python
# prediction_service.py
# This shows how a FastAPI service would use the feature store

def predict_purchase(customer_id, online_store, model, feature_columns):
    """
    Make a real-time purchase prediction for a customer.

    1. Get latest features from the online store
    2. Feed them into the model
    3. Return the prediction
    """
    import time
```

```
    start_time = time.time()

    # Step 1: Get features from online store
    features, fetch_latency = get_online_features(
        online_store,
        [customer_id],
        feature_columns
    )

    if customer_id not in features or features[customer_id] is None:
        return {"error": "Customer not found", "customer_id": customer_id}

    # Step 2: Prepare feature vector
    customer_features = features[customer_id]
    feature_vector = np.array([
        [customer_features[col] for col in feature_columns]
    ])

    # Step 3: Make prediction
    prediction = model.predict(feature_vector)[0]
    probability = model.predict_proba(feature_vector)[0]

    total_latency = (time.time() - start_time) * 1000

    return {
        "customer_id": customer_id,
        "will_purchase": bool(prediction),
        "purchase_probability": float(probability[1]),
        "features_used": customer_features,
        "feature_fetch_ms": round(fetch_latency, 2),
        "total_latency_ms": round(total_latency, 2),
    }

# Simulate API requests
print("=" * 60)
print("PREDICTION API RESPONSES")
print("=" * 60)

for cid in [101, 202, 303]:
    result = predict_purchase(cid, online_store, model, feature_columns)
    print(f"\nCustomer {cid}:")
    print(f"  Will purchase: {result['will_purchase']}")
    print(f"  Probability: {result['purchase_probability']:.3f}")
    print(f"  Feature fetch: {result['feature_fetch_ms']} ms")
    print(f"  Total latency: {result['total_latency_ms']} ms")
```

```
Output:
============================================================
PREDICTION API RESPONSES
============================================================

Customer 101:
  Will purchase: True
```

```
  Probability: 0.670
  Feature fetch: 0.89 ms
  Total latency: 1.45 ms

Customer 202:
  Will purchase: False
  Probability: 0.230
  Feature fetch: 0.76 ms
  Total latency: 1.12 ms

Customer 303:
  Will purchase: True
  Probability: 0.810
  Feature fetch: 0.92 ms
  Total latency: 1.67 ms
```

```
COMPLETE FLOW WITH FEATURE STORE:

User visits       API receives      Feature Store       Model           Response
website           request           serves features     predicts        sent back
   |                |                  |                 |               |
   | "Show me       | customer_id=101  | total_purchases  | P(buy)=0.67   | "Show
   |  products"     |                  | avg_order_value  |               |  recommended
   |                |                  | email_open_rate  |               |  products"
   |                |                  |                  |               |
   +-----1ms--------+------1ms---------+------1ms---------+-----1ms------+

                            Total: ~4ms (fast enough!)
```

Feature Store Best Practices

Naming Conventions

```python
# GOOD feature names: clear, descriptive, include time window
good_names = [
    "customer_total_purchases_lifetime",
    "customer_avg_order_value_90d",
    "customer_email_open_rate_30d",
    "product_view_count_7d",
    "customer_days_since_last_purchase",
]

# BAD feature names: vague, inconsistent, no context
bad_names = [
```

```
    "feat1",           # What is this?
    "value",           # Too generic
    "avg",             # Average of what?
    "purchases",       # Total? Recent? Count? Amount?
    "x_var_23",        # Meaningless
]

print("GOOD Feature Names:")
for name in good_names:
    print(f"  ✓ {name}")

print("\nBAD Feature Names:")
for name in bad_names:
    print(f"  ✗ {name}")
```

```
Output:
GOOD Feature Names:
  ✓ customer_total_purchases_lifetime
  ✓ customer_avg_order_value_90d
  ✓ customer_email_open_rate_30d
  ✓ product_view_count_7d
  ✓ customer_days_since_last_purchase

BAD Feature Names:
  ✗ feat1
  ✗ value
  ✗ avg
  ✗ purchases
  ✗ x_var_23
```

Organizing Feature Views

```
ORGANIZE FEATURES BY DOMAIN:

+---------------------+    +---------------------+    +---------------------+
| Customer Profile    |    | Customer Behavior   |    | Product Features    |
| Features            |    | Features            |    |                     |
|                     |    |                     |    |                     |
| - age               |    | - pages_visited_7d  |    | - price             |
| - location          |    | - cart_abandon_rate |    | - category          |
| - signup_date       |    | - purchase_freq     |    | - avg_rating        |
| - is_loyalty_member |    | - session_duration  |    | - review_count      |
+---------------------+    +---------------------+    +---------------------+

Each domain gets its own Feature View.
This makes features easier to find, manage, and reuse.
```

Common Mistakes

1. Not using point-in-time correctness — Using future data to predict the past causes data leakage. Your model looks great in testing but fails in production.
2. Setting TTL too short — If your TTL (time-to-live) is 1 day but features update weekly, most lookups return null. Set TTL based on how often your data updates.
3. Putting all features in one Feature View — This makes it hard to manage. Group features by domain (customer profile, customer behavior, product features).
4. Not materializing features before serving — The online store needs to be populated before you can serve features. Run materialization on a schedule.
5. Ignoring feature data types — Storing a float as a string wastes memory and slows lookups. Always define explicit data types.

Best Practices

1. Define features once, use everywhere — Every model that needs "customer lifetime value" should get it from the same feature definition.
2. Version your feature definitions — Use Git to track changes to feature definitions just like application code.
3. Monitor feature freshness — Set up alerts when features have not been updated recently. Stale features lead to bad predictions.
4. Document every feature — Include a description, unit, and expected range for each feature. Future team members will thank you.
5. Start simple — Begin with a file-based offline store and SQLite online store. Upgrade to Redis and cloud storage when you need scale.

Quick Summary

A feature store is a centralized system that manages ML features. It solves three main problems: training-serving skew (features differ between training and prediction), duplicated work (multiple people computing the same features differently), and slow feature serving (cannot run complex queries in real time).

Feast is the most popular open source feature store. You define features using Entities (what you track), Feature Views (groups of features), and Data Sources (where data lives). The

offline store serves historical features for training with point-in-time correctness. The online store serves the latest features for real-time prediction in milliseconds.

Key Points

- A feature store is a centralized system for managing, storing, and serving ML features
- Training-serving skew occurs when features differ between training and prediction
- Point-in-time correctness prevents data leakage by only using features available at prediction time
- The offline store holds historical data for training (high volume, higher latency)
- The online store holds latest values for real-time serving (low latency, single records)
- Feast is the leading open source feature store with Python-native definitions
- Feature Views group related features from the same data source
- Entities are the primary keys that features are associated with (customer, product, etc.)
- TTL (time-to-live) controls how old feature data can be before it is considered stale
- Materializing features copies data from the offline store to the online store

---

Practice Questions

1. What is training-serving skew and why does it cause problems in production ML systems?
2. Explain the difference between the online store and offline store in a feature store. When would you use each one?
3. What is point-in-time correctness? Give an example of what goes wrong without it.
4. You have 50 features across customer demographics, purchase behavior, and product information. How would you organize them into Feature Views?
5. Your online store has features that are 3 days old, but your Feature View has a TTL of 1 day. What happens when you request features? How would you fix this?

Exercises

Exercise 1: Define a Feature View for Products

Create a Feast Feature View for product features with the following fields: product_id, price, category, average_rating, total_reviews, and days_since_restock. Choose appropriate data types and set a reasonable TTL.

Exercise 2: Build a Feature Pipeline

Write Python code that:

1. Reads raw transaction data from a CSV file
2. Computes aggregated features (total purchases, average order value, most frequent category) per customer
3. Saves the features in parquet format ready for Feast
4. Includes timestamps for point-in-time correctness

Exercise 3: Compare Online vs Offline Performance

Create a benchmark that:

1. Generates 10,000 customer records
2. Measures the time to retrieve features for 1 customer (simulating online serving)
3. Measures the time to retrieve features for all 10,000 customers (simulating offline training)
4. Compares the latency and explains when each approach is appropriate

---

What Is Next?

Now that you know how to manage and serve features consistently, the next challenge is monitoring those features in production. What happens when the data your model receives starts to change? In the next chapter, we will explore Model Monitoring — how to detect data drift, concept drift, and performance degradation before they cause problems in your ML system.

Chapter 15: Model Monitoring — Detecting Drift and Degradation

What You Will Learn

In this chapter, you will learn:

- What data drift is and why it causes models to fail silently
- What concept drift is and how it differs from data drift
- How to detect performance degradation in production models
- Statistical methods for detecting drift
- How to set up monitoring alerts
- How to use Evidently AI for automated model monitoring

Why This Chapter Matters

Imagine you build a weather prediction model that works perfectly in summer. Autumn arrives, and the model keeps predicting sunny weather because it has never seen fall data. The model did not break — the world changed, but the model did not adapt.

This is exactly what happens to ML models in production. The data they receive changes over time. Customer behavior shifts. Market conditions evolve. New products appear. Your model was trained on the past, but it serves the present.

Without monitoring, a broken model looks the same as a working one from the outside. It still returns predictions. It still responds fast. But the predictions are wrong, and nobody knows until a customer complains or revenue drops.

Think of model monitoring like a health checkup for your ML system. You do not wait until you feel sick to visit a doctor. You get regular checkups to catch problems early. Model monitoring is that regular checkup for your predictions.

Types of Drift

There are three main types of problems that cause models to degrade in production.

```
TYPES OF PRODUCTION PROBLEMS:

+------------------+------------------+------------------+
|   DATA DRIFT     |  CONCEPT DRIFT   |  PERFORMANCE     |
|                  |                  |  DEGRADATION     |
|                  |                  |                  |
| Input data       | Relationship     | Model accuracy   |
| distribution     | between inputs   | drops over       |
| changes          | and outputs      | time             |
|                  | changes          |                  |
|                  |                  |                  |
| Example:         | Example:         | Example:         |
| Customers got    | "Luxury" used    | Accuracy went    |
| younger on       | to mean >$500,   | from 95% to 72%  |
| average          | now means >$200  | over 3 months    |
+------------------+------------------+------------------+
```

Data Drift Explained

Data drift (also called feature drift or covariate shift) happens when the input data your model receives in production looks different from the data it was trained on.

```
DATA DRIFT EXAMPLE:

Training Data (2023):           Production Data (2024):
+---------------------+        +---------------------+
| Age: mostly 30-50   |        | Age: mostly 18-30   |
| Income: $50K-$100K  |        | Income: $30K-$60K   |
| Location: suburban   |        | Location: urban      |
+---------------------+        +---------------------+

The MODEL did not change.
The DATA changed.
The model was not trained for this new data!
```

Think of it like a chef who learned to cook Italian food. If customers suddenly start ordering Thai food, the chef's skills have not changed — but the orders have, and the chef cannot

serve them well.

Detecting Data Drift

```python
import numpy as np
import pandas as pd
from scipy import stats

# Simulate training data distribution
np.random.seed(42)
training_ages = np.random.normal(loc=40, scale=10, size=1000)
training_incomes = np.random.normal(loc=75000, scale=20000, size=1000)

# Simulate production data with drift
# Ages shifted younger, incomes shifted lower
production_ages = np.random.normal(loc=32, scale=12, size=1000)
production_incomes = np.random.normal(loc=55000, scale=18000, size=1000)

print("=" * 50)
print("DATA DRIFT DETECTION REPORT")
print("=" * 50)

# Compare distributions using statistics
print("\n--- Age Feature ---")
print(f"Training:   mean={training_ages.mean():.1f}, "
      f"std={training_ages.std():.1f}")
print(f"Production: mean={production_ages.mean():.1f}, "
      f"std={production_ages.std():.1f}")

# Kolmogorov-Smirnov test
# This test checks if two samples come from the same distribution
# Small p-value (< 0.05) = distributions are different = DRIFT
ks_stat, ks_pvalue = stats.ks_2samp(training_ages, production_ages)
print(f"KS test:    statistic={ks_stat:.4f}, p-value={ks_pvalue:.6f}")
print(f"Drift detected: {'YES' if ks_pvalue < 0.05 else 'NO'}")

print("\n--- Income Feature ---")
print(f"Training:   mean={training_incomes.mean():.0f}, "
      f"std={training_incomes.std():.0f}")
print(f"Production: mean={production_incomes.mean():.0f}, "
      f"std={production_incomes.std():.0f}")

ks_stat, ks_pvalue = stats.ks_2samp(training_incomes, production_incomes)
print(f"KS test:    statistic={ks_stat:.4f}, p-value={ks_pvalue:.6f}")
print(f"Drift detected: {'YES' if ks_pvalue < 0.05 else 'NO'}")
```

```
Output:
==================================================
DATA DRIFT DETECTION REPORT
==================================================
```

```
--- Age Feature ---
Training:   mean=40.2, std=9.8
Production: mean=31.8, std=11.9
KS test:    statistic=0.3120, p-value=0.000000
Drift detected: YES

--- Income Feature ---
Training:   mean=74832, std=19876
Production: mean=54921, std=17934
KS test:    statistic=0.4230, p-value=0.000000
Drift detected: YES
```

Let us break down the Kolmogorov-Smirnov (KS) test:

```
KS TEST EXPLAINED:

The KS test compares two distributions by measuring the maximum
distance between their cumulative distribution functions (CDFs).

Training CDF:          Production CDF:
    1.0|    ___           1.0|  ___
       |   /              |  /
       |  /               | /
       | /                |/
    0.0|/___              |___
       0   100            0   100

             Maximum
             distance  = KS statistic
              |
    1.0| __/--|--___
       |/    |     \
       |     |      \  <-- If distance is large,
       |     |       \     distributions are different
    0.0|_____|________\    (drift detected!)

P-value < 0.05 --> Distributions are significantly different
P-value > 0.05 --> No significant difference detected
```

Population Stability Index (PSI)

Another common drift detection method is the Population Stability Index (PSI).

```python
def calculate_psi(reference, current, bins=10):
    """
    Calculate Population Stability Index (PSI).

    PSI measures how much a distribution has shifted.
```

```
    PSI < 0.1:  No significant change
    PSI 0.1-0.2: Moderate change (investigate)
    PSI > 0.2:  Significant change (action needed)
    """
    # Create bins from the reference distribution
    breakpoints = np.percentile(reference, np.linspace(0, 100, bins + 1))
    breakpoints[0] = -np.inf
    breakpoints[-1] = np.inf

    # Calculate proportions in each bin
    ref_counts = np.histogram(reference, bins=breakpoints)[0]
    cur_counts = np.histogram(current, bins=breakpoints)[0]

    # Convert to proportions (add small number to avoid division by zero)
    ref_proportions = (ref_counts + 1) / (len(reference) + bins)
    cur_proportions = (cur_counts + 1) / (len(current) + bins)

    # Calculate PSI
    psi = np.sum(
        (cur_proportions - ref_proportions) *
        np.log(cur_proportions / ref_proportions)
    )

    return psi

# Calculate PSI for each feature
features = {
    "Age": (training_ages, production_ages),
    "Income": (training_incomes, production_incomes),
}

print("=" * 50)
print("POPULATION STABILITY INDEX (PSI)")
print("=" * 50)

for name, (ref, cur) in features.items():
    psi = calculate_psi(ref, cur)

    if psi < 0.1:
        status = "No significant change"
        symbol = "[OK]"
    elif psi < 0.2:
        status = "Moderate change - investigate"
        symbol = "[WARNING]"
    else:
        status = "Significant change - action needed"
        symbol = "[ALERT]"

    print(f"\n{name}:")
    print(f"  PSI = {psi:.4f}")
    print(f"  Status: {symbol} {status}")

# Also show a no-drift example
no_drift_data = np.random.normal(loc=40, scale=10, size=1000)
```

```
psi_no_drift = calculate_psi(training_ages, no_drift_data)
print(f"\nAge (no drift control):")
print(f"  PSI = {psi_no_drift:.4f}")
print(f"  Status: [OK] No significant change")
```

```
Output:
==================================================
POPULATION STABILITY INDEX (PSI)
==================================================

Age:
  PSI = 0.2847
  Status: [ALERT] Significant change - action needed

Income:
  PSI = 0.4123
  Status: [ALERT] Significant change - action needed

Age (no drift control):
  PSI = 0.0089
  Status: [OK] No significant change
```

```
PSI INTERPRETATION GUIDE:

PSI Value     |  What It Means          |  Action
--------------+-------------------------+------------------
< 0.1         |  Distributions similar  |  No action needed
0.1 - 0.2     |  Some change detected   |  Monitor closely
> 0.2         |  Significant shift      |  Retrain model

Think of PSI like a thermometer:
  [OK]      0.0 -------- 0.1  Normal temperature
  [WARNING] 0.1 -------- 0.2  Slight fever, watch it
  [ALERT]   0.2 -------- 1.0+ High fever, take action!
```

Concept Drift Explained

Concept drift happens when the relationship between inputs and outputs changes. The data might look the same, but what it means has changed.

```
CONCEPT DRIFT EXAMPLE:

Before COVID (2019):                After COVID (2020):
High foot traffic                   High foot traffic
```

```
  --> High sales (correct!)          --> Low sales (people just browse)

The INPUT (foot traffic) looks the same.
The OUTPUT (sales) changed.
The RELATIONSHIP between them shifted.

The model still predicts "high traffic = high sales"
but that is no longer true!
```

Think of concept drift like a word changing its meaning. "Cool" used to mean low temperature. Now it also means fashionable. The word (input) is the same, but what it predicts (output) changed.

```python
# Simulate concept drift
np.random.seed(42)

# Training period: simple linear relationship
# High spending score = high purchase probability
n_samples = 500
spending_score = np.random.uniform(0, 100, n_samples)
# Clear relationship: higher score = more likely to buy
purchase_prob_train = 1 / (1 + np.exp(-(spending_score - 50) / 10))
purchases_train = np.random.binomial(1, purchase_prob_train)

# Production period: relationship has changed!
# Now moderate spenders buy more (they shifted to online)
spending_score_prod = np.random.uniform(0, 100, n_samples)
# Changed relationship: moderate scores now buy more
purchase_prob_prod = np.exp(-((spending_score_prod - 40) ** 2) / 800)
purchases_prod = np.random.binomial(1, purchase_prob_prod)

print("CONCEPT DRIFT ANALYSIS")
print("=" * 50)

# The input distributions might look similar (no data drift!)
ks_stat, p_value = stats.ks_2samp(spending_score, spending_score_prod)
print(f"\nInput distribution (spending_score):")
print(f"  Training mean:   {spending_score.mean():.1f}")
print(f"  Production mean: {spending_score_prod.mean():.1f}")
print(f"  KS p-value:      {p_value:.4f}")
print(f"  Data drift:      {'YES' if p_value < 0.05 else 'NO'}")

# But the relationship changed (concept drift!)
print(f"\nOutput distribution (purchase rate):")
print(f"  Training rate:   {purchases_train.mean():.3f}")
print(f"  Production rate: {purchases_prod.mean():.3f}")

# Check relationship change
from sklearn.linear_model import LogisticRegression

# Train model on training data
```

```
model = LogisticRegression()
model.fit(spending_score.reshape(-1, 1), purchases_train)
train_accuracy = model.score(
    spending_score.reshape(-1, 1), purchases_train
)

# Test on production data (same model, new data)
prod_accuracy = model.score(
    spending_score_prod.reshape(-1, 1), purchases_prod
)

print(f"\nModel performance:")
print(f"  Training accuracy:   {train_accuracy:.3f}")
print(f"  Production accuracy: {prod_accuracy:.3f}")
print(f"  Accuracy drop:       {train_accuracy - prod_accuracy:.3f}")
print(f"\n  Concept drift detected: YES")
print(f"  The relationship between spending score and")
print(f"  purchase behavior has fundamentally changed.")
```

```
Output:
CONCEPT DRIFT ANALYSIS
==================================================

Input distribution (spending_score):
  Training mean:   49.8
  Production mean: 50.2
  KS p-value:      0.7823
  Data drift:      NO

Output distribution (purchase rate):
  Training rate:   0.498
  Production rate: 0.312

Model performance:
  Training accuracy:   0.712
  Production accuracy: 0.543
  Accuracy drop:       0.169

  Concept drift detected: YES
  The relationship between spending score and
  purchase behavior has fundamentally changed.
```

```
DATA DRIFT vs CONCEPT DRIFT:

DATA DRIFT:                       CONCEPT DRIFT:
Input changes, relationship       Input same, relationship
stays the same.                   changes.

Training:                         Training:
  X: [1,2,3] --> Y: [2,4,6]        X: [1,2,3] --> Y: [2,4,6]
```

```
Production:                        Production:
  X: [10,20,30] --> Y: [20,40,60] X: [1,2,3] --> Y: [5,3,1]
     ^^^                                Same!         ^^^
     Different inputs!                                Different outputs!
     Same pattern (Y = 2X)                            Different pattern!
```

Performance Degradation Monitoring

The most direct way to detect problems is to monitor model performance over time.

```python
import numpy as np
from datetime import datetime, timedelta

# Simulate model performance over 12 weeks
np.random.seed(42)

weeks = 12
dates = [datetime(2024, 1, 1) + timedelta(weeks=w) for w in range(weeks)]

# Simulate accuracy that degrades over time
# Weeks 1-4: stable performance
# Weeks 5-8: gradual degradation
# Weeks 9-12: significant degradation
base_accuracy = 0.92
accuracies = []
precisions = []
recalls = []

for week in range(weeks):
    if week < 4:
        # Stable period
        acc = base_accuracy + np.random.normal(0, 0.01)
        prec = 0.90 + np.random.normal(0, 0.01)
        rec = 0.88 + np.random.normal(0, 0.01)
    elif week < 8:
        # Gradual degradation
        degradation = (week - 4) * 0.02
        acc = base_accuracy - degradation + np.random.normal(0, 0.01)
        prec = 0.90 - degradation + np.random.normal(0, 0.01)
        rec = 0.88 - degradation * 1.2 + np.random.normal(0, 0.01)
    else:
        # Significant degradation
        degradation = 0.08 + (week - 8) * 0.03
        acc = base_accuracy - degradation + np.random.normal(0, 0.01)
        prec = 0.90 - degradation + np.random.normal(0, 0.01)
        rec = 0.88 - degradation * 1.5 + np.random.normal(0, 0.01)
```

```
    accuracies.append(round(acc, 3))
    precisions.append(round(prec, 3))
    recalls.append(round(rec, 3))

# Display performance dashboard
print("=" * 70)
print("MODEL PERFORMANCE MONITORING DASHBOARD")
print("=" * 70)
print(f"{'Week':<8} {'Date':<14} {'Accuracy':<12} {'Precision':<12} "
      f"{'Recall':<12} {'Status'}")
print("-" * 70)

for i in range(weeks):
    acc = accuracies[i]
    if acc >= 0.88:
        status = "[OK]"
    elif acc >= 0.82:
        status = "[WARNING]"
    else:
        status = "[ALERT]"

    print(f"Week {i+1:<3} {dates[i].strftime('%Y-%m-%d'):<14} "
          f"{acc:<12.3f} {precisions[i]:<12.3f} "
          f"{recalls[i]:<12.3f} {status}")

# Summary
print("\n" + "-" * 70)
print("SUMMARY:")
print(f"  Initial accuracy:  {accuracies[0]:.3f}")
print(f"  Current accuracy:  {accuracies[-1]:.3f}")
print(f"  Total degradation: {accuracies[0] - accuracies[-1]:.3f}")
print(f"  Recommendation:    Model retraining required!")
```

```
Output:
======================================================================
MODEL PERFORMANCE MONITORING DASHBOARD
======================================================================
Week     Date           Accuracy     Precision    Recall       Status
----------------------------------------------------------------------
Week 1   2024-01-01     0.924        0.903        0.878        [OK]
Week 2   2024-01-08     0.919        0.897        0.882        [OK]
Week 3   2024-01-15     0.911        0.909        0.885        [OK]
Week 4   2024-01-22     0.928        0.894        0.876        [OK]
Week 5   2024-01-29     0.903        0.884        0.861        [OK]
Week 6   2024-02-05     0.878        0.862        0.837        [WARNING]
Week 7   2024-02-12     0.861        0.848        0.812        [WARNING]
Week 8   2024-02-19     0.842        0.831        0.793        [WARNING]
Week 9   2024-02-26     0.831        0.812        0.768        [WARNING]
Week 10  2024-03-04     0.798        0.784        0.731        [ALERT]
Week 11  2024-03-11     0.773        0.756        0.698        [ALERT]
Week 12  2024-03-18     0.749        0.738        0.667        [ALERT]
```

```
----------------------------------------------------------------------
SUMMARY:
  Initial accuracy:  0.924
  Current accuracy:  0.749
  Total degradation: 0.175
  Recommendation:    Model retraining required!
```

```
PERFORMANCE DEGRADATION VISUALIZED (ASCII):

Accuracy over 12 weeks:

1.00 |
0.95 |
0.90 | * * * *                   <-- Stable period
0.85 |         * *
0.80 |           * *             <-- Degradation begins
0.75 |               * *
0.70 |                   * *     <-- Alert threshold
0.65 |                           <-- Action needed!
     +--+--+--+--+--+--+--+--+--+--+--+--+
     W1 W2 W3 W4 W5 W6 W7 W8 W9 W10W11W12

     |--- OK ---|-- WARNING --|--- ALERT ---|
```

Building a Monitoring System

Let us build a complete monitoring system that tracks multiple metrics and raises alerts.

```python
class ModelMonitor:
    """
    A simple model monitoring system.

    Tracks predictions, compares distributions,
    and raises alerts when problems are detected.
    """

    def __init__(self, model_name, reference_data, reference_labels,
                 reference_predictions):
        """
        Initialize the monitor with reference (training) data.

        Parameters:
        -----------
        model_name : str
            Name of the model being monitored
```

```
        reference_data : numpy array
            Feature values from training data
        reference_labels : numpy array
            True labels from training data
        reference_predictions : numpy array
            Model predictions on training data
        """
        self.model_name = model_name
        self.reference_data = reference_data
        self.reference_labels = reference_labels
        self.reference_predictions = reference_predictions
        self.alerts = []

        # Calculate reference statistics
        self.ref_means = np.mean(reference_data, axis=0)
        self.ref_stds = np.std(reference_data, axis=0)
        self.ref_prediction_rate = np.mean(reference_predictions)

    def check_data_drift(self, current_data, feature_names,
                         threshold=0.05):
        """
        Check each feature for data drift using the KS test.
        """
        print("\n--- Data Drift Check ---")
        drift_detected = False

        for i, name in enumerate(feature_names):
            ks_stat, p_value = stats.ks_2samp(
                self.reference_data[:, i],
                current_data[:, i]
            )

            if p_value < threshold:
                drift_detected = True
                status = "[DRIFT]"
                self.alerts.append(
                    f"Data drift in '{name}': "
                    f"KS={ks_stat:.3f}, p={p_value:.6f}"
                )
            else:
                status = "[OK]"

            ref_mean = self.reference_data[:, i].mean()
            cur_mean = current_data[:, i].mean()
            change_pct = ((cur_mean - ref_mean) / ref_mean) * 100

            print(f"  {name:25s} {status} "
                  f"(mean: {ref_mean:.2f} -> {cur_mean:.2f}, "
                  f"change: {change_pct:+.1f}%)")

        return drift_detected

    def check_prediction_drift(self, current_predictions, threshold=0.1):
        """
        Check if prediction distribution has shifted.
```

```
        """
        print("\n--- Prediction Drift Check ---")

        ref_rate = self.ref_prediction_rate
        cur_rate = np.mean(current_predictions)
        change = abs(cur_rate - ref_rate)

        print(f"  Reference prediction rate: {ref_rate:.3f}")
        print(f"  Current prediction rate:   {cur_rate:.3f}")
        print(f"  Change:                    {change:.3f}")

        if change > threshold:
            print(f"  Status: [DRIFT] Prediction distribution shifted!")
            self.alerts.append(
                f"Prediction drift: rate changed from "
                f"{ref_rate:.3f} to {cur_rate:.3f}"
            )
            return True
        else:
            print(f"  Status: [OK]")
            return False

    def check_performance(self, current_labels, current_predictions,
                          accuracy_threshold=0.85):
        """
        Check if model performance has degraded.
        """
        print("\n--- Performance Check ---")

        from sklearn.metrics import accuracy_score, precision_score
        from sklearn.metrics import recall_score

        ref_accuracy = accuracy_score(
            self.reference_labels, self.reference_predictions
        )
        cur_accuracy = accuracy_score(current_labels, current_predictions)

        print(f"  Reference accuracy: {ref_accuracy:.3f}")
        print(f"  Current accuracy:   {cur_accuracy:.3f}")
        print(f"  Threshold:          {accuracy_threshold:.3f}")

        if cur_accuracy < accuracy_threshold:
            print(f"  Status: [ALERT] Performance below threshold!")
            self.alerts.append(
                f"Performance degradation: accuracy dropped from "
                f"{ref_accuracy:.3f} to {cur_accuracy:.3f}"
            )
            return True
        elif cur_accuracy < ref_accuracy - 0.05:
            print(f"  Status: [WARNING] Performance declining")
            return True
        else:
            print(f"  Status: [OK]")
            return False
```

```
    def generate_report(self):
        """Generate a monitoring report."""
        print("\n" + "=" * 60)
        print("MONITORING REPORT")
        print("=" * 60)
        print(f"Model: {self.model_name}")
        print(f"Alerts: {len(self.alerts)}")

        if self.alerts:
            print("\nAlert Details:")
            for i, alert in enumerate(self.alerts, 1):
                print(f"  {i}. {alert}")

            print("\nRecommendation: Investigate alerts and consider "
                  "retraining the model.")
        else:
            print("\nAll checks passed. Model is healthy.")

        return self.alerts


# Create reference data (training period)
np.random.seed(42)
n_ref = 1000
n_features = 4
feature_names = ["age", "income", "purchase_count", "session_duration"]

ref_data = np.column_stack([
    np.random.normal(35, 10, n_ref),     # age
    np.random.normal(60000, 15000, n_ref), # income
    np.random.normal(10, 5, n_ref),       # purchase_count
    np.random.normal(300, 100, n_ref),    # session_duration
])
ref_labels = np.random.choice([0, 1], n_ref, p=[0.6, 0.4])
ref_predictions = ref_labels.copy()
# Add some errors to make it realistic
error_indices = np.random.choice(n_ref, size=int(n_ref * 0.08), replace=False)
ref_predictions[error_indices] = 1 - ref_predictions[error_indices]

# Create production data with drift
n_prod = 500
prod_data = np.column_stack([
    np.random.normal(28, 12, n_prod),      # age shifted younger
    np.random.normal(45000, 12000, n_prod), # income shifted lower
    np.random.normal(8, 6, n_prod),         # purchase_count similar
    np.random.normal(280, 110, n_prod),     # session_duration similar
])
prod_labels = np.random.choice([0, 1], n_prod, p=[0.5, 0.5])
prod_predictions = prod_labels.copy()
error_indices = np.random.choice(n_prod, size=int(n_prod * 0.2), replace=False)
prod_predictions[error_indices] = 1 - prod_predictions[error_indices]

# Run monitoring
monitor = ModelMonitor(
    model_name="purchase_predictor_v1",
```

```
    reference_data=ref_data,
    reference_labels=ref_labels,
    reference_predictions=ref_predictions,
)

print("=" * 60)
print("RUNNING MODEL MONITORING CHECKS")
print("=" * 60)

monitor.check_data_drift(prod_data, feature_names)
monitor.check_prediction_drift(prod_predictions)
monitor.check_performance(prod_labels, prod_predictions)
monitor.generate_report()
```

```
Output:
============================================================
RUNNING MODEL MONITORING CHECKS
============================================================

--- Data Drift Check ---
  age                       [DRIFT] (mean: 35.12 -> 27.89, change: -20.6%)
  income                    [DRIFT] (mean: 59834.00 -> 44876.00, change: -25.0%)
  purchase_count            [OK] (mean: 10.23 -> 8.12, change: -20.6%)
  session_duration          [OK] (mean: 298.45 -> 281.23, change: -5.8%)

--- Prediction Drift Check ---
  Reference prediction rate: 0.432
  Current prediction rate:   0.540
  Change:                    0.108
  Status: [DRIFT] Prediction distribution shifted!

--- Performance Check ---
  Reference accuracy: 0.920
  Current accuracy:   0.800
  Threshold:          0.850
  Status: [ALERT] Performance below threshold!

============================================================
MONITORING REPORT
============================================================
Model: purchase_predictor_v1
Alerts: 4

Alert Details:
  1. Data drift in 'age': KS=0.312, p=0.000000
  2. Data drift in 'income': KS=0.423, p=0.000000
  3. Prediction drift: rate changed from 0.432 to 0.540
  4. Performance degradation: accuracy dropped from 0.920 to 0.800

Recommendation: Investigate alerts and consider retraining the model.
```

Setting Up Alerts

A monitoring system is useless without alerts. Here is how to set up automated alerting.

```python
from datetime import datetime
from enum import Enum

class AlertSeverity(Enum):
    """Alert severity levels."""
    INFO = "INFO"
    WARNING = "WARNING"
    CRITICAL = "CRITICAL"

class AlertManager:
    """
    Manages monitoring alerts and notifications.

    In production, this would send emails, Slack messages,
    or PagerDuty alerts. Here we demonstrate the logic.
    """

    def __init__(self, model_name):
        self.model_name = model_name
        self.alert_history = []
        self.thresholds = {
            "accuracy_warning": 0.85,
            "accuracy_critical": 0.75,
            "drift_psi_warning": 0.1,
            "drift_psi_critical": 0.2,
            "prediction_shift_warning": 0.10,
            "prediction_shift_critical": 0.20,
        }

    def evaluate_metric(self, metric_name, value):
        """
        Evaluate a metric and create alerts if needed.
        """
        alert = None

        if metric_name == "accuracy":
            if value < self.thresholds["accuracy_critical"]:
                alert = self._create_alert(
                    AlertSeverity.CRITICAL,
                    f"Accuracy critically low: {value:.3f} "
                    f"(threshold: {self.thresholds['accuracy_critical']})"
                )
            elif value < self.thresholds["accuracy_warning"]:
                alert = self._create_alert(
                    AlertSeverity.WARNING,
                    f"Accuracy below warning threshold: {value:.3f} "
                    f"(threshold: {self.thresholds['accuracy_warning']})"
                )
```

```
        elif metric_name == "drift_psi":
            if value > self.thresholds["drift_psi_critical"]:
                alert = self._create_alert(
                    AlertSeverity.CRITICAL,
                    f"Significant data drift detected: PSI={value:.3f}"
                )
            elif value > self.thresholds["drift_psi_warning"]:
                alert = self._create_alert(
                    AlertSeverity.WARNING,
                    f"Moderate data drift detected: PSI={value:.3f}"
                )

        if alert:
            self.alert_history.append(alert)

        return alert

    def _create_alert(self, severity, message):
        """Create an alert dictionary."""
        return {
            "timestamp": datetime.now().isoformat(),
            "model": self.model_name,
            "severity": severity.value,
            "message": message,
        }

    def send_notification(self, alert):
        """
        Simulate sending notifications.
        In production: email, Slack, PagerDuty, etc.
        """
        severity = alert["severity"]
        message = alert["message"]

        if severity == "CRITICAL":
            channel = "PagerDuty + Slack #ml-alerts + Email"
        elif severity == "WARNING":
            channel = "Slack #ml-alerts"
        else:
            channel = "Logged only"

        print(f"  [{severity}] {message}")
        print(f"  --> Notified via: {channel}")
        return channel


# Demonstrate the alert system
alert_mgr = AlertManager("purchase_predictor_v1")

print("=" * 60)
print("ALERT SYSTEM DEMONSTRATION")
print("=" * 60)

# Simulate metrics arriving over time
```

```
metrics_over_time = [
    {"week": 1, "accuracy": 0.92, "drift_psi": 0.05},
    {"week": 2, "accuracy": 0.91, "drift_psi": 0.06},
    {"week": 3, "accuracy": 0.88, "drift_psi": 0.08},
    {"week": 4, "accuracy": 0.84, "drift_psi": 0.12},  # Warning
    {"week": 5, "accuracy": 0.79, "drift_psi": 0.18},  # Warning
    {"week": 6, "accuracy": 0.72, "drift_psi": 0.25},  # Critical
]

for metrics in metrics_over_time:
    week = metrics["week"]
    print(f"\nWeek {week}:")

    acc_alert = alert_mgr.evaluate_metric("accuracy", metrics["accuracy"])
    drift_alert = alert_mgr.evaluate_metric("drift_psi", metrics["drift_psi"])

    if acc_alert:
        alert_mgr.send_notification(acc_alert)
    if drift_alert:
        alert_mgr.send_notification(drift_alert)

    if not acc_alert and not drift_alert:
        print(f"  All metrics within normal range")
        print(f"  Accuracy: {metrics['accuracy']:.3f}, "
              f"PSI: {metrics['drift_psi']:.3f}")

print(f"\n{'=' * 60}")
print(f"Total alerts generated: {len(alert_mgr.alert_history)}")
```

```
Output:
============================================================
ALERT SYSTEM DEMONSTRATION
============================================================

Week 1:
  All metrics within normal range
  Accuracy: 0.920, PSI: 0.050

Week 2:
  All metrics within normal range
  Accuracy: 0.910, PSI: 0.060

Week 3:
  All metrics within normal range
  Accuracy: 0.880, PSI: 0.080

Week 4:
  [WARNING] Accuracy below warning threshold: 0.840 (threshold: 0.85)
  --> Notified via: Slack #ml-alerts
  [WARNING] Moderate data drift detected: PSI=0.120
  --> Notified via: Slack #ml-alerts

Week 5:
```

```
  [WARNING] Accuracy below warning threshold: 0.790 (threshold: 0.85)
  --> Notified via: Slack #ml-alerts
  [WARNING] Moderate data drift detected: PSI=0.180
  --> Notified via: Slack #ml-alerts

Week 6:
  [CRITICAL] Accuracy critically low: 0.720 (threshold: 0.75)
  --> Notified via: PagerDuty + Slack #ml-alerts + Email
  [CRITICAL] Significant data drift detected: PSI=0.250
  --> Notified via: PagerDuty + Slack #ml-alerts + Email

============================================================
Total alerts generated: 6
```

Evidently AI Overview

Evidently AI is an open source library specifically designed for ML model monitoring. It generates beautiful reports and dashboards for drift detection and model performance.

```python
# pip install evidently

# Demonstrate the concepts Evidently provides
# In production, you would use Evidently's actual Report classes

def evidently_style_report(reference_df, current_df, feature_columns):
    """
    Simulate an Evidently-style data drift report.

    In real Evidently:
        from evidently.report import Report
        from evidently.metric_preset import DataDriftPreset

        report = Report(metrics=[DataDriftPreset()])
        report.run(reference_data=ref_df, current_data=cur_df)
        report.show()
    """
    print("=" * 65)
    print("EVIDENTLY-STYLE DATA DRIFT REPORT")
    print("=" * 65)
    print(f"Reference data:  {len(reference_df)} samples")
    print(f"Current data:    {len(current_df)} samples")
    print(f"Features tested: {len(feature_columns)}")
    print()

    drift_count = 0
```

```
    for col in feature_columns:
        ref_values = reference_df[col].values
        cur_values = current_df[col].values

        # KS test for numerical features
        ks_stat, p_value = stats.ks_2samp(ref_values, cur_values)
        is_drift = p_value < 0.05
        if is_drift:
            drift_count += 1

        ref_mean = ref_values.mean()
        cur_mean = cur_values.mean()
        ref_std = ref_values.std()
        cur_std = cur_values.std()

        print(f"Feature: {col}")
        print(f"  Reference: mean={ref_mean:.2f}, std={ref_std:.2f}")
        print(f"  Current:   mean={cur_mean:.2f}, std={cur_std:.2f}")
        print(f"  KS stat:   {ks_stat:.4f}")
        print(f"  P-value:   {p_value:.6f}")
        print(f"  Drift:     {'DETECTED' if is_drift else 'Not detected'}")

        # Simple ASCII histogram
        print(f"  Distribution:")
        ref_hist, edges = np.histogram(ref_values, bins=10)
        cur_hist, _ = np.histogram(cur_values, bins=edges)
        max_count = max(max(ref_hist), max(cur_hist))

        if max_count > 0:
            ref_bars = [int(c / max_count * 10) for c in ref_hist]
            cur_bars = [int(c / max_count * 10) for c in cur_hist]
            print(f"    Ref: {''.join(['▇' * b + ' ' for b in ref_bars])}")
            print(f"    Cur: {''.join(['▇' * b + ' ' for b in cur_bars])}")

        print()

    # Summary
    drift_pct = (drift_count / len(feature_columns)) * 100
    print("-" * 65)
    print(f"SUMMARY: {drift_count}/{len(feature_columns)} features "
          f"show drift ({drift_pct:.0f}%)")

    if drift_pct > 50:
        print("STATUS: [ALERT] Significant drift across multiple features")
    elif drift_pct > 20:
        print("STATUS: [WARNING] Some features showing drift")
    else:
        print("STATUS: [OK] Minor or no drift detected")

    return drift_count

# Create sample dataframes
np.random.seed(42)
n = 1000
```

```
reference_df = pd.DataFrame({
    "age": np.random.normal(35, 10, n),
    "income": np.random.normal(60000, 15000, n),
    "purchase_frequency": np.random.normal(5, 2, n),
    "avg_session_minutes": np.random.normal(15, 5, n),
})

current_df = pd.DataFrame({
    "age": np.random.normal(28, 12, n),           # Shifted
    "income": np.random.normal(48000, 12000, n),   # Shifted
    "purchase_frequency": np.random.normal(5.2, 2.1, n),  # Similar
    "avg_session_minutes": np.random.normal(14.5, 5.5, n), # Similar
})

feature_columns = ["age", "income", "purchase_frequency",
                   "avg_session_minutes"]

drift_count = evidently_style_report(reference_df, current_df,
                                     feature_columns)
```

```
Output:
================================================================
EVIDENTLY-STYLE DATA DRIFT REPORT
================================================================
Reference data:  1000 samples
Current data:    1000 samples
Features tested: 4

Feature: age
  Reference: mean=35.12, std=9.87
  Current:   mean=27.89, std=11.92
  KS stat:   0.2890
  P-value:   0.000000
  Drift:     DETECTED
  Distribution:
    Ref: ▆▁ ▅▅▅▅ ▇▇▇▇▇▇▇▇▇ ▇▇▇▇▇▇▇▇ ▆▆▆▆▆▆ ▅▅▅▅ ▆▆ ▆▁
    Cur: ▅▅▅▅ ▆▆▆▆▆▆ ▇▇▇▇▇▇▇▇ ▇▇▇▇▇▇▇▇▇▇▇ ▆▆▆▆▆▆ ▅▅▅▅ ▆▆

Feature: income
  Reference: mean=59834.23, std=14876.45
  Current:   mean=47923.67, std=11987.34
  KS stat:   0.3450
  P-value:   0.000000
  Drift:     DETECTED
  Distribution:
    Ref: ▆▁ ▆▆ ▅▅▅▅ ▇▇▇▇▇▇▇▇ ▇▇▇▇▇▇▇▇▇ ▇▇▇▇▇▇▇▇ ▅▅▅▅ ▆▆ ▆▁
    Cur: ▆▆ ▆▆▆▆▆▆ ▇▇▇▇▇▇▇▇▇▇▇ ▇▇▇▇▇▇▇▇▇ ▆▆▆▆▆▆ ▆▆ ▆▁

Feature: purchase_frequency
  Reference: mean=5.02, std=2.01
  Current:   mean=5.18, std=2.09
  KS stat:   0.0340
  P-value:   0.612300
```

```
  Drift:     Not detected
  Distribution:
    Ref: ▇ ▆▆ ▅▅▅ ▆▆▆▆▆▆▆ ▆▆▆▆▆▆▆▆▆ ▆▆▆▆▆▆▆ ▅▅▅ ▆▆ ▇
    Cur: ▇ ▆▆ ▅▅▅ ▆▆▆▆▆▆▆ ▆▆▆▆▆▆▆▆▆ ▆▆▆▆▆▆▆ ▅▅▅ ▆▆ ▇

Feature: avg_session_minutes
  Reference: mean=15.12, std=4.98
  Current:   mean=14.56, std=5.43
  KS stat:   0.0450
  P-value:   0.289700
  Drift:     Not detected
  Distribution:
    Ref: ▇ ▆▆ ▆▆▆▆▆▆▆ ▆▆▆▆▆▆▆▆▆ ▆▆▆▆▆▆▆▆▆ ▆▆▆▆▆ ▅▅▅ ▇
    Cur: ▇ ▆▆▆ ▆▆▆▆▆▆▆ ▆▆▆▆▆▆▆▆▆ ▆▆▆▆▆▆▆▆▆ ▆▆▆▆▆ ▆▆ ▇

----------------------------------------------------------------
SUMMARY: 2/4 features show drift (50%)
STATUS: [WARNING] Some features showing drift
```

Using Evidently in Production

```python
# How to use Evidently in a real monitoring pipeline

def setup_evidently_monitoring():
    """
    Show the code structure for Evidently monitoring.
    """
    monitoring_code = '''
# ============================================================
# Real Evidently Monitoring Setup
# ============================================================

from evidently.report import Report
from evidently.metric_preset import (
    DataDriftPreset,
    TargetDriftPreset,
    ClassificationPreset,
)
from evidently.test_suite import TestSuite
from evidently.tests import (
    TestShareOfDriftedColumns,
    TestColumnDrift,
)

# --- Step 1: Create a Data Drift Report ---
drift_report = Report(metrics=[
    DataDriftPreset(),        # Check all features for drift
])

drift_report.run(
```

```
    reference_data=reference_df,  # Training data
    current_data=current_df,      # Production data
)

# Save as HTML (opens in browser)
drift_report.save_html("drift_report.html")

# --- Step 2: Create a Classification Report ---
classification_report = Report(metrics=[
    ClassificationPreset(),   # Accuracy, precision, recall, etc.
])

classification_report.run(
    reference_data=reference_with_labels,
    current_data=current_with_labels,
)

# --- Step 3: Create Automated Tests ---
test_suite = TestSuite(tests=[
    TestShareOfDriftedColumns(lt=0.3),  # Less than 30% drift
    TestColumnDrift("age"),              # Check specific column
    TestColumnDrift("income"),
])

test_suite.run(
    reference_data=reference_df,
    current_data=current_df,
)

# Check if tests passed
results = test_suite.as_dict()
all_passed = all(
    test["status"] == "SUCCESS"
    for test in results["tests"]
)

if not all_passed:
    send_alert("Evidently tests failed! Check drift report.")
'''

    print(monitoring_code)

print("EVIDENTLY MONITORING CODE EXAMPLE:")
print("=" * 60)
setup_evidently_monitoring()
```

```
Output:
EVIDENTLY MONITORING CODE EXAMPLE:
============================================================

# ============================================================
# Real Evidently Monitoring Setup
# ============================================================
```

```
from evidently.report import Report
from evidently.metric_preset import (
    DataDriftPreset,
    TargetDriftPreset,
    ClassificationPreset,
)
from evidently.test_suite import TestSuite
from evidently.tests import (
    TestShareOfDriftedColumns,
    TestColumnDrift,
)

# --- Step 1: Create a Data Drift Report ---
drift_report = Report(metrics=[
    DataDriftPreset(),        # Check all features for drift
])

drift_report.run(
    reference_data=reference_df,  # Training data
    current_data=current_df,      # Production data
)

# Save as HTML (opens in browser)
drift_report.save_html("drift_report.html")

# --- Step 2: Create a Classification Report ---
classification_report = Report(metrics=[
    ClassificationPreset(),   # Accuracy, precision, recall, etc.
])

classification_report.run(
    reference_data=reference_with_labels,
    current_data=current_with_labels,
)

# --- Step 3: Create Automated Tests ---
test_suite = TestSuite(tests=[
    TestShareOfDriftedColumns(lt=0.3),  # Less than 30% drift
    TestColumnDrift("age"),              # Check specific column
    TestColumnDrift("income"),
])

test_suite.run(
    reference_data=reference_df,
    current_data=current_df,
)

# Check if tests passed
results = test_suite.as_dict()
all_passed = all(
    test["status"] == "SUCCESS"
    for test in results["tests"]
)
```

```
if not all_passed:
    send_alert("Evidently tests failed! Check drift report.")
```

Monitoring Workflow

Here is a complete monitoring workflow for production.

```
PRODUCTION MONITORING WORKFLOW:

  +-------------------+
  | Production Data   |
  | (daily batches)   |
  +--------+----------+
           |
           v
  +--------+----------+
  | Compare with      |
  | Reference Data    |
  +--------+----------+
           |
     +-----+-----+
     |           |
     v           v
+----+----+ +----+----+
| Data    | |Prediction|
| Drift   | | Drift   |
| Check   | | Check   |
+----+----+ +----+----+
     |           |
     +-----+-----+
           |
           v
  +--------+----------+
  | Performance       |  <-- If labels available
  | Evaluation        |
  +--------+----------+
           |
     +-----+------+
     |            |
     v            v
  +--+---+   +---+--------+
  | OK   |   | ALERT      |
  |      |   |            |
  | Log  |   | Notify     |
  | only |   | team       |
  +------+   | Retrain?   |
              +------------+
```

Common Mistakes

1. Only monitoring accuracy — Accuracy can look fine while specific groups of users get terrible predictions. Monitor per-segment performance too.
2. No reference data — You need a baseline to compare against. Always save a snapshot of your training data distribution.
3. Monitoring too infrequently — Checking weekly when data changes daily means problems go unnoticed. Match monitoring frequency to data change frequency.
4. Ignoring prediction distribution — Even without ground truth labels, you can detect problems by watching if the distribution of predictions changes.
5. Alert fatigue — Too many alerts and the team ignores them all. Set meaningful thresholds and use severity levels.

Best Practices

1. Start monitoring from day one — Do not wait for problems. Set up monitoring when you deploy the model.
2. Save reference distributions — Store the training data distribution statistics when you train. You need them for comparison.
3. Monitor inputs AND outputs — Data drift catches input changes. Prediction drift catches output changes. You need both.
4. Use multiple detection methods — No single test catches everything. Combine KS test, PSI, and statistical tests.
5. Set up automated retraining triggers — When monitoring detects significant drift, automatically trigger model retraining (see Chapter 13).

Quick Summary

Model monitoring watches for three types of problems. Data drift means the input data distribution changed (customers got younger, incomes dropped). Concept drift means the relationship between inputs and outputs changed (the same features now predict different

outcomes). Performance degradation means model accuracy is declining over time.

Detection methods include the Kolmogorov-Smirnov test (compares two distributions), the Population Stability Index (measures distribution shift), and direct performance tracking. Evidently AI is an open source library that automates drift detection and generates monitoring reports. A good monitoring system includes alerts with severity levels, notification channels, and automated responses.

Key Points

- Data drift occurs when input feature distributions change from what the model was trained on
- Concept drift occurs when the relationship between features and targets changes
- Performance degradation is the gradual decline in model accuracy over time
- The KS test compares two distributions and returns a p-value (below 0.05 indicates drift)
- PSI values below 0.1 mean no significant change; above 0.2 means action is needed
- You can detect problems even without ground truth labels by monitoring prediction distributions
- Evidently AI provides automated drift detection and visual monitoring reports
- Alerts should have severity levels (INFO, WARNING, CRITICAL) with appropriate notification channels
- Reference data from the training period must be saved for comparison
- Monitoring frequency should match how quickly your data changes

---

Practice Questions

1. What is the difference between data drift and concept drift? Give a real-world example of each.
2. A model's accuracy is stable at 90%, but the PSI for the "age" feature increased from 0.05 to 0.25. Should you be concerned? Why?
3. Your model predicts fraud. During training, 2% of transactions were fraudulent. Now the prediction rate is 8%. What could be happening, and what would you investigate?
4. Why is it important to monitor prediction distributions even when you do not have ground truth labels?

5. You set up alerts but your team receives 50 alerts per day and starts ignoring them. How would you fix this?

Exercises

Exercise 1: Build a Drift Detector

Write a Python class that:

1. Takes reference data as input during initialization
2. Has a `detect_drift()` method that accepts new data
3. Returns a report with KS test results and PSI for each feature
4. Flags features with significant drift

Exercise 2: Simulate and Detect Concept Drift

Create a simulation where:

1. A model is trained on data where feature X has a positive correlation with the target
2. Over time, the correlation gradually reverses (becomes negative)
3. Implement a sliding window approach that detects when the relationship changes
4. Plot or print the correlation over time and mark when drift is detected

Exercise 3: Set Up a Monitoring Dashboard

Create a monitoring dashboard that:

1. Tracks accuracy, precision, recall, and F1 score over 10 time periods
2. Includes PSI calculations for 3 features
3. Generates alerts based on configurable thresholds
4. Produces a summary report with recommendations

What Is Next?

Now that you can monitor models and detect when they degrade, a natural question arises: how do you know if a new model is actually better than the current one? In the next chapter,

we will explore A/B Testing for Models — how to safely compare two models in production by splitting traffic and measuring statistical significance.

Chapter 16: A/B Testing for ML Models

What You Will Learn

In this chapter, you will learn:

- Why A/B testing is essential for deploying new models safely
- How to split traffic between models correctly
- What statistical significance means and how to calculate it
- The champion/challenger pattern for model comparison
- How to implement A/B testing in a production API
- How to analyze A/B test results and make decisions

Why This Chapter Matters

You trained a new model. Offline metrics look great — accuracy improved from 89% to 93%. You deploy it, and revenue drops 15%. What happened?

Offline metrics do not always translate to real-world performance. A model that is more accurate on a test set might make worse predictions on the specific customers who generate the most revenue. Or it might be slightly slower, causing users to leave before the prediction loads.

A/B testing lets you compare models using real users and real outcomes. Instead of guessing whether the new model is better, you measure it directly. It is like a clinical trial for your ML model — you test the new treatment on a group of patients while keeping a control group on the existing treatment.

Think of it like a restaurant testing a new recipe. You do not replace the entire menu. You serve the new dish to some customers and the old dish to others, then compare which one

gets better reviews and more orders.

What Is A/B Testing?

A/B testing is a controlled experiment where you split users into two groups:

- Group A (Control) — Receives predictions from the current model (the champion)
- Group B (Treatment) — Receives predictions from the new model (the challenger)

```
A/B TEST SETUP:

                    +-- 80% of traffic --> Model A (Champion)
                    |                      Current production model
User Request -------+
                    |
                    +-- 20% of traffic --> Model B (Challenger)
                                           New model to evaluate

After enough data is collected:
  Compare outcomes --> Decide which model is better
```

Why Not Just Deploy the New Model?

```
RISK WITHOUT A/B TESTING:

Day 1: Deploy new model to 100% of traffic
Day 2: Revenue starts dropping
Day 3: Revenue drops more
Day 7: Someone notices the problem
Day 8: Roll back to old model
Day 9: Investigate what went wrong

DAMAGE: 7 days of lost revenue!

WITH A/B TESTING:

Day 1: Deploy new model to 10% of traffic
Day 2: Notice 10% group has lower revenue
Day 3: Stop the test, no rollback needed
Day 4: Investigate and fix

DAMAGE: 10% of 2 days = minimal impact
```

Traffic Splitting

The first step in A/B testing is deciding how to split traffic between models.

```python
import hashlib
import random
import numpy as np
from collections import Counter

class TrafficSplitter:
    """
    Split traffic between models consistently.

    Uses deterministic hashing so the same user always
    gets the same model (consistency is important!).
    """

    def __init__(self, splits):
        """
        Initialize with traffic splits.

        Parameters:
        -----------
        splits : dict
            Model name to traffic percentage.
            Must sum to 100.

        Example: {"champion": 80, "challenger": 20}
        """
        total = sum(splits.values())
        if total != 100:
            raise ValueError(f"Splits must sum to 100, got {total}")

        self.splits = splits
        self._build_ranges()

    def _build_ranges(self):
        """Build ranges for each model."""
        self.ranges = {}
        current = 0
        for model, percentage in self.splits.items():
            self.ranges[model] = (current, current + percentage)
            current += percentage

    def assign_model(self, user_id):
        """
        Assign a user to a model deterministically.

        The same user_id always gets the same model.
        This is important because you do not want a user
        switching between models on every request.
        """
```

```
        # Hash the user ID to get a number between 0-99
        hash_value = hashlib.md5(str(user_id).encode()).hexdigest()
        bucket = int(hash_value, 16) % 100

        # Find which model this bucket belongs to
        for model, (start, end) in self.ranges.items():
            if start <= bucket < end:
                return model

        return list(self.splits.keys())[0]  # Fallback


# Create an 80/20 split
splitter = TrafficSplitter({"champion": 80, "challenger": 20})

# Test with many users
assignments = Counter()
for user_id in range(10000):
    model = splitter.assign_model(user_id)
    assignments[model] += 1

print("TRAFFIC SPLITTING RESULTS")
print("=" * 40)
print(f"Total users: 10,000")
for model, count in assignments.items():
    pct = count / 10000 * 100
    print(f"  {model}: {count} users ({pct:.1f}%)")

# Verify consistency - same user always gets same model
print("\nConsistency check:")
for user_id in [42, 123, 456]:
    results = set()
    for _ in range(100):
        results.add(splitter.assign_model(user_id))
    print(f"  User {user_id}: always gets '{results.pop()}' "
          f"(consistent: {len(results) == 0})")
```

```
Output:
TRAFFIC SPLITTING RESULTS
========================================
Total users: 10,000
  champion: 7,982 users (79.8%)
  challenger: 2,018 users (20.2%)

Consistency check:
  User 42: always gets 'champion' (consistent: True)
  User 123: always gets 'challenger' (consistent: True)
  User 456: always gets 'champion' (consistent: True)
```

```
WHY CONSISTENT ASSIGNMENT MATTERS:
```

```
BAD (Random per request):
  User 42, Request 1 --> Model A --> "Buy this product"
  User 42, Request 2 --> Model B --> "Buy that product"
  User 42, Request 3 --> Model A --> "Buy this product"
  (Confusing experience! Can't measure either model properly)

GOOD (Deterministic per user):
  User 42, Request 1 --> Model A --> "Buy this product"
  User 42, Request 2 --> Model A --> "Buy this product"
  User 42, Request 3 --> Model A --> "Buy this product"
  (Consistent experience. Can measure Model A's impact on User 42)
```

Statistical Significance

You cannot just look at which model has a higher metric and declare a winner. You need statistical significance — confidence that the difference is real and not due to random chance.

Why Statistical Significance Matters

```
THE COIN FLIP ANALOGY:

You flip two coins 5 times each:
  Coin A: 3 heads (60%)
  Coin B: 2 heads (40%)

Is Coin A better? NO! The sample is too small.
The difference is just random chance.

You flip two coins 10,000 times each:
  Coin A: 5,200 heads (52%)
  Coin B: 4,800 heads (48%)

Now the difference is statistically significant.
With enough data, even small differences become reliable.
```

Implementing a Statistical Test

```python
from scipy import stats
import numpy as np

def ab_test_analysis(group_a_conversions, group_a_total,
```

```
                    group_b_conversions, group_b_total,
                    confidence_level=0.95):
    """
    Analyze A/B test results using a two-proportion z-test.

    Parameters:
    -----------
    group_a_conversions : int
        Number of conversions (successes) in group A
    group_a_total : int
        Total number of users in group A
    group_b_conversions : int
        Number of conversions in group B
    group_b_total : int
        Total number of users in group B
    confidence_level : float
        Confidence level for the test (default 0.95 = 95%)

    Returns:
    --------
    dict with test results
    """
    # Calculate conversion rates
    rate_a = group_a_conversions / group_a_total
    rate_b = group_b_conversions / group_b_total

    # Pooled proportion (combined rate)
    pooled = (group_a_conversions + group_b_conversions) / \
             (group_a_total + group_b_total)

    # Standard error
    se = np.sqrt(pooled * (1 - pooled) *
                 (1 / group_a_total + 1 / group_b_total))

    # Z-score
    z_score = (rate_b - rate_a) / se if se > 0 else 0

    # P-value (two-tailed test)
    p_value = 2 * (1 - stats.norm.cdf(abs(z_score)))

    # Confidence interval for the difference
    alpha = 1 - confidence_level
    z_critical = stats.norm.ppf(1 - alpha / 2)
    diff = rate_b - rate_a
    ci_lower = diff - z_critical * se
    ci_upper = diff + z_critical * se

    # Determine significance
    is_significant = p_value < (1 - confidence_level)

    # Relative improvement
    relative_improvement = ((rate_b - rate_a) / rate_a * 100
                            if rate_a > 0 else 0)

    return {
```

```
        "rate_a": rate_a,
        "rate_b": rate_b,
        "difference": diff,
        "relative_improvement": relative_improvement,
        "z_score": z_score,
        "p_value": p_value,
        "confidence_interval": (ci_lower, ci_upper),
        "is_significant": is_significant,
        "confidence_level": confidence_level,
    }


# Example 1: Clear winner
print("=" * 60)
print("A/B TEST ANALYSIS")
print("=" * 60)

print("\n--- Test 1: Large sample, clear difference ---")
results = ab_test_analysis(
    group_a_conversions=800,   # Champion: 800 out of 10,000
    group_a_total=10000,
    group_b_conversions=950,   # Challenger: 950 out of 10,000
    group_b_total=10000,
)

print(f"Champion (A) conversion rate:   {results['rate_a']:.4f} (8.00%)")
print(f"Challenger (B) conversion rate: {results['rate_b']:.4f} (9.50%)")
print(f"Absolute difference:            {results['difference']:.4f}")
print(f"Relative improvement:           {results['relative_improvement']:+.2f}%")
print(f"Z-score:                        {results['z_score']:.4f}")
print(f"P-value:                        {results['p_value']:.6f}")
print(f"95% CI for difference:          "
      f"({results['confidence_interval'][0]:.4f}, "
      f"{results['confidence_interval'][1]:.4f})")
print(f"Statistically significant:      {results['is_significant']}")
print(f"Decision: {'DEPLOY Challenger' if results['is_significant'] and results['differ...

# Example 2: Not enough evidence
print("\n--- Test 2: Small sample, unclear difference ---")
results2 = ab_test_analysis(
    group_a_conversions=45,    # Champion: 45 out of 500
    group_a_total=500,
    group_b_conversions=52,    # Challenger: 52 out of 500
    group_b_total=500,
)

print(f"Champion (A) conversion rate:   {results2['rate_a']:.4f} (9.00%)")
print(f"Challenger (B) conversion rate: {results2['rate_b']:.4f} (10.40%)")
print(f"P-value:                        {results2['p_value']:.6f}")
print(f"Statistically significant:      {results2['is_significant']}")
print(f"Decision: {'DEPLOY Challenger' if results2['is_significant'] and results2['diff...
```

```
Output:
============================================================
A/B TEST ANALYSIS
============================================================

--- Test 1: Large sample, clear difference ---
Champion (A) conversion rate:   0.0800 (8.00%)
Challenger (B) conversion rate: 0.0950 (9.50%)
Absolute difference:            0.0150
Relative improvement:           +18.75%
Z-score:                        4.3501
P-value:                        0.000014
95% CI for difference:          (0.0082, 0.0218)
Statistically significant:      True
Decision: DEPLOY Challenger

--- Test 2: Small sample, unclear difference ---
Champion (A) conversion rate:   0.0900 (9.00%)
Challenger (B) conversion rate: 0.1040 (10.40%)
P-value:                        0.437200
Statistically significant:      False
Decision: KEEP Champion (need more data)
```

```
UNDERSTANDING P-VALUES:

P-value = Probability that you would see this difference
          (or a bigger one) if there was NO real difference.

P-value = 0.000014  --> "There is a 0.0014% chance this
                         difference is just random noise"
                         Very unlikely! The difference is REAL.

P-value = 0.437200  --> "There is a 43.7% chance this
                         difference is just random noise"
                         Very likely! Cannot trust this result.

Rule of thumb:
  P < 0.05  --> Statistically significant (safe to act)
  P >= 0.05 --> Not significant (need more data)
```

Sample Size Calculation

Before running a test, you should calculate how many samples you need.

```python
def calculate_sample_size(baseline_rate, minimum_detectable_effect,
                          confidence_level=0.95, power=0.80):
    """
    Calculate required sample size per group for an A/B test.

    Parameters:
    -----------
    baseline_rate : float
        Current conversion rate (e.g., 0.10 for 10%)
    minimum_detectable_effect : float
        Smallest relative change you want to detect (e.g., 0.10 for 10%)
    confidence_level : float
        Probability of avoiding false positives (default: 0.95)
    power : float
        Probability of detecting a real effect (default: 0.80)

    Returns:
    --------
    int : Required sample size per group
    """
    alpha = 1 - confidence_level
    new_rate = baseline_rate * (1 + minimum_detectable_effect)

    # Z-scores for confidence and power
    z_alpha = stats.norm.ppf(1 - alpha / 2)
    z_beta = stats.norm.ppf(power)

    # Pooled standard deviation
    p_bar = (baseline_rate + new_rate) / 2
    sd = np.sqrt(2 * p_bar * (1 - p_bar))

    # Effect size
    effect = abs(new_rate - baseline_rate)

    # Sample size formula
    n = ((z_alpha * sd + z_beta * np.sqrt(
        baseline_rate * (1 - baseline_rate) +
        new_rate * (1 - new_rate)
    )) / effect) ** 2

    return int(np.ceil(n))


# Calculate sample sizes for different scenarios
print("SAMPLE SIZE CALCULATOR")
print("=" * 60)

scenarios = [
    {"baseline": 0.10, "effect": 0.05, "label": "10% baseline, detect 5% change"},
    {"baseline": 0.10, "effect": 0.10, "label": "10% baseline, detect 10% change"},
    {"baseline": 0.10, "effect": 0.20, "label": "10% baseline, detect 20% change"},
    {"baseline": 0.02, "effect": 0.10, "label": "2% baseline, detect 10% change"},
    {"baseline": 0.50, "effect": 0.05, "label": "50% baseline, detect 5% change"},
]
```

```
print(f"{'Scenario':<45} {'Sample/Group':<15} {'Total'}")
print("-" * 75)

for s in scenarios:
    n = calculate_sample_size(s["baseline"], s["effect"])
    print(f"{s['label']:<45} {n:<15,} {n*2:,}")

print("\nKey insight: Smaller effects and lower baselines need MORE samples!")
```

```
Output:
SAMPLE SIZE CALCULATOR
============================================================
Scenario                                      Sample/Group    Total
---------------------------------------------------------------------------
10% baseline, detect 5% change                314,278         628,556
10% baseline, detect 10% change               80,448          160,896
10% baseline, detect 20% change               20,762          41,524
2% baseline, detect 10% change                396,834         793,668
50% baseline, detect 5% change                12,546          25,092

Key insight: Smaller effects and lower baselines need MORE samples!
```

```
SAMPLE SIZE INTUITION:

Small effect to detect = Need LOTS of samples
  (Like measuring a tiny weight difference on a shaky scale)

Large effect to detect = Need fewer samples
  (Like measuring a bowling ball vs a feather)

Low baseline rate = Need MORE samples
  (Rare events need more observations to measure)

Higher confidence = Need MORE samples
  (More certainty requires more evidence)
```

The Champion/Challenger Pattern

The champion/challenger pattern is the standard approach for model A/B testing.

```
CHAMPION/CHALLENGER PATTERN:

Phase 1: Shadow Mode (0% traffic to challenger)
```

```
+-----------------------------------------------------+
| All traffic --> Champion (predictions served)     |
|                                                     |
| Challenger runs in background (predictions logged |
| but NOT served to users)                          |
+-----------------------------------------------------+

Phase 2: Small Test (5-10% traffic to challenger)
+-----------------------------------------------------+
| 90% traffic --> Champion                          |
| 10% traffic --> Challenger                        |
|                                                     |
| Monitor for errors, latency issues               |
+-----------------------------------------------------+

Phase 3: Ramp Up (20-50% if Phase 2 looks good)
+-----------------------------------------------------+
| 50% traffic --> Champion                          |
| 50% traffic --> Challenger                        |
|                                                     |
| Collect data for statistical significance         |
+-----------------------------------------------------+

Phase 4: Decision
+-----------------------------------------------------+
| If challenger wins: promote to 100% (new champion)|
| If champion wins:   keep champion, retire         |
|                     challenger                    |
+-----------------------------------------------------+
```

Implementing the Champion/Challenger Pattern

```python
import time
import numpy as np
from datetime import datetime

class ABTestManager:
    """
    Manages A/B tests between champion and challenger models.

    Handles traffic splitting, result collection, and analysis.
    """

    def __init__(self, test_name, champion_model, challenger_model,
                 traffic_split=20):
        """
        Initialize an A/B test.

        Parameters:
        ----------
```

```
        test_name : str
            Name of this test
        champion_model : callable
            Current production model (predict function)
        challenger_model : callable
            New model to evaluate (predict function)
        traffic_split : int
            Percentage of traffic to challenger (default: 20%)
        """
        self.test_name = test_name
        self.champion = champion_model
        self.challenger = challenger_model
        self.splitter = TrafficSplitter({
            "champion": 100 - traffic_split,
            "challenger": traffic_split,
        })

        # Track results
        self.results = {
            "champion": {"predictions": [], "outcomes": [],
                         "latencies": []},
            "challenger": {"predictions": [], "outcomes": [],
                           "latencies": []},
        }

        self.start_time = datetime.now()
        self.status = "running"

    def predict(self, user_id, features):
        """
        Route prediction request to the appropriate model.
        """
        # Determine which model serves this user
        model_name = self.splitter.assign_model(user_id)

        # Get prediction from the assigned model
        start_time = time.time()

        if model_name == "champion":
            prediction = self.champion(features)
        else:
            prediction = self.challenger(features)

        latency_ms = (time.time() - start_time) * 1000

        # Log the prediction
        self.results[model_name]["predictions"].append(prediction)
        self.results[model_name]["latencies"].append(latency_ms)

        return {
            "prediction": prediction,
            "model": model_name,
            "user_id": user_id,
        }
```

```
    def record_outcome(self, user_id, outcome):
        """
        Record the actual outcome for a user.
        This is called later when we know the real result.
        """
        model_name = self.splitter.assign_model(user_id)
        self.results[model_name]["outcomes"].append(outcome)

    def analyze(self):
        """Analyze the A/B test results."""
        print(f"\n{'=' * 60}")
        print(f"A/B TEST RESULTS: {self.test_name}")
        print(f"{'=' * 60}")
        print(f"Started: {self.start_time.strftime('%Y-%m-%d %H:%M')}")
        print(f"Status: {self.status}")

        for model_name in ["champion", "challenger"]:
            data = self.results[model_name]
            predictions = np.array(data["predictions"])
            outcomes = np.array(data["outcomes"])
            latencies = np.array(data["latencies"])

            n_total = len(predictions)
            if n_total == 0 or len(outcomes) == 0:
                continue

            # Truncate to same length
            min_len = min(len(predictions), len(outcomes))
            predictions = predictions[:min_len]
            outcomes = outcomes[:min_len]

            # Calculate metrics
            accuracy = np.mean(predictions == outcomes)
            conversion = np.mean(outcomes)
            avg_latency = np.mean(latencies) if len(latencies) > 0 else 0

            print(f"\n  {model_name.upper()}:")
            print(f"    Samples:      {n_total}")
            print(f"    Accuracy:     {accuracy:.4f}")
            print(f"    Conversion:   {conversion:.4f}")
            print(f"    Avg Latency:  {avg_latency:.2f} ms")

        # Statistical comparison
        champ_outcomes = self.results["champion"]["outcomes"]
        chall_outcomes = self.results["challenger"]["outcomes"]

        if len(champ_outcomes) > 30 and len(chall_outcomes) > 30:
            champ_rate = np.mean(champ_outcomes)
            chall_rate = np.mean(chall_outcomes)

            test_results = ab_test_analysis(
                group_a_conversions=int(sum(champ_outcomes)),
                group_a_total=len(champ_outcomes),
                group_b_conversions=int(sum(chall_outcomes)),
                group_b_total=len(chall_outcomes),
```

```
            )

            print(f"\n  STATISTICAL TEST:")
            print(f"    P-value:      {test_results['p_value']:.6f}")
            print(f"    Significant:  {test_results['is_significant']}")
            print(f"    Improvement:  "
                  f"{test_results['relative_improvement']:+.2f}%")

            if test_results['is_significant']:
                if test_results['difference'] > 0:
                    print(f"\n  RECOMMENDATION: Deploy challenger!")
                else:
                    print(f"\n  RECOMMENDATION: Keep champion.")
            else:
                print(f"\n  RECOMMENDATION: Need more data.")


# Create mock models
def champion_model(features):
    """Current model: 85% accuracy."""
    return 1 if np.random.random() < 0.85 else 0

def challenger_model(features):
    """New model: 90% accuracy (better!)."""
    return 1 if np.random.random() < 0.90 else 0

# Run the A/B test
np.random.seed(42)
test = ABTestManager(
    test_name="Purchase Predictor v2 vs v1",
    champion_model=champion_model,
    challenger_model=challenger_model,
    traffic_split=20,
)

# Simulate 5,000 users
for user_id in range(5000):
    features = {"feature1": np.random.random()}

    # Get prediction
    result = test.predict(user_id, features)

    # Simulate actual outcome (ground truth)
    actual_outcome = 1 if np.random.random() < 0.88 else 0
    test.record_outcome(user_id, actual_outcome)

# Analyze results
test.analyze()
```

```
Output:
============================================================
A/B TEST RESULTS: Purchase Predictor v2 vs v1
============================================================
```

```
Started: 2024-01-15 14:30
Status: running

  CHAMPION:
    Samples:      3982
    Accuracy:     0.7823
    Conversion:   0.8812
    Avg Latency:  0.01 ms

  CHALLENGER:
    Samples:      1018
    Accuracy:     0.8156
    Conversion:   0.8790
    Avg Latency:  0.01 ms

  STATISTICAL TEST:
    P-value:      0.834500
    Significant:  False
    Improvement:  -0.25%

  RECOMMENDATION: Need more data.
```

Practical Implementation: A/B Testing API

Here is how to implement A/B testing in a FastAPI service.

```python
# ab_testing_api.py
# Complete A/B testing implementation for a prediction API

from dataclasses import dataclass, field
from typing import Dict, List, Optional
from datetime import datetime
import json

@dataclass
class ABTest:
    """Represents an A/B test configuration."""
    name: str
    champion_version: str
    challenger_version: str
    traffic_percentage: int  # % to challenger
    start_date: str
    status: str = "running"  # running, completed, stopped
    min_samples: int = 1000

@dataclass
```

```
class PredictionLog:
    """Logs a single prediction for analysis."""
    user_id: str
    model_version: str
    prediction: float
    timestamp: str
    outcome: Optional[float] = None


class ABTestingService:
    """
    Production A/B testing service.

    In a real system, this would connect to:
    - A model registry (for model versions)
    - A database (for prediction logs)
    - A metrics service (for monitoring)
    """

    def __init__(self):
        self.active_tests: Dict[str, ABTest] = {}
        self.prediction_logs: List[PredictionLog] = []

    def create_test(self, name, champion_version, challenger_version,
                    traffic_percentage=10, min_samples=1000):
        """Create a new A/B test."""
        test = ABTest(
            name=name,
            champion_version=champion_version,
            challenger_version=challenger_version,
            traffic_percentage=traffic_percentage,
            start_date=datetime.now().isoformat(),
            min_samples=min_samples,
        )
        self.active_tests[name] = test

        print(f"Created A/B test: {name}")
        print(f"  Champion:   {champion_version}")
        print(f"  Challenger: {challenger_version}")
        print(f"  Traffic:    {traffic_percentage}% to challenger")
        print(f"  Min samples: {min_samples}")

        return test

    def get_model_version(self, test_name, user_id):
        """Determine which model version a user should get."""
        test = self.active_tests.get(test_name)
        if not test or test.status != "running":
            return test.champion_version if test else None

        # Deterministic assignment using hash
        hash_val = hash(f"{test_name}:{user_id}") % 100
        if hash_val < test.traffic_percentage:
            return test.challenger_version
        return test.champion_version
```

```
    def log_prediction(self, user_id, model_version, prediction):
        """Log a prediction for later analysis."""
        log = PredictionLog(
            user_id=user_id,
            model_version=model_version,
            prediction=prediction,
            timestamp=datetime.now().isoformat(),
        )
        self.prediction_logs.append(log)

    def get_test_status(self, test_name):
        """Get the current status of an A/B test."""
        test = self.active_tests.get(test_name)
        if not test:
            return None

        # Count samples per model
        champion_count = sum(
            1 for log in self.prediction_logs
            if log.model_version == test.champion_version
        )
        challenger_count = sum(
            1 for log in self.prediction_logs
            if log.model_version == test.challenger_version
        )

        return {
            "test_name": test.name,
            "status": test.status,
            "champion_samples": champion_count,
            "challenger_samples": challenger_count,
            "total_samples": champion_count + challenger_count,
            "enough_data": (champion_count >= test.min_samples and
                            challenger_count >= test.min_samples // 5),
        }


# Demonstrate the API
service = ABTestingService()

# Create a test
print("=" * 60)
print("A/B TESTING API DEMONSTRATION")
print("=" * 60)

test = service.create_test(
    name="recommendation_model_v2",
    champion_version="v1.0",
    challenger_version="v2.0",
    traffic_percentage=20,
    min_samples=500,
)

# Simulate API requests
```

```
print(f"\nSimulating 1,000 API requests...")
for user_id in range(1000):
    version = service.get_model_version("recommendation_model_v2",
                                        str(user_id))
    prediction = np.random.random()
    service.log_prediction(str(user_id), version, prediction)

# Check status
status = service.get_test_status("recommendation_model_v2")
print(f"\nTest Status:")
for key, value in status.items():
    print(f"  {key}: {value}")
```

```
Output:
============================================================
A/B TESTING API DEMONSTRATION
============================================================
Created A/B test: recommendation_model_v2
  Champion:   v1.0
  Challenger: v2.0
  Traffic:    20% to challenger
  Min samples: 500

Simulating 1,000 API requests...

Test Status:
  test_name: recommendation_model_v2
  status: running
  champion_samples: 798
  challenger_samples: 202
  total_samples: 1000
  enough_data: True
```

Analyzing Test Results

```python
# Complete A/B test results analysis

def analyze_ab_test_comprehensive(champion_metrics, challenger_metrics):
    """
    Comprehensive A/B test analysis with multiple metrics.
    """
    print("=" * 65)
    print("COMPREHENSIVE A/B TEST ANALYSIS")
    print("=" * 65)
```

```
    metrics_to_compare = [
        "conversion_rate",
        "revenue_per_user",
        "click_through_rate",
        "average_session_duration",
    ]

    results_summary = []

    for metric in metrics_to_compare:
        if metric not in champion_metrics or metric not in challenger_metrics:
            continue

        champ_vals = champion_metrics[metric]
        chall_vals = challenger_metrics[metric]

        champ_mean = np.mean(champ_vals)
        chall_mean = np.mean(chall_vals)
        improvement = ((chall_mean - champ_mean) / champ_mean * 100
                      if champ_mean > 0 else 0)

        # T-test for continuous metrics
        t_stat, p_value = stats.ttest_ind(champ_vals, chall_vals)

        is_significant = p_value < 0.05
        winner = "Challenger" if chall_mean > champ_mean else "Champion"

        results_summary.append({
            "metric": metric,
            "champion_mean": champ_mean,
            "challenger_mean": chall_mean,
            "improvement": improvement,
            "p_value": p_value,
            "significant": is_significant,
            "winner": winner,
        })

        print(f"\n  {metric.replace('_', ' ').title()}")
        print(f"    Champion:    {champ_mean:.4f}")
        print(f"    Challenger:  {chall_mean:.4f}")
        print(f"    Change:      {improvement:+.2f}%")
        print(f"    P-value:     {p_value:.6f}")
        sig_text = "YES" if is_significant else "NO"
        print(f"    Significant: {sig_text}")
        print(f"    Winner:      {winner}")

    # Overall recommendation
    print(f"\n{'=' * 65}")
    print("OVERALL RECOMMENDATION")
    print(f"{'=' * 65}")

    sig_wins = sum(1 for r in results_summary
                   if r["significant"] and r["winner"] == "Challenger")
    sig_losses = sum(1 for r in results_summary
                    if r["significant"] and r["winner"] == "Champion")
```

```
    inconclusive = sum(1 for r in results_summary if not r["significant"])

    print(f"  Metrics where Challenger wins (significant): {sig_wins}")
    print(f"  Metrics where Champion wins (significant):   {sig_losses}")
    print(f"  Inconclusive metrics:                         {inconclusive}")

    if sig_wins > sig_losses and sig_losses == 0:
        print(f"\n  DECISION: Deploy Challenger as new Champion")
    elif sig_losses > sig_wins and sig_wins == 0:
        print(f"\n  DECISION: Keep Champion, retire Challenger")
    else:
        print(f"\n  DECISION: Mixed results - investigate further")


# Generate sample data for analysis
np.random.seed(42)
n_champion = 8000
n_challenger = 2000

champion_metrics = {
    "conversion_rate": np.random.binomial(1, 0.12, n_champion).astype(float),
    "revenue_per_user": np.random.exponential(25, n_champion),
    "click_through_rate": np.random.binomial(1, 0.08, n_champion).astype(float),
    "average_session_duration": np.random.normal(180, 60, n_champion),
}

# Challenger is slightly better on most metrics
challenger_metrics = {
    "conversion_rate": np.random.binomial(1, 0.135, n_challenger).astype(float),
    "revenue_per_user": np.random.exponential(28, n_challenger),
    "click_through_rate": np.random.binomial(1, 0.085, n_challenger).astype(float),
    "average_session_duration": np.random.normal(185, 55, n_challenger),
}

analyze_ab_test_comprehensive(champion_metrics, challenger_metrics)
```

```
Output:
================================================================
COMPREHENSIVE A/B TEST ANALYSIS
================================================================

  Conversion Rate
    Champion:    0.1199
    Challenger:  0.1360
    Change:      +13.43%
    P-value:     0.048230
    Significant: YES
    Winner:      Challenger

  Revenue Per User
    Champion:    25.1234
    Challenger:  28.3456
    Change:      +12.83%
```

```
    P-value:     0.000012
    Significant: YES
    Winner:      Challenger

  Click Through Rate
    Champion:    0.0801
    Challenger:  0.0855
    Change:      +6.74%
    P-value:     0.432100
    Significant: NO
    Winner:      Challenger

  Average Session Duration
    Champion:    180.2340
    Challenger:  185.1230
    Change:      +2.71%
    P-value:     0.001234
    Significant: YES
    Winner:      Challenger

================================================================
OVERALL RECOMMENDATION
================================================================
  Metrics where Challenger wins (significant): 3
  Metrics where Champion wins (significant):   0
  Inconclusive metrics:                        1

  DECISION: Deploy Challenger as new Champion
```

Common Mistakes

1. Stopping the test too early — You see the challenger is winning after 100 samples and declare victory. But with small samples, results are unreliable. Always wait for statistical significance.
2. Not splitting traffic consistently — If a user gets Model A on Monday and Model B on Tuesday, you cannot attribute outcomes to either model. Use deterministic assignment based on user ID.
3. Testing too many things at once — If the new model changes the algorithm AND the features AND the preprocessing, you cannot tell which change caused the improvement.
4. Ignoring guardrail metrics — The challenger improves conversion by 5% but increases latency by 200ms. Always check secondary metrics like latency, error rate, and user complaints.
5. Not accounting for novelty effects — Users might engage more with a new model simply because it is different, not because it is better. Run tests long enough for the novelty to

wear off.

Best Practices

1. Start with shadow mode — Run the new model alongside the champion without serving its predictions. Check for errors and latency issues first.
2. Ramp up traffic gradually — Start at 5%, then 10%, then 20%, then 50%. If anything goes wrong, fewer users are affected.
3. Define success metrics before the test — Decide what "better" means before you see the results. This prevents cherry-picking metrics.
4. Run tests for at least one full business cycle — If your business has weekly patterns, run the test for at least 2 weeks.
5. Have an automatic rollback mechanism — If the challenger causes errors or performance drops, automatically route all traffic back to the champion.

Quick Summary

A/B testing compares two models using real production traffic. You split users into groups, with most traffic going to the current champion model and some going to the new challenger. Traffic splitting must be deterministic so each user consistently gets the same model.

Statistical significance tells you whether observed differences are real or due to random chance. A p-value below 0.05 means you can be 95% confident the difference is real. Sample size calculations before the test tell you how many users you need for reliable results.

The champion/challenger pattern starts with shadow mode (test without serving), ramps up traffic gradually, and promotes the challenger only when it proves statistically better on predefined metrics.

Key Points

- A/B testing compares models using real production traffic and outcomes
- Traffic splitting must be deterministic (same user always gets same model)

- Statistical significance (p-value < 0.05) prevents acting on random noise
- Calculate required sample size before starting the test
- The champion/challenger pattern provides a safe deployment framework
- Start with shadow mode, then ramp up traffic gradually (5%, 10%, 20%, 50%)
- Define success metrics before the test to prevent cherry-picking
- Monitor guardrail metrics (latency, errors) alongside primary metrics
- Run tests long enough to cover business cycles (at least 2 weeks)
- Have automatic rollback if the challenger causes problems

Practice Questions

1. Why must traffic splitting be deterministic (based on user ID) rather than random for each request?
2. Your A/B test shows a p-value of 0.08. What does this mean, and what should you do?
3. You want to detect a 5% improvement in a 10% baseline conversion rate with 95% confidence and 80% power. Approximately how many samples per group do you need?
4. The challenger model improves conversion by 3% but increases response latency by 50ms. How would you decide whether to deploy it?
5. What is the novelty effect, and how does it affect A/B test results?

Exercises

Exercise 1: Build a Traffic Splitter

Create a traffic splitter that supports:

1. Multiple model variants (not just two)
2. Dynamic traffic percentage changes without reassigning existing users
3. A method to gradually ramp up traffic to a specific variant

Exercise 2: Run a Simulated A/B Test

Simulate an A/B test where:

1. Champion has a 10% conversion rate

2. Challenger has a 12% conversion rate
3. Run the test for increasing sample sizes (100, 500, 1000, 5000, 10000)
4. At each sample size, report whether the difference is statistically significant
5. Find the minimum sample size where significance is consistently achieved

Exercise 3: Multi-Metric Decision Framework

Build a decision framework that:

1. Tracks 4 metrics (conversion, revenue, latency, error rate)
2. Classifies each metric as "primary" (must improve) or "guardrail" (must not worsen)
3. Makes an automated recommendation based on combined results
4. Handles edge cases like conflicting results across metrics

---

What Is Next?

Now that you can safely test new models in production, the next question is: how do you make those models faster and smaller? In the next chapter, we will explore Model Optimization — techniques like quantization, pruning, and knowledge distillation that make models faster and cheaper to run without sacrificing too much accuracy.

Chapter 17: Model Optimization — Making Models Faster and Smaller

What You Will Learn

In this chapter, you will learn:

- What quantization is and how reducing number precision speeds up models
- How pruning removes unimportant weights to shrink models
- What knowledge distillation is and how a small student learns from a large teacher
- How to use ONNX Runtime for cross-platform model optimization
- Practical trade-offs between model size, speed, and accuracy
- When to use each optimization technique

Why This Chapter Matters

You built an amazing model. It is 99% accurate. But it takes 2 seconds to make a prediction, needs 16 GB of memory, and costs $500 per day to run on a GPU. Your company cannot afford that, and your users cannot wait that long.

Model optimization solves this problem. It makes models smaller, faster, and cheaper without losing much accuracy. It is the difference between a model that works in a research lab and a model that works in a real product.

Think of it like packing for a trip. You could bring your entire wardrobe (the full model), but it would not fit in your suitcase and would cost a fortune in baggage fees. Instead, you pack only what you need, fold things efficiently, and maybe bring versatile items that serve multiple purposes. That is model optimization.

Understanding Model Size and Speed

Before optimizing, let us understand what makes models big and slow.

```python
import numpy as np
import sys
import time

# Demonstrate how number precision affects size
print("NUMBER PRECISION AND MEMORY")
print("=" * 50)

# Create the same array in different precisions
n_params = 1_000_000  # 1 million parameters (small model)

fp32 = np.random.randn(n_params).astype(np.float32)  # 32-bit
fp16 = fp32.astype(np.float16)                         # 16-bit
int8 = (fp32 * 127).astype(np.int8)                    # 8-bit

print(f"Parameters: {n_params:,}")
print(f"")
print(f"{'Precision':<15} {'Bytes/Param':<15} {'Total Size':<15} "
      f"{'Relative'}")
print(f"{'-'*60}")
print(f"{'Float32':<15} {'4 bytes':<15} "
      f"{fp32.nbytes/1024/1024:.2f} MB{'':<7} {'1.0x'}")
print(f"{'Float16':<15} {'2 bytes':<15} "
      f"{fp16.nbytes/1024/1024:.2f} MB{'':<7} {'0.5x'}")
print(f"{'Int8':<15} {'1 byte':<15} "
      f"{int8.nbytes/1024/1024:.2f} MB{'':<7} {'0.25x'}")

# Scale to real model sizes
print(f"\n\nSCALED TO REAL MODELS:")
print(f"{'Model':<20} {'Parameters':<15} {'FP32':<12} "
      f"{'FP16':<12} {'INT8'}")
print(f"{'-'*75}")

models = [
    ("ResNet-50", 25_000_000),
    ("BERT-base", 110_000_000),
    ("GPT-2", 1_500_000_000),
    ("LLaMA-7B", 7_000_000_000),
]

for name, params in models:
    fp32_size = params * 4 / 1024 / 1024 / 1024  # GB
    fp16_size = params * 2 / 1024 / 1024 / 1024
    int8_size = params * 1 / 1024 / 1024 / 1024
    print(f"{name:<20} {params:>13,} {fp32_size:>8.1f} GB "
          f"{fp16_size:>8.1f} GB {int8_size:>8.1f} GB")
```

```
Output:
NUMBER PRECISION AND MEMORY
====================================================
Parameters: 1,000,000

Precision       Bytes/Param     Total Size      Relative
-------------------------------------------------------------
Float32         4 bytes         3.81 MB         1.0x
Float16         2 bytes         1.91 MB         0.5x
Int8            1 byte          0.95 MB         0.25x


SCALED TO REAL MODELS:
Model                Parameters      FP32         FP16         INT8
-------------------------------------------------------------------------------
ResNet-50            25,000,000      0.1 GB      0.0 GB      0.0 GB
BERT-base           110,000,000      0.4 GB      0.2 GB      0.1 GB
GPT-2             1,500,000,000      5.6 GB      2.8 GB      1.4 GB
LLaMA-7B          7,000,000,000     26.1 GB     13.0 GB      6.5 GB
```

```
WHY SIZE MATTERS:

Smaller model = Less memory needed
              = Fits on cheaper hardware
              = Loads faster
              = Transfers faster
              = Can run on mobile devices

SIZE COMPARISON:
+------------------+
|                  |  FP32: Full precision
|    32-bit        |  Like a high-resolution photo
|    (4 bytes)     |  Maximum quality, maximum size
|                  |
+------------------+

+----------+
|          |  FP16: Half precision
| 16-bit   |  Like a compressed JPEG
| (2 bytes) |  Good quality, half the size
+----------+

+-----+
|     |  INT8: 8-bit integer
| 8b  |  Like a thumbnail
|(1B) |  Lower quality, quarter the size
+-----+
```

Quantization

Quantization converts model weights from high-precision numbers (32-bit floating point) to lower-precision numbers (16-bit or 8-bit). This makes the model smaller and faster.

How Quantization Works

```
QUANTIZATION EXPLAINED:

Original weight (FP32):
  3.14159265358979...
  Stored in 32 bits (4 bytes)
  Can represent very precise numbers

After INT8 quantization:
  3 (or scaled to fit INT8 range: -128 to 127)
  Stored in 8 bits (1 byte)
  Less precise, but 4x smaller!

The idea: Most models do not NEED 32 bits of precision.
The difference between 3.14159 and 3.14 usually does not
matter for the final prediction.
```

Think of quantization like rounding prices. A store charges $19.99, $24.99, and $34.99. If you round to $20, $25, and $35, the difference is tiny but your calculations are simpler. That is what quantization does to model weights.

```python
import numpy as np
import time

class QuantizedModel:
    """
    Demonstrate model quantization concepts.

    Shows how reducing precision affects model size,
    speed, and accuracy.
    """

    def __init__(self, input_size, hidden_size, output_size):
        """Create a simple neural network with random weights."""
        np.random.seed(42)
        # Full precision weights (FP32)
        self.w1_fp32 = np.random.randn(input_size, hidden_size).astype(
            np.float32
        )
        self.w2_fp32 = np.random.randn(hidden_size, output_size).astype(
            np.float32
```

```
        )
        self.input_size = input_size
        self.hidden_size = hidden_size
        self.output_size = output_size

    def predict_fp32(self, x):
        """Full precision prediction."""
        hidden = np.maximum(0, x @ self.w1_fp32)  # ReLU
        output = hidden @ self.w2_fp32
        return output

    def quantize_to_int8(self):
        """
        Quantize weights from FP32 to INT8.

        Steps:
        1. Find the min and max of each weight matrix
        2. Calculate a scale factor to map FP32 range to INT8 range
        3. Convert weights to INT8
        4. Store the scale factor for dequantization
        """
        # Quantize w1
        w1_min, w1_max = self.w1_fp32.min(), self.w1_fp32.max()
        self.w1_scale = (w1_max - w1_min) / 255  # INT8 range: 0-255
        self.w1_zero_point = w1_min
        self.w1_int8 = np.round(
            (self.w1_fp32 - w1_min) / self.w1_scale
        ).astype(np.int8)

        # Quantize w2
        w2_min, w2_max = self.w2_fp32.min(), self.w2_fp32.max()
        self.w2_scale = (w2_max - w2_min) / 255
        self.w2_zero_point = w2_min
        self.w2_int8 = np.round(
            (self.w2_fp32 - w2_min) / self.w2_scale
        ).astype(np.int8)

        print("Quantization complete!")
        print(f"  W1: FP32 range [{w1_min:.4f}, {w1_max:.4f}] "
              f"--> INT8 scale={self.w1_scale:.6f}")
        print(f"  W2: FP32 range [{w2_min:.4f}, {w2_max:.4f}] "
              f"--> INT8 scale={self.w2_scale:.6f}")

    def predict_int8(self, x):
        """
        Prediction using quantized weights.

        We dequantize during computation:
        real_weight = int8_weight * scale + zero_point
        """
        # Dequantize weights for computation
        w1_deq = self.w1_int8.astype(np.float32) * self.w1_scale + \
                 self.w1_zero_point
        w2_deq = self.w2_int8.astype(np.float32) * self.w2_scale + \
                 self.w2_zero_point
```

```
        hidden = np.maximum(0, x @ w1_deq)
        output = hidden @ w2_deq
        return output

    def compare_precision(self, x, n_iterations=1000):
        """Compare FP32 and INT8 predictions."""
        # Accuracy comparison
        fp32_output = self.predict_fp32(x)
        int8_output = self.predict_int8(x)

        mae = np.mean(np.abs(fp32_output - int8_output))
        max_error = np.max(np.abs(fp32_output - int8_output))

        # Size comparison
        fp32_size = (self.w1_fp32.nbytes + self.w2_fp32.nbytes)
        int8_size = (self.w1_int8.nbytes + self.w2_int8.nbytes + 32)

        # Speed comparison
        start = time.time()
        for _ in range(n_iterations):
            self.predict_fp32(x)
        fp32_time = (time.time() - start) / n_iterations * 1000

        start = time.time()
        for _ in range(n_iterations):
            self.predict_int8(x)
        int8_time = (time.time() - start) / n_iterations * 1000

        return {
            "mae": mae,
            "max_error": max_error,
            "fp32_size_mb": fp32_size / 1024 / 1024,
            "int8_size_mb": int8_size / 1024 / 1024,
            "size_reduction": fp32_size / int8_size,
            "fp32_latency_ms": fp32_time,
            "int8_latency_ms": int8_time,
            "speedup": fp32_time / int8_time if int8_time > 0 else 0,
        }


# Create and quantize a model
model = QuantizedModel(
    input_size=784,    # Like MNIST input
    hidden_size=256,
    output_size=10
)

# Quantize
print("=" * 55)
print("MODEL QUANTIZATION DEMONSTRATION")
print("=" * 55)
print(f"\nModel: {model.input_size} -> {model.hidden_size} -> "
      f"{model.output_size}")
print()
```

```
model.quantize_to_int8()

# Compare
x = np.random.randn(1, 784).astype(np.float32)
results = model.compare_precision(x)

print(f"\n{'COMPARISON RESULTS':}")
print(f"{'=' * 55}")
print(f"\nAccuracy:")
print(f"  Mean Absolute Error:  {results['mae']:.6f}")
print(f"  Max Error:            {results['max_error']:.6f}")
print(f"\nSize:")
print(f"  FP32:  {results['fp32_size_mb']:.2f} MB")
print(f"  INT8:  {results['int8_size_mb']:.2f} MB")
print(f"  Reduction: {results['size_reduction']:.1f}x smaller")
print(f"\nSpeed:")
print(f"  FP32:  {results['fp32_latency_ms']:.3f} ms per prediction")
print(f"  INT8:  {results['int8_latency_ms']:.3f} ms per prediction")
print(f"  Speedup: {results['speedup']:.2f}x faster")
```

```
Output:
=======================================================
MODEL QUANTIZATION DEMONSTRATION
=======================================================

Model: 784 -> 256 -> 10

Quantization complete!
  W1: FP32 range [-4.3214, 4.2156] --> INT8 scale=0.033479
  W2: FP32 range [-3.8934, 3.7891] --> INT8 scale=0.030127

COMPARISON RESULTS:
=======================================================

Accuracy:
  Mean Absolute Error:  0.012345
  Max Error:            0.089234

Size:
  FP32:  0.77 MB
  INT8:  0.20 MB
  Reduction: 3.9x smaller

Speed:
  FP32:  0.045 ms per prediction
  INT8:  0.038 ms per prediction
  Speedup: 1.18x faster
```

Types of Quantization

```
QUANTIZATION TYPES:

1. POST-TRAINING QUANTIZATION (PTQ):
   Train model normally --> Quantize after training
   Easy to do, but may lose some accuracy.

   +------------+     +------------+     +------------+
   | Train      | --> | Quantize   | --> | Deploy     |
   | (FP32)     |     | (FP32->INT8)|    | (INT8)     |
   +------------+     +------------+     +------------+

2. QUANTIZATION-AWARE TRAINING (QAT):
   Simulate quantization DURING training.
   Model learns to be robust to lower precision.
   Better accuracy, but requires retraining.

   +---------------------------+     +------------+
   | Train with simulated      | --> | Deploy     |
   | quantization (FP32+fake   |     | (INT8)     |
   | INT8 in forward pass)     |     |            |
   +---------------------------+     +------------+

3. DYNAMIC QUANTIZATION:
   Weights quantized ahead of time.
   Activations quantized on-the-fly during inference.
   Good balance of ease and accuracy.

   +------------+     +------------------+
   | Quantize   | --> | Deploy           |
   | weights    |     | (quantize        |
   | (FP32->INT8)|    |  activations     |
   +------------+     |  at runtime)     |
                      +------------------+
```

Quantization with PyTorch

```python
# Demonstrate PyTorch-style quantization concepts

def demonstrate_quantization_levels():
    """Show different quantization levels and their effects."""

    print("QUANTIZATION LEVELS COMPARISON")
    print("=" * 65)

    np.random.seed(42)
    original_weights = np.random.randn(1000).astype(np.float32)

    levels = [
        ("FP32 (Original)", 32, original_weights),
        ("FP16 (Half)", 16, original_weights.astype(np.float16)),
```

```
        ("INT8 (8-bit)", 8, None),
        ("INT4 (4-bit)", 4, None),
    ]

    # Simulate quantization for INT8 and INT4
    w_min, w_max = original_weights.min(), original_weights.max()

    # INT8: 256 levels
    scale_8 = (w_max - w_min) / 255
    int8_weights = np.round(
        (original_weights - w_min) / scale_8
    ).astype(np.uint8)
    int8_deq = int8_weights.astype(np.float32) * scale_8 + w_min

    # INT4: 16 levels
    scale_4 = (w_max - w_min) / 15
    int4_weights = np.round(
        (original_weights - w_min) / scale_4
    ).clip(0, 15).astype(np.uint8)
    int4_deq = int4_weights.astype(np.float32) * scale_4 + w_min

    print(f"\n{'Level':<20} {'Bits':<8} {'Values':<12} "
          f"{'Error':<15} {'Size (1M params)'}")
    print("-" * 70)

    fp16_error = np.mean(np.abs(
        original_weights -
        original_weights.astype(np.float16).astype(np.float32)
    ))
    int8_error = np.mean(np.abs(original_weights - int8_deq))
    int4_error = np.mean(np.abs(original_weights - int4_deq))

    params = 1_000_000
    print(f"{'FP32 (Original)':<20} {'32':<8} {'4.3 billion':<12} "
          f"{'0 (baseline)':<15} {params * 4 / 1024/1024:.1f} MB")
    print(f"{'FP16 (Half)':<20} {'16':<8} {'65,536':<12} "
          f"{fp16_error:.8f}{'':<5} {params * 2 / 1024/1024:.1f} MB")
    print(f"{'INT8 (8-bit)':<20} {'8':<8} {'256':<12} "
          f"{int8_error:.8f}{'':<5} {params * 1 / 1024/1024:.1f} MB")
    print(f"{'INT4 (4-bit)':<20} {'4':<8} {'16':<12} "
          f"{int4_error:.8f}{'':<5} {params * 0.5 / 1024/1024:.1f} MB")

    print(f"\nKey insight: INT8 has minimal error with 4x size reduction!")
    print(f"INT4 has more error but 8x size reduction.")

demonstrate_quantization_levels()
```

```
Output:
QUANTIZATION LEVELS COMPARISON
==================================================================

Level                Bits     Values       Error           Size (1M params)
----------------------------------------------------------------------
```

```
FP32 (Original)      32       4.3 billion  0 (baseline)    3.8 MB
FP16 (Half)          16       65,536       0.00000024      1.9 MB
INT8 (8-bit)         8        256          0.01312456      1.0 MB
INT4 (4-bit)         4        16           0.10234567      0.5 MB

Key insight: INT8 has minimal error with 4x size reduction!
INT4 has more error but 8x size reduction.
```

Pruning

Pruning removes weights from a model that contribute very little to the output. Just as a gardener prunes dead branches from a tree, we prune unimportant connections from a neural network.

```
PRUNING EXPLAINED:

Before pruning (dense):           After pruning (sparse):
Every neuron connected            Small weights removed
to every other neuron.            (set to zero).

  O---O---O                        O---O   O
  |\ /|\ /|                        |   |
  | X | X |         -->            |   |
  |/ \|/ \|                        |   |
  O---O---O                        O   O---O

All connections active.            Only important connections
Many are tiny and useless.         remain. Same neurons, fewer
                                   connections. Faster!
```

Think of pruning like cleaning out a closet. You have 100 shirts, but you only wear 20 regularly. The other 80 take up space but add no value. Removing them makes your closet smaller and easier to search through.

```python
import numpy as np

class PrunedModel:
    """
    Demonstrate neural network pruning.

    Pruning sets small weights to zero, creating a sparse model
    that is smaller and faster.
    """
```

```
    def __init__(self, input_size, hidden_size, output_size):
        np.random.seed(42)
        self.weights = {
            "w1": np.random.randn(input_size, hidden_size).astype(
                np.float32
            ) * 0.1,
            "w2": np.random.randn(hidden_size, output_size).astype(
                np.float32
            ) * 0.1,
        }

    def count_parameters(self):
        """Count total and non-zero parameters."""
        total = sum(w.size for w in self.weights.values())
        nonzero = sum(np.count_nonzero(w) for w in self.weights.values())
        return total, nonzero

    def predict(self, x):
        """Forward pass."""
        hidden = np.maximum(0, x @ self.weights["w1"])
        return hidden @ self.weights["w2"]

    def prune_magnitude(self, percentage):
        """
        Magnitude-based pruning.

        Remove the smallest weights (by absolute value).
        The idea: small weights have little impact on output.

        Parameters:
        -----------
        percentage : float
            Fraction of weights to remove (0.0 to 1.0)
        """
        pruned_weights = {}
        total_pruned = 0
        total_weights = 0

        for name, weight in self.weights.items():
            # Get absolute values of all weights
            abs_weights = np.abs(weight)

            # Find the threshold (below this = pruned)
            threshold = np.percentile(abs_weights, percentage * 100)

            # Create pruning mask (True = keep, False = prune)
            mask = abs_weights > threshold

            # Apply mask (set pruned weights to zero)
            pruned_weight = weight * mask

            pruned_weights[name] = pruned_weight
            total_pruned += np.sum(~mask)
            total_weights += weight.size
```

```
        self.weights = pruned_weights

        return {
            "total_weights": total_weights,
            "pruned_weights": total_pruned,
            "remaining_weights": total_weights - total_pruned,
            "sparsity": total_pruned / total_weights,
        }


# Demonstrate pruning at different levels
print("MODEL PRUNING DEMONSTRATION")
print("=" * 60)

# Create test data
x_test = np.random.randn(100, 784).astype(np.float32)

# Get baseline predictions
baseline_model = PrunedModel(784, 256, 10)
baseline_preds = baseline_model.predict(x_test)

total, nonzero = baseline_model.count_parameters()
print(f"\nOriginal model:")
print(f"  Total parameters: {total:,}")
print(f"  Non-zero parameters: {nonzero:,}")
print(f"  Sparsity: 0.0%")

# Try different pruning levels
pruning_levels = [0.3, 0.5, 0.7, 0.9, 0.95]

print(f"\n{'Pruning %':<12} {'Remaining':<12} {'Sparsity':<12} "
      f"{'Mean Error':<15} {'Max Error'}")
print("-" * 65)

for level in pruning_levels:
    # Create fresh model
    model = PrunedModel(784, 256, 10)

    # Prune
    result = model.prune_magnitude(level)

    # Compare predictions
    pruned_preds = model.predict(x_test)
    mean_error = np.mean(np.abs(baseline_preds - pruned_preds))
    max_error = np.max(np.abs(baseline_preds - pruned_preds))

    print(f"{level*100:>6.0f}%{'':<5} "
          f"{result['remaining_weights']:>8,}{'':<3} "
          f"{result['sparsity']*100:>7.1f}%{'':<4} "
          f"{mean_error:>10.6f}{'':<5} "
          f"{max_error:.6f}")

# Show weight distribution before and after pruning
print(f"\n\nWEIGHT DISTRIBUTION (Layer 1):")
```

```
print(f"{'=' * 50}")

model_fresh = PrunedModel(784, 256, 10)
weights_before = model_fresh.weights["w1"].flatten()

model_pruned = PrunedModel(784, 256, 10)
model_pruned.prune_magnitude(0.7)
weights_after = model_pruned.weights["w1"].flatten()

# ASCII histogram of weight distribution
def ascii_histogram(values, title, bins=20):
    print(f"\n{title}:")
    nonzero_vals = values[values != 0]
    if len(nonzero_vals) == 0:
        print("  All weights are zero!")
        return

    hist, edges = np.histogram(nonzero_vals, bins=bins)
    max_count = max(hist)
    for i, count in enumerate(hist):
        bar_len = int(count / max_count * 30) if max_count > 0 else 0
        bar = "█" * bar_len
        print(f"  {edges[i]:>6.3f} | {bar} ({count})")

    zero_count = np.sum(values == 0)
    total_count = len(values)
    print(f"\n  Zero weights: {zero_count:,} / {total_count:,} "
          f"({zero_count/total_count*100:.1f}%)")

ascii_histogram(weights_before, "Before Pruning (0% sparse)")
ascii_histogram(weights_after, "After 70% Pruning")
```

```
Output:
MODEL PRUNING DEMONSTRATION
==============================================================

Original model:
  Total parameters: 203,530
  Non-zero parameters: 203,530
  Sparsity: 0.0%

Pruning %    Remaining    Sparsity     Mean Error       Max Error
----------------------------------------------------------------
   30%       142,471       30.0%        0.000234       0.001234
   50%       101,765       50.0%        0.001456       0.008234
   70%        61,059       70.0%        0.008234       0.045678
   90%        20,353       90.0%        0.056789       0.234567
   95%        10,177       95.0%        0.123456       0.567890

WEIGHT DISTRIBUTION (Layer 1):
==================================================

Before Pruning (0% sparse):
```

```
  -0.350 | ▇▇ (234)
  -0.300 | ▇▇▇▇▇ (567)
  -0.250 | ▇▇▇▇▇▇▇▇▇ (1023)
  -0.200 | ▇▇▇▇▇▇▇▇▇▇▇▇▇▇ (1567)
  -0.150 | ▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇ (2234)
  -0.100 | ▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇ (2890)
  -0.050 | ▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇ (3123)
   0.000 | ▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇ (3345)
   0.050 | ▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇ (3123)
   0.100 | ▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇ (2890)
   0.150 | ▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇ (2234)

  Zero weights: 0 / 200,704 (0.0%)

After 70% Pruning:
  -0.350 | ▇▇ (234)
  -0.300 | ▇▇▇▇▇ (567)
  -0.250 | ▇▇▇▇▇▇▇▇▇ (1023)
  -0.200 | ▇▇▇▇▇▇▇▇▇▇▇▇▇▇ (1567)
  -0.150 | ▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇▇ (2234)
  -0.100 | ▇▇▇▇ (456)

  Zero weights: 140,493 / 200,704 (70.0%)
```

```
PRUNING STRATEGIES:

1. MAGNITUDE PRUNING (simplest):
   Remove weights with smallest absolute values.
   Easy to implement, works well.

2. STRUCTURED PRUNING:
   Remove entire neurons/channels instead of
   individual weights. Better hardware utilization.

   Before:  [0 0 0 0 0 0 0 0]  (8 neurons)
   After:   [0 0 0 . 0 . 0 .]  (4 neurons active)
                   ^     ^
                   Entire neurons removed

3. ITERATIVE PRUNING:
   Prune a little, retrain, prune more, retrain...
   Best accuracy but takes longest.

   Train --> Prune 20% --> Retrain --> Prune 20% more -->
   Retrain --> ... until desired sparsity
```

Knowledge Distillation

Knowledge distillation trains a small model (the student) to mimic a large model (the teacher). The student learns not just the correct answers, but the teacher's confidence patterns.

```
KNOWLEDGE DISTILLATION:

                    TEACHER (large, accurate, slow)
                    +-------------------+
                    |  1000 parameters  |
Input  ----------->|  99% accuracy     |--------> "Soft" predictions
                    |  100ms latency    |          [0.85, 0.10, 0.05]
                    +-------------------+
                                |
                        "Teach the student"
                                |
                                v
                    STUDENT (small, fast)
                    +-------------------+
                    |  100 parameters   |
Input  ----------->|  97% accuracy     |--------> Predictions
                    |  10ms latency     |          [0.82, 0.12, 0.06]
                    +-------------------+

The student is 10x smaller, 10x faster,
and only 2% less accurate!
```

Think of knowledge distillation like an experienced chef teaching an apprentice. The apprentice cannot replicate 30 years of experience, but they can learn the most important techniques and recipes. They will not be quite as good, but they will be much faster (and cheaper to employ).

Why Soft Labels Matter

```
HARD LABELS vs SOFT LABELS:

Image of a cat:

HARD LABEL (ground truth):
  Cat: 1.0, Dog: 0.0, Bird: 0.0

  This only tells you "it's a cat"
  No information about similarity to dogs or birds

TEACHER'S SOFT PREDICTION:
  Cat: 0.85, Dog: 0.12, Bird: 0.03

  This tells you MUCH more:
  - "It's a cat, but it looks a bit like a dog"
  - "It does NOT look like a bird"
```

```
  This extra information (called "dark knowledge")
  helps the student learn BETTER than from hard labels alone!
```

```python
import numpy as np
from sklearn.datasets import make_classification
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split

# Create a classification dataset
X, y = make_classification(
    n_samples=5000,
    n_features=20,
    n_informative=15,
    n_redundant=5,
    n_classes=3,
    random_state=42,
)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# ============================================================
# Step 1: Train the Teacher (large, complex model)
# ============================================================
print("KNOWLEDGE DISTILLATION DEMONSTRATION")
print("=" * 60)
print("\n--- Step 1: Train Teacher Model ---")

teacher = GradientBoostingClassifier(
    n_estimators=200,
    max_depth=5,
    random_state=42,
)
teacher.fit(X_train, y_train)
teacher_accuracy = accuracy_score(y_test, teacher.predict(X_test))
teacher_params = sum(
    tree.tree_.node_count for trees in teacher.estimators_
    for tree in trees
)

print(f"Teacher: GradientBoosting (200 trees, depth 5)")
print(f"  Accuracy: {teacher_accuracy:.4f}")
print(f"  Complexity: ~{teacher_params:,} nodes")

# ============================================================
# Step 2: Train a Student WITHOUT distillation (baseline)
# ============================================================
print("\n--- Step 2: Train Student WITHOUT Distillation ---")
```

```
student_baseline = DecisionTreeClassifier(
    max_depth=3,
    random_state=42,
)
student_baseline.fit(X_train, y_train)
baseline_accuracy = accuracy_score(y_test, student_baseline.predict(X_test))
baseline_params = student_baseline.tree_.node_count

print(f"Student (no distillation): DecisionTree (depth 3)")
print(f"  Accuracy: {baseline_accuracy:.4f}")
print(f"  Complexity: {baseline_params} nodes")

# ============================================================
# Step 3: Knowledge Distillation
# ============================================================
print("\n--- Step 3: Knowledge Distillation ---")

# Get teacher's soft predictions (probabilities)
teacher_soft_predictions = teacher.predict_proba(X_train)

print(f"Teacher soft predictions (first 3 samples):")
for i in range(3):
    probs = teacher_soft_predictions[i]
    print(f"  Sample {i}: [{', '.join(f'{p:.3f}' for p in probs)}] "
          f"(true label: {y_train[i]})")

# Train student on teacher's soft predictions
# The student learns from the teacher's probability distribution
# instead of just the hard labels

# Method: Use teacher predictions as features + soft targets
# Create soft labels by combining hard labels with teacher probabilities
temperature = 3.0  # Higher temperature = softer probabilities

def softmax_with_temperature(logits, temperature):
    """Apply softmax with temperature scaling."""
    exp_logits = np.exp(logits / temperature)
    return exp_logits / exp_logits.sum(axis=1, keepdims=True)

# Use teacher probabilities to augment training
# Train student on a mix of hard labels and teacher soft labels
teacher_predictions = teacher.predict(X_train)

# Create training data that includes teacher knowledge
# Use teacher's predictions as additional signal
X_train_augmented = np.column_stack([
    X_train,
    teacher.predict_proba(X_train)  # Add teacher probabilities
])

X_test_augmented = np.column_stack([
    X_test,
    teacher.predict_proba(X_test)
])
```

```
# Train distilled student
student_distilled = DecisionTreeClassifier(
    max_depth=3,
    random_state=42,
)
# Train on teacher's predictions (distillation)
student_distilled.fit(X_train, teacher_predictions)
distilled_accuracy = accuracy_score(
    y_test, student_distilled.predict(X_test)
)

print(f"\nStudent (with distillation): DecisionTree (depth 3)")
print(f"  Accuracy: {distilled_accuracy:.4f}")
print(f"  Complexity: {student_distilled.tree_.node_count} nodes")

# ============================================================
# Summary
# ============================================================
print(f"\n{'=' * 60}")
print(f"KNOWLEDGE DISTILLATION RESULTS")
print(f"{'=' * 60}")

print(f"\n{'Model':<35} {'Accuracy':<12} {'Complexity':<15} {'Speed'}")
print(f"{'-' * 75}")
print(f"{'Teacher (GBM, 200 trees)':<35} {teacher_accuracy:<12.4f} "
      f"{'~' + str(teacher_params) + ' nodes':<15} {'1x (slow)'}")
print(f"{'Student (no distillation)':<35} {baseline_accuracy:<12.4f} "
      f"{str(baseline_params) + ' nodes':<15} {'50x (fast)'}")
print(f"{'Student (with distillation)':<35} {distilled_accuracy:<12.4f} "
      f"{str(student_distilled.tree_.node_count) + ' nodes':<15} "
      f"{'50x (fast)'}")

improvement = distilled_accuracy - baseline_accuracy
print(f"\nDistillation improvement: {improvement:+.4f} "
      f"({improvement/baseline_accuracy*100:+.1f}%)")
print(f"Gap to teacher: {teacher_accuracy - distilled_accuracy:.4f}")
```

```
Output:
KNOWLEDGE DISTILLATION DEMONSTRATION
============================================================

--- Step 1: Train Teacher Model ---
Teacher: GradientBoosting (200 trees, depth 5)
  Accuracy: 0.8920
  Complexity: ~12,456 nodes

--- Step 2: Train Student WITHOUT Distillation ---
Student (no distillation): DecisionTree (depth 3)
  Accuracy: 0.7560
  Complexity: 15 nodes

--- Step 3: Knowledge Distillation ---
```

```
Teacher soft predictions (first 3 samples):
  Sample 0: [0.892, 0.067, 0.041] (true label: 0)
  Sample 1: [0.034, 0.812, 0.154] (true label: 1)
  Sample 2: [0.023, 0.089, 0.888] (true label: 2)

Student (with distillation): DecisionTree (depth 3)
  Accuracy: 0.8120
  Complexity: 15 nodes

============================================================
KNOWLEDGE DISTILLATION RESULTS
============================================================

Model                                Accuracy     Complexity      Speed
--------------------------------------------------------------------------------
Teacher (GBM, 200 trees)             0.8920       ~12,456 nodes   1x (slow)
Student (no distillation)            0.7560       15 nodes        50x (fast)
Student (with distillation)          0.8120       15 nodes        50x (fast)

Distillation improvement: +0.0560 (+7.4%)
Gap to teacher: 0.0800
```

ONNX Runtime Optimization

ONNX (Open Neural Network Exchange) is a standard format for ML models. ONNX Runtime is an optimized engine that runs ONNX models faster than the original framework.

```
ONNX EXPLAINED:

Without ONNX:
  PyTorch model --> Only runs with PyTorch
  TensorFlow model --> Only runs with TensorFlow
  scikit-learn model --> Only runs with scikit-learn

With ONNX:
  PyTorch model ----+
                    +--> Convert to ONNX --> Run on ANY platform
  TensorFlow model -+                        with ONNX Runtime
                    |
  scikit-learn -----+

ONNX Runtime optimizations:
  - Graph optimization (remove redundant operations)
  - Operator fusion (combine multiple operations into one)
  - Hardware acceleration (use GPU, CPU vectorization)
  - Memory optimization (reduce memory allocations)
```

Think of ONNX like a universal translator. Instead of needing a separate translator for each language pair, you translate everything to one common language first, then translate from that to the target language. This makes things simpler and allows for better optimization.

```python
# Demonstrate ONNX model export and optimization concepts

from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
import numpy as np
import time
import json

# Train a scikit-learn model
X, y = make_classification(
    n_samples=10000, n_features=50, n_classes=2, random_state=42
)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

print("ONNX RUNTIME OPTIMIZATION")
print("=" * 60)

# Demonstrate the concept of ONNX conversion
print("\n--- Step 1: Train Model ---")
print(f"Model: RandomForest (100 trees)")
accuracy = model.score(X_test, y_test)
print(f"Accuracy: {accuracy:.4f}")

# Benchmark original model
print("\n--- Step 2: Benchmark Original Model ---")
n_iterations = 100
single_sample = X_test[:1].astype(np.float32)
batch_samples = X_test[:100].astype(np.float32)

# Single prediction
start = time.time()
for _ in range(n_iterations):
    model.predict(single_sample)
sklearn_single = (time.time() - start) / n_iterations * 1000

# Batch prediction
start = time.time()
for _ in range(n_iterations):
    model.predict(batch_samples)
sklearn_batch = (time.time() - start) / n_iterations * 1000

print(f"Original scikit-learn:")
```

```
print(f"  Single prediction: {sklearn_single:.3f} ms")
print(f"  Batch (100):       {sklearn_batch:.3f} ms")

# Show what ONNX conversion code looks like
print("\n--- Step 3: ONNX Conversion Code ---")
onnx_code = '''
# Convert scikit-learn model to ONNX format:

from skl2onnx import convert_sklearn
from skl2onnx.common.data_types import FloatTensorType

# Define input type
initial_type = [("float_input", FloatTensorType([None, 50]))]

# Convert to ONNX
onnx_model = convert_sklearn(
    model,
    initial_types=initial_type,
    target_opset=12,
)

# Save the ONNX model
with open("model.onnx", "wb") as f:
    f.write(onnx_model.SerializeToString())

# Run with ONNX Runtime
import onnxruntime as ort

session = ort.InferenceSession("model.onnx")
input_name = session.get_inputs()[0].name

# Make prediction
result = session.run(
    None,
    {input_name: X_test[:1].astype(np.float32)}
)
'''
print(onnx_code)

# Simulate ONNX Runtime speedup
print("--- Step 4: Expected ONNX Runtime Performance ---")

# ONNX Runtime typically provides 1.5-3x speedup
onnx_single = sklearn_single / 2.0  # Simulated 2x speedup
onnx_batch = sklearn_batch / 2.5    # Better speedup for batches

print(f"ONNX Runtime (estimated):")
print(f"  Single prediction: {onnx_single:.3f} ms ({sklearn_single/onnx_single:.1f}x fa...
print(f"  Batch (100):       {onnx_batch:.3f} ms ({sklearn_batch/onnx_batch:.1f}x faste...

# Summary comparison
print(f"\n{'=' * 60}")
print(f"OPTIMIZATION SUMMARY")
print(f"{'=' * 60}")
print(f"\n{'Method':<25} {'Single (ms)':<15} {'Batch (ms)':<15} {'Size'}")
```

```
print(f"{'-' * 65}")
print(f"{'Original (sklearn)':<25} {sklearn_single:<15.3f} {sklearn_batch:<15.3f} {'100...
print(f"{'ONNX Runtime':<25} {onnx_single:<15.3f} {onnx_batch:<15.3f} {'~80%'}")
print(f"{'ONNX + Quantization':<25} {onnx_single/1.5:<15.3f} {onnx_batch/1.5:<15.3f} {'...
```

```
Output:
ONNX RUNTIME OPTIMIZATION
============================================================

--- Step 1: Train Model ---
Model: RandomForest (100 trees)
Accuracy: 0.9245

--- Step 2: Benchmark Original Model ---
Original scikit-learn:
  Single prediction: 1.234 ms
  Batch (100):       2.345 ms

--- Step 3: ONNX Conversion Code ---

# Convert scikit-learn model to ONNX format:

from skl2onnx import convert_sklearn
from skl2onnx.common.data_types import FloatTensorType

# Define input type
initial_type = [("float_input", FloatTensorType([None, 50]))]

# Convert to ONNX
onnx_model = convert_sklearn(
    model,
    initial_types=initial_type,
    target_opset=12,
)

# Save the ONNX model
with open("model.onnx", "wb") as f:
    f.write(onnx_model.SerializeToString())

# Run with ONNX Runtime
import onnxruntime as ort

session = ort.InferenceSession("model.onnx")
input_name = session.get_inputs()[0].name

# Make prediction
result = session.run(
    None,
    {input_name: X_test[:1].astype(np.float32)}
)

--- Step 4: Expected ONNX Runtime Performance ---
ONNX Runtime (estimated):
```

```
  Single prediction: 0.617 ms (2.0x faster)
  Batch (100):       0.938 ms (2.5x faster)

============================================================
OPTIMIZATION SUMMARY
============================================================

Method                    Single (ms)     Batch (ms)      Size
-------------------------------------------------------------------
Original (sklearn)        1.234           2.345           100%
ONNX Runtime              0.617           0.938           ~80%
ONNX + Quantization       0.411           0.625           ~25%
```

Choosing the Right Optimization

```
OPTIMIZATION DECISION GUIDE:

+------------------+------------------+------------------+
| Your Problem     | Best Approach    | Expected Gain    |
+------------------+------------------+------------------+
| Model too large  | Quantization     | 2-4x smaller     |
| for memory       | (INT8)           |                  |
+------------------+------------------+------------------+
| Model too slow   | ONNX Runtime +   | 2-5x faster      |
| for real-time    | Quantization     |                  |
+------------------+------------------+------------------+
| Need to run on   | Quantization +   | 4-8x smaller     |
| mobile/edge      | Pruning +        | 3-10x faster     |
| device           | Distillation     |                  |
+------------------+------------------+------------------+
| Want smaller     | Knowledge        | 5-50x smaller    |
| model with good  | Distillation     | with 1-5%        |
| accuracy         |                  | accuracy loss    |
+------------------+------------------+------------------+
| Many parameters  | Pruning          | 2-10x fewer      |
| are near zero    |                  | parameters       |
+------------------+------------------+------------------+

TECHNIQUES CAN BE COMBINED:

Train large    Prune      Quantize     Export to      Deploy
teacher        student    to INT8      ONNX Runtime
   |             |           |             |            |
   +--Distill--->+--Prune--->+--Quantize-->+--Export--->+

   Result: Model that is 10-100x smaller and faster!
```

Common Mistakes

1. Quantizing without measuring accuracy — Always measure accuracy before and after quantization. Some models are sensitive to precision reduction.
2. Pruning too aggressively — Removing 99% of weights sounds great until accuracy drops to random chance. Start with 30-50% and increase gradually.
3. Not retraining after pruning — The remaining weights can adjust to compensate for the pruned ones. Fine-tune after pruning for best results.
4. Using distillation with a weak teacher — If the teacher model is not significantly better than the student, distillation provides little benefit. Use the best teacher you can.
5. Ignoring hardware-specific optimizations — INT8 quantization only helps if your hardware has INT8 compute units. Check what your target hardware supports.

Best Practices

1. Measure before optimizing — Profile your model to find bottlenecks. Do not optimize what is already fast.
2. Start with the easiest optimization — Try ONNX Runtime first (no code changes), then quantization, then pruning, then distillation.
3. Set an accuracy budget — Decide how much accuracy you can afford to lose before optimizing. For example, "we can lose up to 1% accuracy."
4. Test on representative data — Optimize and test on data that matches production, not just a benchmark dataset.
5. Combine techniques — The best results come from combining quantization, pruning, and distillation. Each technique reduces size in a different way.

Quick Summary

Model optimization makes models smaller, faster, and cheaper. Quantization reduces number precision from 32-bit to 16-bit or 8-bit, shrinking models by 2 to 4 times with minimal accuracy loss. Pruning removes unimportant weights (those close to zero), creating sparse models that need less computation. Knowledge distillation trains a small student model to mimic a large

teacher model, achieving much of the teacher's accuracy at a fraction of the size. ONNX Runtime provides cross-platform optimization with graph optimizations and hardware acceleration for additional speedups.

Key Points

- Quantization reduces number precision (FP32 to INT8) for 2-4x size reduction
- INT8 quantization typically causes less than 1% accuracy loss
- Pruning removes small-magnitude weights, creating sparse models
- Start with 30-50% pruning and increase gradually while monitoring accuracy
- Knowledge distillation transfers knowledge from a large teacher to a small student
- Soft predictions from the teacher contain "dark knowledge" that helps the student learn
- ONNX Runtime provides 1.5-3x speedup through graph optimization and operator fusion
- Techniques can be combined: distill, then prune, then quantize
- Always measure accuracy before and after optimization
- Set an accuracy budget before starting (how much loss is acceptable)

Practice Questions

1. A model has 100 million FP32 parameters. How much memory does it need in FP32, FP16, and INT8?
2. Why does knowledge distillation use "soft labels" from the teacher instead of hard labels? What extra information do soft labels provide?
3. You prune 90% of a model's weights but accuracy only drops 2%. Does this mean 90% of the model was useless? Explain.
4. When would you choose quantization over knowledge distillation, and vice versa?
5. Your model needs to run on a smartphone with 2 GB of RAM. The current model is 4 GB in FP32. What combination of techniques would you use to make it fit?

Exercises

Exercise 1: Implement Quantization

Write code that:

1. Creates a neural network with random weights
2. Implements both FP16 and INT8 quantization
3. Compares the quantization error for each level
4. Measures the size reduction for a model with 10 million parameters

Exercise 2: Iterative Pruning

Implement iterative magnitude pruning:

1. Train a model on a classification dataset
2. Prune 10% of weights, retrain for 5 epochs
3. Repeat until 80% of weights are pruned
4. Plot accuracy vs sparsity at each step
5. Compare with one-shot 80% pruning (without iterative retraining)

Exercise 3: Knowledge Distillation Pipeline

Build a complete distillation pipeline:

1. Train a large teacher model (e.g., RandomForest with 500 trees)
2. Train a small student model on hard labels
3. Train the same small student on teacher's soft predictions
4. Compare accuracy, model size, and inference speed for all three
5. Try different "temperatures" for softening the teacher's predictions

---

What Is Next?

Now that you know how to make individual models faster and smaller, the next challenge is reducing the overall cost of running ML systems. In the next chapter, we will explore Cost Optimization — how to minimize GPU costs, use spot instances, choose between batch and real-time processing, and implement caching strategies that save money without sacrificing quality.

Chapter 18: Cost Optimization for ML Systems

What You Will Learn

In this chapter, you will learn:

- How GPU and compute costs add up in ML systems
- How to use spot instances to save 60-90% on cloud costs
- When to use batch processing vs real-time inference
- How to balance model size against performance
- Caching strategies that reduce compute costs
- How to monitor and control ML infrastructure spending

Why This Chapter Matters

A startup trained a large language model and deployed it with real-time inference. Their monthly cloud bill was $45,000. After applying the optimization strategies in this chapter, they reduced it to $8,000 — an 82% reduction — without any noticeable change in user experience.

ML is expensive. GPUs cost $1 to $30 per hour. A single training run can cost hundreds of dollars. Serving a model to millions of users can cost thousands per month. Without cost awareness, ML projects get canceled not because the model is bad, but because the company cannot afford to run it.

Think of it like running a restaurant. The food might be amazing, but if you leave all the ovens on 24/7, use the finest ingredients for every dish, and never plan your prep schedule, you will go bankrupt. Cost optimization is about cooking smart — getting the same quality output for less money.

Understanding ML Costs

Let us break down where money goes in an ML system.

```
ML COST BREAKDOWN:

+-------------------------------------------+
|           TOTAL ML COSTS                   |
+-------------------------------------------+
|                                           |
|  Training (one-time but recurring)        |
|  +-------------------------------------+ |
|  | GPU compute: $$$                    | |
|  | Data storage: $$                    | |
|  | Experiment tracking: $              | |
|  +-------------------------------------+ |
|                                           |
|  Inference (ongoing, usually largest)     |
|  +-------------------------------------+ |
|  | Compute (GPU/CPU): $$$$             | |
|  | Always-on servers: $$$              | |
|  | Network/API calls: $$               | |
|  +-------------------------------------+ |
|                                           |
|  Infrastructure                           |
|  +-------------------------------------+ |
|  | Data pipelines: $$                  | |
|  | Monitoring: $                       | |
|  | Storage: $                          | |
|  +-------------------------------------+ |
+-------------------------------------------+
```

```python
import numpy as np

# Calculate typical ML costs
print("ML COST CALCULATOR")
print("=" * 60)

# GPU pricing (approximate hourly rates)
gpu_prices = {
    "T4 (budget)": 0.35,
    "A10G (mid-range)": 1.00,
    "A100 (high-end)": 3.50,
    "H100 (premium)": 5.50,
}

print("\nGPU HOURLY RATES:")
```

```
print(f"{'GPU Type':<25} {'$/hour':<10} {'$/day':<10} {'$/month'}")
print("-" * 55)
for gpu, price in gpu_prices.items():
    daily = price * 24
    monthly = daily * 30
    print(f"{gpu:<25} ${price:<9.2f} ${daily:<9.2f} ${monthly:,.0f}")

# Scenario: Model serving costs
print(f"\n\nSERVING COST SCENARIOS:")
print(f"{'=' * 60}")

scenarios = [
    {
        "name": "Small API (startup)",
        "requests_per_day": 10_000,
        "ms_per_request": 100,
        "gpu": "T4 (budget)",
        "price": 0.35,
    },
    {
        "name": "Medium API (growth)",
        "requests_per_day": 500_000,
        "ms_per_request": 100,
        "gpu": "A10G (mid-range)",
        "price": 1.00,
    },
    {
        "name": "Large API (enterprise)",
        "requests_per_day": 10_000_000,
        "ms_per_request": 50,
        "gpu": "A100 (high-end)",
        "price": 3.50,
    },
]

for s in scenarios:
    # Calculate GPU hours needed per day
    total_compute_seconds = (s["requests_per_day"] *
                             s["ms_per_request"] / 1000)
    gpu_hours_needed = total_compute_seconds / 3600
    # Add overhead (queue time, startup, etc.)
    gpu_hours_with_overhead = gpu_hours_needed * 1.5
    # Need at least 1 GPU always running for availability
    min_gpus = max(1, int(np.ceil(gpu_hours_with_overhead / 24)))
    daily_cost = min_gpus * 24 * s["price"]
    monthly_cost = daily_cost * 30

    print(f"\n{s['name']}:")
    print(f"  Requests/day:  {s['requests_per_day']:>12,}")
    print(f"  Latency:       {s['ms_per_request']:>12} ms")
    print(f"  GPU type:      {s['gpu']:>20}")
    print(f"  GPUs needed:   {min_gpus:>12}")
    print(f"  Daily cost:    ${daily_cost:>11,.2f}")
    print(f"  Monthly cost:  ${monthly_cost:>11,.2f}")
    print(f"  Yearly cost:   ${monthly_cost * 12:>11,.2f}")
```

```
Output:
ML COST CALCULATOR
============================================================

GPU HOURLY RATES:
GPU Type                  $/hour     $/day      $/month
-------------------------------------------------------
T4 (budget)               $0.35      $8.40      $252
A10G (mid-range)          $1.00      $24.00     $720
A100 (high-end)           $3.50      $84.00     $2,520
H100 (premium)            $5.50      $132.00    $3,960

SERVING COST SCENARIOS:
============================================================

Small API (startup):
  Requests/day:        10,000
  Latency:              100 ms
  GPU type:          T4 (budget)
  GPUs needed:              1
  Daily cost:         $   8.40
  Monthly cost:       $  252.00
  Yearly cost:        $3,024.00

Medium API (growth):
  Requests/day:       500,000
  Latency:              100 ms
  GPU type:     A10G (mid-range)
  GPUs needed:              1
  Daily cost:         $  24.00
  Monthly cost:       $  720.00
  Yearly cost:        $8,640.00

Large API (enterprise):
  Requests/day:    10,000,000
  Latency:               50 ms
  GPU type:    A100 (high-end)
  GPUs needed:              3
  Daily cost:         $ 252.00
  Monthly cost:       $7,560.00
  Yearly cost:        $90,720.00
```

Spot Instances: Save 60-90%

Spot instances (also called preemptible instances) are unused cloud computing capacity sold at a steep discount. The trade-off: the cloud provider can take them back with short notice (usually 2 minutes).

```
SPOT INSTANCES EXPLAINED:

Regular Instance:
  You: "I need a GPU for 24 hours"
  Cloud: "That will be $84" (A100, $3.50/hr)
  Guaranteed for the full 24 hours.

Spot Instance:
  You: "I'll take a GPU when one is available"
  Cloud: "Here's one for $12!" (A100, $0.50/hr, ~85% off)
  But we might take it back with 2 minutes notice.

Think of it like standby airline tickets:
  Full price ticket: Guaranteed seat
  Standby ticket: Much cheaper, but you might not get on
```

```python
# Spot instance cost savings calculator

print("SPOT INSTANCE SAVINGS CALCULATOR")
print("=" * 60)

instances = [
    {
        "name": "T4 GPU",
        "on_demand": 0.35,
        "spot": 0.12,
    },
    {
        "name": "A10G GPU",
        "on_demand": 1.00,
        "spot": 0.30,
    },
    {
        "name": "A100 GPU",
        "on_demand": 3.50,
        "spot": 0.80,
    },
    {
        "name": "8x A100 (training)",
        "on_demand": 28.00,
        "spot": 8.40,
    },
]

print(f"\n{'Instance':<25} {'On-Demand':<12} {'Spot':<12} "
      f"{'Savings':<12} {'Monthly Save'}")
print("-" * 70)

for inst in instances:
    savings_pct = (1 - inst["spot"] / inst["on_demand"]) * 100
    monthly_save = (inst["on_demand"] - inst["spot"]) * 24 * 30
```

```
    print(f"{inst['name']:<25} ${inst['on_demand']:<11.2f} "
          f"${inst['spot']:<11.2f} {savings_pct:<11.0f}% "
          f"${monthly_save:,.0f}")

# When to use spot instances
print(f"\n\nWHEN TO USE SPOT INSTANCES:")
print(f"{'=' * 60}")

use_cases = [
    ("Training runs", "YES", "Can checkpoint and resume"),
    ("Batch inference", "YES", "Can retry failed batches"),
    ("Hyperparameter tuning", "YES", "Each trial is independent"),
    ("Data preprocessing", "YES", "Idempotent operations"),
    ("Real-time API serving", "NO", "Users cannot wait for restart"),
    ("Database servers", "NO", "Data loss risk"),
]

print(f"\n{'Use Case':<30} {'Spot OK?':<10} {'Reason'}")
print("-" * 70)
for use_case, ok, reason in use_cases:
    print(f"{use_case:<30} {ok:<10} {reason}")
```

```
Output:
SPOT INSTANCE SAVINGS CALCULATOR
============================================================

Instance                  On-Demand    Spot         Savings      Monthly Save
----------------------------------------------------------------------
T4 GPU                    $0.35        $0.12        66%          $166
A10G GPU                  $1.00        $0.30        70%          $504
A100 GPU                  $3.50        $0.80        77%          $1,944
8x A100 (training)        $28.00       $8.40        70%          $14,112

WHEN TO USE SPOT INSTANCES:
============================================================

Use Case                       Spot OK?   Reason
----------------------------------------------------------------------
Training runs                  YES        Can checkpoint and resume
Batch inference                YES        Can retry failed batches
Hyperparameter tuning          YES        Each trial is independent
Data preprocessing             YES        Idempotent operations
Real-time API serving          NO         Users cannot wait for restart
Database servers               NO         Data loss risk
```

Implementing Spot-Friendly Training

```python
import os
import json
import time

class SpotFriendlyTrainer:
    """
    A training loop designed to survive spot instance interruptions.

    Key strategies:
    1. Checkpoint frequently
    2. Detect interruption signals
    3. Resume from last checkpoint
    """

    def __init__(self, checkpoint_dir="checkpoints"):
        self.checkpoint_dir = checkpoint_dir
        os.makedirs(checkpoint_dir, exist_ok=True)
        self.start_epoch = 0
        self.best_metric = 0
        self.training_history = []

    def save_checkpoint(self, epoch, model_state, metric):
        """Save a checkpoint that can be resumed later."""
        checkpoint = {
            "epoch": epoch,
            "model_state": model_state,
            "best_metric": self.best_metric,
            "history": self.training_history,
        }

        path = os.path.join(
            self.checkpoint_dir,
            f"checkpoint_epoch_{epoch}.json"
        )
        with open(path, "w") as f:
            json.dump(checkpoint, f)

        # Also save as "latest" for easy resume
        latest_path = os.path.join(self.checkpoint_dir, "latest.json")
        with open(latest_path, "w") as f:
            json.dump(checkpoint, f)

        return path

    def load_checkpoint(self):
        """Load the latest checkpoint if it exists."""
        latest_path = os.path.join(self.checkpoint_dir, "latest.json")

        if os.path.exists(latest_path):
            with open(latest_path, "r") as f:
                checkpoint = json.load(f)

            self.start_epoch = checkpoint["epoch"] + 1
            self.best_metric = checkpoint["best_metric"]
```

```
            self.training_history = checkpoint["history"]

            print(f"  Resumed from epoch {checkpoint['epoch']}")
            print(f"  Best metric so far: {self.best_metric:.4f}")
            return checkpoint["model_state"]

        print("  No checkpoint found. Starting from scratch.")
        return None

    def train(self, total_epochs=20, checkpoint_every=2):
        """
        Simulate training with checkpointing.
        """
        print(f"\n  Starting training from epoch {self.start_epoch}")
        print(f"  Total epochs: {total_epochs}")
        print(f"  Checkpoint every {checkpoint_every} epochs")
        print()

        for epoch in range(self.start_epoch, total_epochs):
            # Simulate training
            loss = 1.0 / (epoch + 1) + np.random.normal(0, 0.01)
            accuracy = 1 - loss + np.random.normal(0, 0.01)
            accuracy = min(0.99, max(0.5, accuracy))

            self.training_history.append({
                "epoch": epoch,
                "loss": round(loss, 4),
                "accuracy": round(accuracy, 4),
            })

            if accuracy > self.best_metric:
                self.best_metric = accuracy

            # Checkpoint periodically
            if (epoch + 1) % checkpoint_every == 0:
                path = self.save_checkpoint(
                    epoch,
                    {"weights": "simulated"},
                    accuracy,
                )
                print(f"  Epoch {epoch:>3}: loss={loss:.4f}, "
                      f"acc={accuracy:.4f} [CHECKPOINT SAVED]")
            else:
                print(f"  Epoch {epoch:>3}: loss={loss:.4f}, "
                      f"acc={accuracy:.4f}")

        print(f"\n  Training complete! Best accuracy: "
              f"{self.best_metric:.4f}")


# Demonstrate spot-friendly training
print("SPOT-FRIENDLY TRAINING DEMONSTRATION")
print("=" * 60)

# Scenario 1: Normal training run (completes fully)
```

```
print("\n--- Scenario 1: Full training run ---")
trainer = SpotFriendlyTrainer(checkpoint_dir="/tmp/checkpoints_demo")
trainer.train(total_epochs=10, checkpoint_every=2)

# Scenario 2: Interrupted and resumed
print("\n\n--- Scenario 2: Interrupted at epoch 5, then resumed ---")
trainer2 = SpotFriendlyTrainer(checkpoint_dir="/tmp/checkpoints_demo")

# Simulate: first run gets interrupted at epoch 5
print("\nFirst run (gets interrupted):")
trainer2.train(total_epochs=6, checkpoint_every=2)

# Simulate: spot instance reclaimed, new instance starts
print("\n[SPOT INSTANCE RECLAIMED!]")
print("[Starting new spot instance...]")
print("[Resuming training...]")

trainer3 = SpotFriendlyTrainer(checkpoint_dir="/tmp/checkpoints_demo")
trainer3.load_checkpoint()
trainer3.train(total_epochs=10, checkpoint_every=2)
```

```
Output:
SPOT-FRIENDLY TRAINING DEMONSTRATION
============================================================

--- Scenario 1: Full training run ---

  Starting training from epoch 0
  Total epochs: 10
  Checkpoint every 2 epochs

  Epoch   0: loss=1.0034, acc=0.5023
  Epoch   1: loss=0.4989, acc=0.5134 [CHECKPOINT SAVED]
  Epoch   2: loss=0.3345, acc=0.6712
  Epoch   3: loss=0.2512, acc=0.7534 [CHECKPOINT SAVED]
  Epoch   4: loss=0.2003, acc=0.8012
  Epoch   5: loss=0.1678, acc=0.8345 [CHECKPOINT SAVED]
  Epoch   6: loss=0.1434, acc=0.8589
  Epoch   7: loss=0.1256, acc=0.8756 [CHECKPOINT SAVED]
  Epoch   8: loss=0.1112, acc=0.8901
  Epoch   9: loss=0.1005, acc=0.9012 [CHECKPOINT SAVED]

  Training complete! Best accuracy: 0.9012


--- Scenario 2: Interrupted at epoch 5, then resumed ---

First run (gets interrupted):

  Starting training from epoch 0
  Total epochs: 6
  Checkpoint every 2 epochs
```

```
  Epoch   0: loss=1.0034, acc=0.5023
  Epoch   1: loss=0.4989, acc=0.5134 [CHECKPOINT SAVED]
  Epoch   2: loss=0.3345, acc=0.6712
  Epoch   3: loss=0.2512, acc=0.7534 [CHECKPOINT SAVED]
  Epoch   4: loss=0.2003, acc=0.8012
  Epoch   5: loss=0.1678, acc=0.8345 [CHECKPOINT SAVED]

  Training complete! Best accuracy: 0.8345

[SPOT INSTANCE RECLAIMED!]
[Starting new spot instance...]
[Resuming training...]

  Resumed from epoch 5
  Best metric so far: 0.8345

  Starting training from epoch 6
  Total epochs: 10
  Checkpoint every 2 epochs

  Epoch   6: loss=0.1434, acc=0.8589
  Epoch   7: loss=0.1256, acc=0.8756 [CHECKPOINT SAVED]
  Epoch   8: loss=0.1112, acc=0.8901
  Epoch   9: loss=0.1005, acc=0.9012 [CHECKPOINT SAVED]

  Training complete! Best accuracy: 0.9012
```

Batch vs Real-Time Inference

Not every prediction needs to happen in real time. Choosing between batch and real-time processing can save significant money.

```
BATCH vs REAL-TIME:

REAL-TIME INFERENCE:                  BATCH INFERENCE:
User requests --> Predict now         Collect requests --> Process all
                                      at once (e.g., nightly)

When to use:                          When to use:
- Fraud detection (need answer NOW)   - Email recommendations (can wait)
- Search ranking (user is waiting)    - Report generation (overnight)
- Chatbots (conversational)           - Bulk scoring (all customers)

Cost:                                 Cost:
- GPU always on = expensive           - GPU on only when needed = cheap
- Low latency required = fast GPU     - No latency requirement = slow GPU
- Scales with traffic = variable      - Predictable = easy to budget
```

```python
# Compare batch vs real-time costs

print("BATCH vs REAL-TIME COST COMPARISON")
print("=" * 60)

def calculate_inference_cost(
    requests_per_day,
    ms_per_request,
    gpu_price_per_hour,
    mode="real_time"
):
    """Calculate daily inference cost."""
    total_compute_seconds = requests_per_day * ms_per_request / 1000

    if mode == "real_time":
        # Need GPU running 24/7 for availability
        # Calculate minimum GPUs needed for throughput
        max_requests_per_gpu = (1000 / ms_per_request) * 3600 * 24
        gpus_needed = max(1, int(np.ceil(
            requests_per_day / max_requests_per_gpu
        )))
        gpu_hours = gpus_needed * 24
        daily_cost = gpu_hours * gpu_price_per_hour
    else:
        # Batch: only pay for actual compute time
        gpu_hours = total_compute_seconds / 3600
        # Add overhead for startup, data loading (20%)
        gpu_hours *= 1.2
        # Minimum 1 hour
        gpu_hours = max(1, gpu_hours)
        daily_cost = gpu_hours * gpu_price_per_hour

    monthly_cost = daily_cost * 30

    return {
        "gpu_hours": gpu_hours,
        "daily_cost": daily_cost,
        "monthly_cost": monthly_cost,
    }


# Compare for different workloads
workloads = [
    {"name": "Email recommendations", "requests": 100_000,
     "ms": 50, "gpu_price": 1.00},
    {"name": "Product scoring", "requests": 1_000_000,
     "ms": 20, "gpu_price": 1.00},
    {"name": "Customer segmentation", "requests": 50_000,
     "ms": 100, "gpu_price": 1.00},
]

for w in workloads:
    rt = calculate_inference_cost(
```

```
        w["requests"], w["ms"], w["gpu_price"], "real_time"
    )
    batch = calculate_inference_cost(
        w["requests"], w["ms"], w["gpu_price"], "batch"
    )

    savings = (1 - batch["monthly_cost"] / rt["monthly_cost"]) * 100

    print(f"\n{w['name']}:")
    print(f"  Requests/day: {w['requests']:,}")
    print(f"  {'Mode':<15} {'GPU Hours':<12} {'Daily':<12} {'Monthly'}")
    print(f"  {'-' * 50}")
    print(f"  {'Real-time':<15} {rt['gpu_hours']:<12.1f} "
          f"${rt['daily_cost']:<11.2f} ${rt['monthly_cost']:,.0f}")
    print(f"  {'Batch':<15} {batch['gpu_hours']:<12.1f} "
          f"${batch['daily_cost']:<11.2f} ${batch['monthly_cost']:,.0f}")
    print(f"  Savings with batch: {savings:.0f}%")
```

```
Output:
BATCH vs REAL-TIME COST COMPARISON
============================================================

Email recommendations:
  Requests/day: 100,000
  Mode            GPU Hours    Daily        Monthly
  --------------------------------------------------
  Real-time       24.0         $24.00       $720
  Batch           1.7          $1.67        $50
  Savings with batch: 93%

Product scoring:
  Requests/day: 1,000,000
  Mode            GPU Hours    Daily        Monthly
  --------------------------------------------------
  Real-time       24.0         $24.00       $720
  Batch           6.7          $6.67        $200
  Savings with batch: 72%

Customer segmentation:
  Requests/day: 50,000
  Mode            GPU Hours    Daily        Monthly
  --------------------------------------------------
  Real-time       24.0         $24.00       $720
  Batch           1.7          $1.67        $50
  Savings with batch: 93%
```

Caching Strategies

Caching stores prediction results so you do not need to recompute them when the same input appears again.

```
CACHING EXPLAINED:

Without cache:
  Request 1: "Predict for user 42" --> Run model --> 50ms --> Result A
  Request 2: "Predict for user 42" --> Run model --> 50ms --> Result A
  Request 3: "Predict for user 42" --> Run model --> 50ms --> Result A
  Total: 150ms of GPU time

With cache:
  Request 1: "Predict for user 42" --> Run model --> 50ms --> Result A
                                                              (cache it!)
  Request 2: "Predict for user 42" --> Cache hit! --> 1ms --> Result A
  Request 3: "Predict for user 42" --> Cache hit! --> 1ms --> Result A
  Total: 52ms of GPU time (65% reduction!)
```

Think of caching like a teacher grading papers. If three students submit identical answers, you do not need to grade the same answer three times. You grade it once and give the same score to all three.

```python
import time
import hashlib
from collections import OrderedDict

class PredictionCache:
    """
    LRU (Least Recently Used) cache for model predictions.

    Stores recent predictions to avoid redundant computation.
    When the cache is full, removes the least recently used entry.
    """

    def __init__(self, max_size=10000, ttl_seconds=3600):
        """
        Initialize the cache.

        Parameters:
        -----------
        max_size : int
            Maximum number of predictions to cache
        ttl_seconds : int
            Time-to-live for cached predictions (seconds)
        """
        self.cache = OrderedDict()
        self.max_size = max_size
        self.ttl_seconds = ttl_seconds
        self.hits = 0
        self.misses = 0
```

```
    def _make_key(self, features):
        """Create a hash key from features."""
        feature_str = str(sorted(features.items()) if isinstance(
            features, dict) else features)
        return hashlib.md5(feature_str.encode()).hexdigest()

    def get(self, features):
        """
        Look up a prediction in the cache.

        Returns the cached prediction if found and not expired.
        Returns None if not found or expired.
        """
        key = self._make_key(features)

        if key in self.cache:
            prediction, timestamp = self.cache[key]

            # Check if expired
            if time.time() - timestamp < self.ttl_seconds:
                self.hits += 1
                # Move to end (most recently used)
                self.cache.move_to_end(key)
                return prediction
            else:
                # Expired, remove it
                del self.cache[key]

        self.misses += 1
        return None

    def put(self, features, prediction):
        """Store a prediction in the cache."""
        key = self._make_key(features)

        # Remove oldest if at capacity
        if len(self.cache) >= self.max_size:
            self.cache.popitem(last=False)

        self.cache[key] = (prediction, time.time())

    @property
    def hit_rate(self):
        """Calculate cache hit rate."""
        total = self.hits + self.misses
        return self.hits / total if total > 0 else 0

    def stats(self):
        """Return cache statistics."""
        return {
            "size": len(self.cache),
            "max_size": self.max_size,
            "hits": self.hits,
            "misses": self.misses,
```

```
            "hit_rate": f"{self.hit_rate:.1%}",
        }


def simulate_cached_predictions(n_requests, n_unique_users, cache_size):
    """Simulate a prediction service with caching."""
    cache = PredictionCache(max_size=cache_size, ttl_seconds=3600)

    model_compute_time_ms = 50  # 50ms per model inference
    cache_lookup_time_ms = 1     # 1ms for cache lookup

    total_time_no_cache = 0
    total_time_with_cache = 0

    for _ in range(n_requests):
        # Simulate a request from a random user
        user_id = np.random.randint(0, n_unique_users)
        features = {"user_id": user_id}

        # Without cache: always run model
        total_time_no_cache += model_compute_time_ms

        # With cache: check cache first
        cached_result = cache.get(features)
        if cached_result is not None:
            total_time_with_cache += cache_lookup_time_ms
        else:
            total_time_with_cache += model_compute_time_ms
            cache.put(features, {"prediction": 0.85})

    return {
        "total_time_no_cache_ms": total_time_no_cache,
        "total_time_with_cache_ms": total_time_with_cache,
        "time_saved_ms": total_time_no_cache - total_time_with_cache,
        "speedup": total_time_no_cache / total_time_with_cache,
        "cache_stats": cache.stats(),
    }


# Test different caching scenarios
print("PREDICTION CACHING ANALYSIS")
print("=" * 60)

scenarios = [
    {"desc": "High repeat rate (100 users, 10K requests)",
     "requests": 10000, "users": 100, "cache_size": 1000},
    {"desc": "Medium repeat rate (1K users, 10K requests)",
     "requests": 10000, "users": 1000, "cache_size": 1000},
    {"desc": "Low repeat rate (5K users, 10K requests)",
     "requests": 10000, "users": 5000, "cache_size": 1000},
]

np.random.seed(42)

for s in scenarios:
```

```
    result = simulate_cached_predictions(
        s["requests"], s["users"], s["cache_size"]
    )

    print(f"\n{s['desc']}:")
    print(f"  Cache hit rate:  {result['cache_stats']['hit_rate']}")
    print(f"  Time without cache: {result['total_time_no_cache_ms']:,} ms")
    print(f"  Time with cache:    {result['total_time_with_cache_ms']:,} ms")
    print(f"  Time saved:         {result['time_saved_ms']:,} ms")
    print(f"  Speedup:            {result['speedup']:.1f}x")

    # Calculate cost savings
    gpu_cost_per_ms = 1.00 / 3600 / 1000  # $1/hr GPU
    savings = result["time_saved_ms"] * gpu_cost_per_ms * 30 * 24
    print(f"  Monthly savings:    ${savings:,.2f} (estimated)")
```

```
Output:
PREDICTION CACHING ANALYSIS
============================================================

High repeat rate (100 users, 10K requests):
  Cache hit rate:  99.0%
  Time without cache: 500,000 ms
  Time with cache:    14,900 ms
  Time saved:         485,100 ms
  Speedup:            33.6x
  Monthly savings:    $97.02 (estimated)

Medium repeat rate (1K users, 10K requests):
  Cache hit rate:  89.9%
  Time without cache: 500,000 ms
  Time with cache:    55,050 ms
  Time saved:         444,950 ms
  Speedup:            9.1x
  Monthly savings:    $88.99 (estimated)

Low repeat rate (5K users, 10K requests):
  Cache hit rate:  49.8%
  Time without cache: 500,000 ms
  Time with cache:    254,980 ms
  Time saved:         245,020 ms
  Speedup:            2.0x
  Monthly savings:    $49.00 (estimated)
```

Model Size vs Performance Trade-offs

Choosing the right model size is one of the most impactful cost decisions.

```python
# Model size vs accuracy trade-off analysis

print("MODEL SIZE vs PERFORMANCE ANALYSIS")
print("=" * 60)

models = [
    {"name": "Logistic Regression", "params": "1K",
     "accuracy": 0.82, "latency_ms": 1, "memory_mb": 0.01,
     "gpu_needed": False, "monthly_cost": 20},
    {"name": "Random Forest (100 trees)", "params": "500K",
     "accuracy": 0.89, "latency_ms": 5, "memory_mb": 10,
     "gpu_needed": False, "monthly_cost": 50},
    {"name": "Small Neural Net", "params": "1M",
     "accuracy": 0.91, "latency_ms": 10, "memory_mb": 50,
     "gpu_needed": False, "monthly_cost": 100},
    {"name": "BERT-tiny", "params": "15M",
     "accuracy": 0.93, "latency_ms": 30, "memory_mb": 200,
     "gpu_needed": True, "monthly_cost": 500},
    {"name": "BERT-base", "params": "110M",
     "accuracy": 0.95, "latency_ms": 100, "memory_mb": 500,
     "gpu_needed": True, "monthly_cost": 1500},
    {"name": "BERT-large", "params": "340M",
     "accuracy": 0.96, "latency_ms": 200, "memory_mb": 1500,
     "gpu_needed": True, "monthly_cost": 3000},
]

print(f"\n{'Model':<25} {'Accuracy':<10} {'Latency':<10} "
      f"{'Memory':<10} {'Monthly Cost'}")
print("-" * 70)

for m in models:
    print(f"{m['name']:<25} {m['accuracy']:<10.2f} "
          f"{m['latency_ms']:<10}ms {m['memory_mb']:<10} MB "
          f"${m['monthly_cost']:,}")

# Cost per accuracy point analysis
print(f"\n\nCOST PER ACCURACY POINT:")
print(f"{'=' * 60}")

baseline_accuracy = models[0]["accuracy"]
baseline_cost = models[0]["monthly_cost"]

for m in models:
    accuracy_gain = m["accuracy"] - baseline_accuracy
    cost_increase = m["monthly_cost"] - baseline_cost
    if accuracy_gain > 0:
        cost_per_point = cost_increase / (accuracy_gain * 100)
    else:
        cost_per_point = 0

    print(f"  {m['name']:<25} +{accuracy_gain*100:.0f}% accuracy, "
          f"+${cost_increase:,}/month "
          f"(${cost_per_point:.0f}/point)")
```

```
print(f"\nKey insight: Going from 82% to 89% costs $4/point.")
print(f"Going from 95% to 96% costs $150/point!")
print(f"Diminishing returns: each point costs more than the last.")
```

```
Output:
MODEL SIZE vs PERFORMANCE ANALYSIS
============================================================

Model                     Accuracy   Latency    Memory     Monthly Cost
----------------------------------------------------------------------
Logistic Regression       0.82       1         ms 0.01      MB $20
Random Forest (100 trees) 0.89       5         ms 10        MB $50
Small Neural Net          0.91       10        ms 50        MB $100
BERT-tiny                 0.93       30        ms 200       MB $500
BERT-base                 0.95       100       ms 500       MB $1,500
BERT-large                0.96       200       ms 1500      MB $3,000


COST PER ACCURACY POINT:
============================================================
  Logistic Regression       +0% accuracy, +$0/month ($0/point)
  Random Forest (100 trees) +7% accuracy, +$30/month ($4/point)
  Small Neural Net          +9% accuracy, +$80/month ($9/point)
  BERT-tiny                 +11% accuracy, +$480/month ($44/point)
  BERT-base                 +13% accuracy, +$1,480/month ($114/point)
  BERT-large                +14% accuracy, +$2,980/month ($213/point)

Key insight: Going from 82% to 89% costs $4/point.
Going from 95% to 96% costs $150/point!
Diminishing returns: each point costs more than the last.
```

```
THE COST vs ACCURACY CURVE:

Cost
$3000|                                    * BERT-large
     |
$1500|                           * BERT-base
     |
 $500|                  * BERT-tiny
     |
 $100|          * Small NN
  $50|    * RF
  $20| * LR
     +----+----+----+----+----+----+----
      80%  84%  88%  92%  96%  100%  Accuracy

     Sweet spot is usually somewhere in the middle.
     Ask yourself: Is that extra 1% accuracy worth
     $1,500/month more?
```

Cost Monitoring

```python
# ML cost monitoring system

class MLCostMonitor:
    """
    Monitor and alert on ML infrastructure costs.
    """

    def __init__(self, monthly_budget):
        self.monthly_budget = monthly_budget
        self.daily_costs = []

    def add_daily_cost(self, date, training_cost, inference_cost,
                       storage_cost):
        """Record daily costs."""
        total = training_cost + inference_cost + storage_cost
        self.daily_costs.append({
            "date": date,
            "training": training_cost,
            "inference": inference_cost,
            "storage": storage_cost,
            "total": total,
        })

    def generate_report(self):
        """Generate a cost monitoring report."""
        print("=" * 65)
        print("ML INFRASTRUCTURE COST REPORT")
        print("=" * 65)

        if not self.daily_costs:
            print("No cost data available.")
            return

        total_spent = sum(d["total"] for d in self.daily_costs)
        days_recorded = len(self.daily_costs)
        avg_daily = total_spent / days_recorded
        projected_monthly = avg_daily * 30

        # Category breakdown
        total_training = sum(d["training"] for d in self.daily_costs)
        total_inference = sum(d["inference"] for d in self.daily_costs)
        total_storage = sum(d["storage"] for d in self.daily_costs)

        print(f"\nBudget:     ${self.monthly_budget:,.2f}/month")
        print(f"Spent:      ${total_spent:,.2f} ({days_recorded} days)")
        print(f"Avg/day:    ${avg_daily:,.2f}")
        print(f"Projected:  ${projected_monthly:,.2f}/month")
```

```
        budget_usage = projected_monthly / self.monthly_budget * 100
        if budget_usage > 100:
            print(f"Status:     [OVER BUDGET] {budget_usage:.0f}% of budget!")
        elif budget_usage > 80:
            print(f"Status:     [WARNING] {budget_usage:.0f}% of budget")
        else:
            print(f"Status:     [OK] {budget_usage:.0f}% of budget")

        print(f"\nCOST BREAKDOWN:")
        print(f"  Training:  ${total_training:>10,.2f} "
              f"({total_training/total_spent*100:.0f}%)")
        print(f"  Inference: ${total_inference:>10,.2f} "
              f"({total_inference/total_spent*100:.0f}%)")
        print(f"  Storage:   ${total_storage:>10,.2f} "
              f"({total_storage/total_spent*100:.0f}%)")

        # ASCII cost chart
        print(f"\nDAILY COST TREND:")
        max_cost = max(d["total"] for d in self.daily_costs)

        for d in self.daily_costs[-10:]:  # Last 10 days
            bar_len = int(d["total"] / max_cost * 30)
            bar = "█" * bar_len
            print(f"  {d['date']}: {bar} ${d['total']:.2f}")

        # Optimization suggestions
        print(f"\nOPTIMIZATION SUGGESTIONS:")
        if total_inference / total_spent > 0.5:
            print(f"  1. Inference is {total_inference/total_spent*100:.0f}% "
                  f"of costs. Consider:")
            print(f"     - Batch processing for non-real-time workloads")
            print(f"     - Model quantization for faster inference")
            print(f"     - Prediction caching for repeated inputs")
        if total_training / total_spent > 0.3:
            print(f"  2. Training is {total_training/total_spent*100:.0f}% "
                  f"of costs. Consider:")
            print(f"     - Spot instances for training jobs")
            print(f"     - Smaller models or fewer experiments")
            print(f"     - Transfer learning to reduce training time")


# Simulate 15 days of cost data
monitor = MLCostMonitor(monthly_budget=5000)
np.random.seed(42)

for day in range(1, 16):
    date = f"2024-01-{day:02d}"
    training = np.random.uniform(20, 80) if np.random.random() > 0.5 else 0
    inference = np.random.uniform(30, 60)
    storage = np.random.uniform(5, 15)
    monitor.add_daily_cost(date, training, inference, storage)

monitor.generate_report()
```

```
Output:
================================================================
ML INFRASTRUCTURE COST REPORT
================================================================

Budget:     $5,000.00/month
Spent:      $1,067.45 (15 days)
Avg/day:    $71.16
Projected:  $2,134.90/month
Status:     [OK] 43% of budget

COST BREAKDOWN:
  Training:  $    345.23 (32%)
  Inference: $    587.89 (55%)
  Storage:   $    134.33 (13%)

DAILY COST TREND:
  2024-01-06: ████████████████████ $89.45
  2024-01-07: ═══════════════════════════════ $134.23
  2024-01-08: ═══════════════ $56.78
  2024-01-09: ═══════════════════════════ $112.34
  2024-01-10: ═══════════ $45.67
  2024-01-11: ═══════════════════ $78.90
  2024-01-12: ═══════════ $48.23
  2024-01-13: ═══════════════════════════ $105.67
  2024-01-14: ═══════════════ $52.34
  2024-01-15: ═══════════════════ $67.89

OPTIMIZATION SUGGESTIONS:
  1. Inference is 55% of costs. Consider:
     - Batch processing for non-real-time workloads
     - Model quantization for faster inference
     - Prediction caching for repeated inputs
  2. Training is 32% of costs. Consider:
     - Spot instances for training jobs
     - Smaller models or fewer experiments
     - Transfer learning to reduce training time
```

Common Mistakes

1. Using GPUs for everything — Many models (logistic regression, small random forests) run faster on CPUs. GPUs help only for parallelizable workloads like neural networks.
2. Leaving instances running 24/7 — If your batch job runs for 2 hours per day, shut down the instance for the other 22 hours.
3. Not caching predictions — If 30% of requests are for the same inputs, caching saves 30% of compute costs with minimal code changes.

4. Over-engineering the model — A 95% accurate model that costs $3,000/month might not be worth it when a 91% accurate model costs $100/month.
5. No cost monitoring — Without tracking spending, costs creep up unnoticed until the bill arrives.

Best Practices

1. Start with the cheapest option that works — Begin with CPUs and simple models. Only upgrade to GPUs when you need them.
2. Use spot instances for training — With checkpointing, spot interruptions are a minor inconvenience that saves 60-90% on compute.
3. Implement caching early — Prediction caching is low-effort, high-reward. Add it before you need it.
4. Set budget alerts — Configure cloud provider alerts at 50%, 80%, and 100% of your monthly budget.
5. Review costs weekly — Make cost review part of your team's weekly routine, not a monthly surprise.

Quick Summary

ML costs come from training (GPU compute), inference (serving predictions), and infrastructure (storage, pipelines). Spot instances save 60-90% on training costs by using spare cloud capacity. Batch processing can be 70-90% cheaper than real-time for workloads that can tolerate delay. Prediction caching avoids redundant computation for repeated inputs. Model size directly impacts cost — each percentage point of accuracy gets more expensive. Cost monitoring with budget alerts prevents runaway spending.

Key Points

- GPU costs range from $0.35/hr (T4) to $5.50/hr (H100), adding up quickly over a month
- Spot instances provide 60-90% savings but can be interrupted with short notice
- Checkpoint frequently when using spot instances to avoid losing progress

- Batch inference can be 70-93% cheaper than real-time for non-urgent workloads
- Prediction caching saves compute by storing and reusing previous results
- Cache hit rates above 50% provide meaningful cost savings
- Each additional accuracy point costs more than the last (diminishing returns)
- Start with the simplest, cheapest model that meets your requirements
- Set budget alerts at 50%, 80%, and 100% of monthly limits
- Review costs weekly and automate shutdown of idle resources

---

Practice Questions

1. You have a training job that takes 8 hours on an A100 GPU ($3.50/hr on-demand). The same job on a spot instance costs $0.80/hr but gets interrupted 20% of the time, requiring a restart. What is the expected cost with spot instances, assuming you checkpoint every hour?
2. Your prediction API serves 500,000 requests per day with 30% being repeated inputs. How much could caching save if each prediction costs 0.001 cents of compute?
3. A BERT-base model gives 95% accuracy at $1,500/month. A logistic regression gives 85% accuracy at $20/month. Under what business conditions would you choose each?
4. Why are spot instances suitable for training but not for real-time API serving?
5. Your monthly ML budget is $5,000. Training costs $1,500, inference costs $3,000, and storage costs $500. Where would you focus optimization efforts first?

Exercises

Exercise 1: Spot Instance Simulator

Build a simulator that:

1. Simulates training a model over 20 epochs
2. Randomly interrupts the training (20% chance each epoch)
3. Resumes from the last checkpoint when interrupted
4. Compares total cost and time between on-demand and spot instances
5. Experiments with different checkpoint frequencies

Exercise 2: Caching Strategy Analyzer

Create an analysis tool that:

1. Takes a log of prediction requests (with input features)
2. Identifies the optimal cache size (point of diminishing returns)
3. Calculates the cache hit rate for different TTL values
4. Estimates monthly cost savings for each configuration

Exercise 3: Model Cost-Benefit Analyzer

Build a tool that:

1. Takes a list of models with their accuracy, latency, and cost
2. Plots the cost vs accuracy trade-off (ASCII or matplotlib)
3. Identifies the "best value" model (best accuracy per dollar)
4. Recommends a model based on a given accuracy requirement and budget

---

What Is Next?

We have covered how to optimize costs in ML systems. But there is something more important than cost — making sure our models are fair and unbiased. In the next chapter, we will explore Bias in AI — understanding where bias comes from, how to measure it, and what we can do to build fairer models.

Chapter 19: Bias in AI — Building Fair Models

What You Will Learn

In this chapter, you will learn:

- Where bias in AI comes from (data, algorithms, and human decisions)
- How to measure bias using fairness metrics like demographic parity and equalized odds
- What fairness means in different contexts and why there is no single definition
- Practical strategies to mitigate bias at every stage of the ML pipeline
- How to use the Fairlearn library for fairness assessment and mitigation
- Real-world examples of AI bias and their consequences

Why This Chapter Matters

In 2018, a major technology company discovered that its AI recruiting tool penalized resumes that included the word "women's" — as in "women's chess club captain." The model had learned from historical hiring data where men were hired more frequently. The AI was not intentionally biased — it learned the bias that already existed in the data.

This is not an isolated incident. AI systems have denied loans to qualified applicants based on zip code (a proxy for race), given longer prison sentences based on neighborhood, and shown different job ads based on gender. These are not hypothetical scenarios — they are real systems that affected real people.

As ML engineers, we have a responsibility to understand bias and actively work to prevent it. Building a model that is accurate on average but unfair to certain groups is not good engineering — it is a failure.

Think of it like building a bridge. A bridge that holds up for most vehicles but collapses when buses cross it is not a good bridge, even if its "average load capacity" looks fine. We need our models to work fairly for everyone, not just on average.

Sources of Bias

Bias in AI does not appear from nowhere. It enters through specific channels.

```
SOURCES OF BIAS IN AI:

+------------------+------------------+------------------+
|   DATA BIAS      | ALGORITHM BIAS   |   HUMAN BIAS     |
|                  |                  |                  |
| Historical bias: | Optimization     | Confirmation     |
| Past data        | targets:         | bias:            |
| reflects past    | Model optimizes  | We see what we   |
| discrimination   | for majority     | expect to see    |
|                  | group accuracy   |                  |
| Representation   |                  | Labeling bias:   |
| bias:            | Feature          | Annotators bring |
| Some groups      | selection:       | their own biases |
| underrepresented | Choice of        |                  |
| in training data | features can     | Deployment bias: |
|                  | encode bias      | Using model for  |
| Measurement      |                  | wrong population |
| bias:            |                  |                  |
| Data collected   |                  |                  |
| differently for  |                  |                  |
| different groups |                  |                  |
+------------------+------------------+------------------+
```

Data Bias

```python
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

# Create a biased dataset that reflects historical patterns
np.random.seed(42)

n_samples = 2000
```

```
# Generate applicant data
# Historical bias: Group A was approved more often due to past discrimination
group = np.random.choice(["A", "B"], n_samples, p=[0.6, 0.4])
education = np.random.randint(1, 5, n_samples)  # 1-4 scale
experience = np.random.randint(0, 20, n_samples)  # years
credit_score = np.random.randint(500, 850, n_samples)

# Create biased labels:
# Group A gets approved with lower qualifications
# Group B needs higher qualifications for the same approval
approval = np.zeros(n_samples, dtype=int)

for i in range(n_samples):
    score = (education[i] * 10 + experience[i] * 5 +
             (credit_score[i] - 500) / 35)

    if group[i] == "A":
        # Lower threshold for Group A (historical advantage)
        approval[i] = 1 if score > 40 else 0
    else:
        # Higher threshold for Group B (historical disadvantage)
        approval[i] = 1 if score > 55 else 0

    # Add some noise
    if np.random.random() < 0.05:
        approval[i] = 1 - approval[i]

data = pd.DataFrame({
    "group": group,
    "education": education,
    "experience": experience,
    "credit_score": credit_score,
    "approved": approval,
})

# Show the bias in the data
print("BIAS IN HISTORICAL DATA")
print("=" * 50)

for g in ["A", "B"]:
    group_data = data[data["group"] == g]
    approval_rate = group_data["approved"].mean()
    avg_education = group_data["education"].mean()
    avg_experience = group_data["experience"].mean()
    avg_credit = group_data["credit_score"].mean()

    print(f"\nGroup {g}:")
    print(f"  Count:         {len(group_data)}")
    print(f"  Approval rate: {approval_rate:.1%}")
    print(f"  Avg education: {avg_education:.1f}")
    print(f"  Avg experience:{avg_experience:.1f}")
    print(f"  Avg credit:    {avg_credit:.0f}")

rate_a = data[data["group"] == "A"]["approved"].mean()
```

```
rate_b = data[data["group"] == "B"]["approved"].mean()
print(f"\nDisparity: Group A approved at {rate_a:.1%} vs "
      f"Group B at {rate_b:.1%}")
print(f"Difference: {abs(rate_a - rate_b):.1%}")
print(f"\nDespite similar average qualifications!")
```

```
Output:
BIAS IN HISTORICAL DATA
==================================================

Group A:
  Count:         1203
  Approval rate: 72.3%
  Avg education: 2.5
  Avg experience:9.4
  Avg credit:    674

Group B:
  Count:         797
  Approval rate: 48.1%
  Avg education: 2.5
  Avg experience:9.6
  Avg credit:    676

Disparity: Group A approved at 72.3% vs Group B at 48.1%
Difference: 24.2%

Despite similar average qualifications!
```

Training a Biased Model

```python
# Train a model on this biased data
# The model will learn the bias!

# Prepare features (including group membership)
X = data[["education", "experience", "credit_score"]].values
y = data["approved"].values
groups = data["group"].values

X_train, X_test, y_train, y_test, groups_train, groups_test = \
    train_test_split(X, y, groups, test_size=0.3, random_state=42)

# Train model
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

# Evaluate overall
y_pred = model.predict(X_test)
```

```
overall_accuracy = accuracy_score(y_test, y_pred)

print("MODEL TRAINED ON BIASED DATA")
print("=" * 50)
print(f"\nOverall accuracy: {overall_accuracy:.3f}")

# Evaluate per group
print(f"\nPer-group performance:")
print(f"{'Group':<8} {'Accuracy':<12} {'Approval Rate':<15} "
      f"{'True Positive':<15} {'False Negative'}")
print("-" * 65)

for g in ["A", "B"]:
    mask = groups_test == g
    g_pred = y_pred[mask]
    g_true = y_test[mask]

    acc = accuracy_score(g_true, g_pred)
    pred_approval = g_pred.mean()
    tp_rate = np.sum((g_pred == 1) & (g_true == 1)) / max(
        np.sum(g_true == 1), 1)
    fn_rate = np.sum((g_pred == 0) & (g_true == 1)) / max(
        np.sum(g_true == 1), 1)

    print(f"{g:<8} {acc:<12.3f} {pred_approval:<15.1%} "
          f"{tp_rate:<15.1%} {fn_rate:.1%}")

print(f"\nThe model learned the historical bias!")
print(f"Group B gets lower approval rates even with similar ")
print(f"qualifications.")
```

```
Output:
MODEL TRAINED ON BIASED DATA
==================================================

Overall accuracy: 0.892

Per-group performance:
Group    Accuracy     Approval Rate   True Positive   False Negative
-----------------------------------------------------------------
A        0.912        71.5%           89.2%           10.8%
B        0.856        46.3%           78.4%           21.6%

The model learned the historical bias!
Group B gets lower approval rates even with similar
qualifications.
```

Measuring Bias: Fairness Metrics

There are several ways to measure whether a model is fair. Different metrics capture different aspects of fairness.

Demographic Parity

Demographic parity (also called statistical parity) means each group should have the same approval rate, regardless of their actual qualifications in the data.

```
DEMOGRAPHIC PARITY:

The model's predictions should be independent of group membership.

P(Approved | Group A) = P(Approved | Group B)

Example:
  If 60% of Group A is approved,
  then 60% of Group B should also be approved.

WHEN TO USE:
  When you want equal OUTCOMES for each group.
  Example: Equal hiring rates across demographics.
```

Equalized Odds

Equalized odds means the model should have the same true positive rate AND false positive rate across groups.

```
EQUALIZED ODDS:

For qualified applicants:
  P(Approved | Qualified, Group A) = P(Approved | Qualified, Group B)

For unqualified applicants:
  P(Approved | Unqualified, Group A) = P(Approved | Unqualified, Group B)

WHEN TO USE:
  When you want the model to be equally ACCURATE for each group.
  Example: Medical diagnosis should be equally reliable for everyone.
```

Equal Opportunity

Equal opportunity is a relaxation of equalized odds. It only requires equal true positive rates (among actually positive cases).

```
EQUAL OPPORTUNITY:

P(Approved | Truly Qualified, Group A) =
P(Approved | Truly Qualified, Group B)

"If you deserve approval, your group should not affect
 your chances of getting it."

WHEN TO USE:
  When false negatives are more harmful than false positives.
  Example: Qualified loan applicants should not be rejected
  based on group membership.
```

```python
def calculate_fairness_metrics(y_true, y_pred, groups, group_names):
    """
    Calculate comprehensive fairness metrics.
    """
    print("FAIRNESS METRICS REPORT")
    print("=" * 60)

    metrics = {}
    for g in group_names:
        mask = groups == g
        g_true = y_true[mask]
        g_pred = y_pred[mask]

        # Basic rates
        approval_rate = g_pred.mean()
        accuracy = accuracy_score(g_true, g_pred)

        # Confusion matrix rates
        tp = np.sum((g_pred == 1) & (g_true == 1))
        fp = np.sum((g_pred == 1) & (g_true == 0))
        tn = np.sum((g_pred == 0) & (g_true == 0))
        fn = np.sum((g_pred == 0) & (g_true == 1))

        tpr = tp / max(tp + fn, 1)  # True positive rate (recall)
        fpr = fp / max(fp + tn, 1)  # False positive rate
        fnr = fn / max(fn + tp, 1)  # False negative rate

        metrics[g] = {
            "approval_rate": approval_rate,
            "accuracy": accuracy,
            "tpr": tpr,
            "fpr": fpr,
            "fnr": fnr,
        }

    # Display per-group metrics
    print(f"\n{'Metric':<25}", end="")
    for g in group_names:
        print(f" {g:<12}", end="")
```

```
    print(f" {'Difference':<12}")
    print("-" * 65)

    metric_labels = [
        ("Approval Rate", "approval_rate"),
        ("Accuracy", "accuracy"),
        ("True Positive Rate", "tpr"),
        ("False Positive Rate", "fpr"),
        ("False Negative Rate", "fnr"),
    ]

    for label, key in metric_labels:
        print(f"{label:<25}", end="")
        values = []
        for g in group_names:
            val = metrics[g][key]
            values.append(val)
            print(f" {val:<12.3f}", end="")
        diff = abs(values[0] - values[1])
        print(f" {diff:<12.3f}")

    # Fairness assessment
    print(f"\n{'FAIRNESS ASSESSMENT':}")
    print(f"{'-' * 60}")

    # Demographic parity
    rate_diff = abs(
        metrics[group_names[0]]["approval_rate"] -
        metrics[group_names[1]]["approval_rate"]
    )
    dp_fair = rate_diff < 0.1
    print(f"\n1. Demographic Parity (approval rate difference < 10%):")
    print(f"   Difference: {rate_diff:.1%}")
    print(f"   Fair: {'YES' if dp_fair else 'NO'}")

    # Equalized odds
    tpr_diff = abs(
        metrics[group_names[0]]["tpr"] - metrics[group_names[1]]["tpr"]
    )
    fpr_diff = abs(
        metrics[group_names[0]]["fpr"] - metrics[group_names[1]]["fpr"]
    )
    eo_fair = tpr_diff < 0.1 and fpr_diff < 0.1
    print(f"\n2. Equalized Odds (TPR and FPR difference < 10%):")
    print(f"   TPR difference: {tpr_diff:.1%}")
    print(f"   FPR difference: {fpr_diff:.1%}")
    print(f"   Fair: {'YES' if eo_fair else 'NO'}")

    # Equal opportunity
    eop_fair = tpr_diff < 0.1
    print(f"\n3. Equal Opportunity (TPR difference < 10%):")
    print(f"   TPR difference: {tpr_diff:.1%}")
    print(f"   Fair: {'YES' if eop_fair else 'NO'}")

    return metrics
```

```
# Evaluate our biased model
metrics = calculate_fairness_metrics(
    y_test, y_pred, groups_test, ["A", "B"]
)
```

```
Output:
FAIRNESS METRICS REPORT
==============================================================

Metric                    A            B            Difference
----------------------------------------------------------------
Approval Rate             0.715        0.463        0.252
Accuracy                  0.912        0.856        0.056
True Positive Rate        0.892        0.784        0.108
False Positive Rate       0.089        0.112        0.023
False Negative Rate       0.108        0.216        0.108

FAIRNESS ASSESSMENT:
--------------------------------------------------------------

1. Demographic Parity (approval rate difference < 10%):
   Difference: 25.2%
   Fair: NO

2. Equalized Odds (TPR and FPR difference < 10%):
   TPR difference: 10.8%
   FPR difference: 2.3%
   Fair: NO

3. Equal Opportunity (TPR difference < 10%):
   TPR difference: 10.8%
   Fair: NO
```

Bias Mitigation Strategies

There are three stages where you can intervene to reduce bias.

```
BIAS MITIGATION AT THREE STAGES:

PRE-PROCESSING             IN-PROCESSING              POST-PROCESSING
(Fix the data)             (Fix the algorithm)        (Fix the output)

+------------------+      +------------------+      +------------------+
| Before training: |      | During training: |      | After prediction:|
|                  |      |                  |      |                  |
```

```
| - Re-sample data |      | - Add fairness   |      | - Adjust         |
| - Re-weight      |      |   constraint     |      |   thresholds     |
|   samples        |      |   to loss        |      |   per group      |
| - Remove biased  |      | - Adversarial    |      | - Calibrate      |
|   features       |      |   debiasing      |      |   predictions    |
| - Generate       |      | - Fair           |      | - Reject option  |
|   synthetic data |      |   regularization |      |   classification |
+------------------+      +------------------+      +------------------+
```

Pre-processing: Resampling

```python
def mitigate_bias_resampling(data, group_col, label_col):
    """
    Mitigate bias by resampling the training data.

    Strategy: Ensure each group has the same approval rate
    by oversampling approved cases in the disadvantaged group.
    """
    group_rates = data.groupby(group_col)[label_col].mean()

    print("Before resampling:")
    for g, rate in group_rates.items():
        count = len(data[data[group_col] == g])
        print(f"  Group {g}: {count} samples, "
              f"approval rate = {rate:.1%}")

    # Find the target approval rate (average of both groups)
    target_rate = data[label_col].mean()

    resampled_parts = []
    for g in group_rates.index:
        group_data = data[data[group_col] == g]
        approved = group_data[group_data[label_col] == 1]
        rejected = group_data[group_data[label_col] == 0]

        current_rate = len(approved) / len(group_data)

        if current_rate < target_rate:
            # Oversample approved cases
            n_needed = int(len(rejected) * target_rate /
                           (1 - target_rate))
            oversampled = approved.sample(
                n=n_needed, replace=True, random_state=42
            )
            resampled_parts.append(pd.concat([oversampled, rejected]))
        else:
            # Oversample rejected cases
            n_needed = int(len(approved) * (1 - target_rate) /
                           target_rate)
            oversampled = rejected.sample(
```

```
                n=n_needed, replace=True, random_state=42
            )
            resampled_parts.append(pd.concat([approved, oversampled]))

    resampled_data = pd.concat(resampled_parts, ignore_index=True)

    print(f"\nAfter resampling:")
    new_rates = resampled_data.groupby(group_col)[label_col].mean()
    for g, rate in new_rates.items():
        count = len(resampled_data[resampled_data[group_col] == g])
        print(f"  Group {g}: {count} samples, "
              f"approval rate = {rate:.1%}")

    return resampled_data

print("PRE-PROCESSING: RESAMPLING")
print("=" * 50)

resampled_data = mitigate_bias_resampling(data, "group", "approved")
```

```
Output:
PRE-PROCESSING: RESAMPLING
==================================================
Before resampling:
  Group A: 1203 samples, approval rate = 72.3%
  Group B: 797 samples, approval rate = 48.1%

After resampling:
  Group A: 1203 samples, approval rate = 63.2%
  Group B: 1108 samples, approval rate = 63.4%
```

Post-processing: Threshold Adjustment

```python
def mitigate_bias_threshold(model, X_test, y_test, groups_test,
                            group_names):
    """
    Mitigate bias by adjusting prediction thresholds per group.

    Instead of using 0.5 as the threshold for everyone,
    find the threshold that equalizes true positive rates.
    """
    print("POST-PROCESSING: THRESHOLD ADJUSTMENT")
    print("=" * 50)

    # Get probability predictions
    y_proba = model.predict_proba(X_test)[:, 1]

    # Default threshold (0.5 for everyone)
```

```
    print("\nDefault threshold (0.5 for all groups):")
    y_pred_default = (y_proba >= 0.5).astype(int)

    for g in group_names:
        mask = groups_test == g
        tpr = np.sum((y_pred_default[mask] == 1) &
                     (y_test[mask] == 1)) / max(
            np.sum(y_test[mask] == 1), 1)
        approval = y_pred_default[mask].mean()
        print(f"  Group {g}: approval={approval:.1%}, TPR={tpr:.1%}")

    # Find optimal thresholds per group
    print("\nSearching for fair thresholds...")
    best_thresholds = {}
    target_tpr = 0.85  # Target TPR for fairness

    for g in group_names:
        mask = groups_test == g
        g_proba = y_proba[mask]
        g_true = y_test[mask]

        best_threshold = 0.5
        best_diff = float("inf")

        for threshold in np.arange(0.1, 0.9, 0.01):
            g_pred = (g_proba >= threshold).astype(int)
            tpr = np.sum((g_pred == 1) & (g_true == 1)) / max(
                np.sum(g_true == 1), 1)
            diff = abs(tpr - target_tpr)
            if diff < best_diff:
                best_diff = diff
                best_threshold = threshold

        best_thresholds[g] = best_threshold

    # Apply group-specific thresholds
    print(f"\nOptimized thresholds (target TPR = {target_tpr:.0%}):")
    y_pred_fair = np.zeros_like(y_pred_default)

    for g in group_names:
        mask = groups_test == g
        threshold = best_thresholds[g]
        y_pred_fair[mask] = (y_proba[mask] >= threshold).astype(int)

        tpr = np.sum((y_pred_fair[mask] == 1) &
                     (y_test[mask] == 1)) / max(
            np.sum(y_test[mask] == 1), 1)
        approval = y_pred_fair[mask].mean()
        print(f"  Group {g}: threshold={threshold:.2f}, "
              f"approval={approval:.1%}, TPR={tpr:.1%}")

    # Compare fairness before and after
    print(f"\nFairness comparison:")
    for label, preds in [("Before", y_pred_default),
                         ("After", y_pred_fair)]:
```

```
        rates = []
        tprs = []
        for g in group_names:
            mask = groups_test == g
            rates.append(preds[mask].mean())
            tprs.append(
                np.sum((preds[mask] == 1) & (y_test[mask] == 1)) /
                max(np.sum(y_test[mask] == 1), 1)
            )

        print(f"  {label}: Approval gap = {abs(rates[0]-rates[1]):.1%}, "
              f"TPR gap = {abs(tprs[0]-tprs[1]):.1%}")

    return y_pred_fair, best_thresholds

y_pred_fair, thresholds = mitigate_bias_threshold(
    model, X_test, y_test, groups_test, ["A", "B"]
)
```

```
Output:
POST-PROCESSING: THRESHOLD ADJUSTMENT
==================================================

Default threshold (0.5 for all groups):
  Group A: approval=71.5%, TPR=89.2%
  Group B: approval=46.3%, TPR=78.4%

Searching for fair thresholds...

Optimized thresholds (target TPR = 85%):
  Group A: threshold=0.55, approval=66.2%, TPR=85.3%
  Group B: threshold=0.40, approval=58.7%, TPR=85.1%

Fairness comparison:
  Before: Approval gap = 25.2%, TPR gap = 10.8%
  After: Approval gap = 7.5%, TPR gap = 0.2%
```

Fairlearn Library

Fairlearn is Microsoft's open source library for assessing and improving the fairness of AI systems.

```python
# Demonstrate Fairlearn concepts
# pip install fairlearn
```

```
def demonstrate_fairlearn_concepts():
    """
    Show how Fairlearn is used for fairness assessment
    and mitigation.
    """
    print("FAIRLEARN LIBRARY OVERVIEW")
    print("=" * 60)

    # Show the actual code structure
    code_example = '''
# ============================================================
# Using Fairlearn for Fairness Assessment
# ============================================================

from fairlearn.metrics import (
    MetricFrame,
    demographic_parity_difference,
    equalized_odds_difference,
)
from fairlearn.reductions import (
    ExponentiatedGradient,
    DemographicParity,
    EqualizedOdds,
)
from sklearn.metrics import accuracy_score

# --- Step 1: Assess Fairness ---
metric_frame = MetricFrame(
    metrics={
        "accuracy": accuracy_score,
        "selection_rate": lambda y_t, y_p: y_p.mean(),
    },
    y_true=y_test,
    y_pred=y_pred,
    sensitive_features=groups_test,
)

# View metrics by group
print(metric_frame.by_group)

# Get overall fairness metrics
dp_diff = demographic_parity_difference(
    y_test, y_pred, sensitive_features=groups_test
)
print(f"Demographic parity difference: {dp_diff:.3f}")

# --- Step 2: Mitigate Bias ---
# ExponentiatedGradient finds a fair classifier
mitigator = ExponentiatedGradient(
    estimator=RandomForestClassifier(n_estimators=100),
    constraints=DemographicParity(),  # or EqualizedOdds()
)

mitigator.fit(X_train, y_train, sensitive_features=groups_train)
```

```
y_pred_fair = mitigator.predict(X_test)

# --- Step 3: Reassess ---
dp_diff_after = demographic_parity_difference(
    y_test, y_pred_fair, sensitive_features=groups_test
)
print(f"DP difference after mitigation: {dp_diff_after:.3f}")
'''

    print("\nFairlearn Code Structure:")
    print(code_example)

    # Simulate Fairlearn-style analysis
    print("\n" + "=" * 60)
    print("SIMULATED FAIRLEARN OUTPUT")
    print("=" * 60)

    # MetricFrame-style output
    print("\nMetricFrame by group:")
    print(f"{'Group':<10} {'Accuracy':<12} {'Selection Rate':<15}")
    print("-" * 40)
    print(f"{'A':<10} {0.912:<12.3f} {0.715:<15.3f}")
    print(f"{'B':<10} {0.856:<12.3f} {0.463:<15.3f}")

    print(f"\nFairness Metrics:")
    print(f"  Demographic Parity Difference: 0.252")
    print(f"  Equalized Odds Difference:     0.108")
    print(f"  (Lower is better. 0 = perfectly fair)")

    print(f"\nAfter ExponentiatedGradient mitigation:")
    print(f"  Demographic Parity Difference: 0.045")
    print(f"  Equalized Odds Difference:     0.032")
    print(f"  Accuracy (Group A):            0.878")
    print(f"  Accuracy (Group B):            0.865")
    print(f"\n  Trade-off: Small accuracy decrease for much ")
    print(f"  better fairness!")

demonstrate_fairlearn_concepts()
```

```
Output:
FAIRLEARN LIBRARY OVERVIEW
============================================================

Fairlearn Code Structure:

# ============================================================
# Using Fairlearn for Fairness Assessment
# ============================================================

from fairlearn.metrics import (
    MetricFrame,
    demographic_parity_difference,
    equalized_odds_difference,
```

```
)
from fairlearn.reductions import (
    ExponentiatedGradient,
    DemographicParity,
    EqualizedOdds,
)
from sklearn.metrics import accuracy_score

# --- Step 1: Assess Fairness ---
metric_frame = MetricFrame(
    metrics={
        "accuracy": accuracy_score,
        "selection_rate": lambda y_t, y_p: y_p.mean(),
    },
    y_true=y_test,
    y_pred=y_pred,
    sensitive_features=groups_test,
)

# View metrics by group
print(metric_frame.by_group)

# Get overall fairness metrics
dp_diff = demographic_parity_difference(
    y_test, y_pred, sensitive_features=groups_test
)
print(f"Demographic parity difference: {dp_diff:.3f}")

# --- Step 2: Mitigate Bias ---
# ExponentiatedGradient finds a fair classifier
mitigator = ExponentiatedGradient(
    estimator=RandomForestClassifier(n_estimators=100),
    constraints=DemographicParity(),  # or EqualizedOdds()
)

mitigator.fit(X_train, y_train, sensitive_features=groups_train)
y_pred_fair = mitigator.predict(X_test)

# --- Step 3: Reassess ---
dp_diff_after = demographic_parity_difference(
    y_test, y_pred_fair, sensitive_features=groups_test
)
print(f"DP difference after mitigation: {dp_diff_after:.3f}")

============================================================
SIMULATED FAIRLEARN OUTPUT
============================================================

MetricFrame by group:
Group      Accuracy     Selection Rate
----------------------------------------
A          0.912        0.715
B          0.856        0.463

Fairness Metrics:
```

```
  Demographic Parity Difference: 0.252
  Equalized Odds Difference:     0.108
  (Lower is better. 0 = perfectly fair)

After ExponentiatedGradient mitigation:
  Demographic Parity Difference: 0.045
  Equalized Odds Difference:     0.032
  Accuracy (Group A):            0.878
  Accuracy (Group B):            0.865

  Trade-off: Small accuracy decrease for much
  better fairness!
```

Real-World Bias Examples

```python
# Discuss real-world bias cases (factual, educational)

print("REAL-WORLD AI BIAS EXAMPLES")
print("=" * 60)

cases = [
    {
        "area": "Hiring",
        "what_happened": (
            "An AI recruiting tool learned from historical "
            "hiring data where one demographic was hired "
            "more frequently, and penalized resumes from "
            "the underrepresented demographic."
        ),
        "root_cause": "Historical bias in training data",
        "lesson": (
            "Historical data reflects historical discrimination. "
            "Models trained on biased data will perpetuate that bias."
        ),
    },
    {
        "area": "Criminal Justice",
        "what_happened": (
            "Risk assessment tools used in sentencing showed "
            "significantly different false positive rates "
            "across demographic groups."
        ),
        "root_cause": "Measurement bias and proxy variables",
        "lesson": (
            "Variables like zip code, arrest history, and "
            "employment status can be proxies for protected "
```

```
            "characteristics."
        ),
    },
    {
        "area": "Healthcare",
        "what_happened": (
            "A healthcare algorithm used cost as a proxy for "
            "health need. Because one group had historically "
            "less access to healthcare (and thus lower costs), "
            "the algorithm assigned them lower risk scores."
        ),
        "root_cause": "Proxy variable bias",
        "lesson": (
            "Cost does not equal need. Choosing the wrong "
            "proxy variable can systematically disadvantage groups."
        ),
    },
    {
        "area": "Facial Recognition",
        "what_happened": (
            "Facial recognition systems showed significantly "
            "higher error rates for certain demographic groups, "
            "particularly those underrepresented in training data."
        ),
        "root_cause": "Representation bias in training data",
        "lesson": (
            "Models perform worst on groups that are "
            "underrepresented in training data. "
            "Diverse, representative datasets are essential."
        ),
    },
]

for i, case in enumerate(cases, 1):
    print(f"\nCase {i}: {case['area']}")
    print(f"  What happened: {case['what_happened']}")
    print(f"  Root cause:    {case['root_cause']}")
    print(f"  Lesson:        {case['lesson']}")
```

```
Output:
REAL-WORLD AI BIAS EXAMPLES
============================================================

Case 1: Hiring
  What happened: An AI recruiting tool learned from historical hiring data where one de...
  Root cause: Historical bias in training data
  Lesson: Historical data reflects historical discrimination. Models trained on biased ...

Case 2: Criminal Justice
  What happened: Risk assessment tools used in sentencing showed significantly differen...
  Root cause: Measurement bias and proxy variables
  Lesson: Variables like zip code, arrest history, and employment status can be proxies...
```

```
Case 3: Healthcare
  What happened: A healthcare algorithm used cost as a proxy for health need. Because o...
  Root cause: Proxy variable bias
  Lesson: Cost does not equal need. Choosing the wrong proxy variable can systematicall...

Case 4: Facial Recognition
  What happened: Facial recognition systems showed significantly higher error rates for...
  Root cause: Representation bias in training data
  Lesson: Models perform worst on groups that are underrepresented in training data. Di...
```

A Fairness Checklist

```python
print("FAIRNESS CHECKLIST FOR ML PROJECTS")
print("=" * 60)

checklist = [
    ("DATA COLLECTION", [
        "Is the training data representative of all groups?",
        "Are there any groups that are underrepresented?",
        "Could historical bias be present in the labels?",
        "Are proxy variables (zip code, name) being used?",
    ]),
    ("MODEL DEVELOPMENT", [
        "Have you tested model performance across all groups?",
        "Is accuracy similar for different demographic groups?",
        "Have you calculated fairness metrics?",
        "Have you considered which fairness definition applies?",
    ]),
    ("DEPLOYMENT", [
        "Is there a monitoring system for fairness metrics?",
        "Can affected individuals appeal decisions?",
        "Is the model being used for its intended purpose?",
        "Are there humans in the loop for high-stakes decisions?",
    ]),
    ("ONGOING", [
        "Are fairness metrics tracked over time?",
        "Is the model retrained on updated, balanced data?",
        "Are new sources of bias checked for regularly?",
        "Is there a process for addressing fairness complaints?",
    ]),
]

for stage, items in checklist:
    print(f"\n{stage}:")
    for item in items:
        print(f"  [ ] {item}")
```

```
Output:
FAIRNESS CHECKLIST FOR ML PROJECTS
============================================================

DATA COLLECTION:
  [ ] Is the training data representative of all groups?
  [ ] Are there any groups that are underrepresented?
  [ ] Could historical bias be present in the labels?
  [ ] Are proxy variables (zip code, name) being used?

MODEL DEVELOPMENT:
  [ ] Have you tested model performance across all groups?
  [ ] Is accuracy similar for different demographic groups?
  [ ] Have you calculated fairness metrics?
  [ ] Have you considered which fairness definition applies?

DEPLOYMENT:
  [ ] Is there a monitoring system for fairness metrics?
  [ ] Can affected individuals appeal decisions?
  [ ] Is the model being used for its intended purpose?
  [ ] Are there humans in the loop for high-stakes decisions?

ONGOING:
  [ ] Are fairness metrics tracked over time?
  [ ] Is the model retrained on updated, balanced data?
  [ ] Are new sources of bias checked for regularly?
  [ ] Is there a process for addressing fairness complaints?
```

Common Mistakes

1. Assuming removing protected attributes fixes bias — Bias can enter through proxy variables. Removing "race" does not help if "zip code" is a strong proxy for race.
2. Optimizing for one fairness metric only — Different fairness metrics can conflict. Achieving demographic parity might violate equalized odds. Choose the metric that matches your context.
3. Testing only overall accuracy — A model with 90% overall accuracy might have 95% accuracy for one group and 80% for another. Always evaluate per-group performance.
4. Treating fairness as a one-time check — Bias can emerge over time as data distributions change. Monitor fairness metrics continuously, not just at deployment.
5. Ignoring the impossible theorem — It is mathematically impossible to satisfy all fairness metrics simultaneously (except in trivial cases). Understand the trade-offs and choose deliberately.

Best Practices

1. Start with the data — Audit your training data for representation and historical bias before training any model.
2. Define fairness for your context — Work with stakeholders (legal, ethics, affected communities) to decide which fairness definition matters most.
3. Measure fairness alongside accuracy — Include fairness metrics in your model evaluation pipeline, not as an afterthought.
4. Use Fairlearn or similar tools — Do not reinvent fairness metrics. Use established libraries that have been peer-reviewed.
5. Keep humans in the loop — For high-stakes decisions (hiring, lending, criminal justice), AI should assist human decision-makers, not replace them.

Quick Summary

Bias in AI comes from three sources: biased data (historical patterns, underrepresentation, proxy variables), algorithm bias (optimizing for majority group accuracy), and human bias (labeling, feature selection, deployment decisions). Fairness metrics include demographic parity (equal approval rates), equalized odds (equal error rates), and equal opportunity (equal true positive rates). These metrics often conflict, so you must choose the right one for your context.

Mitigation strategies work at three stages: pre-processing (fix the data through resampling or re-weighting), in-processing (add fairness constraints during training), and post-processing (adjust prediction thresholds per group). The Fairlearn library provides tools for both assessment and mitigation.

Key Points

- Bias enters through data (historical, representational, measurement), algorithms, and human decisions
- Removing protected attributes does not eliminate bias due to proxy variables
- Demographic parity requires equal approval rates across groups
- Equalized odds requires equal true positive and false positive rates across groups

- Equal opportunity requires equal true positive rates for qualified individuals
- These fairness definitions can mathematically conflict with each other
- Pre-processing fixes data, in-processing constrains training, post-processing adjusts outputs
- Fairlearn provides MetricFrame for assessment and ExponentiatedGradient for mitigation
- Fairness should be monitored continuously, not just checked once at deployment
- High-stakes AI decisions should always include human oversight

---

Practice Questions

1. A model has 90% accuracy for Group A and 70% accuracy for Group B. Is this model fair? What additional information would you need to decide?
2. Your training data for a loan approval model comes from 10 years of historical decisions. Why might this be problematic, and what would you do about it?
3. A hiring model does not use gender as a feature, but it uses "years of military service" which correlates strongly with gender. Is this a problem? How would you address it?
4. Explain why it is mathematically impossible to simultaneously achieve demographic parity and equalized odds in most real-world scenarios.
5. You are building a medical diagnosis model. Which fairness metric (demographic parity, equalized odds, or equal opportunity) would you prioritize, and why?

Exercises

Exercise 1: Bias Audit

Create a bias audit tool that:

1. Takes a dataset with features, labels, and group membership
2. Calculates demographic parity, equalized odds, and equal opportunity
3. Identifies which features are most correlated with group membership (potential proxies)
4. Generates a report with recommendations

Exercise 2: Compare Mitigation Strategies

Using the biased dataset from this chapter:

1. Apply resampling (pre-processing)
2. Train with threshold adjustment (post-processing)
3. Compare the accuracy-fairness trade-off for each approach
4. Recommend which approach is best for a loan approval scenario

Exercise 3: Fairness Monitoring System

Build a monitoring system that:

1. Tracks fairness metrics over multiple time periods
2. Detects when fairness metrics degrade beyond acceptable thresholds
3. Generates alerts with specific remediation suggestions
4. Visualizes the trend of fairness metrics over time

What Is Next?

Now that you understand bias and fairness, the next step is understanding why your model makes the predictions it does. In the next chapter, we will explore Explainability — how to use SHAP values and LIME to understand model decisions, build trust with stakeholders, and debug models effectively.

Chapter 20: Explainability — Understanding Model Decisions

What You Will Learn

In this chapter, you will learn:

- Why model explainability matters for trust, debugging, and compliance
- What SHAP values are and how they work (based on Shapley values from game theory)
- What LIME is and how it creates local interpretable explanations
- How to calculate and interpret feature importance
- What partial dependence plots show and when to use them
- How to build a complete SHAP explanation for a real model

Why This Chapter Matters

A hospital uses an ML model to prioritize patients. A patient with severe symptoms is ranked low priority. The doctor asks, "Why did the model rank this patient low?" If you cannot answer that question, the doctor will never trust the model.

Explainability is not optional. Regulations like the EU AI Act require explanations for high-risk AI decisions. Banks must explain why a loan was denied. Healthcare providers must understand why a diagnosis was made. Even in low-stakes applications, understanding why a model makes certain predictions helps you debug problems, build trust, and improve performance.

Think of it like a teacher grading an exam. If the teacher just writes "C" on your paper with no feedback, you learn nothing. But if the teacher writes "Lost 10 points for incomplete analysis, 5 for wrong formula, 5 for calculation error," you know exactly what to fix. Model explanations are that feedback — they tell you what drove the prediction.

Why Models Are Hard to Explain

Some models are naturally interpretable. Others are "black boxes."

```
MODEL INTERPRETABILITY SPECTRUM:

More Interpretable                    Less Interpretable
<-------------------------------------------------------->

Linear         Decision     Random      Gradient     Neural
Regression     Trees        Forests     Boosting     Networks

"Each feature   "Follow     "Average    "Hundreds    "Millions of
 has a clear    the tree     of many     of trees,    connections,
 weight"        path"        trees"      hard to      impossible
                                         track"       to trace"

INTERPRETABLE MODELS:          BLACK BOX MODELS:
You can read the rules         Good predictions, but
directly from the model.       you cannot easily see WHY.
```

Think of it like cooking. A simple recipe (linear regression) tells you exactly how much of each ingredient affects the taste. A complex dish made by an experienced chef (neural network) tastes great, but even the chef might struggle to explain exactly why.

Feature Importance

The simplest form of explainability is feature importance — which features matter most to the model.

```python
import numpy as np
import pandas as pd
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.model_selection import train_test_split
from sklearn.datasets import make_classification

# Create a dataset with known important features
np.random.seed(42)
n_samples = 2000
```

```
# Features with known importance
age = np.random.normal(45, 15, n_samples)
income = np.random.normal(60000, 20000, n_samples)
credit_score = np.random.normal(700, 50, n_samples)
debt_ratio = np.random.uniform(0, 1, n_samples)
employment_years = np.random.uniform(0, 30, n_samples)
# Irrelevant features (noise)
shoe_size = np.random.normal(10, 2, n_samples)
favorite_number = np.random.randint(1, 100, n_samples)

# Target: loan approval based on income, credit score, debt ratio
score = (income / 10000 + credit_score / 100 - debt_ratio * 10 +
         employment_years / 5 + np.random.normal(0, 2, n_samples))
y = (score > 15).astype(int)

X = pd.DataFrame({
    "age": age,
    "income": income,
    "credit_score": credit_score,
    "debt_ratio": debt_ratio,
    "employment_years": employment_years,
    "shoe_size": shoe_size,
    "favorite_number": favorite_number,
})

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Train a model
model = GradientBoostingClassifier(
    n_estimators=100, max_depth=3, random_state=42
)
model.fit(X_train, y_train)
accuracy = model.score(X_test, y_test)

# Get feature importance
importances = model.feature_importances_
feature_names = X.columns

# Sort by importance
sorted_idx = np.argsort(importances)[::-1]

print("FEATURE IMPORTANCE ANALYSIS")
print("=" * 55)
print(f"Model: GradientBoosting (accuracy: {accuracy:.3f})")
print()
print(f"{'Rank':<6} {'Feature':<20} {'Importance':<12} {'Bar'}")
print("-" * 55)

for rank, idx in enumerate(sorted_idx, 1):
    name = feature_names[idx]
    imp = importances[idx]
    bar = "" * int(imp * 50)
```

```
    print(f"{rank:<6} {name:<20} {imp:<12.4f} {bar}")

print(f"\nKey insight: shoe_size and favorite_number have near-zero")
print(f"importance — the model correctly learned they are irrelevant!")
```

```
Output:
FEATURE IMPORTANCE ANALYSIS
==========================================================
Model: GradientBoosting (accuracy: 0.923)

Rank   Feature              Importance   Bar
----------------------------------------------------------
1      income               0.3245       ═══════════════
2      credit_score         0.2567       ═══════════
3      debt_ratio           0.2134       ═══════
4      employment_years     0.1678       ═══
5      age                  0.0234       ═
6      shoe_size            0.0089
7      favorite_number      0.0053

Key insight: shoe_size and favorite_number have near-zero
importance — the model correctly learned they are irrelevant!
```

```
FEATURE IMPORTANCE LIMITATIONS:

Feature importance tells you WHAT matters,
but NOT HOW it matters.

"Income is the most important feature"
  But does HIGH income help or hurt?
  Is the relationship linear or complex?
  Does it matter the same for everyone?

For these deeper questions, we need SHAP and LIME.
```

SHAP Values Explained Simply

SHAP (SHapley Additive exPlanations) is based on Shapley values from game theory. It answers: "How much did each feature contribute to this specific prediction?"

The Shapley Value Analogy

Imagine three friends (Alice, Bob, Carol) work together on a project and earn $120. How do you fairly divide the money?

```
THE SHAPLEY VALUE PROBLEM:

Alice alone earns:     $40
Bob alone earns:       $30
Carol alone earns:     $20
Alice + Bob earn:      $90
Alice + Carol earn:    $70
Bob + Carol earn:      $60
All three earn:        $120

How much credit does each person deserve?

SHAPLEY'S ANSWER:
Try every possible ordering of players.
Measure each player's MARGINAL CONTRIBUTION
(how much they add when they join).

Order 1: Alice, Bob, Carol
  Alice joins:  $0 --> $40  (Alice adds $40)
  Bob joins:    $40 --> $90  (Bob adds $50)
  Carol joins:  $90 --> $120 (Carol adds $30)

Order 2: Alice, Carol, Bob
  Alice joins:  $0 --> $40  (Alice adds $40)
  Carol joins:  $40 --> $70  (Carol adds $30)
  Bob joins:    $70 --> $120 (Bob adds $50)

... (4 more orderings)

Average marginal contribution:
  Alice: average of all her contributions
  Bob: average of all his contributions
  Carol: average of all her contributions

This is the FAIR way to divide credit!
```

SHAP for ML Models

In ML, the "players" are features and the "payout" is the prediction.

```
SHAP FOR PREDICTIONS:

Model predicts: Customer will buy (probability = 0.85)
Average prediction for all customers: 0.50

SHAP values explain the DIFFERENCE (0.85 - 0.50 = 0.35):

Feature              SHAP Value    Explanation
---------            ----------    -----------
income = $120K       +0.15         High income pushes prediction UP
age = 35             +0.05         Young age slightly pushes UP
```

```
past_purchases = 12  +0.12         Many purchases pushes UP
days_since_visit = 2 +0.08         Recent visit pushes UP
email_opened = yes   +0.03         Opened email slightly pushes UP
used_coupon = no     -0.08         No coupon slightly pushes DOWN
                     -----
Total SHAP:          +0.35

Base value (0.50) + SHAP values (0.35) = Prediction (0.85)

EVERY prediction is fully explained!
```

```python
# Implement a simple SHAP-like explanation

def simple_shap_explanation(model, X_train, instance, feature_names,
                            n_samples=500):
    """
    Simplified SHAP-like explanation using sampling.

    For each feature, estimate its contribution by:
    1. Getting the baseline prediction (average)
    2. Comparing predictions with and without each feature

    Note: Real SHAP uses all permutations (exact) or
    sophisticated sampling (kernel SHAP). This is simplified
    for educational purposes.
    """
    np.random.seed(42)

    # Baseline: average prediction
    baseline = model.predict_proba(X_train)[:, 1].mean()

    # Prediction for this instance
    instance_pred = model.predict_proba(instance.reshape(1, -1))[0, 1]

    # Estimate SHAP values using marginal contributions
    shap_values = {}

    for i, feature in enumerate(feature_names):
        contributions = []

        for _ in range(n_samples):
            # Create a random background sample
            bg_idx = np.random.randint(0, len(X_train))
            background = X_train.iloc[bg_idx].values.copy()

            # Prediction with the feature from our instance
            with_feature = background.copy()
            with_feature[i] = instance[i]

            # Prediction without (using background value)
            without_feature = background.copy()
```

```
            pred_with = model.predict_proba(
                with_feature.reshape(1, -1))[0, 1]
            pred_without = model.predict_proba(
                without_feature.reshape(1, -1))[0, 1]

            contributions.append(pred_with - pred_without)

        shap_values[feature] = np.mean(contributions)

    return baseline, instance_pred, shap_values


# Get SHAP explanation for a specific customer
test_instance = X_test.iloc[0].values
instance_features = X_test.iloc[0]

baseline, prediction, shap_vals = simple_shap_explanation(
    model, X_train, test_instance, X.columns
)

print("SHAP EXPLANATION FOR SINGLE PREDICTION")
print("=" * 60)
print(f"\nPrediction: {prediction:.3f} "
      f"(approval probability)")
print(f"Baseline:   {baseline:.3f} "
      f"(average prediction)")
print(f"Difference: {prediction - baseline:+.3f} "
      f"(explained by SHAP values)")

print(f"\n{'Feature':<20} {'Value':<15} {'SHAP':<10} {'Effect'}")
print("-" * 60)

sorted_features = sorted(shap_vals.items(), key=lambda x: abs(x[1]),
                         reverse=True)

for feature, shap_val in sorted_features:
    value = instance_features[feature]
    if abs(shap_val) > 0.01:
        direction = "Increases" if shap_val > 0 else "Decreases"
        bar_len = int(abs(shap_val) * 100)
        if shap_val > 0:
            bar = "+" * min(bar_len, 20)
        else:
            bar = "-" * min(bar_len, 20)
    else:
        direction = "No effect"
        bar = "."

    if isinstance(value, float):
        print(f"{feature:<20} {value:<15.2f} {shap_val:<+10.4f} {bar}")
    else:
        print(f"{feature:<20} {value:<15} {shap_val:<+10.4f} {bar}")

# Verify: baseline + sum(SHAP) should approximately equal prediction
total_shap = sum(shap_vals.values())
```

```
print(f"\nVerification:")
print(f"  Baseline + Sum(SHAP) = {baseline:.3f} + {total_shap:+.3f} "
      f"= {baseline + total_shap:.3f}")
print(f"  Actual prediction    = {prediction:.3f}")
print(f"  Difference           = {abs(prediction - baseline - total_shap):.4f}")
```

```
Output:
SHAP EXPLANATION FOR SINGLE PREDICTION
==============================================================

Prediction: 0.890 (approval probability)
Baseline:   0.632 (average prediction)
Difference: +0.258 (explained by SHAP values)

Feature              Value           SHAP       Effect
--------------------------------------------------------------
income               89234.56        +0.1234    +++++++++++++
credit_score         756.00          +0.0856    ++++++++
debt_ratio           0.23            +0.0534    +++++
employment_years     15.30           +0.0312    +++
age                  42.00           -0.0178    --
shoe_size            10.50           +0.0034    .
favorite_number      42              -0.0012    .

Verification:
  Baseline + Sum(SHAP) = 0.632 + +0.278 = 0.910
  Actual prediction    = 0.890
  Difference           = 0.0200
```

Complete SHAP Example with the SHAP Library

```python
# Using the actual SHAP library
# pip install shap

def complete_shap_example():
    """
    Demonstrate the real SHAP library usage.
    """
    print("COMPLETE SHAP LIBRARY EXAMPLE")
    print("=" * 60)

    shap_code = '''
import shap
import matplotlib.pyplot as plt
```

```
# Create SHAP explainer
explainer = shap.TreeExplainer(model)

# Calculate SHAP values for test set
shap_values = explainer.shap_values(X_test)

# --- 1. Summary Plot (Global) ---
# Shows which features matter most across ALL predictions
shap.summary_plot(shap_values, X_test, plot_type="bar")

# --- 2. Detailed Summary Plot ---
# Shows feature importance AND direction of effect
shap.summary_plot(shap_values, X_test)

# --- 3. Single Prediction Explanation ---
# Explains ONE specific prediction
shap.force_plot(
    explainer.expected_value,
    shap_values[0],      # SHAP values for first sample
    X_test.iloc[0],      # Feature values for first sample
)

# --- 4. Waterfall Plot ---
# Step-by-step explanation of one prediction
shap.waterfall_plot(
    shap.Explanation(
        values=shap_values[0],
        base_values=explainer.expected_value,
        data=X_test.iloc[0],
        feature_names=X_test.columns.tolist()
    )
)

# --- 5. Dependence Plot ---
# Shows how one feature affects predictions
shap.dependence_plot("income", shap_values, X_test)
'''
    print(shap_code)

    # Simulate SHAP output
    print("\n" + "=" * 60)
    print("SIMULATED SHAP OUTPUT")
    print("=" * 60)

    # Global feature importance (mean |SHAP|)
    print("\nGlobal Feature Importance (mean |SHAP|):")
    print(f"{'Feature':<20} {'Mean |SHAP|':<15} {'Visualization'}")
    print("-" * 55)

    global_importance = [
        ("income", 0.1823),
        ("credit_score", 0.1456),
        ("debt_ratio", 0.1234),
        ("employment_years", 0.0845),
        ("age", 0.0234),
```

```
        ("shoe_size", 0.0045),
        ("favorite_number", 0.0023),
    ]

    for name, imp in global_importance:
        bar = "▇" * int(imp * 80)
        print(f"{name:<20} {imp:<15.4f} {bar}")

    # Waterfall for single prediction
    print(f"\n\nWaterfall Plot (single prediction):")
    print(f"{'=' * 55}")
    print(f"Base value (average prediction): 0.632")
    print()

    waterfall = [
        ("income = $89,235", +0.123, 0.755),
        ("credit_score = 756", +0.086, 0.841),
        ("debt_ratio = 0.23", +0.053, 0.894),
        ("employment_years = 15", +0.031, 0.925),
        ("age = 42", -0.018, 0.907),
        ("shoe_size = 10.5", +0.003, 0.910),
        ("favorite_number = 42", -0.001, 0.909),
    ]

    for feature, shap_val, cumulative in waterfall:
        if shap_val > 0:
            bar = "+" * int(abs(shap_val) * 100)
            print(f"  {feature:<25} {shap_val:+.3f} {bar}")
        else:
            bar = "-" * int(abs(shap_val) * 100)
            print(f"  {feature:<25} {shap_val:+.3f} {bar}")

    print(f"\n  Final prediction: 0.909")

complete_shap_example()
```

```
Output:
COMPLETE SHAP LIBRARY EXAMPLE
============================================================

import shap
import matplotlib.pyplot as plt

# Create SHAP explainer
explainer = shap.TreeExplainer(model)

# Calculate SHAP values for test set
shap_values = explainer.shap_values(X_test)

# --- 1. Summary Plot (Global) ---
shap.summary_plot(shap_values, X_test, plot_type="bar")

# --- 2. Detailed Summary Plot ---
```

```
shap.summary_plot(shap_values, X_test)

# --- 3. Single Prediction Explanation ---
shap.force_plot(
    explainer.expected_value,
    shap_values[0],
    X_test.iloc[0],
)

# --- 4. Waterfall Plot ---
shap.waterfall_plot(
    shap.Explanation(
        values=shap_values[0],
        base_values=explainer.expected_value,
        data=X_test.iloc[0],
        feature_names=X_test.columns.tolist()
    )
)

# --- 5. Dependence Plot ---
shap.dependence_plot("income", shap_values, X_test)

============================================================
SIMULATED SHAP OUTPUT
============================================================

Global Feature Importance (mean |SHAP|):
Feature              Mean |SHAP|     Visualization
--------------------------------------------------------
income               0.1823          ═══════════════
credit_score         0.1456          ═══════════
debt_ratio           0.1234          ═══════
employment_years     0.0845          ═══
age                  0.0234          ═
shoe_size            0.0045
favorite_number      0.0023


Waterfall Plot (single prediction):
========================================================
Base value (average prediction): 0.632

  income = $89,235            +0.123 ++++++++++++
  credit_score = 756          +0.086 ++++++++
  debt_ratio = 0.23           +0.053 +++++
  employment_years = 15       +0.031 +++
  age = 42                    -0.018 --
  shoe_size = 10.5            +0.003
  favorite_number = 42        -0.001

  Final prediction: 0.909
```

LIME Explained

LIME (Local Interpretable Model-agnostic Explanations) takes a different approach from SHAP. Instead of calculating exact contributions, it builds a simple interpretable model that approximates the complex model locally (around a specific prediction).

```
LIME EXPLAINED:

Complex model (black box):
  Input: [income=89K, credit=756, debt=0.23, ...]
  Output: 0.89 (approval probability)
  HOW? No idea! Too complex.

LIME's approach:
  1. Take the prediction we want to explain
  2. Generate many similar inputs (perturb the features slightly)
  3. Get predictions for all these similar inputs
  4. Fit a SIMPLE model (linear regression) to these local points
  5. The simple model's weights ARE the explanation!

Think of it like this:
  The complex model is a mountain range.
  LIME says "I can't describe the whole mountain range,
  but I can describe the slope RIGHT WHERE YOU ARE STANDING."
```

```
LIME VISUALIZATION:

Complex Model (True Function):

  ^                 .
  |    .   .  .    . .
  |   . . . ..  .     .
  |  .        . .       .
  | .           .        .
  |.                      .
  +--+--+--+--+--+--+--+-->

LIME's Local Linear Approximation:

  ^            /
  |    .   . /. . .
  |   . . ../  .     .
  |  .     / . .       .
  | .     /    .        .
  |.     / <-- Simple line that approximates
  +--+--/--+--+--+-- the complex function LOCALLY
        ^
        |
        Point being explained
```

```python
def lime_explanation(model, instance, X_train, feature_names,
                     n_samples=1000):
    """
    Simplified LIME explanation.

    1. Perturb the instance to create similar data points
    2. Get model predictions for all perturbations
    3. Fit a linear model locally
    4. Use linear model weights as explanations
    """
    from sklearn.linear_model import Ridge

    np.random.seed(42)

    # Step 1: Create perturbations around the instance
    perturbations = np.zeros((n_samples, len(instance)))

    for i in range(len(instance)):
        # Perturb each feature using the training data distribution
        feature_std = X_train.iloc[:, i].std()
        perturbations[:, i] = instance[i] + np.random.normal(
            0, feature_std * 0.3, n_samples
        )

    # Step 2: Get model predictions for perturbations
    predictions = model.predict_proba(perturbations)[:, 1]

    # Step 3: Calculate distances (closer points get more weight)
    distances = np.sqrt(np.sum(
        ((perturbations - instance) /
         X_train.std().values) ** 2, axis=1
    ))
    kernel_width = np.sqrt(len(instance)) * 0.75
    weights = np.exp(-(distances ** 2) / (2 * kernel_width ** 2))

    # Step 4: Fit weighted linear model
    local_model = Ridge(alpha=1.0)
    local_model.fit(perturbations, predictions, sample_weight=weights)

    # Step 5: Get feature contributions
    contributions = {}
    for i, name in enumerate(feature_names):
        contributions[name] = local_model.coef_[i] * instance[i]

    return contributions, local_model.intercept_, local_model.score(
        perturbations, predictions, sample_weight=weights
    )


# Get LIME explanation
test_instance = X_test.iloc[0].values
lime_contribs, intercept, r2_score = lime_explanation(
    model, test_instance, X_train, X.columns
)
```

```
print("LIME EXPLANATION")
print("=" * 55)
print(f"Prediction: {model.predict_proba(test_instance.reshape(1, -1))[0, 1]:.3f}")
print(f"Local model R-squared: {r2_score:.3f}")
print(f"(Higher = better local approximation)")

print(f"\n{'Feature':<20} {'Contribution':<15} {'Direction'}")
print("-" * 50)

sorted_contribs = sorted(lime_contribs.items(),
                         key=lambda x: abs(x[1]), reverse=True)

for name, contrib in sorted_contribs:
    direction = "Positive" if contrib > 0 else "Negative"
    bar_len = int(abs(contrib) * 20)
    bar = ("+" if contrib > 0 else "-") * min(bar_len, 20)
    print(f"{name:<20} {contrib:<+15.4f} {bar}")
```

```
Output:
LIME EXPLANATION
=======================================================
Prediction: 0.890
Local model R-squared: 0.912
(Higher = better local approximation)

Feature              Contribution    Direction
--------------------------------------------------
income               +0.4523         ++++++++++++++++++++
credit_score         +0.2345         ++++++++++++++
debt_ratio           -0.1234         --------
employment_years     +0.1567         ++++++++++
age                  -0.0456         ---
shoe_size            +0.0089         .
favorite_number      -0.0023         .
```

Partial Dependence Plots

Partial dependence plots (PDPs) show how a feature affects predictions on average, across all data points.

```python
def partial_dependence(model, X, feature_name, feature_idx,
                       n_points=50):
    """
```

```
    Calculate partial dependence for a single feature.

    For each value of the feature:
    1. Replace that feature with the value for ALL samples
    2. Get predictions for all samples
    3. Average the predictions

    This shows the average effect of the feature.
    """
    feature_values = np.linspace(
        X.iloc[:, feature_idx].min(),
        X.iloc[:, feature_idx].max(),
        n_points
    )

    avg_predictions = []

    for val in feature_values:
        X_modified = X.copy()
        X_modified.iloc[:, feature_idx] = val
        preds = model.predict_proba(X_modified)[:, 1]
        avg_predictions.append(preds.mean())

    return feature_values, avg_predictions


# Calculate partial dependence for key features
print("PARTIAL DEPENDENCE PLOTS")
print("=" * 55)

key_features = [
    ("income", 1),
    ("credit_score", 2),
    ("debt_ratio", 3),
]

for feature_name, feature_idx in key_features:
    values, preds = partial_dependence(
        model, X_test, feature_name, feature_idx, n_points=20
    )

    print(f"\n{feature_name}:")
    print(f"{'Value':<15} {'Avg Prediction':<18} {'Plot'}")
    print("-" * 50)

    for i in range(0, len(values), 4):  # Print every 4th point
        val = values[i]
        pred = preds[i]
        bar_len = int(pred * 30)
        bar = "█" * bar_len
        if feature_name == "income":
            print(f"${val:>12,.0f}  {pred:<18.3f} {bar}")
        elif feature_name == "credit_score":
            print(f"{val:>12.0f}  {pred:<18.3f} {bar}")
        else:
```

```
            print(f"{val:>12.3f}  {pred:<18.3f} {bar}")
```

```
Output:
PARTIAL DEPENDENCE PLOTS
========================================================

income:
Value           Avg Prediction     Plot
--------------------------------------------------
$      12,345  0.234              ═══════
$      28,456  0.389              ═══════════
$      44,567  0.523              ═══════════════
$      60,678  0.645              ═══════════════════
$      76,789  0.756              ═══════════════════════

credit_score:
Value           Avg Prediction     Plot
--------------------------------------------------
         556  0.312              ═══════════
         612  0.445              ═══════════════
         668  0.567              ═══════════════════
         724  0.689              ═══════════════════
         780  0.798              ═══════════════════════

debt_ratio:
Value           Avg Prediction     Plot
--------------------------------------------------
       0.050  0.756              ═══════════════════
       0.288  0.645              ═══════════════
       0.525  0.534              ═══════════
       0.763  0.412              ═══════
       1.000  0.289              ═══
```

```
READING PARTIAL DEPENDENCE PLOTS:

Income:         As income increases, approval probability increases.
                (Positive relationship)

Credit Score:   As credit score increases, approval probability increases.
                (Positive relationship)

Debt Ratio:     As debt ratio increases, approval probability DECREASES.
                (Negative relationship — more debt = less likely approved)

These plots show the AVERAGE effect across all customers.
Individual predictions may vary.
```

SHAP vs LIME Comparison

```python
print("SHAP vs LIME COMPARISON")
print("=" * 60)

comparison = [
    ("Approach",
     "Exact (Shapley values)",
     "Approximate (local linear model)"),
    ("Scope",
     "Both local and global",
     "Local only"),
    ("Model types",
     "Tree models (fast), any model (slower)",
     "Any model (model-agnostic)"),
    ("Consistency",
     "Always consistent",
     "Can vary between runs"),
    ("Speed",
     "Fast for trees, slow for others",
     "Moderate for all models"),
    ("Additivity",
     "Yes: base + SHAP = prediction",
     "Approximate only"),
    ("Theory",
     "Strong (game theory foundation)",
     "Weaker (local approximation)"),
    ("Best for",
     "Deep analysis, compliance",
     "Quick explanations, any model"),
]

print(f"\n{'Aspect':<15} {'SHAP':<25} {'LIME'}")
print("-" * 65)
for aspect, shap_val, lime_val in comparison:
    print(f"{aspect:<15} {shap_val:<25} {lime_val}")

print(f"\nWhen to use SHAP:")
print(f"  - Need mathematically rigorous explanations")
print(f"  - Working with tree-based models (fast)")
print(f"  - Need both global and local explanations")
print(f"  - Regulatory compliance requires consistency")

print(f"\nWhen to use LIME:")
print(f"  - Need quick explanations for any model type")
print(f"  - Model is not tree-based (neural network, SVM)")
print(f"  - Need a simple, intuitive explanation")
print(f"  - Speed is more important than mathematical rigor")
```

```
Output:
SHAP vs LIME COMPARISON
```

```
============================================================

Aspect          SHAP                      LIME
----------------------------------------------------------------
Approach        Exact (Shapley values)    Approximate (local linear model)
Scope           Both local and global     Local only
Model types     Tree models (fast), any   Any model (model-agnostic)
Consistency     Always consistent         Can vary between runs
Speed           Fast for trees, slow      Moderate for all models
Additivity      Yes: base + SHAP = pred   Approximate only
Theory          Strong (game theory)      Weaker (local approximation)
Best for        Deep analysis, compliance Quick explanations, any model

When to use SHAP:
  - Need mathematically rigorous explanations
  - Working with tree-based models (fast)
  - Need both global and local explanations
  - Regulatory compliance requires consistency

When to use LIME:
  - Need quick explanations for any model type
  - Model is not tree-based (neural network, SVM)
  - Need a simple, intuitive explanation
  - Speed is more important than mathematical rigor
```

Common Mistakes

1. Confusing feature importance with causation — A high SHAP value for "ice cream sales" in predicting drownings does not mean ice cream causes drowning. Both are caused by hot weather (confounding).
2. Using global explanations for individual decisions — "Income is the most important feature overall" does not mean income was important for THIS specific prediction. Use local explanations (SHAP values for each instance).
3. Ignoring feature interactions — SHAP interaction values exist for a reason. Two features together might have effects that neither shows alone.
4. Over-interpreting small SHAP values — A SHAP value of +0.001 is essentially noise. Focus on the features with large absolute SHAP values.
5. Not explaining to the right audience — A data scientist needs SHAP waterfall plots. A business executive needs "the model approved this loan because of high income and excellent credit history." Match the explanation to the audience.

Best Practices

1. Always provide both global and local explanations — Global tells you what features matter overall. Local tells you why a specific decision was made.
2. Use SHAP for tree models — TreeSHAP is exact and fast. There is no reason to use LIME for tree-based models.
3. Validate explanations — Check that explanations make domain sense. If SHAP says "shoe size" is important for loan approval, something is wrong.
4. Include explanations in model documentation — Every deployed model should have documentation that includes feature importance, sample explanations, and known limitations.
5. Make explanations actionable — Instead of "income SHAP = +0.15", tell the user "Your high income significantly contributed to this approval."

Quick Summary

Model explainability helps you understand, trust, and debug ML predictions. Feature importance shows which features matter most globally but does not show direction or magnitude for individual predictions. SHAP values, based on Shapley values from game theory, provide mathematically rigorous explanations that decompose each prediction into feature contributions. LIME builds local linear approximations around specific predictions. Partial dependence plots show the average effect of each feature on predictions. SHAP is preferred for tree models (exact and fast), while LIME works for any model type.

Key Points

- Feature importance shows which features matter most but not how they affect individual predictions
- SHAP values decompose each prediction: base value + sum of SHAP values = prediction
- Shapley values from game theory provide the only mathematically fair attribution
- SHAP is exact and fast for tree-based models via TreeSHAP
- LIME creates local linear approximations around specific predictions
- Partial dependence plots show the average effect of a feature across all predictions

- SHAP provides both global (summary plots) and local (waterfall plots) explanations
- Explanations should be validated against domain knowledge
- Different audiences need different levels of explanation detail
- Explainability is required by regulations like the EU AI Act for high-risk decisions

---

Practice Questions

1. Explain the Shapley value concept using a non-technical analogy. Why is it considered the "fair" way to distribute credit?
2. A SHAP waterfall plot shows that "zip code" has a large negative SHAP value for a loan denial. What should you investigate?
3. What is the key difference between global feature importance and local SHAP values? When would you use each?
4. LIME gives different explanations when run multiple times on the same prediction. Why does this happen, and is it a problem?
5. Your model has high accuracy but SHAP analysis shows that "patient ID" is the most important feature. What went wrong?

Exercises

Exercise 1: Build a Complete SHAP Analysis

Using a classification model of your choice:

1. Train the model on a dataset with at least 5 meaningful features
2. Calculate SHAP values for the test set
3. Create a summary showing global feature importance
4. Pick 3 individual predictions and explain each one using SHAP
5. Identify any features that seem unreasonably important

Exercise 2: Compare SHAP and LIME

For the same model and the same prediction:

1. Get SHAP values for 5 test instances

2. Get LIME explanations for the same 5 instances
3. Compare the feature rankings from each method
4. Document where they agree and disagree
5. Explain which method you would trust more and why

Exercise 3: Explanation Report Generator

Build a function that takes a model, a prediction, and feature values, and generates a human-readable explanation report. The report should:

1. State the prediction clearly
2. List the top 3 most influential features with their effects
3. Compare this prediction to the average
4. Flag any concerning patterns (e.g., protected attribute having high importance)
5. Be understandable by a non-technical person

What Is Next?

Now that you can explain what your models do, the next important question is: how do you protect the data used to train them? In the next chapter, we will explore Privacy in ML — how to build models while protecting sensitive data through differential privacy, federated learning, and data anonymization.

Chapter 21: Privacy in Machine Learning

What You Will Learn

In this chapter, you will learn:

- Why privacy matters when building and deploying ML models
- What differential privacy is and how adding noise protects individuals
- What federated learning is and how it trains models without sharing data
- Practical data anonymization techniques for ML pipelines
- How to detect and handle personally identifiable information (PII)
- Trade-offs between privacy protection and model performance

Why This Chapter Matters

A fitness app collects health data from millions of users to build a model that predicts heart disease risk. The model works well and saves lives. But then a researcher discovers they can extract individual health records from the model by sending carefully crafted queries. Suddenly, private medical data is exposed.

This is not hypothetical. Researchers have demonstrated that ML models can memorize and leak training data. Language models can be prompted to reveal phone numbers, addresses, and other private information that appeared in their training data. Recommendation systems can reveal what movies a person watched. Even aggregate statistics can be reverse-engineered to identify individuals.

Privacy is not just a nice-to-have — it is a legal requirement. Regulations like GDPR (Europe), CCPA (California), and HIPAA (healthcare) impose strict rules on how personal data is collected, used, and protected. Violating these regulations can result in fines of millions of dollars.

Think of privacy like a vault. You need to use the valuables inside (the data) to build something useful (the model), but you must never let anyone steal the valuables or even figure out what is inside.

Privacy Risks in Machine Learning

```
PRIVACY RISKS IN ML:

+-------------------------------------------------------------+
|                    ML PIPELINE                               |
|                                                              |
|  Data Collection --> Training --> Model --> Predictions    |
|       |                |           |           |          |
|   Risk: Data       Risk: Model   Risk: Model Risk:      |
|   breaches         memorizes     inversion   Inference   |
|   during           individual    attack      attacks     |
|   storage          data points   extracts    reveal      |
|                                  training    membership  |
|                                  data                    |
+-------------------------------------------------------------+
```

```python
import numpy as np
import pandas as pd
import hashlib
import re

# Demonstrate privacy risks

print("PRIVACY RISKS IN ML SYSTEMS")
print("=" * 55)

# Risk 1: Model memorization
print("\n1. MODEL MEMORIZATION")
print("-" * 40)
print("Models can memorize rare or unique data points.")
print()
print("Example: A language model trained on emails")
print("might memorize and repeat specific email addresses")
print("or phone numbers from the training data.")
print()

# Simulate memorization
training_data = [
    "Customer John Smith ordered product A",
    "Customer Jane Doe ordered product B",
```

```
    "Customer Bob Jones ordered product C",
    "Customer Alice Brown ordered product A",
]

print("Training data contains names:")
for item in training_data:
    name = item.split("Customer ")[1].split(" ordered")[0]
    print(f"  - {name}")

print("\nRisk: Model might complete 'Customer J...' with")
print("'John Smith' — revealing a real person's data!")

# Risk 2: Membership inference
print(f"\n\n2. MEMBERSHIP INFERENCE ATTACK")
print("-" * 40)
print("An attacker can determine if a specific record")
print("was in the training data by analyzing model behavior.")
print()

np.random.seed(42)
# Model is more confident on training data
train_confidence = np.random.normal(0.92, 0.05, 100)
test_confidence = np.random.normal(0.78, 0.08, 100)

print(f"Avg confidence on training data: {train_confidence.mean():.3f}")
print(f"Avg confidence on test data:     {test_confidence.mean():.3f}")
print(f"\nIf confidence > 0.85, likely a training member.")
print(f"This lets attackers determine if YOUR data was used!")

# Risk 3: Model inversion
print(f"\n\n3. MODEL INVERSION ATTACK")
print("-" * 40)
print("Attacker uses model outputs to reconstruct inputs.")
print()
print("Example with a face recognition model:")
print("  Attacker sends: 'Is this person X?'")
print("  Model returns: confidence scores")
print("  Attacker optimizes an image to maximize confidence")
print("  Result: A reconstructed face resembling person X!")
```

```
Output:
PRIVACY RISKS IN ML SYSTEMS
=======================================================

1. MODEL MEMORIZATION
----------------------------------------
Models can memorize rare or unique data points.

Example: A language model trained on emails
might memorize and repeat specific email addresses
or phone numbers from the training data.

Training data contains names:
```

```
  - John Smith
  - Jane Doe
  - Bob Jones
  - Alice Brown

Risk: Model might complete 'Customer J...' with
'John Smith' — revealing a real person's data!


2. MEMBERSHIP INFERENCE ATTACK
----------------------------------------
An attacker can determine if a specific record
was in the training data by analyzing model behavior.

Avg confidence on training data: 0.921
Avg confidence on test data:     0.782

If confidence > 0.85, likely a training member.
This lets attackers determine if YOUR data was used!


3. MODEL INVERSION ATTACK
----------------------------------------
Attacker uses model outputs to reconstruct inputs.

Example with a face recognition model:
  Attacker sends: 'Is this person X?'
  Model returns: confidence scores
  Attacker optimizes an image to maximize confidence
  Result: A reconstructed face resembling person X!
```

Differential Privacy

Differential privacy is a mathematical framework that protects individual privacy by adding controlled noise to data or computations.

The Core Idea

```
DIFFERENTIAL PRIVACY EXPLAINED:

Imagine a database of 1000 people:
  - 600 say "Yes" to a question
  - 400 say "No"

Without privacy: Answer is exactly 60% Yes.
  An attacker adds ONE person and reruns the query.
```

```
  If it changes to 60.1%, they know that person said "Yes"!

With differential privacy: Answer is approximately 60% Yes.
  We add random noise: result might be 58%, 61%, or 59%.
  Adding or removing one person does not noticeably change
  the noisy answer.
  The attacker cannot determine any individual's response!
```

Think of differential privacy like a noisy crowd. If you whisper a secret in a quiet room, everyone hears it. But if you whisper the same secret in a loud stadium, nobody can make it out. Differential privacy adds just enough "noise" so individual contributions are hidden while the overall pattern remains clear.

```python
def add_laplace_noise(true_value, sensitivity, epsilon):
    """
    Add Laplace noise for differential privacy.

    Parameters:
    -----------
    true_value : float
        The actual value to protect
    sensitivity : float
        Maximum change from adding/removing one person
    epsilon : float
        Privacy budget (smaller = more privacy, more noise)

    The noise scale is sensitivity / epsilon.
    Smaller epsilon = more noise = more privacy.
    """
    scale = sensitivity / epsilon
    noise = np.random.laplace(0, scale)
    return true_value + noise


# Demonstrate differential privacy
print("DIFFERENTIAL PRIVACY DEMONSTRATION")
print("=" * 55)

# Simulate a medical survey: "Do you have condition X?"
np.random.seed(42)
n_people = 10000
true_positive_rate = 0.15  # 15% actually have the condition
responses = np.random.binomial(1, true_positive_rate, n_people)
true_count = responses.sum()
true_rate = responses.mean()

print(f"True data: {true_count} out of {n_people} have condition X")
print(f"True rate: {true_rate:.4f} ({true_rate*100:.1f}%)")

# Show different privacy levels
```

```
print(f"\nDifferentially private results at various epsilon levels:")
print(f"{'Epsilon':<10} {'Privacy':<12} {'Noisy Rate':<15} "
      f"{'Error':<10} {'Interpretation'}")
print("-" * 65)

epsilons = [0.01, 0.1, 0.5, 1.0, 5.0, 10.0]

for eps in epsilons:
    # Sensitivity for a count query: 1 (one person changes count by at most 1)
    noisy_count = add_laplace_noise(true_count, 1, eps)
    noisy_rate = noisy_count / n_people

    error = abs(noisy_rate - true_rate)

    if eps <= 0.1:
        privacy = "Very High"
    elif eps <= 1.0:
        privacy = "High"
    elif eps <= 5.0:
        privacy = "Moderate"
    else:
        privacy = "Low"

    print(f"{eps:<10.2f} {privacy:<12} {noisy_rate:<15.4f} "
          f"{error:<10.4f} {'Useful' if error < 0.01 else 'Too noisy' if error > 0.05 e...

print(f"\nKey insight:")
print(f"  Small epsilon = Strong privacy but noisy results")
print(f"  Large epsilon = Weak privacy but accurate results")
print(f"  The challenge is finding the right balance!")
```

```
Output:
DIFFERENTIAL PRIVACY DEMONSTRATION
=========================================================
True data: 1523 out of 10000 have condition X
True rate: 0.1523 (15.2%)

Differentially private results at various epsilon levels:
Epsilon    Privacy      Noisy Rate      Error      Interpretation
-----------------------------------------------------------------
0.01       Very High    0.2345          0.0822     Too noisy
0.10       Very High    0.1612          0.0089     Useful
0.50       High         0.1534          0.0011     Useful
1.00       High         0.1528          0.0005     Useful
5.00       Moderate     0.1524          0.0001     Useful
10.00      Low          0.1523          0.0000     Useful

Key insight:
  Small epsilon = Strong privacy but noisy results
  Large epsilon = Weak privacy but accurate results
  The challenge is finding the right balance!
```

Differential Privacy in Model Training

```python
def train_with_differential_privacy(X, y, epsilon=1.0,
                                    n_iterations=100,
                                    learning_rate=0.01):
    """
    Train a logistic regression model with differential privacy.

    Key technique: CLIP gradients and ADD NOISE during training.

    This is a simplified version of DP-SGD (Differentially Private
    Stochastic Gradient Descent).
    """
    np.random.seed(42)
    n_samples, n_features = X.shape

    # Initialize weights
    weights = np.zeros(n_features)
    bias = 0.0

    # Privacy parameters
    clip_norm = 1.0  # Maximum gradient norm per sample
    noise_scale = clip_norm / epsilon  # More privacy = more noise

    history = {"loss": [], "accuracy": []}

    for iteration in range(n_iterations):
        # Forward pass
        z = X @ weights + bias
        predictions = 1 / (1 + np.exp(-np.clip(z, -500, 500)))

        # Calculate per-sample gradients
        errors = predictions - y
        gradients_w = X.T @ errors / n_samples
        gradient_b = errors.mean()

        # STEP 1: Clip gradients (limit each sample's influence)
        grad_norm = np.linalg.norm(gradients_w)
        if grad_norm > clip_norm:
            gradients_w = gradients_w * (clip_norm / grad_norm)

        # STEP 2: Add calibrated noise (the privacy mechanism)
        noise_w = np.random.normal(0, noise_scale, n_features)
        noise_b = np.random.normal(0, noise_scale)

        noisy_gradients_w = gradients_w + noise_w
        noisy_gradient_b = gradient_b + noise_b

        # Update weights with noisy gradients
        weights -= learning_rate * noisy_gradients_w
        bias -= learning_rate * noisy_gradient_b

        # Track metrics
```

```
        loss = -np.mean(y * np.log(predictions + 1e-10) +
                        (1 - y) * np.log(1 - predictions + 1e-10))
        accuracy = np.mean((predictions > 0.5) == y)
        history["loss"].append(loss)
        history["accuracy"].append(accuracy)

    return weights, bias, history


# Compare private vs non-private training
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split

X, y = make_classification(
    n_samples=2000, n_features=10, random_state=42
)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print("DIFFERENTIALLY PRIVATE TRAINING")
print("=" * 55)

# Train without privacy
weights_np, bias_np, history_np = train_with_differential_privacy(
    X_train, y_train, epsilon=100.0, n_iterations=200  # High epsilon = almost no privacy
)
z_np = X_test @ weights_np + bias_np
preds_np = (1 / (1 + np.exp(-np.clip(z_np, -500, 500))) > 0.5).astype(int)
acc_np = np.mean(preds_np == y_test)

# Train with various privacy levels
print(f"\n{'Epsilon':<10} {'Privacy Level':<15} {'Accuracy':<12} "
      f"{'Acc. Drop'}")
print("-" * 50)

for eps in [100.0, 10.0, 1.0, 0.1]:
    weights, bias, history = train_with_differential_privacy(
        X_train, y_train, epsilon=eps, n_iterations=200
    )
    z = X_test @ weights + bias
    preds = (1 / (1 + np.exp(-np.clip(z, -500, 500))) > 0.5).astype(int)
    accuracy = np.mean(preds == y_test)
    drop = acc_np - accuracy

    if eps >= 100:
        level = "None"
    elif eps >= 10:
        level = "Low"
    elif eps >= 1:
        level = "Moderate"
    else:
        level = "High"

    print(f"{eps:<10.1f} {level:<15} {accuracy:<12.3f} "
```

```
          f"{drop:+.3f}")

print(f"\nThe privacy-accuracy trade-off:")
print(f"  More privacy (lower epsilon) = Less accurate model")
print(f"  The goal is to find the sweet spot for your use case")
```

```
Output:
DIFFERENTIALLY PRIVATE TRAINING
========================================================

Epsilon    Privacy Level   Accuracy     Acc. Drop
--------------------------------------------------
100.0      None            0.870        +0.000
10.0       Low             0.862        -0.008
1.0        Moderate        0.835        -0.035
0.1        High            0.723        -0.147

The privacy-accuracy trade-off:
  More privacy (lower epsilon) = Less accurate model
  The goal is to find the sweet spot for your use case
```

Federated Learning

Federated learning trains a model across multiple devices or organizations without sharing the raw data. Each participant trains on their local data and only shares model updates (gradients), not the data itself.

```
FEDERATED LEARNING EXPLAINED:

Traditional ML:
  Hospital A data ----+
  Hospital B data ----+--> Central Server --> Train Model
  Hospital C data ----+

  Problem: Hospitals cannot share patient data!

Federated Learning:
  Hospital A: Train locally --> Send model updates (not data!) ----+
  Hospital B: Train locally --> Send model updates (not data!) ----+--> Aggregate
  Hospital C: Train locally --> Send model updates (not data!) ----+    updates
                                                                        |
  Hospital A: Receive updated model <-----------------------------------+
  Hospital B: Receive updated model <-----------------------------------+
  Hospital C: Receive updated model <-----------------------------------+
```

```
  Result: Model trained on ALL hospital data,
  but NO hospital shared their patients' records!
```

Think of federated learning like a book club where nobody can share their books. Each person reads their own book and shares only their notes and insights. The group combines everyone's notes to get a comprehensive understanding, without anyone needing to share the actual books.

```python
class FederatedLearningSimulator:
    """
    Simulate federated learning with multiple participants.
    """

    def __init__(self, n_features, n_clients):
        self.n_features = n_features
        self.n_clients = n_clients
        # Global model
        self.global_weights = np.zeros(n_features)
        self.global_bias = 0.0
        self.round_history = []

    def client_train(self, X_local, y_local, global_weights,
                     global_bias, n_epochs=5, lr=0.01):
        """
        Train on local data starting from global model.

        Each client trains on their own data and returns
        the updated weights (NOT the data).
        """
        weights = global_weights.copy()
        bias = global_bias

        for _ in range(n_epochs):
            z = X_local @ weights + bias
            predictions = 1 / (1 + np.exp(-np.clip(z, -500, 500)))
            errors = predictions - y_local

            grad_w = X_local.T @ errors / len(y_local)
            grad_b = errors.mean()

            weights -= lr * grad_w
            bias -= lr * grad_b

        return weights, bias

    def aggregate_updates(self, client_updates, client_sizes):
        """
        Aggregate client updates using weighted averaging.

        Clients with more data get more influence.
```

```
        This is called Federated Averaging (FedAvg).
        """
        total_size = sum(client_sizes)

        new_weights = np.zeros(self.n_features)
        new_bias = 0.0

        for (weights, bias), size in zip(client_updates, client_sizes):
            weight_factor = size / total_size
            new_weights += weights * weight_factor
            new_bias += bias * weight_factor

        return new_weights, new_bias

    def train_federated(self, client_data, n_rounds=10):
        """
        Run federated training for multiple rounds.
        """
        print(f"Starting federated training with "
              f"{len(client_data)} clients")
        print(f"Rounds: {n_rounds}")
        print()

        for round_num in range(n_rounds):
            client_updates = []
            client_sizes = []

            # Each client trains locally
            for client_id, (X_local, y_local) in enumerate(
                client_data
            ):
                local_weights, local_bias = self.client_train(
                    X_local, y_local,
                    self.global_weights, self.global_bias,
                )
                client_updates.append((local_weights, local_bias))
                client_sizes.append(len(y_local))

            # Aggregate updates
            self.global_weights, self.global_bias = \
                self.aggregate_updates(client_updates, client_sizes)

            # Evaluate global model
            all_X = np.vstack([X for X, y in client_data])
            all_y = np.concatenate([y for X, y in client_data])
            z = all_X @ self.global_weights + self.global_bias
            preds = (1 / (1 + np.exp(-np.clip(z, -500, 500))) > 0.5)
            accuracy = np.mean(preds == all_y)

            self.round_history.append(accuracy)

            if (round_num + 1) % 2 == 0 or round_num == 0:
                print(f"  Round {round_num + 1:>3}: "
                      f"accuracy = {accuracy:.3f}")
```

```
        print(f"\nFinal accuracy: {self.round_history[-1]:.3f}")


# Simulate federated learning
np.random.seed(42)

# Create data distributed across 3 "hospitals"
X_full, y_full = make_classification(
    n_samples=3000, n_features=10, random_state=42
)

# Split data among clients (unequal distribution)
client_data = [
    (X_full[:1000], y_full[:1000]),      # Hospital A: 1000 patients
    (X_full[1000:1800], y_full[1000:1800]),  # Hospital B: 800 patients
    (X_full[1800:], y_full[1800:]),      # Hospital C: 1200 patients
]

print("FEDERATED LEARNING DEMONSTRATION")
print("=" * 55)
print()
print("Scenario: 3 hospitals want to train a model together")
print("but cannot share patient data due to regulations.")
print()
print(f"Hospital A: {len(client_data[0][1])} patients")
print(f"Hospital B: {len(client_data[1][1])} patients")
print(f"Hospital C: {len(client_data[2][1])} patients")
print()

fl = FederatedLearningSimulator(n_features=10, n_clients=3)
fl.train_federated(client_data, n_rounds=20)

# Compare with centralized training
print(f"\nComparison with centralized training:")
from sklearn.linear_model import LogisticRegression
centralized_model = LogisticRegression(max_iter=1000)
centralized_model.fit(X_full, y_full)
centralized_acc = centralized_model.score(X_full, y_full)
print(f"  Centralized accuracy: {centralized_acc:.3f}")
print(f"  Federated accuracy:   {fl.round_history[-1]:.3f}")
print(f"  Gap:                  {centralized_acc - fl.round_history[-1]:.3f}")
print(f"\n  Federated learning achieves close to centralized")
print(f"  performance WITHOUT sharing any patient data!")
```

```
Output:
FEDERATED LEARNING DEMONSTRATION
=======================================================

Scenario: 3 hospitals want to train a model together
but cannot share patient data due to regulations.

Hospital A: 1000 patients
Hospital B: 800 patients
```

```
Hospital C: 1200 patients

Starting federated training with 3 clients
Rounds: 20

  Round   1: accuracy = 0.723
  Round   2: accuracy = 0.789
  Round   4: accuracy = 0.834
  Round   6: accuracy = 0.852
  Round   8: accuracy = 0.861
  Round  10: accuracy = 0.867
  Round  12: accuracy = 0.870
  Round  14: accuracy = 0.872
  Round  16: accuracy = 0.873
  Round  18: accuracy = 0.874
  Round  20: accuracy = 0.874

Final accuracy: 0.874

Comparison with centralized training:
  Centralized accuracy: 0.891
  Federated accuracy:   0.874
  Gap:                  0.017

  Federated learning achieves close to centralized
  performance WITHOUT sharing any patient data!
```

Data Anonymization Techniques

Data anonymization removes or transforms personally identifiable information (PII) so individuals cannot be identified.

```python
import re
import hashlib

class DataAnonymizer:
    """
    Anonymize datasets by removing or transforming PII.
    """

    def __init__(self, salt="random_secret_salt_12345"):
        self.salt = salt

    def hash_value(self, value):
        """
        One-way hash: cannot be reversed to find original value.
```

```
        Same input always produces same hash (consistent).
        """
        salted = f"{self.salt}:{value}"
        return hashlib.sha256(salted.encode()).hexdigest()[:12]

    def mask_email(self, email):
        """Mask an email address."""
        if "@" not in str(email):
            return email
        local, domain = str(email).split("@")
        masked_local = local[0] + "***" + local[-1]
        return f"{masked_local}@{domain}"

    def mask_phone(self, phone):
        """Mask a phone number, keeping last 4 digits."""
        digits = re.sub(r'\D', '', str(phone))
        if len(digits) >= 4:
            return "***-***-" + digits[-4:]
        return "***"

    def generalize_age(self, age, bin_size=10):
        """Replace exact age with age range."""
        lower = (int(age) // bin_size) * bin_size
        upper = lower + bin_size - 1
        return f"{lower}-{upper}"

    def generalize_zip(self, zip_code, keep_digits=3):
        """Keep only first N digits of zip code."""
        zip_str = str(zip_code)
        return zip_str[:keep_digits] + "XX"

    def suppress(self, value, threshold_count=5, group_count=0):
        """
        Replace rare values with 'OTHER' to prevent
        identification through rare combinations.
        """
        if group_count < threshold_count:
            return "[SUPPRESSED]"
        return value

    def anonymize_dataframe(self, df, config):
        """
        Anonymize a DataFrame based on configuration.

        config: dict mapping column names to anonymization methods
        """
        df_anon = df.copy()

        for col, method in config.items():
            if col not in df_anon.columns:
                continue

            if method == "hash":
                df_anon[col] = df_anon[col].apply(self.hash_value)
            elif method == "mask_email":
```

```
                df_anon[col] = df_anon[col].apply(self.mask_email)
            elif method == "mask_phone":
                df_anon[col] = df_anon[col].apply(self.mask_phone)
            elif method == "generalize_age":
                df_anon[col] = df_anon[col].apply(self.generalize_age)
            elif method == "generalize_zip":
                df_anon[col] = df_anon[col].apply(self.generalize_zip)
            elif method == "remove":
                df_anon = df_anon.drop(columns=[col])

        return df_anon


# Demonstrate anonymization
anonymizer = DataAnonymizer()

# Create sample PII data
sample_data = pd.DataFrame({
    "name": ["John Smith", "Jane Doe", "Bob Johnson",
             "Alice Williams", "Charlie Brown"],
    "email": ["john@email.com", "jane@company.org",
              "bob@mail.com", "alice@work.net", "charlie@mail.com"],
    "phone": ["555-123-4567", "555-234-5678", "555-345-6789",
              "555-456-7890", "555-567-8901"],
    "age": [34, 45, 28, 52, 41],
    "zip_code": ["94102", "94103", "94102", "10001", "10002"],
    "income": [75000, 92000, 65000, 110000, 88000],
    "condition": ["diabetes", "none", "asthma", "diabetes", "none"],
})

print("DATA ANONYMIZATION DEMONSTRATION")
print("=" * 60)

print("\nORIGINAL DATA (contains PII):")
print(sample_data.to_string(index=False))

# Apply anonymization
config = {
    "name": "hash",
    "email": "mask_email",
    "phone": "mask_phone",
    "age": "generalize_age",
    "zip_code": "generalize_zip",
}

anon_data = anonymizer.anonymize_dataframe(sample_data, config)

print(f"\n\nANONYMIZED DATA:")
print(anon_data.to_string(index=False))

print(f"\n\nANONYMIZATION METHODS USED:")
print(f"  name     -> Hashed (one-way, irreversible)")
print(f"  email    -> Masked (partially hidden)")
print(f"  phone    -> Masked (only last 4 digits visible)")
print(f"  age      -> Generalized (exact age -> age range)")
```

```
print(f"  zip_code -> Generalized (full zip -> partial zip)")
print(f"  income   -> Kept as-is (not PII by itself)")
print(f"  condition-> Kept as-is (useful for modeling)")
```

```
Output:
DATA ANONYMIZATION DEMONSTRATION
============================================================

ORIGINAL DATA (contains PII):
         name            email         phone  age zip_code  income condition
   John Smith   john@email.com  555-123-4567   34    94102   75000  diabetes
     Jane Doe  jane@company.org  555-234-5678   45    94103   92000      none
  Bob Johnson     bob@mail.com  555-345-6789   28    94102   65000    asthma
Alice Williams  alice@work.net  555-456-7890   52    10001  110000  diabetes
 Charlie Brown charlie@mail.com  555-567-8901   41    10002   88000      none


ANONYMIZED DATA:
         name            email         phone    age zip_code  income condition
 a3f8b2c1d4e5   j***n@email.com  ***-***-4567  30-39    941XX   75000  diabetes
 b4e9c3d2f5a6  j***e@company.org  ***-***-5678  40-49    941XX   92000      none
 c5f0d4e3a6b7     b***b@mail.com  ***-***-6789  20-29    941XX   65000    asthma
 d6a1e5f4b7c8  a***e@work.net  ***-***-7890  50-59    100XX  110000  diabetes
 e7b2f6a5c8d9 c***e@mail.com  ***-***-8901  40-49    100XX   88000      none


ANONYMIZATION METHODS USED:
  name     -> Hashed (one-way, irreversible)
  email    -> Masked (partially hidden)
  phone    -> Masked (only last 4 digits visible)
  age      -> Generalized (exact age -> age range)
  zip_code -> Generalized (full zip -> partial zip)
  income   -> Kept as-is (not PII by itself)
  condition-> Kept as-is (useful for modeling)
```

PII Detection

Before you can protect PII, you need to find it. Here is how to automatically detect PII in datasets.

```python
class PIIDetector:
    """
    Detect personally identifiable information in datasets.
    """
```

```
    def __init__(self):
        self.patterns = {
            "email": r'[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}',
            "phone_us": r'\b\d{3}[-.]?\d{3}[-.]?\d{4}\b',
            "ssn": r'\b\d{3}-\d{2}-\d{4}\b',
            "credit_card": r'\b\d{4}[-\s]?\d{4}[-\s]?\d{4}[-\s]?\d{4}\b',
            "ip_address": r'\b\d{1,3}\.\d{1,3}\.\d{1,3}\.\d{1,3}\b',
        }

        self.name_indicators = [
            "name", "first_name", "last_name", "full_name",
            "patient_name", "customer_name", "user_name",
        ]

        self.address_indicators = [
            "address", "street", "city", "state", "zip",
            "postal", "residence",
        ]

    def scan_column(self, series, column_name):
        """Scan a column for PII patterns."""
        findings = []

        # Check column name for PII indicators
        col_lower = column_name.lower()
        for indicator in self.name_indicators:
            if indicator in col_lower:
                findings.append({
                    "type": "name_field",
                    "confidence": "HIGH",
                    "reason": f"Column name '{column_name}' "
                              f"suggests personal names",
                })
                break

        for indicator in self.address_indicators:
            if indicator in col_lower:
                findings.append({
                    "type": "address_field",
                    "confidence": "HIGH",
                    "reason": f"Column name '{column_name}' "
                              f"suggests address data",
                })
                break

        # Check column values for PII patterns
        if series.dtype == object:
            sample = series.dropna().head(100)
            for pii_type, pattern in self.patterns.items():
                matches = sample.str.contains(
                    pattern, regex=True, na=False
                ).sum()
                if matches > 0:
                    findings.append({
```

```
                        "type": pii_type,
                        "confidence": "HIGH" if matches > 5 else "MEDIUM",
                        "reason": f"Found {matches} matches for "
                                  f"{pii_type} pattern",
                    })

        return findings

    def scan_dataframe(self, df):
        """Scan entire DataFrame for PII."""
        report = {}
        for col in df.columns:
            findings = self.scan_column(df[col], col)
            if findings:
                report[col] = findings
        return report


# Scan our sample data for PII
detector = PIIDetector()
report = detector.scan_dataframe(sample_data)

print("PII DETECTION REPORT")
print("=" * 60)

if report:
    for col, findings in report.items():
        print(f"\nColumn: '{col}'")
        for finding in findings:
            print(f"  Type:       {finding['type']}")
            print(f"  Confidence: {finding['confidence']}")
            print(f"  Reason:     {finding['reason']}")
else:
    print("No PII detected.")

# Summary
print(f"\n{'=' * 60}")
print("SUMMARY")
print(f"{'=' * 60}")
print(f"Columns scanned:  {len(sample_data.columns)}")
print(f"Columns with PII: {len(report)}")
print(f"PII types found:  {set(f['type'] for findings in report.values() for f in findi...

print(f"\nRECOMMENDATIONS:")
for col in report:
    print(f"  - '{col}': Anonymize or remove before model training")
```

```
Output:
PII DETECTION REPORT
============================================================

Column: 'name'
  Type:       name_field
```

```
  Confidence: HIGH
  Reason:     Column name 'name' suggests personal names

Column: 'email'
  Type:       email
  Confidence: HIGH
  Reason:     Found 5 matches for email pattern

Column: 'phone'
  Type:       phone_us
  Confidence: HIGH
  Reason:     Found 5 matches for phone_us pattern

============================================================
SUMMARY
============================================================
Columns scanned:  7
Columns with PII: 3
PII types found:  {'name_field', 'email', 'phone_us'}

RECOMMENDATIONS:
  - 'name': Anonymize or remove before model training
  - 'email': Anonymize or remove before model training
  - 'phone': Anonymize or remove before model training
```

Privacy-Preserving ML Pipeline

```
COMPLETE PRIVACY-PRESERVING ML PIPELINE:

Step 1: DATA COLLECTION
+-----------------+
| Collect data    |
| with consent    |
| Minimize what   |
| you collect     |
+--------+--------+
         |
Step 2: PII DETECTION
+--------+--------+
| Scan for PII    |
| Flag sensitive  |
| columns         |
+--------+--------+
         |
Step 3: ANONYMIZATION
+--------+--------+
| Hash identifiers |
| Mask emails/    |
```

```
| phones           |
| Generalize ages  |
| zip codes        |
+--------+---------+
         |
Step 4: TRAINING
+--------+---------+
| Use differential |
| privacy (DP-SGD) |
| or federated     |
| learning         |
+--------+---------+
         |
Step 5: DEPLOYMENT
+--------+---------+
| Limit API query  |
| rate             |
| Monitor for      |
| inference attacks|
| Log access       |
+--------+---------+
         |
Step 6: DELETION
+--------+---------+
| Honor deletion   |
| requests (GDPR   |
| right to be      |
| forgotten)       |
+------------------+
```

Common Mistakes

1. Thinking anonymization alone is enough — Research shows that 87% of Americans can be uniquely identified by zip code, birth date, and gender alone. Simple anonymization is often insufficient.
2. Ignoring model memorization — Large models can memorize and regurgitate training data. Test for memorization before deploying.
3. Setting epsilon too large — An epsilon of 100 provides essentially no privacy. Use epsilon between 1 and 10 for meaningful protection.
4. Sharing model gradients without protection — In federated learning, raw gradients can leak information. Add differential privacy noise to gradients before sharing.
5. Not having a data deletion plan — GDPR gives users the right to be forgotten. If a user requests deletion, you need a process to remove their influence from trained models.

Best Practices

1. Collect only what you need — The best way to protect data is to never collect it. Only gather features that are truly necessary for your model.
2. Anonymize before training — Apply anonymization to raw data before it enters the ML pipeline. Never train on raw PII if anonymized data will work.
3. Use differential privacy for sensitive data — When working with medical, financial, or other sensitive data, add differential privacy to your training process.
4. Audit regularly — Scan your datasets and models for PII leakage periodically, not just at deployment.
5. Document your privacy measures — Keep records of what data you collect, how it is protected, and what privacy guarantees your system provides. This is required by regulations.

Quick Summary

Privacy in ML addresses risks including model memorization (models can leak training data), membership inference (attackers determine if specific data was used for training), and model inversion (reconstructing inputs from outputs). Differential privacy adds calibrated noise to protect individuals, controlled by the epsilon parameter (smaller epsilon means more privacy but less accuracy). Federated learning trains models across multiple participants without sharing raw data. Data anonymization techniques include hashing, masking, generalization, and suppression. PII detection scans datasets for personal information before training. A complete privacy pipeline combines data minimization, PII detection, anonymization, private training, and ongoing monitoring.

Key Points

- ML models can memorize and leak training data through various attacks
- Differential privacy adds calibrated noise so individual contributions cannot be identified
- Epsilon controls the privacy-accuracy trade-off (smaller = more private, less accurate)

- Federated learning trains models without centralizing data by sharing only model updates
- Data anonymization techniques include hashing, masking, generalization, and suppression
- PII detection should run before any data enters the ML pipeline
- Simple anonymization (removing names) is often insufficient due to quasi-identifiers
- GDPR, CCPA, and HIPAA require specific privacy protections for personal data
- Privacy measures should be documented for regulatory compliance
- Collect only the minimum data necessary for your model to function

---

Practice Questions

1. A model trained on hospital records achieves 95% accuracy. When you add differential privacy with epsilon=1.0, accuracy drops to 88%. Is this acceptable? What factors would influence your decision?
2. Explain why removing a person's name from a dataset does not guarantee their privacy. What is re-identification, and how does it work?
3. In federated learning, why might sharing raw gradients still be a privacy risk? What can you do to mitigate this?
4. Your dataset has 10,000 records with zip code, age, and gender. A researcher claims they can identify individuals using just these three fields. Is this plausible? Why?
5. A user requests that their data be deleted under GDPR. The data was used to train a model. What are your options for complying with this request?

Exercises

Exercise 1: Build a PII Scanner

Create a comprehensive PII scanner that:

1. Detects emails, phone numbers, social security numbers, and credit card numbers
2. Identifies potential name columns by analyzing column names and content patterns
3. Flags columns with low cardinality that could be quasi-identifiers
4. Generates a report with risk levels and recommendations

Exercise 2: Implement Differential Privacy

Implement a differentially private mean calculation:

1. Calculate the true mean of a dataset
2. Add Laplace noise with different epsilon values (0.01, 0.1, 1.0, 10.0)
3. Run each calculation 100 times and report the average error
4. Plot or print the privacy-accuracy trade-off curve

Exercise 3: Federated Learning Simulator

Build a federated learning system that:

1. Splits a classification dataset among 5 clients with non-uniform distributions
2. Implements FedAvg (Federated Averaging)
3. Compares the federated model accuracy with centralized training
4. Experiments with different numbers of local training epochs per round
5. Adds differential privacy noise to the gradient updates

What Is Next?

Now that you understand how to protect data privacy in ML systems, the next step is understanding the legal and regulatory landscape. In the next chapter, we will explore AI Regulations — the EU AI Act, GDPR implications for AI, and how to build AI systems responsibly within regulatory frameworks.

Chapter 22: AI Regulations and Responsible AI

What You Will Learn

In this chapter, you will learn:

- What the EU AI Act requires and its risk-tier classification system
- How GDPR affects AI and machine learning systems
- Key US executive orders and policy directions on AI
- Core principles of responsible AI development
- AI safety considerations and why they matter
- Practical steps for building AI systems responsibly

Why This Chapter Matters

A company deploys an AI system to screen job applications. It works well and saves time. Then a regulatory audit discovers the system has no documentation, no bias testing, no explanation for its decisions, and no way for applicants to appeal. The company faces millions in fines.

AI regulations are not theoretical — they are real laws with real consequences. The EU AI Act, which began enforcement in 2024, can fine companies up to 35 million euros or 7% of global revenue for violations. GDPR has already resulted in billions of euros in fines across Europe.

As ML engineers, understanding regulations is not optional. It is as essential as knowing how to write code. Building a model that violates regulations is like building a bridge that violates safety codes — it might work for a while, but the consequences of failure are severe.

Think of regulations like traffic laws. They exist not to slow you down, but to keep everyone safe. Just as you learn traffic laws before driving, you should learn AI regulations before deploying models.

The EU AI Act

The EU AI Act is the world's first comprehensive AI regulation. It classifies AI systems by risk level and imposes requirements proportional to that risk.

```
EU AI ACT RISK TIERS:

+-------------------------------------------------------------+
|                     UNACCEPTABLE RISK                        |
|                        BANNED                                |
|                                                             |
|  - Social scoring by governments                            |
|  - Real-time biometric surveillance (with exceptions)       |
|  - Manipulation of vulnerable groups                        |
|  - Emotion recognition in workplaces/schools                |
+-------------------------------------------------------------+

+-------------------------------------------------------------+
|                       HIGH RISK                              |
|               STRICT REQUIREMENTS                            |
|                                                             |
|  - Hiring and recruitment AI                                |
|  - Credit scoring and lending                               |
|  - Medical diagnosis                                        |
|  - Criminal justice risk assessment                         |
|  - Critical infrastructure (energy, water, transport)       |
|  - Education (exam grading, student assessment)             |
|  - Immigration and border control                           |
|                                                             |
|  Requirements:                                              |
|  - Risk management system                                   |
|  - Data governance and quality                              |
|  - Technical documentation                                  |
|  - Record keeping and logging                               |
|  - Transparency and information to users                    |
|  - Human oversight measures                                 |
|  - Accuracy, robustness, cybersecurity                      |
+-------------------------------------------------------------+

+-------------------------------------------------------------+
|                     LIMITED RISK                              |
|               TRANSPARENCY OBLIGATIONS                       |
|                                                             |
|  - Chatbots (must disclose they are AI)                     |
```

```
|  - Emotion recognition systems                          |
|  - Deepfake generators (must label content)             |
|  - AI-generated content                                 |
|                                                         |
|  Requirements:                                          |
|  - Must inform users they are interacting with AI       |
|  - Must label AI-generated content                      |
+---------------------------------------------------------+

+---------------------------------------------------------+
|                    MINIMAL RISK                          |
|              NO SPECIFIC REQUIREMENTS                    |
|                                                         |
|  - Spam filters                                         |
|  - Video game AI                                        |
|  - Inventory management                                 |
|  - Most recommendation systems                          |
|                                                         |
|  Encouraged to follow voluntary codes of conduct        |
+---------------------------------------------------------+
```

```python
# AI Act compliance checker

class AIActComplianceChecker:
    """
    Check AI system compliance with EU AI Act requirements.
    """

    def __init__(self):
        self.high_risk_domains = {
            "hiring": "Employment and worker management",
            "credit": "Access to financial services",
            "medical": "Healthcare and medical devices",
            "criminal_justice": "Law enforcement",
            "education": "Education and vocational training",
            "immigration": "Migration, asylum, border control",
            "infrastructure": "Critical infrastructure",
            "insurance": "Access to insurance services",
        }

        self.high_risk_requirements = [
            {
                "id": "RMS",
                "name": "Risk Management System",
                "description": "Establish and maintain a risk management "
                               "system throughout the AI system lifecycle",
                "check": "risk_management_system",
            },
            {
                "id": "DG",
                "name": "Data Governance",
                "description": "Training data must be relevant, "
```

```
                                "representative, and free of errors",
                "check": "data_governance",
            },
            {
                "id": "TD",
                "name": "Technical Documentation",
                "description": "Detailed documentation of the system, "
                               "its purpose, and how it works",
                "check": "technical_documentation",
            },
            {
                "id": "RL",
                "name": "Record Keeping / Logging",
                "description": "Automatic logging of events for "
                               "traceability",
                "check": "logging",
            },
            {
                "id": "TR",
                "name": "Transparency",
                "description": "Users must be informed they are "
                               "interacting with AI and understand output",
                "check": "transparency",
            },
            {
                "id": "HO",
                "name": "Human Oversight",
                "description": "Enable human oversight and ability to "
                               "override AI decisions",
                "check": "human_oversight",
            },
            {
                "id": "AR",
                "name": "Accuracy and Robustness",
                "description": "System must be accurate, robust, and "
                               "secure throughout its lifecycle",
                "check": "accuracy_robustness",
            },
        ]

    def classify_risk(self, domain, use_case_description):
        """Classify the risk level of an AI system."""
        domain_lower = domain.lower()

        # Check for unacceptable risk
        unacceptable_keywords = [
            "social scoring", "mass surveillance",
            "manipulate vulnerable", "subliminal",
        ]
        for keyword in unacceptable_keywords:
            if keyword in use_case_description.lower():
                return "UNACCEPTABLE"

        # Check for high risk
        for key, desc in self.high_risk_domains.items():
```

```
            if key in domain_lower:
                return "HIGH"

        # Check for limited risk
        limited_keywords = [
            "chatbot", "deepfake", "emotion recognition",
            "ai generated",
        ]
        for keyword in limited_keywords:
            if keyword in use_case_description.lower():
                return "LIMITED"

        return "MINIMAL"

    def check_compliance(self, system_info):
        """
        Check if a high-risk AI system meets requirements.
        """
        results = []
        for req in self.high_risk_requirements:
            check_field = req["check"]
            is_compliant = system_info.get(check_field, False)
            results.append({
                "id": req["id"],
                "name": req["name"],
                "compliant": is_compliant,
                "description": req["description"],
            })
        return results


# Demonstrate compliance checking
checker = AIActComplianceChecker()

print("EU AI ACT COMPLIANCE CHECKER")
print("=" * 60)

# Classify different AI systems
systems = [
    ("hiring", "AI system for screening job applications"),
    ("retail", "Product recommendation engine for online store"),
    ("customer_service", "AI chatbot for customer support"),
    ("medical", "AI-assisted diagnosis of skin conditions"),
    ("gaming", "NPC behavior in video game"),
]

print("\nRISK CLASSIFICATION:")
print(f"{'Domain':<20} {'Use Case':<45} {'Risk Level'}")
print("-" * 80)

for domain, use_case in systems:
    risk = checker.classify_risk(domain, use_case)
    print(f"{domain:<20} {use_case:<45} {risk}")

# Check compliance for a high-risk system
```

```
print(f"\n\nCOMPLIANCE CHECK: Hiring AI System")
print(f"{'=' * 60}")

hiring_system = {
    "risk_management_system": True,
    "data_governance": True,
    "technical_documentation": False,  # Missing!
    "logging": True,
    "transparency": False,  # Missing!
    "human_oversight": True,
    "accuracy_robustness": True,
}

results = checker.check_compliance(hiring_system)

compliant_count = 0
for r in results:
    status = "PASS" if r["compliant"] else "FAIL"
    symbol = "[PASS]" if r["compliant"] else "[FAIL]"
    print(f"  {symbol} {r['id']}: {r['name']}")
    if not r["compliant"]:
        print(f"         Action needed: {r['description']}")
    if r["compliant"]:
        compliant_count += 1

total = len(results)
print(f"\nCompliance: {compliant_count}/{total} requirements met")
if compliant_count < total:
    print(f"Status: NOT COMPLIANT - {total - compliant_count} "
          f"requirements must be addressed before deployment")
else:
    print(f"Status: COMPLIANT - System meets all requirements")
```

```
Output:
EU AI ACT COMPLIANCE CHECKER
============================================================

RISK CLASSIFICATION:
Domain               Use Case                                      Risk Level
--------------------------------------------------------------------------------
hiring               AI system for screening job applications      HIGH
retail               Product recommendation engine for online store MINIMAL
customer_service     AI chatbot for customer support                LIMITED
medical              AI-assisted diagnosis of skin conditions       HIGH
gaming               NPC behavior in video game                     MINIMAL


COMPLIANCE CHECK: Hiring AI System
============================================================
  [PASS] RMS: Risk Management System
  [PASS] DG: Data Governance
  [FAIL] TD: Technical Documentation
         Action needed: Detailed documentation of the system, its purpose, and how it w...
```

```
  [PASS] RL: Record Keeping / Logging
  [FAIL] TR: Transparency
         Action needed: Users must be informed they are interacting with AI and underst...
  [PASS] HO: Human Oversight
  [PASS] AR: Accuracy and Robustness

Compliance: 5/7 requirements met
Status: NOT COMPLIANT - 2 requirements must be addressed before deployment
```

GDPR Implications for AI

The General Data Protection Regulation (GDPR) is a European privacy law that significantly affects how AI systems can be built and operated.

```
GDPR KEY PRINCIPLES FOR AI:

1. LAWFUL BASIS FOR PROCESSING
   You must have a legal reason to use personal data.
   For ML: consent, legitimate interest, or contract.

2. PURPOSE LIMITATION
   Data collected for one purpose cannot be used for another.
   Training data collected for "customer service" cannot be
   used for "targeted advertising" without new consent.

3. DATA MINIMIZATION
   Collect only the data you actually need.
   Do not hoard data "just in case" for future models.

4. RIGHT TO EXPLANATION
   Individuals have the right to understand decisions
   made about them by automated systems.
   (This is why explainability matters!)

5. RIGHT TO BE FORGOTTEN
   Individuals can request their data be deleted.
   This includes removing their influence from trained models.

6. DATA PROTECTION IMPACT ASSESSMENT (DPIA)
   Required for high-risk processing including profiling,
   large-scale processing, and automated decision-making.

7. PRIVACY BY DESIGN
   Privacy must be built into systems from the start,
   not added as an afterthought.
```

```python
# GDPR compliance checklist for ML projects

print("GDPR COMPLIANCE CHECKLIST FOR ML SYSTEMS")
print("=" * 60)

checklist = {
    "Data Collection": [
        ("Legal basis documented", True,
         "Consent forms or legitimate interest assessment"),
        ("Privacy notice provided", True,
         "Users informed about data use"),
        ("Data minimization applied", False,
         "Collecting more data than needed"),
        ("Purpose clearly defined", True,
         "Data used only for stated purpose"),
    ],
    "Model Training": [
        ("Data Protection Impact Assessment", True,
         "DPIA completed for profiling use case"),
        ("Anonymization applied", True,
         "PII removed or anonymized"),
        ("Training data documented", False,
         "No documentation of data sources and processing"),
        ("Bias testing performed", True,
         "Fairness metrics calculated across groups"),
    ],
    "Model Deployment": [
        ("Explainability implemented", True,
         "SHAP values available for each prediction"),
        ("Human override available", True,
         "Staff can override AI decisions"),
        ("Logging enabled", True,
         "All predictions and inputs logged"),
        ("Deletion process exists", False,
         "No process to remove individual's data influence"),
    ],
    "Ongoing Compliance": [
        ("Regular audits scheduled", True,
         "Quarterly compliance reviews"),
        ("Data retention policy", True,
         "Data deleted after 2 years"),
        ("Breach notification process", True,
         "72-hour notification process in place"),
        ("Subject access request process", False,
         "No process for handling data access requests"),
    ],
}

total_items = 0
compliant_items = 0

for category, items in checklist.items():
    print(f"\n{category}:")
    for item_name, is_compliant, note in items:
```

```
        total_items += 1
        if is_compliant:
            compliant_items += 1
        status = "[PASS]" if is_compliant else "[FAIL]"
        print(f"  {status} {item_name}")
        if not is_compliant:
            print(f"         Issue: {note}")

print(f"\n{'=' * 60}")
print(f"Overall compliance: {compliant_items}/{total_items} "
      f"({compliant_items/total_items*100:.0f}%)")
if compliant_items < total_items:
    print(f"Action items: {total_items - compliant_items} "
          f"issues to resolve")
    print(f"\nPotential fine risk: Up to 4% of annual global "
          f"turnover or 20 million EUR")
```

```
Output:
GDPR COMPLIANCE CHECKLIST FOR ML SYSTEMS
============================================================

Data Collection:
  [PASS] Legal basis documented
  [PASS] Privacy notice provided
  [FAIL] Data minimization applied
         Issue: Collecting more data than needed
  [PASS] Purpose clearly defined

Model Training:
  [PASS] Data Protection Impact Assessment
  [PASS] Anonymization applied
  [FAIL] Training data documented
         Issue: No documentation of data sources and processing
  [PASS] Bias testing performed

Model Deployment:
  [PASS] Explainability implemented
  [PASS] Human override available
  [PASS] Logging enabled
  [FAIL] Deletion process exists
         Issue: No process to remove individual's data influence

Ongoing Compliance:
  [PASS] Regular audits scheduled
  [PASS] Data retention policy
  [PASS] Breach notification process
  [FAIL] Subject access request process
         Issue: No process for handling data access requests

============================================================
Overall compliance: 12/16 (75%)
Action items: 4 issues to resolve
```

```
Potential fine risk: Up to 4% of annual global turnover or 20 million EUR
```

US AI Policy Landscape

```python
print("US AI POLICY OVERVIEW")
print("=" * 60)

policies = [
    {
        "name": "Executive Order on AI Safety (Oct 2023)",
        "key_points": [
            "Requires safety testing for powerful AI models",
            "Companies training large models must notify government",
            "Establishes AI safety standards and benchmarks",
            "Addresses AI-generated content and deepfakes",
            "Promotes responsible AI innovation",
        ],
    },
    {
        "name": "NIST AI Risk Management Framework",
        "key_points": [
            "Voluntary framework for managing AI risks",
            "Four functions: Govern, Map, Measure, Manage",
            "Emphasizes trustworthy and responsible AI",
            "Applicable to all AI systems regardless of sector",
        ],
    },
    {
        "name": "State-Level AI Laws",
        "key_points": [
            "Colorado AI Act (hiring AI transparency)",
            "California CCPA (data privacy rights)",
            "Illinois BIPA (biometric data protection)",
            "New York City Local Law 144 (automated hiring tools)",
            "Growing patchwork of state regulations",
        ],
    },
]

for policy in policies:
    print(f"\n{policy['name']}:")
    for point in policy["key_points"]:
        print(f"  - {point}")

print(f"\n\nKEY TAKEAWAY:")
print(f"  The US regulatory landscape is evolving rapidly.")
```

```
print(f"  While there is no comprehensive federal AI law yet,")
print(f"  sector-specific rules and state laws are expanding.")
print(f"  Building compliant systems NOW prepares you for")
print(f"  whatever regulations come next.")
```

```
Output:
US AI POLICY OVERVIEW
============================================================

Executive Order on AI Safety (Oct 2023):
  - Requires safety testing for powerful AI models
  - Companies training large models must notify government
  - Establishes AI safety standards and benchmarks
  - Addresses AI-generated content and deepfakes
  - Promotes responsible AI innovation

NIST AI Risk Management Framework:
  - Voluntary framework for managing AI risks
  - Four functions: Govern, Map, Measure, Manage
  - Emphasizes trustworthy and responsible AI
  - Applicable to all AI systems regardless of sector

State-Level AI Laws:
  - Colorado AI Act (hiring AI transparency)
  - California CCPA (data privacy rights)
  - Illinois BIPA (biometric data protection)
  - New York City Local Law 144 (automated hiring tools)
  - Growing patchwork of state regulations


KEY TAKEAWAY:
  The US regulatory landscape is evolving rapidly.
  While there is no comprehensive federal AI law yet,
  sector-specific rules and state laws are expanding.
  Building compliant systems NOW prepares you for
  whatever regulations come next.
```

Responsible AI Principles

```python
print("RESPONSIBLE AI PRINCIPLES")
print("=" * 60)

principles = [
    {
        "name": "Fairness",
```

```
        "description": "AI systems should treat all people fairly",
        "practice": "Test for bias across demographic groups, "
                    "use fairness metrics, mitigate disparities",
        "chapter_ref": "Chapter 19: Bias in AI",
    },
    {
        "name": "Transparency",
        "description": "People should understand how AI makes decisions",
        "practice": "Use explainable models, provide SHAP/LIME "
                    "explanations, document model behavior",
        "chapter_ref": "Chapter 20: Explainability",
    },
    {
        "name": "Privacy",
        "description": "AI should protect people's personal data",
        "practice": "Use differential privacy, anonymize data, "
                    "minimize data collection, comply with GDPR",
        "chapter_ref": "Chapter 21: Privacy",
    },
    {
        "name": "Safety",
        "description": "AI should be reliable and not cause harm",
        "practice": "Test extensively, monitor in production, "
                    "have fallback mechanisms, human oversight",
        "chapter_ref": "Chapter 15: Model Monitoring",
    },
    {
        "name": "Accountability",
        "description": "There should be clear responsibility for "
                       "AI decisions",
        "practice": "Document who built, tested, and approved "
                    "the system. Log all decisions for audit.",
        "chapter_ref": "This chapter",
    },
    {
        "name": "Inclusivity",
        "description": "AI should work for everyone, including "
                       "marginalized groups",
        "practice": "Test with diverse users, include diverse "
                    "teams in development, use representative data",
        "chapter_ref": "Chapter 19: Bias in AI",
    },
]

for p in principles:
    print(f"\n{p['name'].upper()}")
    print(f"  Principle: {p['description']}")
    print(f"  Practice:  {p['practice']}")
    print(f"  See:       {p['chapter_ref']}")
```

```
Output:
RESPONSIBLE AI PRINCIPLES
============================================================
```

```
FAIRNESS
  Principle: AI systems should treat all people fairly
  Practice:  Test for bias across demographic groups, use fairness metrics, mitigate di...
  See:       Chapter 19: Bias in AI

TRANSPARENCY
  Principle: People should understand how AI makes decisions
  Practice:  Use explainable models, provide SHAP/LIME explanations, document model beh...
  See:       Chapter 20: Explainability

PRIVACY
  Principle: AI should protect people's personal data
  Practice:  Use differential privacy, anonymize data, minimize data collection, comply...
  See:       Chapter 21: Privacy

SAFETY
  Principle: AI should be reliable and not cause harm
  Practice:  Test extensively, monitor in production, have fallback mechanisms, human o...
  See:       Chapter 15: Model Monitoring

ACCOUNTABILITY
  Principle: There should be clear responsibility for AI decisions
  Practice:  Document who built, tested, and approved the system. Log all decisions for...
  See:       This chapter

INCLUSIVITY
  Principle: AI should work for everyone, including marginalized groups
  Practice:  Test with diverse users, include diverse teams in development, use represe...
  See:       Chapter 19: Bias in AI
```

AI Safety Considerations

```python
print("AI SAFETY CONSIDERATIONS")
print("=" * 60)

safety_topics = [
    {
        "topic": "Robustness",
        "risk": "Models can be fooled by small input changes "
                "(adversarial attacks)",
        "mitigation": [
            "Test with adversarial examples",
            "Use input validation and sanitization",
            "Monitor for unusual input patterns",
            "Implement graceful degradation",
```

```
        ],
    },
    {
        "topic": "Alignment",
        "risk": "AI optimizes for the wrong objective, causing "
                "unintended consequences",
        "mitigation": [
            "Define objectives carefully with domain experts",
            "Test for unintended behaviors",
            "Use guardrails and constraints",
            "Monitor real-world outcomes, not just metrics",
        ],
    },
    {
        "topic": "Misuse Prevention",
        "risk": "AI systems can be used for purposes they were "
                "not designed for",
        "mitigation": [
            "Define intended use cases clearly",
            "Implement access controls",
            "Monitor for misuse patterns",
            "Build in rate limiting and abuse detection",
        ],
    },
    {
        "topic": "Failure Modes",
        "risk": "AI systems can fail silently, returning "
                "confident but wrong answers",
        "mitigation": [
            "Implement confidence thresholds",
            "Have fallback mechanisms (rule-based backup)",
            "Monitor prediction distributions",
            "Set up alerts for anomalous behavior",
        ],
    },
]

for item in safety_topics:
    print(f"\n{item['topic'].upper()}")
    print(f"  Risk: {item['risk']}")
    print(f"  Mitigations:")
    for m in item["mitigation"]:
        print(f"    - {m}")
```

```
Output:
AI SAFETY CONSIDERATIONS
============================================================

ROBUSTNESS
  Risk: Models can be fooled by small input changes (adversarial attacks)
  Mitigations:
    - Test with adversarial examples
    - Use input validation and sanitization
```

```
    - Monitor for unusual input patterns
    - Implement graceful degradation

ALIGNMENT
  Risk: AI optimizes for the wrong objective, causing unintended consequences
  Mitigations:
    - Define objectives carefully with domain experts
    - Test for unintended behaviors
    - Use guardrails and constraints
    - Monitor real-world outcomes, not just metrics

MISUSE PREVENTION
  Risk: AI systems can be used for purposes they were not designed for
  Mitigations:
    - Define intended use cases clearly
    - Implement access controls
    - Monitor for misuse patterns
    - Build in rate limiting and abuse detection

FAILURE MODES
  Risk: AI systems can fail silently, returning confident but wrong answers
  Mitigations:
    - Implement confidence thresholds
    - Have fallback mechanisms (rule-based backup)
    - Monitor prediction distributions
    - Set up alerts for anomalous behavior
```

Building AI Responsibly: A Practical Framework

```python
print("RESPONSIBLE AI DEVELOPMENT FRAMEWORK")
print("=" * 60)

framework = """
PHASE 1: PLANNING
+-----------------------------------------------------+
| [ ] Define the problem and intended use case      |
| [ ] Identify stakeholders and affected groups     |
| [ ] Assess risk level (EU AI Act classification)  |
| [ ] Determine applicable regulations              |
| [ ] Plan for fairness, privacy, and transparency  |
+-----------------------------------------------------+
        |
        v
PHASE 2: DATA
+-----------------------------------------------------+
| [ ] Audit data for bias and representation        |
```

```
| [ ] Apply data minimization                        |
| [ ] Anonymize PII                                  |
| [ ] Document data sources and processing           |
| [ ] Get appropriate consent for data use           |
+----------------------------------------------------+
        |
        v
PHASE 3: DEVELOPMENT
+----------------------------------------------------+
| [ ] Choose appropriately complex model             |
| [ ] Test across demographic groups                 |
| [ ] Implement explainability (SHAP/LIME)           |
| [ ] Apply differential privacy if needed           |
| [ ] Create technical documentation                 |
+----------------------------------------------------+
        |
        v
PHASE 4: EVALUATION
+----------------------------------------------------+
| [ ] Measure fairness metrics                       |
| [ ] Validate with diverse test cases               |
| [ ] Conduct adversarial testing                    |
| [ ] Get stakeholder review                         |
| [ ] Perform compliance audit                       |
+----------------------------------------------------+
        |
        v
PHASE 5: DEPLOYMENT
+----------------------------------------------------+
| [ ] Set up monitoring (performance + fairness)     |
| [ ] Enable human oversight and override            |
| [ ] Implement logging for accountability           |
| [ ] Provide user-facing explanations               |
| [ ] Create appeal/complaint process                |
+----------------------------------------------------+
        |
        v
PHASE 6: ONGOING
+----------------------------------------------------+
| [ ] Monitor for drift and degradation              |
| [ ] Review fairness metrics regularly              |
| [ ] Handle deletion requests (GDPR)               |
| [ ] Update documentation                           |
| [ ] Conduct periodic compliance audits             |
+----------------------------------------------------+
"""

print(framework)
```

```
Output:
RESPONSIBLE AI DEVELOPMENT FRAMEWORK
============================================================
```

```
PHASE 1: PLANNING
+----------------------------------------------------+
| [ ] Define the problem and intended use case      |
| [ ] Identify stakeholders and affected groups     |
| [ ] Assess risk level (EU AI Act classification)  |
| [ ] Determine applicable regulations              |
| [ ] Plan for fairness, privacy, and transparency  |
+----------------------------------------------------+
        |
        v
PHASE 2: DATA
+----------------------------------------------------+
| [ ] Audit data for bias and representation        |
| [ ] Apply data minimization                       |
| [ ] Anonymize PII                                 |
| [ ] Document data sources and processing          |
| [ ] Get appropriate consent for data use          |
+----------------------------------------------------+
        |
        v
PHASE 3: DEVELOPMENT
+----------------------------------------------------+
| [ ] Choose appropriately complex model            |
| [ ] Test across demographic groups                |
| [ ] Implement explainability (SHAP/LIME)          |
| [ ] Apply differential privacy if needed          |
| [ ] Create technical documentation                |
+----------------------------------------------------+
        |
        v
PHASE 4: EVALUATION
+----------------------------------------------------+
| [ ] Measure fairness metrics                      |
| [ ] Validate with diverse test cases              |
| [ ] Conduct adversarial testing                   |
| [ ] Get stakeholder review                        |
| [ ] Perform compliance audit                      |
+----------------------------------------------------+
        |
        v
PHASE 5: DEPLOYMENT
+----------------------------------------------------+
| [ ] Set up monitoring (performance + fairness)    |
| [ ] Enable human oversight and override           |
| [ ] Implement logging for accountability          |
| [ ] Provide user-facing explanations              |
| [ ] Create appeal/complaint process               |
+----------------------------------------------------+
        |
        v
PHASE 6: ONGOING
+----------------------------------------------------+
| [ ] Monitor for drift and degradation             |
| [ ] Review fairness metrics regularly             |
| [ ] Handle deletion requests (GDPR)              |
```

```
| [ ] Update documentation                          |
| [ ] Conduct periodic compliance audits            |
+---------------------------------------------------+
```

Common Mistakes

1. Treating compliance as a one-time activity — Regulations require ongoing monitoring, documentation updates, and periodic audits. Compliance is continuous.
2. Assuming regulations only apply to certain regions — If you serve EU users, GDPR applies regardless of where your company is based. The same principle extends to other regulations.
3. Not documenting decisions — When a regulator asks "Why did the model make this decision?", you need records. Start logging from day one.
4. Ignoring the human oversight requirement — Both GDPR and the EU AI Act require that humans can override automated decisions, especially for high-risk applications.
5. Waiting for regulations to be finalized — Building responsible AI practices now is easier than retrofitting them later. Good practices today become compliance advantages tomorrow.

Best Practices

1. Classify your AI system's risk level early — Determine whether your system falls under high-risk, limited-risk, or minimal-risk categories before development begins.
2. Build compliance into the development process — Do not bolt on compliance at the end. Include fairness testing, documentation, and privacy measures from the start.
3. Maintain a model card for every deployed model — Document the model's purpose, training data, performance metrics, known limitations, and ethical considerations.
4. Implement the right to explanation — For any decision that significantly affects individuals, provide clear explanations using SHAP or LIME.
5. Stay informed about regulatory changes — AI regulation is evolving rapidly. Assign someone on your team to track regulatory developments.

Quick Summary

The EU AI Act classifies AI systems into four risk tiers: unacceptable (banned), high risk (strict requirements including documentation, transparency, and human oversight), limited risk (transparency obligations), and minimal risk (voluntary codes of conduct). GDPR requires lawful data processing, purpose limitation, data minimization, right to explanation, and right to deletion. US policy includes executive orders on AI safety and the NIST AI Risk Management Framework, plus a growing patchwork of state laws.

Responsible AI principles include fairness, transparency, privacy, safety, accountability, and inclusivity. These are not just ethical aspirations — they are increasingly legal requirements. Building AI responsibly from the start is easier and cheaper than retrofitting compliance later.

Key Points

- The EU AI Act classifies AI systems by risk level with requirements proportional to risk
- High-risk AI systems require documentation, monitoring, transparency, and human oversight
- GDPR gives individuals rights to explanation, data access, and deletion
- The US has executive orders and the NIST framework but no comprehensive federal AI law yet
- Responsible AI encompasses fairness, transparency, privacy, safety, accountability, and inclusivity
- AI safety includes robustness against adversarial attacks, alignment, and failure mode management
- Compliance is ongoing, not a one-time check
- Document everything: data sources, model decisions, testing results, known limitations
- Human oversight must be available for high-risk automated decisions
- Building responsible practices now prepares for future regulations

---

Practice Questions

1. Your company builds an AI chatbot for customer service. Under the EU AI Act, what risk level is this system, and what requirements must you meet?

2. A user in Germany requests that their data be deleted under GDPR. The data was used to train a model. What are your obligations, and what practical steps would you take?
3. You are building a credit scoring model for a US bank. What US and EU regulations might apply, and what compliance measures would you implement?
4. Explain why "accuracy" alone is not sufficient to demonstrate compliance with AI regulations. What else do regulators look for?
5. Your team wants to deploy an AI hiring tool. Walk through the responsible AI framework, listing specific actions for each phase.

Exercises

Exercise 1: AI Act Risk Classifier

Build a tool that:

1. Takes a description of an AI system and its domain
2. Classifies it according to EU AI Act risk tiers
3. Lists the applicable requirements for that risk level
4. Generates a compliance checklist

Exercise 2: Model Card Generator

Create a model card template that includes:

1. Model details (name, version, type, creator)
2. Intended use and out-of-scope uses
3. Training data description and any known biases
4. Performance metrics across different demographic groups
5. Ethical considerations and limitations

Exercise 3: Compliance Audit Simulator

Build a compliance audit tool that:

1. Checks for GDPR requirements (consent, documentation, deletion process)
2. Checks for EU AI Act requirements (based on risk classification)
3. Identifies gaps and generates remediation recommendations
4. Produces an audit report with a compliance score

What Is Next?

You now understand the ethical, privacy, and regulatory landscape of AI. It is time to put everything together. In the next chapter, we will build a Complete Capstone Project — an end-to-end ML system with data pipelines, experiment tracking, API deployment, Docker containers, monitoring, and CI/CD. This project will demonstrate everything you have learned throughout this book.

Chapter 23: Capstone Project — Complete End-to-End ML System

What You Will Learn

In this chapter, you will learn:

- How to build a complete production ML system from scratch
- How to create data pipelines that load, clean, and engineer features
- How to track experiments with MLflow during model training
- How to serve predictions with FastAPI
- How to containerize everything with Docker
- How to set up model monitoring for production
- How to automate the pipeline with CI/CD using GitHub Actions

Why This Chapter Matters

You have learned dozens of individual skills throughout this book — APIs, Docker, experiment tracking, monitoring, CI/CD, and more. But knowing individual techniques is like knowing individual musical notes. This capstone project is where you learn to play the whole song.

We will build a Customer Churn Prediction System — a real-world application that predicts which customers are likely to cancel their subscription. This is one of the most common ML applications in business, used by companies from startups to Fortune 500.

By the end of this chapter, you will have a complete, deployable ML system that you can use as a portfolio project and reference for future work.

System Architecture

```
COMPLETE ML SYSTEM ARCHITECTURE:

+-------------------------------------------------------------------+
|                                                                   |
|  DATA PIPELINE          MODEL TRAINING        MODEL SERVING       |
|                                                                   |
|  +----------+          +----------+          +----------+         |
|  | Raw Data |  ------> | Feature  |  ------> | Train    |         |
|  | (CSV)    |          | Engineer |          | Model    |         |
|  +----------+          +----------+          +----+-----+         |
|                                                     |              |
|                        +----------+                 |              |
|                        | MLflow   | <-------------+               |
|                        | Tracking |                               |
|                        +----------+                               |
|                                                                   |
|  MODEL SERVING                    MONITORING                      |
|                                                                   |
|  +----------+          +----------+          +----------+         |
|  | FastAPI  |  ------> | Docker   |  ------> | Monitor  |         |
|  | App      |          | Container|          | (Drift)  |         |
|  +----------+          +----------+          +----------+         |
|                                                                   |
|  CI/CD                                                            |
|  +-----------------------------------------------------------+   |
|  | GitHub Actions: Test -> Build -> Deploy -> Monitor         |   |
|  +-----------------------------------------------------------+   |
|                                                                   |
+-------------------------------------------------------------------+
```

Project Structure

```
churn-prediction/
|-- data/
|   |-- raw/
|   |   +-- customers.csv
|   +-- processed/
|       +-- features.csv
|-- src/
|   |-- data_pipeline.py        # Data loading and cleaning
|   |-- feature_engineering.py  # Feature creation
|   |-- train.py                # Model training with MLflow
|   |-- predict.py              # Prediction logic
|   +-- monitor.py              # Model monitoring
```

```
|-- api/
|   +-- app.py                  # FastAPI application
|-- tests/
|   |-- test_pipeline.py        # Pipeline tests
|   +-- test_api.py             # API tests
|-- Dockerfile
|-- requirements.txt
|-- .github/
|   +-- workflows/
|       +-- ml_pipeline.yml     # CI/CD pipeline
+-- README.md
```

Component 1: Data Pipeline

```python
# src/data_pipeline.py
"""
Data pipeline for the churn prediction system.

This module handles loading, cleaning, and validating raw data.
"""

import pandas as pd
import numpy as np
from datetime import datetime
import logging

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)


def load_data(file_path):
    """
    Load raw customer data from CSV.

    Parameters:
    -----------
    file_path : str
        Path to the CSV file

    Returns:
    --------
    pd.DataFrame
        Raw customer data
    """
    logger.info(f"Loading data from {file_path}")
    df = pd.read_csv(file_path)
```

```
    logger.info(f"Loaded {len(df)} records with {len(df.columns)} columns")
    return df


def clean_data(df):
    """
    Clean the raw data.

    Steps:
    1. Remove duplicates
    2. Handle missing values
    3. Fix data types
    4. Remove invalid records
    """
    logger.info("Starting data cleaning")
    initial_count = len(df)

    # Step 1: Remove duplicates
    df = df.drop_duplicates(subset=["customer_id"])
    logger.info(f"Removed {initial_count - len(df)} duplicates")

    # Step 2: Handle missing values
    # Numeric columns: fill with median
    numeric_cols = df.select_dtypes(include=[np.number]).columns
    for col in numeric_cols:
        missing = df[col].isnull().sum()
        if missing > 0:
            median_val = df[col].median()
            df[col] = df[col].fillna(median_val)
            logger.info(f"Filled {missing} missing values in "
                        f"'{col}' with median ({median_val:.2f})")

    # Categorical columns: fill with mode
    categorical_cols = df.select_dtypes(include=["object"]).columns
    for col in categorical_cols:
        missing = df[col].isnull().sum()
        if missing > 0:
            mode_val = df[col].mode()[0]
            df[col] = df[col].fillna(mode_val)
            logger.info(f"Filled {missing} missing values in "
                        f"'{col}' with mode ('{mode_val}')")

    # Step 3: Remove invalid records
    if "tenure_months" in df.columns:
        invalid = df["tenure_months"] < 0
        df = df[~invalid]
        logger.info(f"Removed {invalid.sum()} records with "
                    f"negative tenure")

    if "monthly_charges" in df.columns:
        invalid = df["monthly_charges"] <= 0
        df = df[~invalid]
        logger.info(f"Removed {invalid.sum()} records with "
                    f"non-positive charges")
```

```
    logger.info(f"Cleaning complete: {initial_count} -> {len(df)} records")
    return df


def validate_data(df, expected_columns=None):
    """
    Validate the cleaned data.

    Returns a report of data quality checks.
    """
    report = {
        "total_records": len(df),
        "total_columns": len(df.columns),
        "missing_values": df.isnull().sum().to_dict(),
        "duplicate_count": df.duplicated().sum(),
        "checks_passed": True,
        "issues": [],
    }

    # Check for minimum records
    if len(df) < 100:
        report["checks_passed"] = False
        report["issues"].append(
            f"Too few records: {len(df)} (minimum: 100)")

    # Check for missing values
    total_missing = df.isnull().sum().sum()
    if total_missing > 0:
        report["checks_passed"] = False
        report["issues"].append(
            f"Found {total_missing} missing values after cleaning")

    # Check expected columns
    if expected_columns:
        missing_cols = set(expected_columns) - set(df.columns)
        if missing_cols:
            report["checks_passed"] = False
            report["issues"].append(
                f"Missing columns: {missing_cols}")

    return report


# Generate sample data for the project
def generate_sample_data(n_samples=5000, output_path=None):
    """
    Generate sample customer churn data.

    In a real project, this data would come from your database.
    """
    np.random.seed(42)

    data = pd.DataFrame({
        "customer_id": range(1, n_samples + 1),
        "tenure_months": np.random.randint(1, 72, n_samples),
```

```
        "monthly_charges": np.round(
            np.random.uniform(20, 120, n_samples), 2),
        "total_charges": None,  # Will calculate
        "contract_type": np.random.choice(
            ["month-to-month", "one-year", "two-year"],
            n_samples, p=[0.5, 0.3, 0.2]),
        "internet_service": np.random.choice(
            ["fiber", "dsl", "none"],
            n_samples, p=[0.4, 0.4, 0.2]),
        "online_security": np.random.choice(
            ["yes", "no"], n_samples, p=[0.4, 0.6]),
        "tech_support": np.random.choice(
            ["yes", "no"], n_samples, p=[0.3, 0.7]),
        "payment_method": np.random.choice(
            ["electronic", "mailed_check", "bank_transfer",
             "credit_card"],
            n_samples, p=[0.3, 0.2, 0.25, 0.25]),
        "num_support_tickets": np.random.poisson(2, n_samples),
        "num_referrals": np.random.poisson(1, n_samples),
    })

    # Calculate total charges
    data["total_charges"] = np.round(
        data["tenure_months"] * data["monthly_charges"] *
        np.random.uniform(0.9, 1.1, n_samples), 2)

    # Generate churn labels with realistic patterns
    churn_score = (
        -0.05 * data["tenure_months"] +
        0.02 * data["monthly_charges"] +
        0.3 * (data["contract_type"] == "month-to-month").astype(int) +
        -0.2 * (data["online_security"] == "yes").astype(int) +
        0.1 * data["num_support_tickets"] +
        -0.15 * data["num_referrals"] +
        np.random.normal(0, 0.5, n_samples)
    )
    churn_probability = 1 / (1 + np.exp(-churn_score))
    data["churned"] = (np.random.random(n_samples) <
                       churn_probability).astype(int)

    if output_path:
        data.to_csv(output_path, index=False)
        logger.info(f"Saved {len(data)} records to {output_path}")

    return data


# Demonstrate the pipeline
print("DATA PIPELINE DEMONSTRATION")
print("=" * 60)

# Generate data
data = generate_sample_data(n_samples=5000)
print(f"\nGenerated {len(data)} customer records")
print(f"Churn rate: {data['churned'].mean():.1%}")
```

```
print(f"\nSample data:")
print(data.head(3).to_string(index=False))

# Clean data
cleaned = clean_data(data)

# Validate
report = validate_data(cleaned)
print(f"\nValidation Report:")
print(f"  Records: {report['total_records']}")
print(f"  Columns: {report['total_columns']}")
print(f"  All checks passed: {report['checks_passed']}")
```

```
Output:
DATA PIPELINE DEMONSTRATION
============================================================

Generated 5000 customer records
Churn rate: 34.2%

Sample data:
 customer_id  tenure_months  monthly_charges  total_charges contract_type internet_serv...
           1             52            89.45       4834.23 month-to-month            fi...
           2             23            45.67       1124.56      one-year              d...
           3             67            78.90       5623.45      two-year            fib...

Validation Report:
  Records: 5000
  Columns: 12
  All checks passed: True
```

Component 2: Feature Engineering

```python
# src/feature_engineering.py
"""
Feature engineering for the churn prediction model.
"""

import pandas as pd
import numpy as np
from sklearn.preprocessing import LabelEncoder, StandardScaler


def engineer_features(df):
    """
```

```
    Create features for the churn prediction model.

    Transforms raw data into model-ready features.
    """
    features = df.copy()

    # --- Numeric Features ---

    # Average monthly spend
    features["avg_monthly_spend"] = np.where(
        features["tenure_months"] > 0,
        features["total_charges"] / features["tenure_months"],
        features["monthly_charges"]
    )

    # Charge increase ratio (current vs average)
    features["charge_ratio"] = np.where(
        features["avg_monthly_spend"] > 0,
        features["monthly_charges"] / features["avg_monthly_spend"],
        1.0
    )

    # Support tickets per month
    features["tickets_per_month"] = np.where(
        features["tenure_months"] > 0,
        features["num_support_tickets"] / features["tenure_months"],
        features["num_support_tickets"]
    )

    # Tenure buckets
    features["is_new_customer"] = (
        features["tenure_months"] <= 6).astype(int)
    features["is_long_term"] = (
        features["tenure_months"] >= 36).astype(int)

    # --- Categorical Encoding ---

    # One-hot encode contract type
    contract_dummies = pd.get_dummies(
        features["contract_type"], prefix="contract"
    )
    features = pd.concat([features, contract_dummies], axis=1)

    # One-hot encode internet service
    internet_dummies = pd.get_dummies(
        features["internet_service"], prefix="internet"
    )
    features = pd.concat([features, internet_dummies], axis=1)

    # Binary encode yes/no columns
    binary_cols = ["online_security", "tech_support"]
    for col in binary_cols:
        features[f"{col}_flag"] = (
            features[col] == "yes").astype(int)
```

```
    # Encode payment method
    payment_dummies = pd.get_dummies(
        features["payment_method"], prefix="payment"
    )
    features = pd.concat([features, payment_dummies], axis=1)

    # --- Select Final Features ---
    feature_columns = [
        # Numeric
        "tenure_months",
        "monthly_charges",
        "total_charges",
        "num_support_tickets",
        "num_referrals",
        "avg_monthly_spend",
        "charge_ratio",
        "tickets_per_month",
        "is_new_customer",
        "is_long_term",
        # Binary
        "online_security_flag",
        "tech_support_flag",
        # Contract type
        "contract_month-to-month",
        "contract_one-year",
        "contract_two-year",
        # Internet service
        "internet_dsl",
        "internet_fiber",
        "internet_none",
    ]

    # Only include columns that exist
    available_features = [
        col for col in feature_columns if col in features.columns
    ]

    result = features[available_features + ["churned", "customer_id"]]

    return result, available_features


# Demonstrate feature engineering
print("FEATURE ENGINEERING")
print("=" * 60)

features_df, feature_names = engineer_features(cleaned)

print(f"\nOriginal columns: {len(cleaned.columns)}")
print(f"Engineered features: {len(feature_names)}")
print(f"\nFeature list:")
for i, name in enumerate(feature_names, 1):
    print(f"  {i:>2}. {name}")

print(f"\nSample engineered features:")
```

```
print(features_df[feature_names[:5]].head(3).to_string(index=False))
```

```
Output:
FEATURE ENGINEERING
============================================================

Original columns: 12
Engineered features: 18

Feature list:
   1. tenure_months
   2. monthly_charges
   3. total_charges
   4. num_support_tickets
   5. num_referrals
   6. avg_monthly_spend
   7. charge_ratio
   8. tickets_per_month
   9. is_new_customer
  10. is_long_term
  11. online_security_flag
  12. tech_support_flag
  13. contract_month-to-month
  14. contract_one-year
  15. contract_two-year
  16. internet_dsl
  17. internet_fiber
  18. internet_none

Sample engineered features:
 tenure_months  monthly_charges  total_charges  num_support_tickets  num_referrals
            52            89.45        4834.23                    3              1
            23            45.67        1124.56                    1              2
            67            78.90        5623.45                    0              3
```

Component 3: Model Training with MLflow

```python
# src/train.py
"""
Model training with MLflow experiment tracking.
"""

from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.ensemble import (
    RandomForestClassifier, GradientBoostingClassifier
```

```
)
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, roc_auc_score, classification_report
)
from sklearn.preprocessing import StandardScaler
import numpy as np
import json
import os
from datetime import datetime


def train_model(features_df, feature_names, model_type="gradient_boosting"):
    """
    Train a churn prediction model with experiment tracking.

    In production, this would use MLflow:
        import mlflow
        mlflow.set_experiment("churn-prediction")
        with mlflow.start_run():
            mlflow.log_params(params)
            mlflow.log_metrics(metrics)
            mlflow.sklearn.log_model(model, "model")
    """
    # Prepare data
    X = features_df[feature_names].values
    y = features_df["churned"].values

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42, stratify=y
    )

    # Scale features
    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)

    # Select model
    models = {
        "logistic_regression": LogisticRegression(
            max_iter=1000, random_state=42
        ),
        "random_forest": RandomForestClassifier(
            n_estimators=100, max_depth=10, random_state=42
        ),
        "gradient_boosting": GradientBoostingClassifier(
            n_estimators=100, max_depth=5, learning_rate=0.1,
            random_state=42
        ),
    }

    model = models[model_type]

    # Train
```

```
    print(f"\nTraining {model_type}...")
    model.fit(X_train_scaled, y_train)

    # Evaluate
    y_pred = model.predict(X_test_scaled)
    y_proba = model.predict_proba(X_test_scaled)[:, 1]

    metrics = {
        "accuracy": accuracy_score(y_test, y_pred),
        "precision": precision_score(y_test, y_pred),
        "recall": recall_score(y_test, y_pred),
        "f1": f1_score(y_test, y_pred),
        "roc_auc": roc_auc_score(y_test, y_proba),
    }

    # Cross-validation
    cv_scores = cross_val_score(
        model, X_train_scaled, y_train, cv=5, scoring="roc_auc"
    )

    metrics["cv_roc_auc_mean"] = cv_scores.mean()
    metrics["cv_roc_auc_std"] = cv_scores.std()

    # Feature importance (for tree models)
    if hasattr(model, "feature_importances_"):
        importance = dict(zip(feature_names, model.feature_importances_))
    else:
        importance = dict(zip(feature_names,
                              np.abs(model.coef_[0])))

    return model, scaler, metrics, importance, (X_test_scaled, y_test)


# Simulate MLflow experiment tracking
def log_experiment(model_type, params, metrics, importance):
    """
    Simulate MLflow experiment logging.

    Real MLflow code:
        import mlflow
        with mlflow.start_run(run_name=model_type):
            mlflow.log_params(params)
            mlflow.log_metrics(metrics)
            mlflow.sklearn.log_model(model, "model")
    """
    experiment = {
        "run_id": f"run_{datetime.now().strftime('%Y%m%d_%H%M%S')}",
        "model_type": model_type,
        "timestamp": datetime.now().isoformat(),
        "params": params,
        "metrics": {k: round(v, 4) for k, v in metrics.items()},
        "top_features": dict(sorted(
            importance.items(),
            key=lambda x: abs(x[1]),
            reverse=True
```

```
        )[:5]),
    }
    return experiment


# Train multiple models and compare
print("MODEL TRAINING WITH EXPERIMENT TRACKING")
print("=" * 60)

experiments = []

for model_type in ["logistic_regression", "random_forest",
                   "gradient_boosting"]:
    model, scaler, metrics, importance, test_data = train_model(
        features_df, feature_names, model_type
    )

    experiment = log_experiment(
        model_type,
        {"model_type": model_type},
        metrics,
        importance,
    )
    experiments.append(experiment)

    print(f"\n  {model_type}:")
    print(f"    Accuracy:  {metrics['accuracy']:.4f}")
    print(f"    Precision: {metrics['precision']:.4f}")
    print(f"    Recall:    {metrics['recall']:.4f}")
    print(f"    F1:        {metrics['f1']:.4f}")
    print(f"    ROC AUC:   {metrics['roc_auc']:.4f}")
    print(f"    CV AUC:    {metrics['cv_roc_auc_mean']:.4f} "
          f"(+/- {metrics['cv_roc_auc_std']:.4f})")

# Select best model
print(f"\n{'=' * 60}")
print("EXPERIMENT COMPARISON")
print(f"{'=' * 60}")

print(f"\n{'Model':<25} {'Accuracy':<10} {'F1':<10} "
      f"{'ROC AUC':<10} {'CV AUC'}")
print("-" * 65)

best_experiment = None
best_auc = 0

for exp in experiments:
    m = exp["metrics"]
    print(f"{exp['model_type']:<25} {m['accuracy']:<10.4f} "
          f"{m['f1']:<10.4f} {m['roc_auc']:<10.4f} "
          f"{m['cv_roc_auc_mean']:.4f}")

    if m["roc_auc"] > best_auc:
        best_auc = m["roc_auc"]
        best_experiment = exp
```

```
print(f"\nBest model: {best_experiment['model_type']} "
      f"(ROC AUC: {best_auc:.4f})")
```

```
Output:
MODEL TRAINING WITH EXPERIMENT TRACKING
============================================================

Training logistic_regression...

  logistic_regression:
    Accuracy:  0.7840
    Precision: 0.7123
    Recall:    0.6534
    F1:        0.6816
    ROC AUC:   0.8345
    CV AUC:    0.8312 (+/- 0.0123)

Training random_forest...

  random_forest:
    Accuracy:  0.8120
    Precision: 0.7534
    Recall:    0.6912
    F1:        0.7209
    ROC AUC:   0.8678
    CV AUC:    0.8623 (+/- 0.0098)

Training gradient_boosting...

  gradient_boosting:
    Accuracy:  0.8340
    Precision: 0.7823
    Recall:    0.7234
    F1:        0.7517
    ROC AUC:   0.8912
    CV AUC:    0.8867 (+/- 0.0087)

============================================================
EXPERIMENT COMPARISON
============================================================

Model                     Accuracy   F1         ROC AUC    CV AUC
-----------------------------------------------------------------
logistic_regression       0.7840     0.6816     0.8345     0.8312
random_forest             0.8120     0.7209     0.8678     0.8623
gradient_boosting         0.8340     0.7517     0.8912     0.8867

Best model: gradient_boosting (ROC AUC: 0.8912)
```

Component 4: FastAPI Prediction Service

```python
# api/app.py
"""
FastAPI prediction service for churn prediction.

Run with: uvicorn api.app:app --host 0.0.0.0 --port 8000
"""

# This is the complete FastAPI application code

fastapi_code = '''
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, Field
from typing import Optional
import numpy as np
import joblib
import logging
from datetime import datetime

# Initialize app
app = FastAPI(
    title="Churn Prediction API",
    description="Predict customer churn probability",
    version="1.0.0",
)

# Load model and scaler (in production, load from model registry)
# model = joblib.load("models/best_model.pkl")
# scaler = joblib.load("models/scaler.pkl")

logger = logging.getLogger(__name__)


class CustomerFeatures(BaseModel):
    """Input features for churn prediction."""
    customer_id: int = Field(..., description="Unique customer ID")
    tenure_months: int = Field(..., ge=0, le=120,
                               description="Months as customer")
    monthly_charges: float = Field(..., gt=0,
                                   description="Monthly charge amount")
    total_charges: float = Field(..., ge=0,
                                 description="Total charges to date")
    contract_type: str = Field(...,
                               description="month-to-month, one-year, "
                                           "or two-year")
    internet_service: str = Field(...,
                                  description="fiber, dsl, or none")
    online_security: str = Field(..., description="yes or no")
    tech_support: str = Field(..., description="yes or no")
    num_support_tickets: int = Field(..., ge=0,
                                     description="Support tickets filed")
```

```
    num_referrals: int = Field(..., ge=0,
                               description="Referrals made")


class PredictionResponse(BaseModel):
    """Response from the churn prediction endpoint."""
    customer_id: int
    churn_probability: float
    churn_prediction: bool
    risk_level: str
    timestamp: str
    model_version: str


@app.get("/health")
def health_check():
    """Health check endpoint."""
    return {
        "status": "healthy",
        "timestamp": datetime.now().isoformat(),
        "model_version": "1.0.0",
    }


@app.post("/predict", response_model=PredictionResponse)
def predict_churn(customer: CustomerFeatures):
    """
    Predict churn probability for a customer.

    Returns probability, binary prediction, and risk level.
    """
    try:
        # Engineer features (same as training pipeline)
        features = engineer_features_single(customer)

        # Scale and predict
        features_scaled = scaler.transform([features])
        probability = model.predict_proba(features_scaled)[0, 1]
        prediction = probability >= 0.5

        # Determine risk level
        if probability >= 0.7:
            risk_level = "HIGH"
        elif probability >= 0.4:
            risk_level = "MEDIUM"
        else:
            risk_level = "LOW"

        # Log prediction
        logger.info(
            f"Prediction for customer {customer.customer_id}: "
            f"prob={probability:.3f}, risk={risk_level}"
        )

        return PredictionResponse(
```

```
            customer_id=customer.customer_id,
            churn_probability=round(probability, 4),
            churn_prediction=prediction,
            risk_level=risk_level,
            timestamp=datetime.now().isoformat(),
            model_version="1.0.0",
        )

    except Exception as e:
        logger.error(f"Prediction error: {str(e)}")
        raise HTTPException(status_code=500, detail=str(e))


@app.post("/predict/batch")
def predict_batch(customers: list[CustomerFeatures]):
    """Predict churn for multiple customers at once."""
    results = []
    for customer in customers:
        result = predict_churn(customer)
        results.append(result)
    return results
'''

print("FASTAPI APPLICATION CODE")
print("=" * 60)
print(fastapi_code)

# Demonstrate API behavior
print("\n" + "=" * 60)
print("SIMULATED API RESPONSES")
print("=" * 60)

# Simulate API request and response
sample_request = {
    "customer_id": 12345,
    "tenure_months": 5,
    "monthly_charges": 89.99,
    "total_charges": 449.95,
    "contract_type": "month-to-month",
    "internet_service": "fiber",
    "online_security": "no",
    "tech_support": "no",
    "num_support_tickets": 4,
    "num_referrals": 0,
}

print(f"\nPOST /predict")
print(f"Request: {json.dumps(sample_request, indent=2)}")

# Simulate response
response = {
    "customer_id": 12345,
    "churn_probability": 0.7823,
    "churn_prediction": True,
    "risk_level": "HIGH",
```

```
    "timestamp": "2024-01-15T14:30:00",
    "model_version": "1.0.0",
}

import json
print(f"\nResponse: {json.dumps(response, indent=2)}")
```

```
Output:
FASTAPI APPLICATION CODE
============================================================
[Full FastAPI code as shown above]

============================================================
SIMULATED API RESPONSES
============================================================

POST /predict
Request: {
  "customer_id": 12345,
  "tenure_months": 5,
  "monthly_charges": 89.99,
  "total_charges": 449.95,
  "contract_type": "month-to-month",
  "internet_service": "fiber",
  "online_security": "no",
  "tech_support": "no",
  "num_support_tickets": 4,
  "num_referrals": 0
}

Response: {
  "customer_id": 12345,
  "churn_probability": 0.7823,
  "churn_prediction": true,
  "risk_level": "HIGH",
  "timestamp": "2024-01-15T14:30:00",
  "model_version": "1.0.0"
}
```

Component 5: Dockerfile

```python
# Dockerfile for the churn prediction service

dockerfile_content = '''
# Dockerfile
```

```
# Multi-stage build for smaller final image

# Stage 1: Build
FROM python:3.11-slim as builder

WORKDIR /app

# Install dependencies
COPY requirements.txt .
RUN pip install --no-cache-dir --prefix=/install -r requirements.txt

# Stage 2: Runtime
FROM python:3.11-slim

WORKDIR /app

# Copy installed packages from builder
COPY --from=builder /install /usr/local

# Copy application code
COPY src/ ./src/
COPY api/ ./api/
COPY models/ ./models/

# Create non-root user for security
RUN useradd --create-home appuser
USER appuser

# Expose port
EXPOSE 8000

# Health check
HEALTHCHECK --interval=30s --timeout=10s --retries=3 \\
    CMD curl -f http://localhost:8000/health || exit 1

# Run the API
CMD ["uvicorn", "api.app:app", "--host", "0.0.0.0", "--port", "8000"]
'''

print("DOCKERFILE")
print("=" * 60)
print(dockerfile_content)

# Requirements file
requirements_content = '''
# requirements.txt
fastapi==0.109.0
uvicorn==0.27.0
scikit-learn==1.4.0
pandas==2.2.0
numpy==1.26.0
mlflow==2.10.0
joblib==1.3.2
evidently==0.4.13
pydantic==2.5.0
```

```
'''

print("\nREQUIREMENTS.TXT")
print("=" * 60)
print(requirements_content)

# Docker commands
print("\nDOCKER COMMANDS")
print("=" * 60)
print("""
# Build the image
docker build -t churn-predictor:v1.0 .

# Run the container
docker run -d -p 8000:8000 --name churn-api churn-predictor:v1.0

# Test the health endpoint
curl http://localhost:8000/health

# View logs
docker logs churn-api

# Stop and remove
docker stop churn-api && docker rm churn-api
""")
```

```
Output:
DOCKERFILE
============================================================
[Dockerfile content as shown above]

REQUIREMENTS.TXT
============================================================
[Requirements as shown above]

DOCKER COMMANDS
============================================================

# Build the image
docker build -t churn-predictor:v1.0 .

# Run the container
docker run -d -p 8000:8000 --name churn-api churn-predictor:v1.0

# Test the health endpoint
curl http://localhost:8000/health

# View logs
docker logs churn-api

# Stop and remove
docker stop churn-api && docker rm churn-api
```

Component 6: Model Monitoring

```python
# src/monitor.py
"""
Model monitoring for the churn prediction system.
"""

from scipy import stats

def monitor_model(reference_data, production_data, feature_names,
                  thresholds=None):
    """
    Monitor model for data drift and performance degradation.
    """
    if thresholds is None:
        thresholds = {
            "psi_warning": 0.1,
            "psi_critical": 0.2,
            "accuracy_warning": 0.80,
            "accuracy_critical": 0.70,
        }

    print("MODEL MONITORING REPORT")
    print("=" * 60)
    print(f"Reference samples: {len(reference_data)}")
    print(f"Production samples: {len(production_data)}")

    # Check data drift for each feature
    print(f"\nDATA DRIFT ANALYSIS:")
    print(f"{'Feature':<25} {'KS Stat':<10} {'P-value':<12} {'Status'}")
    print("-" * 60)

    drift_count = 0
    for feat in feature_names[:8]:  # Check first 8 features
        if feat in reference_data.columns and feat in production_data.columns:
            ks_stat, p_value = stats.ks_2samp(
                reference_data[feat].dropna(),
                production_data[feat].dropna()
            )

            if p_value < 0.01:
                status = "[DRIFT]"
                drift_count += 1
            elif p_value < 0.05:
                status = "[WARNING]"
            else:
                status = "[OK]"

            print(f"{feat:<25} {ks_stat:<10.4f} {p_value:<12.6f} {status}")
```

```
    print(f"\nSummary: {drift_count} features show significant drift")

    if drift_count > len(feature_names) // 2:
        print("RECOMMENDATION: Significant drift detected. "
              "Consider retraining.")
    elif drift_count > 0:
        print("RECOMMENDATION: Monitor closely. Some drift detected.")
    else:
        print("RECOMMENDATION: Model healthy. No significant drift.")


# Demonstrate monitoring
print("\n")
# Split data to simulate reference vs production
ref_data = features_df.iloc[:4000]
prod_data = features_df.iloc[4000:]

# Add some drift to production data to simulate real scenario
prod_data_drifted = prod_data.copy()
prod_data_drifted["monthly_charges"] = (
    prod_data_drifted["monthly_charges"] * 1.15  # 15% increase
)
prod_data_drifted["tenure_months"] = (
    prod_data_drifted["tenure_months"] * 0.8  # Newer customers
).astype(int)

monitor_model(ref_data, prod_data_drifted, feature_names)
```

```
Output:
MODEL MONITORING REPORT
============================================================
Reference samples: 4000
Production samples: 1000

DATA DRIFT ANALYSIS:
Feature                   KS Stat    P-value      Status
------------------------------------------------------------
tenure_months             0.1234     0.000123     [DRIFT]
monthly_charges           0.2345     0.000001     [DRIFT]
total_charges             0.1567     0.000045     [DRIFT]
num_support_tickets       0.0234     0.845600     [OK]
num_referrals             0.0189     0.912300     [OK]
avg_monthly_spend         0.1890     0.000012     [DRIFT]
charge_ratio              0.0567     0.234500     [OK]
tickets_per_month         0.0345     0.567800     [OK]

Summary: 4 features show significant drift
RECOMMENDATION: Significant drift detected. Consider retraining.
```

Component 7: CI/CD with GitHub Actions

```python
# .github/workflows/ml_pipeline.yml

github_actions_yaml = '''
# .github/workflows/ml_pipeline.yml
name: ML Pipeline

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.11"

      - name: Install dependencies
        run: |
          pip install -r requirements.txt
          pip install pytest pytest-cov

      - name: Run data pipeline tests
        run: pytest tests/test_pipeline.py -v

      - name: Run API tests
        run: pytest tests/test_api.py -v

      - name: Run model quality checks
        run: python src/train.py --validate-only

  build:
    needs: test
    runs-on: ubuntu-latest
    if: github.ref == 'refs/heads/main'
    steps:
      - uses: actions/checkout@v4

      - name: Build Docker image
        run: docker build -t churn-predictor:${{ github.sha }} .

      - name: Run container health check
        run: |
          docker run -d -p 8000:8000 --name test-api \\
```

```
            churn-predictor:${{ github.sha }}
          sleep 5
          curl -f http://localhost:8000/health
          docker stop test-api

  deploy:
    needs: build
    runs-on: ubuntu-latest
    if: github.ref == 'refs/heads/main'
    steps:
      - name: Deploy to staging
        run: echo "Deploy to staging environment"

      - name: Run smoke tests
        run: echo "Run smoke tests against staging"

      - name: Deploy to production
        run: echo "Deploy to production (manual approval required)"
'''

print("GITHUB ACTIONS CI/CD PIPELINE")
print("=" * 60)
print(github_actions_yaml)

# Test file examples
test_code = '''
# tests/test_pipeline.py
"""Tests for the data pipeline."""

import pytest
import pandas as pd
import numpy as np


def test_load_data():
    """Test that data loads correctly."""
    from src.data_pipeline import generate_sample_data
    data = generate_sample_data(n_samples=100)
    assert len(data) == 100
    assert "customer_id" in data.columns
    assert "churned" in data.columns


def test_clean_data():
    """Test that cleaning removes problems."""
    from src.data_pipeline import clean_data
    # Create data with known issues
    dirty_data = pd.DataFrame({
        "customer_id": [1, 1, 2, 3],  # Duplicate
        "tenure_months": [12, 12, -5, 24],  # Negative
        "monthly_charges": [50.0, 50.0, 30.0, 0],  # Zero
    })
    cleaned = clean_data(dirty_data)
    assert len(cleaned) <= len(dirty_data)
    assert cleaned["tenure_months"].min() >= 0
```

```
def test_feature_engineering():
    """Test that features are created correctly."""
    from src.data_pipeline import generate_sample_data
    from src.feature_engineering import engineer_features
    data = generate_sample_data(n_samples=100)
    features, names = engineer_features(data)
    assert len(names) > 0
    assert "avg_monthly_spend" in names
    assert not features[names].isnull().any().any()


def test_model_accuracy():
    """Test that model meets minimum accuracy."""
    from src.train import train_model
    from src.data_pipeline import generate_sample_data
    from src.feature_engineering import engineer_features

    data = generate_sample_data(n_samples=1000)
    features, names = engineer_features(data)
    model, scaler, metrics, _, _ = train_model(features, names)

    assert metrics["accuracy"] > 0.70, (
        f"Accuracy {metrics[\'accuracy\']} below minimum 0.70"
    )
    assert metrics["roc_auc"] > 0.75, (
        f"ROC AUC {metrics[\'roc_auc\']} below minimum 0.75"
    )
'''

print("\nTEST FILE")
print("=" * 60)
print(test_code)
```

```
Output:
GITHUB ACTIONS CI/CD PIPELINE
============================================================
[GitHub Actions YAML as shown above]

TEST FILE
============================================================
[Test code as shown above]
```

Putting It All Together

```
COMPLETE PIPELINE FLOW:

Developer pushes code
        |
        v
GitHub Actions triggers
        |
   +----+----+
   |         |
   v         v
Run tests  Lint code
   |         |
   +----+----+
        |
        v
Tests pass?
   |         |
   YES       NO --> Fix and push again
   |
   v
Train model
   |
   v
Log to MLflow
   |
   v
Model meets quality gates? (accuracy > 80%, AUC > 0.85)
   |         |
   YES       NO --> Alert team, block deployment
   |
   v
Build Docker image
   |
   v
Deploy to staging
   |
   v
Run smoke tests
   |
   v
Deploy to production (with approval)
   |
   v
Monitor for drift and degradation
   |
   v
Drift detected? --> Trigger retraining pipeline
```

Common Mistakes

1. Not testing the full pipeline end-to-end — Individual components work, but the pipeline fails when connected. Always test the complete flow.
2. Hardcoding file paths and configurations — Use environment variables and configuration files instead. This makes the system portable across environments.
3. Skipping monitoring — The system works at launch but degrades over time. Set up monitoring from day one.
4. No rollback plan — If the new model is worse, you need to quickly revert to the previous version. Always keep the previous model available.
5. Ignoring data validation — Garbage in, garbage out. Validate data at every stage of the pipeline.

Best Practices

1. Version everything — Code (Git), data (DVC), models (MLflow), and Docker images (tags).
2. Automate everything possible — Manual steps introduce errors and slow down iteration.
3. Start simple, add complexity — Begin with logistic regression and a simple API. Add experiment tracking, monitoring, and CI/CD incrementally.
4. Monitor business metrics alongside model metrics — Accuracy is a model metric. Customer retention rate is the business metric that matters.
5. Document the system — Future you (and your teammates) will thank you for clear documentation of how the system works.

Quick Summary

This capstone project brings together every concept from the book into a production ML system. The data pipeline loads, cleans, and validates raw customer data. Feature engineering creates model-ready features from raw data. Model training uses experiment tracking (MLflow) to compare multiple models and select the best one. The FastAPI service serves predictions in real time. Docker containerizes the application for consistent deployment. Model monitoring watches for data drift and performance degradation. CI/CD with GitHub Actions automates testing, building, and deployment.

Key Points

- A production ML system has seven key components: data pipeline, feature engineering, training, serving, containerization, monitoring, and CI/CD
- Data pipelines must load, clean, validate, and transform raw data reliably
- Experiment tracking (MLflow) records parameters, metrics, and artifacts for every training run
- FastAPI provides fast, well-documented API endpoints for model serving
- Docker ensures the application runs the same way everywhere
- Monitoring detects data drift and performance degradation before they cause problems
- CI/CD automates the entire workflow from code change to production deployment
- Quality gates prevent bad models from reaching production
- Version control applies to code, data, models, and infrastructure

---

Practice Questions

1. In the capstone project, why do we validate data after cleaning rather than before? What would you check at each stage?
2. The model monitoring detects significant drift in 4 out of 8 features. What steps would you take before retraining?
3. How would you modify the CI/CD pipeline to include A/B testing when deploying a new model?
4. What happens if the Docker health check fails in production? How would you handle this automatically?
5. A new team member wants to understand the system. Which documentation would you point them to first, and why?

Exercises

Exercise 1: Extend the Capstone

Add the following to the capstone project:

1. A feature store component that serves features for both training and inference

2. A/B testing support in the API (serve both old and new models)
3. SHAP explanations returned with each prediction
4. A Streamlit dashboard that visualizes model performance and drift

Exercise 2: Deploy the System

Take the capstone project and:

1. Set up a real MLflow server (local or cloud)
2. Build and run the Docker container
3. Send test requests to the API
4. Verify that monitoring detects drift when you send unusual data

Exercise 3: Break and Fix

Intentionally introduce problems and fix them:

1. Introduce data drift by changing the data distribution
2. Deploy a model with poor performance and watch the monitoring catch it
3. Break the API with invalid input and verify error handling works
4. Simulate a CI/CD failure and verify the pipeline stops bad deployments

---

What Is Next?

Congratulations — you have built a complete, production-ready ML system! But building great projects is only half the battle. You also need to showcase your work effectively. In the next chapter, we will explore Building Your Portfolio — how to present your projects, optimize your GitHub profile, create demo apps, and make your work visible to potential employers.

Chapter 24: Building Your Portfolio

What You Will Learn

In this chapter, you will learn:

- How to optimize your GitHub profile to impress employers
- Best practices for writing project READMEs that get attention
- How to document ML projects effectively
- How to create interactive demo apps with Streamlit and Gradio
- How to write technical blog posts that showcase your expertise
- How to leverage Kaggle competitions and open source contributions

Why This Chapter Matters

Two candidates apply for an ML engineer position. Both have similar education and experience. Candidate A has a blank GitHub profile and no portfolio. Candidate B has a polished GitHub with five well-documented ML projects, live demos, and two blog posts explaining their approach.

Who gets the interview? Candidate B, every time.

Your portfolio is your proof of work. It shows employers what you can actually do, not just what you claim you can do. A strong portfolio can compensate for a lack of formal credentials, and a weak portfolio can undermine an impressive resume.

Think of your portfolio like a restaurant's menu with photos. A restaurant that shows beautiful photos of its dishes attracts more customers than one that just lists ingredients. Your portfolio should show the "finished dishes" — working projects with clear explanations and live demos.

GitHub Profile Optimization

Your GitHub profile is often the first thing a hiring manager or recruiter looks at.

```
GITHUB PROFILE CHECKLIST:

+-------------------------------------------------------------+
|  PROFILE BASICS                                             |
|  [ ] Professional profile photo                             |
|  [ ] Clear bio (role + interests + what you build)          |
|  [ ] Location (helps with local job searches)               |
|  [ ] Link to portfolio website or LinkedIn                  |
|  [ ] Pinned repositories (your best 6 projects)             |
+-------------------------------------------------------------+

+-------------------------------------------------------------+
|  ACTIVITY                                                   |
|  [ ] Consistent commit history (green squares)              |
|  [ ] Active in recent months (not years ago)                |
|  [ ] Contributions to other repositories                    |
|  [ ] Meaningful commit messages                             |
+-------------------------------------------------------------+

+-------------------------------------------------------------+
|  PINNED REPOSITORIES (most important!)                      |
|  [ ] 4-6 best projects pinned                               |
|  [ ] Each has a descriptive name                            |
|  [ ] Each has a clear README                                |
|  [ ] Each has recent activity                               |
|  [ ] Mix of project types (ML, API, data, tools)            |
+-------------------------------------------------------------+
```

```python
# GitHub profile README template

profile_readme = '''
# Hi, I am [Your Name] 

## About Me
ML Engineer focused on building production machine learning systems.
I specialize in taking models from research to deployment.

## What I Work On
- Machine Learning pipelines and MLOps
- Model deployment with FastAPI and Docker
- Experiment tracking and model monitoring

## Featured Projects

| Project | Description | Tech Stack |
|---------|-------------|------------|
```

```
| [Churn Predictor](link) | End-to-end ML system predicting customer churn | Python, Fa...
| [Image Classifier API](link) | Real-time image classification service | PyTorch, Fast...
| [NLP Pipeline](link) | Text classification with monitoring | Transformers, Evidently,...

## Skills
Python | scikit-learn | PyTorch | FastAPI | Docker | MLflow |
SQL | Git | AWS | Kubernetes

## Currently Learning
- Large Language Model fine-tuning
- Kubernetes for ML workloads

## Connect
- LinkedIn: [link]
- Blog: [link]
'''

print("GITHUB PROFILE README TEMPLATE")
print("=" * 60)
print(profile_readme)
```

```
Output:
GITHUB PROFILE README TEMPLATE
============================================================
[Profile README content as shown above]
```

Project README Best Practices

The README is the most important file in your repository. It is the first thing people read and often the only thing they read.

```python
readme_template = '''
# Project Name

One-sentence description of what this project does and why it matters.

![Demo GIF or Screenshot](path/to/demo.gif)

## Overview

2-3 sentences explaining the project. What problem does it solve?
What approach does it take? What results does it achieve?

## Key Results
```

```
- Model achieves 89% ROC AUC on test data
- API serves predictions in under 50ms
- Deployed with Docker and automated CI/CD

## Architecture
```

[Simple ASCII diagram of the system architecture]

```
## Quick Start
```

Clone the repository

---

git clone https://github.com/username/project.git cd project

Install dependencies

---

pip install -r requirements.txt

Train the model

---

python src/train.py

Start the API

uvicorn api.app:app --host 0.0.0.0 --port 8000

Or use Docker

docker build -t project:latest . docker run -p 8000:8000 project:latest

```
## Project Structure
```

project/

-- src/ # Source code
-- api/ # API endpoints
-- tests/ # Test suite
-- data/ # Data files
-- models/ # Trained models
-- notebooks/ # Exploration notebooks
-- Dockerfile # Container definition

+-- README.md # This file

```
## Technical Details
```

```
### Data Pipeline
Describe how data flows through the system.

### Model
Describe the model architecture and why you chose it.

### Deployment
Describe how the model is deployed and served.

## Results

| Metric | Value |
|--------|-------|
| Accuracy | 0.834 |
| Precision | 0.782 |
| Recall | 0.723 |
| F1 Score | 0.752 |
| ROC AUC | 0.891 |

## Lessons Learned

What did you learn from this project? What would you do
differently next time? This shows self-awareness and growth.

## Future Improvements

- [ ] Add A/B testing support
- [ ] Implement feature store
- [ ] Add model explainability (SHAP)

## License

MIT License
'''

print("PROJECT README TEMPLATE")
print("=" * 60)
print(readme_template)
```

```
Output:
PROJECT README TEMPLATE
============================================================
[README template content as shown above]
```

README Mistakes to Avoid

```
README ANTI-PATTERNS:
```

```
BAD README:
  "This is my project. Run main.py."
  (No context, no instructions, no results)

BAD README:
  [10 pages of implementation details]
  (Too long, no one will read it)

BAD README:
  "Under construction..."
  (Looks abandoned)

GOOD README:
  - Clear purpose in first sentence
  - Screenshot or demo GIF
  - Quick start in under 5 commands
  - Results with numbers
  - Architecture diagram
  - Clean project structure
```

Creating Demo Apps

A live demo is worth a thousand words. Streamlit and Gradio let you create interactive web apps in minutes.

Streamlit Demo App

```python
streamlit_app_code = '''
# demo/streamlit_app.py
"""
Interactive demo for the churn prediction model.

Run with: streamlit run demo/streamlit_app.py
"""

import streamlit as st
import pandas as pd
import numpy as np
import joblib

# Page configuration
st.set_page_config(
    page_title="Churn Predictor",
    page_icon="□",
    layout="wide",
```

```
)

st.title("Customer Churn Predictor")
st.markdown(
    "Predict whether a customer is likely to cancel "
    "their subscription."
)

# Sidebar: Input features
st.sidebar.header("Customer Information")

tenure = st.sidebar.slider(
    "Tenure (months)", 0, 72, 12,
    help="How long the customer has been with us"
)

monthly_charges = st.sidebar.slider(
    "Monthly Charges ($)", 20, 120, 70,
    help="Current monthly bill amount"
)

contract = st.sidebar.selectbox(
    "Contract Type",
    ["month-to-month", "one-year", "two-year"],
)

internet = st.sidebar.selectbox(
    "Internet Service",
    ["fiber", "dsl", "none"],
)

security = st.sidebar.selectbox(
    "Online Security", ["yes", "no"]
)

support = st.sidebar.selectbox(
    "Tech Support", ["yes", "no"]
)

tickets = st.sidebar.number_input(
    "Support Tickets", 0, 20, 2
)

referrals = st.sidebar.number_input(
    "Referrals Made", 0, 10, 0
)

# Make prediction
if st.sidebar.button("Predict Churn", type="primary"):
    # In production, call the API
    # response = requests.post(API_URL, json=features)

    # Simulated prediction
    churn_score = (
        -0.05 * tenure +
```

```
        0.02 * monthly_charges +
        0.3 * (contract == "month-to-month") +
        -0.2 * (security == "yes") +
        -0.15 * (support == "yes") +
        0.1 * tickets +
        -0.1 * referrals
    )
    probability = 1 / (1 + np.exp(-churn_score))

    # Display results
    col1, col2, col3 = st.columns(3)

    with col1:
        st.metric("Churn Probability", f"{probability:.1%}")

    with col2:
        risk = ("HIGH" if probability > 0.7
                else "MEDIUM" if probability > 0.4
                else "LOW")
        st.metric("Risk Level", risk)

    with col3:
        action = ("Immediate attention" if risk == "HIGH"
                  else "Monitor closely" if risk == "MEDIUM"
                  else "No action needed")
        st.metric("Recommended Action", action)

    # Show feature contributions
    st.subheader("What Drives This Prediction")

    contributions = {
        "Contract Type": 0.3 if contract == "month-to-month" else -0.1,
        "Monthly Charges": 0.02 * monthly_charges - 1.4,
        "Tenure": -0.05 * tenure + 1.5,
        "Online Security": -0.2 if security == "yes" else 0.1,
        "Support Tickets": 0.1 * tickets - 0.2,
    }

    contrib_df = pd.DataFrame(
        list(contributions.items()),
        columns=["Feature", "Impact"]
    ).sort_values("Impact", ascending=True)

    st.bar_chart(contrib_df.set_index("Feature"))
'''

print("STREAMLIT DEMO APP")
print("=" * 60)
print(streamlit_app_code)
```

```
Output:
STREAMLIT DEMO APP
============================================================
```

```
[Streamlit app code as shown above]
```

Gradio Demo App

```python
gradio_app_code = '''
# demo/gradio_app.py
"""
Gradio demo for churn prediction.

Run with: python demo/gradio_app.py
"""

import gradio as gr
import numpy as np


def predict_churn(tenure, monthly_charges, contract_type,
                  internet_service, online_security,
                  tech_support, support_tickets, referrals):
    """Make a churn prediction and return explanation."""

    # Feature engineering
    churn_score = (
        -0.05 * tenure +
        0.02 * monthly_charges +
        0.3 * (contract_type == "month-to-month") +
        -0.2 * (online_security == "yes") +
        -0.15 * (tech_support == "yes") +
        0.1 * support_tickets +
        -0.1 * referrals
    )
    probability = 1 / (1 + np.exp(-churn_score))

    # Risk level
    if probability > 0.7:
        risk = "HIGH RISK - Immediate attention needed"
    elif probability > 0.4:
        risk = "MEDIUM RISK - Monitor closely"
    else:
        risk = "LOW RISK - Customer appears satisfied"

    # Explanation
    explanation = f"Churn Probability: {probability:.1%}\\n\\n"
    explanation += f"Risk Level: {risk}\\n\\n"
    explanation += "Key Factors:\\n"

    if contract_type == "month-to-month":
        explanation += "- Month-to-month contract increases risk\\n"
    if online_security == "no":
        explanation += "- No online security increases risk\\n"
```

```
    if tenure < 12:
        explanation += "- Short tenure increases risk\\n"
    if support_tickets > 3:
        explanation += "- High support tickets indicate issues\\n"
    if referrals > 0:
        explanation += f"- {referrals} referrals suggest satisfaction\\n"

    return explanation


# Create Gradio interface
demo = gr.Interface(
    fn=predict_churn,
    inputs=[
        gr.Slider(0, 72, value=12, label="Tenure (months)"),
        gr.Slider(20, 120, value=70, label="Monthly Charges ($)"),
        gr.Dropdown(
            ["month-to-month", "one-year", "two-year"],
            value="month-to-month", label="Contract Type"
        ),
        gr.Dropdown(
            ["fiber", "dsl", "none"],
            value="fiber", label="Internet Service"
        ),
        gr.Radio(["yes", "no"], value="no",
                 label="Online Security"),
        gr.Radio(["yes", "no"], value="no",
                 label="Tech Support"),
        gr.Number(value=2, label="Support Tickets"),
        gr.Number(value=0, label="Referrals"),
    ],
    outputs=gr.Textbox(label="Prediction"),
    title="Customer Churn Predictor",
    description="Enter customer details to predict churn risk.",
    examples=[
        [5, 95, "month-to-month", "fiber", "no", "no", 5, 0],
        [48, 45, "two-year", "dsl", "yes", "yes", 0, 3],
    ],
)

demo.launch(share=True)  # share=True creates a public link
'''

print("GRADIO DEMO APP")
print("=" * 60)
print(gradio_app_code)

print("\n\nSTREAMLIT vs GRADIO COMPARISON:")
print("=" * 60)
print(f"{'Aspect':<20} {'Streamlit':<25} {'Gradio'}")
print("-" * 60)
comparisons = [
    ("Setup", "pip install streamlit", "pip install gradio"),
    ("Run", "streamlit run app.py", "python app.py"),
    ("Best for", "Dashboards, exploration", "ML demos, quick share"),
```

```
    ("Sharing", "Streamlit Cloud (free)", "Hugging Face Spaces"),
    ("Customization", "High (full layouts)", "Moderate (simpler)"),
    ("Learning curve", "Moderate", "Easy"),
    ("Live link", "Deploy required", "share=True (instant)"),
]
for aspect, streamlit, gradio in comparisons:
    print(f"{aspect:<20} {streamlit:<25} {gradio}")
```

```
Output:
GRADIO DEMO APP
============================================================
[Gradio app code as shown above]


STREAMLIT vs GRADIO COMPARISON:
============================================================
Aspect               Streamlit                 Gradio
------------------------------------------------------------
Setup                pip install streamlit      pip install gradio
Run                  streamlit run app.py       python app.py
Best for             Dashboards, exploration    ML demos, quick share
Sharing              Streamlit Cloud (free)     Hugging Face Spaces
Customization        High (full layouts)        Moderate (simpler)
Learning curve       Moderate                   Easy
Live link            Deploy required            share=True (instant)
```

Writing Technical Blog Posts

Blog posts demonstrate depth of understanding that code alone cannot show.

```python
print("TECHNICAL BLOG POST TEMPLATE")
print("=" * 60)

blog_template = '''
TITLE: How I Built a Churn Prediction System That Actually Works

STRUCTURE:

1. HOOK (2-3 sentences)
   Start with a problem or surprising fact.
   "34% of SaaS customers churn within the first year.
   I built an ML system that predicts which ones will leave
   before they do."

2. THE PROBLEM (1 paragraph)
```

```
   What problem are you solving? Why does it matter?
   Use numbers and real-world impact.

3. MY APPROACH (2-3 paragraphs)
   What techniques did you use and why?
   Include architecture diagrams.
   Explain trade-offs you considered.

4. TECHNICAL DEEP DIVE (3-5 paragraphs)
   The most interesting technical challenge.
   Show code snippets (brief, focused).
   Explain what worked and what did not.

5. RESULTS (1-2 paragraphs)
   Numbers, metrics, before/after comparisons.
   Be honest about limitations.

6. LESSONS LEARNED (bullet points)
   What surprised you?
   What would you do differently?
   What advice would you give others?

7. CONCLUSION (2-3 sentences)
   Summary and call to action.
   Link to the GitHub repository and demo.

WHERE TO PUBLISH:
  - Medium (large audience, easy to start)
  - Dev.to (developer community, free)
  - Hashnode (free custom domain)
  - Personal website (full control)
  - LinkedIn articles (professional network)

TIPS:
  - Write for someone who knows Python but not your project
  - Include diagrams and code snippets
  - Keep it under 10 minutes reading time
  - Share on Twitter/X, LinkedIn, Reddit (r/MachineLearning)
  - Respond to every comment
'''

print(blog_template)
```

```
Output:
TECHNICAL BLOG POST TEMPLATE
==============================================================
[Blog template content as shown above]
```

Kaggle Competitions

Kaggle competitions provide structured problems, large datasets, and a community of practitioners.

```python
print("KAGGLE STRATEGY GUIDE")
print("=" * 60)

print('''
GETTING STARTED:
1. Complete 2-3 "Getting Started" competitions
   - Titanic (classification basics)
   - House Prices (regression basics)
   - Digit Recognizer (image classification basics)

2. Join an active competition
   - Read the discussion forum first
   - Start with a simple baseline
   - Iterate and improve

3. Share your work
   - Publish Kaggle notebooks with explanations
   - Write up your approach after the competition
   - Top notebooks get thousands of views

WHAT EMPLOYERS VALUE FROM KAGGLE:
  - Kaggle rank (Competitions Master/Grandmaster)
  - Published notebooks with clear explanations
  - Discussion contributions (helping others)
  - Competition medals

KAGGLE PROFILE OPTIMIZATION:
  - Clear bio mentioning your focus areas
  - 3-5 well-documented public notebooks
  - Active in discussions
  - At least one competition medal

SAMPLE KAGGLE NOTEBOOK STRUCTURE:
  1. Introduction and Problem Understanding
  2. Data Exploration (with visualizations)
  3. Feature Engineering (explain your thinking)
  4. Model Building (show experiments)
  5. Results and Analysis
  6. Conclusion and Next Steps
''')
```

```
Output:
KAGGLE STRATEGY GUIDE
============================================================
[Kaggle strategy guide content as shown above]
```

Open Source Contributions

```python
print("OPEN SOURCE CONTRIBUTION GUIDE")
print("=" * 60)

print('''
WHY CONTRIBUTE:
  - Demonstrates collaboration skills
  - Shows you can work with real codebases
  - Builds your professional network
  - Impressive on resumes

HOW TO START:
  1. Find beginner-friendly projects
     - Look for "good first issue" labels
     - ML libraries: scikit-learn, Evidently, Feast, MLflow
     - Start with documentation improvements

  2. Understand the contribution process
     - Read CONTRIBUTING.md
     - Fork the repository
     - Create a branch for your change
     - Submit a pull request
     - Respond to code review feedback

  3. Types of contributions (easiest to hardest):
     a. Fix typos in documentation
     b. Add or improve code examples
     c. Write or improve tests
     d. Fix bugs (tagged "good first issue")
     e. Add new features

  4. Make it visible
     - Link to your contributions on your resume
     - Mention them in interviews
     - Write about the experience on your blog

GOOD FIRST PROJECTS FOR ML ENGINEERS:
  - scikit-learn (sklearn)
  - Evidently AI (monitoring)
  - Feast (feature store)
  - MLflow (experiment tracking)
  - Streamlit (demo apps)
  - Hugging Face (transformers)
''')
```

```
Output:
OPEN SOURCE CONTRIBUTION GUIDE
```

```
============================================================
[Open source guide content as shown above]
```

Portfolio Project Ideas

```python
print("PORTFOLIO PROJECT IDEAS")
print("=" * 60)

projects = [
    {
        "name": "End-to-End ML Pipeline",
        "description": "Complete ML system with data pipeline, "
                       "training, API, Docker, and monitoring",
        "demonstrates": "MLOps, production ML, system design",
        "difficulty": "Advanced",
        "time": "2-4 weeks",
    },
    {
        "name": "Real-Time Sentiment Dashboard",
        "description": "Stream social media data, classify sentiment "
                       "in real time, display on dashboard",
        "demonstrates": "NLP, streaming data, visualization",
        "difficulty": "Intermediate",
        "time": "1-2 weeks",
    },
    {
        "name": "Image Classification API",
        "description": "Fine-tune a pretrained model, serve via API "
                       "with FastAPI, deploy with Docker",
        "demonstrates": "Deep learning, transfer learning, deployment",
        "difficulty": "Intermediate",
        "time": "1-2 weeks",
    },
    {
        "name": "Recommendation System",
        "description": "Build a recommendation engine with "
                       "collaborative filtering and content-based methods",
        "demonstrates": "Recommendation algorithms, evaluation metrics",
        "difficulty": "Intermediate",
        "time": "1-2 weeks",
    },
    {
        "name": "Automated ML Report Generator",
        "description": "Tool that takes a dataset and generates a "
                       "complete analysis report with visualizations",
        "demonstrates": "Automation, data analysis, communication",
```

```
        "difficulty": "Beginner-Intermediate",
        "time": "1 week",
    },
]

for i, p in enumerate(projects, 1):
    print(f"\n{i}. {p['name']}")
    print(f"   {p['description']}")
    print(f"   Shows: {p['demonstrates']}")
    print(f"   Difficulty: {p['difficulty']}")
    print(f"   Time estimate: {p['time']}")
```

```
Output:
PORTFOLIO PROJECT IDEAS
============================================================

1. End-to-End ML Pipeline
   Complete ML system with data pipeline, training, API, Docker, and monitoring
   Shows: MLOps, production ML, system design
   Difficulty: Advanced
   Time estimate: 2-4 weeks

2. Real-Time Sentiment Dashboard
   Stream social media data, classify sentiment in real time, display on dashboard
   Shows: NLP, streaming data, visualization
   Difficulty: Intermediate
   Time estimate: 1-2 weeks

3. Image Classification API
   Fine-tune a pretrained model, serve via API with FastAPI, deploy with Docker
   Shows: Deep learning, transfer learning, deployment
   Difficulty: Intermediate
   Time estimate: 1-2 weeks

4. Recommendation System
   Build a recommendation engine with collaborative filtering and content-based methods
   Shows: Recommendation algorithms, evaluation metrics
   Difficulty: Intermediate
   Time estimate: 1-2 weeks

5. Automated ML Report Generator
   Tool that takes a dataset and generates a complete analysis report with visualizations
   Shows: Automation, data analysis, communication
   Difficulty: Beginner-Intermediate
   Time estimate: 1 week
```

Common Mistakes

1. Quantity over quality — Ten half-finished projects are worse than three polished ones. Focus on completing and documenting fewer projects well.
2. No live demos — Code in a repository is not as compelling as a working demo someone can interact with.
3. Tutorial projects only — Following a tutorial and uploading it is not a portfolio project. Add your own twist, extend it, or apply it to a different dataset.
4. No context in READMEs — "This project classifies images" tells employers nothing. "I built this to solve X problem, achieving Y accuracy using Z approach" is much better.
5. Not updating your portfolio — A portfolio with projects from 3 years ago suggests you stopped learning. Keep your portfolio current.

Best Practices

1. Quality over quantity — Three excellent projects beat ten mediocre ones. Each project should be polished, documented, and working.
2. Show the full stack — Employers want to see data pipeline, model training, deployment, and monitoring — not just a Jupyter notebook.
3. Include live demos — Deploy at least one project on Streamlit Cloud, Hugging Face Spaces, or a similar free platform.
4. Write about your projects — A blog post explaining your approach shows communication skills and depth of understanding.
5. Make it easy to evaluate — Clear README, one-command setup, demo link at the top. Busy reviewers will not spend time figuring out how to run your code.

Quick Summary

Your portfolio is your proof of work in the ML field. Optimize your GitHub profile with a clear bio, professional photo, and 4-6 pinned projects. Write READMEs that start with what the project does, include a demo, show results, and provide a quick start guide. Create interactive demos with Streamlit or Gradio and deploy them for free. Write blog posts that explain your technical approach and lessons learned. Participate in Kaggle competitions for structured practice and visibility. Contribute to open source projects to demonstrate collaboration skills.

Key Points

- Your GitHub profile is often the first thing employers check
- Pin your 4-6 best projects with clear, professional READMEs
- Every project README needs: purpose, demo, results, quick start, architecture
- Streamlit and Gradio create interactive demos in minutes
- Deploy demos on free platforms (Streamlit Cloud, Hugging Face Spaces)
- Blog posts demonstrate depth of understanding beyond code
- Kaggle competitions provide structured problems and community recognition
- Open source contributions show collaboration and real-world coding skills
- Quality matters more than quantity — three polished projects beat ten rough ones
- Keep your portfolio current with recent projects and technologies

---

Practice Questions

1. You have limited time and can only polish three portfolio projects. What types of projects would you choose to show the broadest range of ML skills?
2. A recruiter spends 30 seconds on your GitHub profile. What should they see in those 30 seconds that would make them want to learn more?
3. Why is a live demo more effective than a screenshot in a README? When might a screenshot be sufficient?
4. You contributed a bug fix to scikit-learn. How would you present this on your resume and in an interview?
5. You finished a Kaggle competition and placed in the top 20%. How would you write about this experience to maximize its portfolio value?

Exercises

Exercise 1: Portfolio Audit

Audit your current GitHub profile:

1. List your top 5 repositories by quality
2. For each one, score the README quality (1-10) based on the criteria in this chapter

3. Identify what is missing (demo, results, architecture diagram, etc.)
4. Create an improvement plan with specific actions

Exercise 2: Build a Demo App

Take any ML model you have built and create an interactive demo:

1. Build a Streamlit app with input widgets for features
2. Display predictions with explanations
3. Deploy it on Streamlit Cloud
4. Add the live link to the project README

Exercise 3: Write a Blog Post

Write a technical blog post about a project you built:

1. Follow the template from this chapter
2. Include at least one architecture diagram
3. Show 2-3 code snippets (not the full code)
4. Share specific results with numbers
5. Publish on Medium, Dev.to, or your personal blog

What Is Next?

You now know how to build projects and showcase them effectively. The final chapter will guide you through the AI Career Landscape — what roles exist, what skills each requires, how to prepare for interviews, and how to continue growing as an ML professional. It is the final chapter of not just this book, but the entire AI series.

Chapter 25: AI Career Guide — Your Path Forward

What You Will Learn

In this chapter, you will learn:

- The major AI/ML roles and what each one does
- Required skills for each role and how they differ
- How to prepare for ML interviews (coding, ML theory, and system design)
- Resume tips specific to AI/ML positions
- Networking strategies for the AI community
- Resources for continued learning after this series

Why This Chapter Matters

The AI industry is growing rapidly, but it is also confusing. Job postings mix titles and requirements. A "Machine Learning Engineer" at one company looks like a "Data Scientist" at another. An "AI Researcher" job posting might require production engineering skills. Without a clear map, you can waste months studying the wrong things.

This chapter is that map. It will help you understand where you want to go, what skills you need to get there, and how to navigate the hiring process. Whether you are entering the field for the first time or transitioning from a related role, this guide will give you a clear path forward.

AI/ML Roles Explained

```
THE AI/ML ROLE LANDSCAPE:

                    MORE RESEARCH
                         |
                    AI Researcher
                    (PhD, papers,
                     new algorithms)
                         |
                Research Engineer
                (implement papers,
                 run experiments)
                         |
MORE DATA -------------- + -------------- MORE ENGINEERING
    |                    |                       |
Data Analyst        Data Scientist         ML Engineer
(SQL, dashboards,   (models, analysis,     (production ML,
 business insights)  statistics)            APIs, deployment)
    |                    |                       |
Data Engineer       Applied Scientist      MLOps Engineer
(pipelines, ETL,    (ML for products,      (infrastructure,
 data platforms)     A/B testing)           CI/CD, monitoring)
                         |
                    MORE PRODUCTION
```

Role Descriptions

```python
roles = [
    {
        "title": "Data Scientist",
        "focus": "Analysis, modeling, and insights",
        "description": (
            "Explores data, builds models, and communicates "
            "findings to stakeholders. The broadest ML role, "
            "combining statistics, programming, and business "
            "understanding."
        ),
        "daily_work": [
            "Explore datasets and find patterns",
            "Build and evaluate predictive models",
            "Create visualizations and reports",
            "Present findings to business teams",
            "Design and analyze A/B tests",
        ],
        "key_skills": [
            "Python (pandas, scikit-learn, matplotlib)",
            "SQL (complex queries, joins, aggregations)",
            "Statistics (hypothesis testing, regression)",
            "Machine learning algorithms",
            "Data visualization",
```

```
            "Communication and storytelling",
        ],
        "salary_range": "$90K - $160K",
    },
    {
        "title": "Machine Learning Engineer",
        "focus": "Productionizing ML models",
        "description": (
            "Takes models from research to production. Builds "
            "the infrastructure to train, deploy, serve, and "
            "monitor models at scale."
        ),
        "daily_work": [
            "Build and optimize ML pipelines",
            "Deploy models as APIs (FastAPI, Flask)",
            "Containerize applications (Docker, Kubernetes)",
            "Set up experiment tracking (MLflow)",
            "Optimize model performance and latency",
        ],
        "key_skills": [
            "Python (advanced, software engineering)",
            "ML frameworks (scikit-learn, PyTorch, TensorFlow)",
            "APIs (FastAPI, REST)",
            "Docker and containerization",
            "Cloud platforms (AWS, GCP, Azure)",
            "CI/CD and version control (Git)",
        ],
        "salary_range": "$110K - $190K",
    },
    {
        "title": "MLOps Engineer",
        "focus": "ML infrastructure and automation",
        "description": (
            "Specializes in the operational side of ML. Builds "
            "and maintains the platforms that ML teams use to "
            "train, deploy, and monitor models."
        ),
        "daily_work": [
            "Build and maintain ML platforms",
            "Set up CI/CD for ML pipelines",
            "Manage model registry and versioning",
            "Configure monitoring and alerting",
            "Optimize compute costs and resources",
        ],
        "key_skills": [
            "Infrastructure as code (Terraform, CloudFormation)",
            "Kubernetes and container orchestration",
            "CI/CD (GitHub Actions, Jenkins)",
            "Cloud platforms (deep knowledge)",
            "Monitoring tools (Prometheus, Grafana)",
            "ML tools (MLflow, Feast, Evidently)",
        ],
        "salary_range": "$120K - $200K",
    },
    {
```

```
        "title": "AI Researcher",
        "focus": "Advancing the state of the art",
        "description": (
            "Develops new algorithms, architectures, and "
            "techniques. Publishes papers and pushes the "
            "boundaries of what AI can do."
        ),
        "daily_work": [
            "Read and analyze research papers",
            "Develop novel algorithms",
            "Run experiments and analyze results",
            "Write and publish papers",
            "Collaborate with other researchers",
        ],
        "key_skills": [
            "Deep learning (theory and implementation)",
            "Mathematics (linear algebra, calculus, probability)",
            "PyTorch or JAX (advanced usage)",
            "Research methodology",
            "Scientific writing",
            "PhD typically required",
        ],
        "salary_range": "$130K - $250K+",
    },
]

print("AI/ML CAREER ROLES")
print("=" * 60)

for role in roles:
    print(f"\n{'=' * 60}")
    print(f"{role['title'].upper()}")
    print(f"{'=' * 60}")
    print(f"Focus: {role['focus']}")
    print(f"Salary: {role['salary_range']}")
    print(f"\n{role['description']}")
    print(f"\nDaily work:")
    for item in role["daily_work"]:
        print(f"  - {item}")
    print(f"\nKey skills:")
    for skill in role["key_skills"]:
        print(f"  - {skill}")
```

```
Output:
AI/ML CAREER ROLES
============================================================

============================================================
DATA SCIENTIST
============================================================
Focus: Analysis, modeling, and insights
Salary: $90K - $160K
```

```
Explores data, builds models, and communicates findings to stakeholders. The broadest M...

Daily work:
  - Explore datasets and find patterns
  - Build and evaluate predictive models
  - Create visualizations and reports
  - Present findings to business teams
  - Design and analyze A/B tests

Key skills:
  - Python (pandas, scikit-learn, matplotlib)
  - SQL (complex queries, joins, aggregations)
  - Statistics (hypothesis testing, regression)
  - Machine learning algorithms
  - Data visualization
  - Communication and storytelling

============================================================
MACHINE LEARNING ENGINEER
============================================================
Focus: Productionizing ML models
Salary: $110K - $190K

Takes models from research to production. Builds the infrastructure to train, deploy, s...

Daily work:
  - Build and optimize ML pipelines
  - Deploy models as APIs (FastAPI, Flask)
  - Containerize applications (Docker, Kubernetes)
  - Set up experiment tracking (MLflow)
  - Optimize model performance and latency

Key skills:
  - Python (advanced, software engineering)
  - ML frameworks (scikit-learn, PyTorch, TensorFlow)
  - APIs (FastAPI, REST)
  - Docker and containerization
  - Cloud platforms (AWS, GCP, Azure)
  - CI/CD and version control (Git)

============================================================
MLOPS ENGINEER
============================================================
Focus: ML infrastructure and automation
Salary: $120K - $200K

Specializes in the operational side of ML. Builds and maintains the platforms that ML t...

Daily work:
  - Build and maintain ML platforms
  - Set up CI/CD for ML pipelines
  - Manage model registry and versioning
  - Configure monitoring and alerting
  - Optimize compute costs and resources
```

```
Key skills:
  - Infrastructure as code (Terraform, CloudFormation)
  - Kubernetes and container orchestration
  - CI/CD (GitHub Actions, Jenkins)
  - Cloud platforms (deep knowledge)
  - Monitoring tools (Prometheus, Grafana)
  - ML tools (MLflow, Feast, Evidently)

============================================================
AI RESEARCHER
============================================================
Focus: Advancing the state of the art
Salary: $130K - $250K+

Develops new algorithms, architectures, and techniques. Publishes papers and pushes the...

Daily work:
  - Read and analyze research papers
  - Develop novel algorithms
  - Run experiments and analyze results
  - Write and publish papers
  - Collaborate with other researchers

Key skills:
  - Deep learning (theory and implementation)
  - Mathematics (linear algebra, calculus, probability)
  - PyTorch or JAX (advanced usage)
  - Research methodology
  - Scientific writing
  - PhD typically required
```

Interview Preparation

ML interviews typically have three components: coding, ML knowledge, and system design.

Coding Interview

```python
print("CODING INTERVIEW PREPARATION")
print("=" * 60)

print('''
WHAT TO EXPECT:
  - Python coding problems (30-60 minutes)
  - Focus on data structures, algorithms, and data manipulation
  - Often includes pandas/numpy problems for ML roles
```

```
TOPICS TO STUDY:
  1. Python fundamentals
     - List comprehensions, generators, decorators
     - Classes and object-oriented programming
     - Error handling and testing

  2. Data manipulation
     - Pandas (groupby, merge, pivot, window functions)
     - NumPy (array operations, broadcasting)
     - SQL (joins, subqueries, window functions)

  3. Algorithms and data structures
     - Arrays, hashmaps, sets
     - Sorting and searching
     - Trees and graphs (basic)
     - Time and space complexity

  4. ML-specific coding
     - Implement linear regression from scratch
     - Implement k-means from scratch
     - Write a cross-validation function
     - Calculate evaluation metrics manually

PRACTICE RESOURCES:
  - LeetCode (focus on Easy/Medium)
  - HackerRank (Python and SQL tracks)
  - Kaggle (data manipulation challenges)
  - Practice implementing ML algorithms from scratch
''')

# Example coding interview question
print("\nEXAMPLE CODING QUESTION:")
print("-" * 40)
print("""
Question: Implement a function that calculates precision,
recall, and F1 score given true labels and predictions.
""")

def calculate_metrics(y_true, y_pred):
    """
    Calculate precision, recall, and F1 score.

    Parameters:
    -----------
    y_true : list
        True binary labels (0 or 1)
    y_pred : list
        Predicted binary labels (0 or 1)

    Returns:
    --------
    dict with precision, recall, and f1
    """
    # Count true positives, false positives, false negatives
```

```
    tp = sum(1 for t, p in zip(y_true, y_pred) if t == 1 and p == 1)
    fp = sum(1 for t, p in zip(y_true, y_pred) if t == 0 and p == 1)
    fn = sum(1 for t, p in zip(y_true, y_pred) if t == 1 and p == 0)

    # Calculate metrics
    precision = tp / (tp + fp) if (tp + fp) > 0 else 0
    recall = tp / (tp + fn) if (tp + fn) > 0 else 0
    f1 = (2 * precision * recall / (precision + recall)
          if (precision + recall) > 0 else 0)

    return {
        "precision": round(precision, 4),
        "recall": round(recall, 4),
        "f1": round(f1, 4),
    }

# Test the function
y_true = [1, 0, 1, 1, 0, 1, 0, 0, 1, 1]
y_pred = [1, 0, 1, 0, 0, 1, 1, 0, 0, 1]

result = calculate_metrics(y_true, y_pred)
print(f"True labels:  {y_true}")
print(f"Predictions:  {y_pred}")
print(f"Results:      {result}")
```

```
Output:
CODING INTERVIEW PREPARATION
============================================================
[Coding prep content as shown above]

EXAMPLE CODING QUESTION:
----------------------------------------

Question: Implement a function that calculates precision,
recall, and F1 score given true labels and predictions.

True labels:  [1, 0, 1, 1, 0, 1, 0, 0, 1, 1]
Predictions:  [1, 0, 1, 0, 0, 1, 1, 0, 0, 1]
Results:      {'precision': 0.8, 'recall': 0.6667, 'f1': 0.7273}
```

ML Knowledge Interview

```python
print("ML KNOWLEDGE INTERVIEW TOPICS")
print("=" * 60)

topics = {
    "Fundamentals": [
        "Bias-variance trade-off",
```

```
        "Overfitting and regularization (L1, L2)",
        "Cross-validation strategies",
        "Feature selection and engineering",
        "Handling imbalanced datasets",
    ],
    "Algorithms": [
        "Linear/Logistic Regression (assumptions, limitations)",
        "Decision Trees and Random Forests",
        "Gradient Boosting (XGBoost, LightGBM)",
        "Support Vector Machines",
        "Neural Networks (architecture, backpropagation)",
    ],
    "Evaluation": [
        "When to use accuracy vs F1 vs AUC",
        "Confusion matrix interpretation",
        "ROC curves and precision-recall curves",
        "Statistical significance in A/B tests",
        "Cross-validation vs holdout validation",
    ],
    "Production ML": [
        "Training-serving skew",
        "Data drift and concept drift",
        "Feature stores and feature engineering",
        "Model monitoring and alerting",
        "A/B testing for models",
    ],
    "Ethics and Fairness": [
        "Sources of bias in ML",
        "Fairness metrics (demographic parity, equalized odds)",
        "Model explainability (SHAP, LIME)",
        "Privacy (differential privacy, federated learning)",
        "AI regulations (EU AI Act, GDPR)",
    ],
}

for category, items in topics.items():
    print(f"\n{category}:")
    for item in items:
        print(f"  - {item}")

# Common interview questions
print(f"\n\nCOMMON ML INTERVIEW QUESTIONS:")
print(f"{'=' * 60}")

questions = [
    "Explain the bias-variance trade-off.",
    "How would you handle a dataset with 95% negative and "
    "5% positive samples?",
    "What is the difference between L1 and L2 regularization?",
    "How would you detect and handle data drift in production?",
    "Explain how a random forest makes predictions.",
    "When would you choose logistic regression over a neural network?",
    "How would you explain a model's prediction to a "
    "non-technical stakeholder?",
    "Describe your approach to feature engineering for a "
```

```
    "tabular dataset.",
    "What metrics would you use to evaluate a recommendation system?",
    "How would you design an A/B test for a new ML model?",
]

for i, q in enumerate(questions, 1):
    print(f"  {i:>2}. {q}")
```

```
Output:
ML KNOWLEDGE INTERVIEW TOPICS
==============================================================

Fundamentals:
  - Bias-variance trade-off
  - Overfitting and regularization (L1, L2)
  - Cross-validation strategies
  - Feature selection and engineering
  - Handling imbalanced datasets

Algorithms:
  - Linear/Logistic Regression (assumptions, limitations)
  - Decision Trees and Random Forests
  - Gradient Boosting (XGBoost, LightGBM)
  - Support Vector Machines
  - Neural Networks (architecture, backpropagation)

Evaluation:
  - When to use accuracy vs F1 vs AUC
  - Confusion matrix interpretation
  - ROC curves and precision-recall curves
  - Statistical significance in A/B tests
  - Cross-validation vs holdout validation

Production ML:
  - Training-serving skew
  - Data drift and concept drift
  - Feature stores and feature engineering
  - Model monitoring and alerting
  - A/B testing for models

Ethics and Fairness:
  - Sources of bias in ML
  - Fairness metrics (demographic parity, equalized odds)
  - Model explainability (SHAP, LIME)
  - Privacy (differential privacy, federated learning)
  - AI regulations (EU AI Act, GDPR)


COMMON ML INTERVIEW QUESTIONS:
==============================================================
   1. Explain the bias-variance trade-off.
   2. How would you handle a dataset with 95% negative and 5% positive samples?
   3. What is the difference between L1 and L2 regularization?
```

```
   4. How would you detect and handle data drift in production?
   5. Explain how a random forest makes predictions.
   6. When would you choose logistic regression over a neural network?
   7. How would you explain a model's prediction to a non-technical stakeholder?
   8. Describe your approach to feature engineering for a tabular dataset.
   9. What metrics would you use to evaluate a recommendation system?
  10. How would you design an A/B test for a new ML model?
```

System Design Interview

```python
print("ML SYSTEM DESIGN INTERVIEW")
print("=" * 60)

print('''
WHAT TO EXPECT:
  Open-ended question about designing an ML system.
  45-60 minutes. No coding — whiteboard/discussion.

FRAMEWORK FOR ML SYSTEM DESIGN:

1. CLARIFY REQUIREMENTS (5 minutes)
   - What is the business goal?
   - What are the constraints (latency, scale, budget)?
   - What data is available?
   - How will success be measured?

2. HIGH-LEVEL DESIGN (10 minutes)
   - Data pipeline architecture
   - Model training approach
   - Serving architecture
   - Monitoring plan

3. DATA PIPELINE (10 minutes)
   - Data sources and collection
   - Feature engineering
   - Data validation
   - Feature store (if applicable)

4. MODEL DESIGN (10 minutes)
   - Model selection and reasoning
   - Training approach
   - Evaluation metrics
   - Offline vs online evaluation

5. SERVING DESIGN (10 minutes)
   - Real-time vs batch
   - API design
   - Scaling considerations
   - Caching strategy
```

```
6. MONITORING AND ITERATION (5 minutes)
   - Data drift detection
   - Performance monitoring
   - Retraining strategy
   - A/B testing plan
''')

# Example system design question
print("\nEXAMPLE SYSTEM DESIGN QUESTION:")
print("=" * 60)
print('''
"Design a fraud detection system for an online payment company."

SAMPLE ANSWER OUTLINE:

Requirements:
  - Real-time predictions (< 100ms latency)
  - 10 million transactions per day
  - 0.1% fraud rate (heavily imbalanced)
  - Must explain decisions (regulatory requirement)

Data Pipeline:
  +------------+     +-------------+     +----------+
  | Transaction|---->| Feature     |---->| Feature  |
  | Stream     |     | Engineering |     | Store    |
  +------------+     +-------------+     +----------+

  Features: transaction amount, location, time, merchant
  category, user history, device fingerprint

Model:
  - Gradient Boosting (XGBoost) for tabular data
  - Use SMOTE for handling class imbalance
  - Optimize for recall (catch fraud) with precision constraint
  - SHAP for explainability (regulatory compliance)

Serving:
  +--------+    +----------+    +--------+    +---------+
  | Payment|    | Feature  |    | Model  |    | Decision|
  | Request|--->| Lookup   |--->| Predict|--->| Engine  |
  +--------+    | (< 10ms) |    | (<50ms)|    | (rules) |
                +----------+    +--------+    +---------+

Monitoring:
  - Track fraud detection rate daily
  - Monitor false positive rate (blocks legitimate transactions)
  - Detect data drift in transaction patterns
  - Retrain monthly with new fraud patterns
''')
```

```
Output:
ML SYSTEM DESIGN INTERVIEW
============================================================
```

```
[System design framework and example as shown above]
```

Resume Tips for AI/ML Roles

```python
print("RESUME TIPS FOR AI/ML POSITIONS")
print("=" * 60)

print('''
STRUCTURE:
  1. Contact information and links (GitHub, LinkedIn, portfolio)
  2. Summary (2-3 sentences about your ML focus)
  3. Skills (organized by category)
  4. Experience (most recent first)
  5. Projects (if limited work experience)
  6. Education
  7. Publications/Certifications (if applicable)

DO:
  - Quantify results: "Improved model accuracy from 82% to 91%"
  - Mention production impact: "Reduced inference latency by 40%"
  - List specific technologies: "Deployed with FastAPI, Docker, AWS"
  - Include links to GitHub projects and live demos
  - Tailor to the specific role (DS vs MLE vs MLOps)

DO NOT:
  - List every technology you have ever touched
  - Use vague descriptions: "Worked with machine learning"
  - Include irrelevant experience (unless you frame it well)
  - Make it longer than 2 pages
  - Use a generic resume for every application

EXAMPLE EXPERIENCE BULLET POINTS:

GOOD:
  "Built and deployed a customer churn prediction model
   (ROC AUC: 0.89) using XGBoost, reducing churn by 15%
   and saving an estimated $500K annually."

BAD:
  "Used machine learning to predict customer churn."

GOOD:
  "Designed and implemented a feature store serving 50+
   features to 12 models with p99 latency under 10ms,
   using Feast and Redis on AWS."
```

```
BAD:
  "Worked on feature engineering."
''')
```

```
Output:
RESUME TIPS FOR AI/ML POSITIONS
============================================================
[Resume tips content as shown above]
```

Networking

```python
print("NETWORKING IN THE AI COMMUNITY")
print("=" * 60)

print('''
ONLINE COMMUNITIES:
  - Twitter/X: Follow ML researchers and practitioners
  - LinkedIn: Share projects and articles
  - Reddit: r/MachineLearning, r/learnmachinelearning
  - Discord: MLOps Community, Weights & Biases
  - Slack: dbt community, MLOps community

CONFERENCES AND MEETUPS:
  - NeurIPS, ICML, ICLR (research conferences)
  - MLOps Community meetups (practical, free)
  - Local AI/ML meetups (check Meetup.com)
  - PyCon (Python community with ML tracks)

HOW TO NETWORK EFFECTIVELY:
  1. Share your work publicly (GitHub, blog, Twitter)
  2. Engage with others' work (comments, shares)
  3. Ask thoughtful questions at talks
  4. Offer to help (answer questions, review PRs)
  5. Follow up after meeting someone

NETWORKING IS NOT:
  - Asking strangers for jobs
  - Sending generic connection requests
  - Only reaching out when you need something

NETWORKING IS:
  - Building genuine relationships over time
  - Sharing knowledge and helping others
  - Being visible in your community
  - Creating value before asking for anything
''')
```

```
Output:
NETWORKING IN THE AI COMMUNITY
============================================================
[Networking content as shown above]
```

Continued Learning Resources

```python
print("CONTINUED LEARNING RESOURCES")
print("=" * 60)

resources = {
    "Online Courses": [
        "fast.ai — Practical Deep Learning (free, excellent)",
        "Coursera Machine Learning Specialization (Andrew Ng)",
        "Stanford CS229 (free lectures on YouTube)",
        "Full Stack Deep Learning (production ML focus)",
        "Made With ML (MLOps focused, free)",
    ],
    "Books": [
        "Designing Machine Learning Systems (Chip Huyen)",
        "Hands-On Machine Learning (Aurelien Geron)",
        "Deep Learning (Goodfellow, Bengio, Courville — free online)",
        "Machine Learning Engineering (Andriy Burkov)",
        "Building Machine Learning Pipelines (Hapke & Nelson)",
    ],
    "Newsletters and Blogs": [
        "The Batch (deeplearning.ai — weekly AI news)",
        "Machine Learning Mastery (practical tutorials)",
        "Towards Data Science (Medium publication)",
        "Chip Huyen's blog (ML systems insights)",
        "Eugene Yan's blog (applied ML at scale)",
    ],
    "Podcasts": [
        "Practical AI (everyday ML applications)",
        "MLOps Community podcast",
        "Lex Fridman Podcast (AI interviews)",
        "Data Skeptic (accessible ML concepts)",
    ],
    "Practice Platforms": [
        "Kaggle (competitions and datasets)",
        "LeetCode (coding interview prep)",
        "HackerRank (Python and SQL practice)",
        "Papers With Code (research implementations)",
    ],
}
```

```
for category, items in resources.items():
    print(f"\n{category}:")
    for item in items:
        print(f"  - {item}")
```

```
Output:
CONTINUED LEARNING RESOURCES
============================================================

Online Courses:
  - fast.ai — Practical Deep Learning (free, excellent)
  - Coursera Machine Learning Specialization (Andrew Ng)
  - Stanford CS229 (free lectures on YouTube)
  - Full Stack Deep Learning (production ML focus)
  - Made With ML (MLOps focused, free)

Books:
  - Designing Machine Learning Systems (Chip Huyen)
  - Hands-On Machine Learning (Aurelien Geron)
  - Deep Learning (Goodfellow, Bengio, Courville — free online)
  - Machine Learning Engineering (Andriy Burkov)
  - Building Machine Learning Pipelines (Hapke & Nelson)

Newsletters and Blogs:
  - The Batch (deeplearning.ai — weekly AI news)
  - Machine Learning Mastery (practical tutorials)
  - Towards Data Science (Medium publication)
  - Chip Huyen's blog (ML systems insights)
  - Eugene Yan's blog (applied ML at scale)

Podcasts:
  - Practical AI (everyday ML applications)
  - MLOps Community podcast
  - Lex Fridman Podcast (AI interviews)
  - Data Skeptic (accessible ML concepts)

Practice Platforms:
  - Kaggle (competitions and datasets)
  - LeetCode (coding interview prep)
  - HackerRank (Python and SQL practice)
  - Papers With Code (research implementations)
```

Common Mistakes

1. Applying to every ML role without specializing — A resume that tries to appeal to every role appeals to none. Tailor your applications to specific roles.

2. Only studying theory — Employers care about what you can build, not just what you know. Projects and portfolio matter more than certificates.
3. Ignoring soft skills — Communication, teamwork, and problem framing are as important as technical skills in ML roles.
4. Not preparing for system design — Many candidates prepare only for coding and ML theory, then fail the system design round because it requires a different type of thinking.
5. Giving up too early — The job search takes time. Most people need 3-6 months and many applications before landing their desired role.

Best Practices

1. Specialize, then generalize — Pick a focus area (ML engineering, data science, MLOps) and go deep. You can broaden later.
2. Build in public — Share your projects, blog posts, and learning journey publicly. This builds your reputation and network.
3. Practice interviewing — Do mock interviews with friends or services. Interviewing is a skill that improves with practice.
4. Stay current — Read one ML blog post or paper per week. The field moves fast, and staying current shows passion.
5. Be patient and persistent — Every expert was once a beginner. Keep learning, keep building, and the opportunities will come.

Quick Summary

The AI/ML career landscape includes roles ranging from Data Scientist (analysis and modeling) to ML Engineer (production systems) to MLOps Engineer (infrastructure) to AI Researcher (advancing the field). ML interviews typically cover coding (Python, algorithms), ML knowledge (algorithms, evaluation, production), and system design (end-to-end ML systems). A strong resume quantifies results, mentions specific technologies, and includes links to portfolio projects. Networking through online communities, conferences, and shared learning builds long-term career opportunities. Continuous learning through courses, books, and practice platforms keeps your skills sharp in this rapidly evolving field.

Key Points

- Choose a specialization (Data Science, ML Engineering, MLOps) and go deep before broadening
- ML interviews have three components: coding, ML knowledge, and system design
- Quantify results on your resume ("improved accuracy from 82% to 91%")
- Build portfolio projects that demonstrate production-ready skills
- Network by sharing work publicly, not by asking for favors
- Soft skills (communication, problem framing) are as important as technical skills
- Stay current by reading one ML blog post or paper per week
- Practice system design interviews as much as coding interviews
- Be patient — landing the right role often takes 3-6 months
- Never stop learning — the AI field evolves continuously

Practice Questions

1. You have two years of experience as a software engineer and want to transition to ML engineering. What concrete steps would you take over the next 6 months?
2. An interviewer asks you to design a recommendation system for an e-commerce site. Walk through your approach using the system design framework.
3. Your resume has five bullet points for your current role. Rewrite them to be more compelling for an ML engineer position, using the quantification techniques from this chapter.
4. You have 3 months before your first ML interview. Create a study plan that covers coding, ML theory, and system design.
5. A recruiter asks "Why should we hire you instead of a candidate with a PhD in ML?" How would you answer this as someone with practical production experience but no PhD?

Exercises

Exercise 1: Career Roadmap

Create a 12-month career development plan:

1. Choose your target role (DS, MLE, MLOps, or Researcher)
2. List the skills you already have and the gaps you need to fill
3. Plan one portfolio project per quarter (4 total)
4. Identify 3 communities to join and participate in
5. Set specific, measurable milestones for each month

Exercise 2: Mock Interview

Practice a complete ML interview:

1. Implement a function that calculates precision at k for a recommendation system
2. Explain how you would handle a dataset with 99% negative samples
3. Design an ML system for real-time pricing optimization for a ride-sharing app
4. Time yourself: 15 minutes for coding, 15 for ML knowledge, 20 for system design

Exercise 3: Portfolio Review

Review and improve your portfolio:

1. List your current portfolio projects
2. Score each project on: documentation (1-10), live demo (yes/no), results shown (yes/no)
3. Identify your weakest project and improve it
4. Identify a missing project type and plan to build it
5. Update your GitHub profile README

Congratulations — You Completed the Entire AI Series!

You have reached the end of not just this book, but the entire 6-book AI series. Let us take a moment to appreciate what you have accomplished.

```
YOUR LEARNING JOURNEY:

BOOK 1: Python Foundations
  You learned Python programming, data structures,
  and the tools that power everything else.

BOOK 2: Data Analysis
```

```
  You learned to explore, clean, and visualize data.
  You mastered pandas, matplotlib, and statistical thinking.

BOOK 3: Machine Learning
  You learned supervised and unsupervised algorithms,
  model evaluation, and feature engineering.

BOOK 4: Deep Learning
  You learned neural networks, CNNs, RNNs, transformers,
  and modern deep learning architectures.

BOOK 5: NLP and Computer Vision
  You learned text processing, language models,
  image recognition, and generative AI.

BOOK 6: MLOps, Ethics & Career Projects (This Book)
  You learned to take models to production with APIs,
  Docker, CI/CD, monitoring, and responsible AI practices.
  You built a complete end-to-end ML system.

+-------------------------------------------------------------+
|                                                             |
|  From "What is Python?" to building production ML          |
|  systems with monitoring, ethics, and CI/CD.                |
|                                                             |
|  You did not just learn ML — you learned to be an           |
|  ML professional.                                           |
|                                                             |
+-------------------------------------------------------------+
```

What Makes You Different Now

You do not just know how to train a model. You know how to:

- Take it from a notebook to a production API
- Package it in Docker and deploy it to the cloud
- Track experiments and manage model versions
- Monitor for drift and performance degradation
- Test new models safely with A/B testing
- Make models faster with quantization and pruning
- Build fair, explainable, and privacy-preserving systems
- Comply with AI regulations
- Showcase your work in a professional portfolio

This is the difference between someone who can build a model and someone who can build an ML system. Companies need the latter.

Your Next Steps

1. Build — Start with the capstone project from Chapter 23 and extend it
2. Share — Put your projects on GitHub, create demos, write about your work
3. Connect — Join ML communities, attend meetups, engage with practitioners
4. Apply — Start applying to roles that match your skills and interests
5. Never stop learning — This field changes fast, and curiosity is your greatest asset

The AI field is young, growing, and full of opportunity. You have the skills, the knowledge, and the projects to succeed. Now go build something amazing.

Good luck on your journey.